

UNIVERSITÉ DE SHERBROOKE
Faculté de génie
Département de génie électrique et de génie informatique

Mise à l'échelle des calculateurs quantiques
grâce aux réseaux de neurones à base de
memristors

Thèse de doctorat
Spécialité : génie électrique

Victor YON

Sherbrooke (Québec) Canada

Octobre 2024

MEMBRES DU JURY

Dominique DROUIN

Directeur

Roger MELKO

Codirecteur

Yann BEILLIARD

Évaluateur

Audrey CORBEIL THERRIEN

Évaluatrice

Sophie ROCHETTE

Évaluatrice

RÉSUMÉ

Cette thèse explore de nouvelles approches pour surmonter les défis de la mise à l'échelle des calculateurs quantiques, en particulier pour les problèmes de calibration automatique des qubits de spin et de correction d'erreur quantique. Les solutions proposées reposent sur des réseaux de neurones artificiels intégrés sur des composants électroniques émergents appelés memristors. Ces derniers exploitent le calcul en mémoire pour implémenter des applications d'apprentissage automatique efficaces, surpassant d'un facteur 100 les processeurs traditionnels en termes de consommation énergétique.

Une méthode de calibration automatique des boîtes quantiques a ainsi été élaborée et validée expérimentalement. Cette approche repose sur la capacité des réseaux de neurones artificiels à identifier des motifs dans les mesures bruitées de courant, afin de confiner un unique électron dans un espace de quelques nanomètres, sans intervention humaine. Une attention particulière a été portée à la quantification de l'incertitude de ces réseaux, permettant une réduction significative des erreurs de calibration. Les résultats obtenus montrent un taux de succès supérieur à 99 % sur un jeu de données hors ligne et 95 % lors des tests en temps réel, démontrant ainsi la robustesse de la méthode proposée.

Dans un second temps, une architecture à base de memristors est proposée pour traiter efficacement les syndromes générés par des codes de correction d'erreur quantique. Ce décodeur répond aux exigences strictes de temps et d'énergie pour une intégration au plus proche des qubits, dans le cryostat, mais au prix d'une réduction de performance induite par des imperfections matérielles. Pour pallier cette limitation, une méthode de ré-entraînement prenant en compte ces imperfections a été développée, permettant de restaurer la fidélité originale du décodeur.

Ces travaux de recherche proposent ainsi des solutions concrètes à des problématiques actuelles de l'informatique quantique, tout en soulignant le potentiel des memristors pour concevoir des applications d'intelligence artificielle à basse énergie.

Mots-clés : memristor, réseau de neurones artificiels, apprentissage automatique, informatique quantique, calibration automatique, boîte quantique, correction d'erreur quantique

ABSTRACT

Scaling-up quantum computers using memristor-based neural networks

This thesis explores new methods to overcome the challenges of scaling quantum computers, focusing specifically on the automatic calibration of spin qubits and quantum error corrections. The proposed solutions rely on artificial neural networks integrated on emerging electronic components known as memristors. These components leverage in-memory computing for machine learning applications, surpassing traditional processors by a factor of 100 in terms of energy efficiency.

In the first phase, an automatic quantum dot calibration method is developed and experimentally validated. This approach relies on artificial neural networks to identify patterns in noisy current measurements, confining a single electron within a nanometer-scale device without human intervention. Special attention is given to quantifying the uncertainty of these networks, leading to a significant reduction in calibration errors. The results show a success rate of over 99 % on an offline dataset and 95 % for a real-time experiment, demonstrating the robustness of the proposed method.

In the second phase, a memristor-based architecture is proposed to efficiently process syndromes generated by quantum error correction codes. This cryogenic decoder architecture meets the strict time and energy requirements necessary for integration near the qubits, though with some performance loss due to hardware imperfections. To address this limitation, a hardware-aware re-training method that accounts for these imperfections is developed, restoring the original fidelity of the decoder.

This research offers concrete solutions to current issues in quantum computing while highlighting the potential of memristors for the design of low-energy artificial intelligence applications.

Keywords: memristor, artificial neural network, machine learning, quantum computing, autotuning, quantum dot, quantum error correction

À ma mère.

REMERCIEMENTS

Sans le soutien de mon entourage, aucune page de cette thèse n'aurait pu exister. À chacun et chacune d'entre vous, j'adresse toute ma gratitude.

À mes amis restés en France : Thibaut, Rebecca, Damien, Solène, Bertrand, Maryan, Guillaume, Charlélie, Myriam, et Jean-Baptiste qui sont restés proches malgré la distance.

Aux amis que j'ai rencontrés au fil de cette aventure : Raphaël, Dorian, Lisa, Pierre-Antoine, Matthieu, Phillipe, Benoît et Tadeáš, pour ces difficultés et succès partagés autour d'innombrables cafés, et bières.

Aux membres de l'équipe d'INPAQT et aux collègues d'Irréversible dont la compagnie et les encouragements ont rendu ce parcours encore plus agréable.

À Dominique et Yann qui m'ont fait confiance tout au long de ce doctorat et qui ont toujours été disponibles pour répondre à mes questions.

À Roger et aux membres de l'équipe PIQUIL pour leur accueil chaleureux et leur bonne humeur.

À Anh pour m'avoir encouragé à entreprendre ce grand projet.

À Megan pour sa précieuse aide tout au long de la rédaction de ce manuscrit.

Aux membres de ma famille : Laurent, Catherine, Thomas, Chantal, Sebastien, Camille, Paloma, Achille et Clara, sur lesquels je peux toujours compter.

À ma sœur qui a toujours été présente pour m'épauler et qui a eu l'amabilité de ne pas terminer sa thèse avant moi !

Et à ma mère, pour son soutien indéfectible depuis 32 ans.

TABLE DES MATIÈRES

1	Introduction	1
1.1	Mise en contexte	1
1.1.1	Innovations matérielles	2
1.1.2	Innovations logicielles	5
1.2	Problématiques	6
1.2.1	Mise à l'échelle des calculateurs quantiques à l'aide de réseaux de neurones	7
1.2.2	Inefficacité énergétique des réseaux de neurones artificiels	8
1.2.3	Les imperfections des memristors	9
1.2.4	Questions de recherche	10
1.3	Objectifs	10
1.3.1	Implémentation de solutions à base de réseaux de neurones artificiels	10
1.3.2	Atténuation des effets indésirables des imperfections des memristors	11
1.4	Contexte du projet	11
1.5	Plan du document	12
2	État de l'art	13
2.1	Calcul quantique	13
2.1.1	Concepts et définitions	13
2.1.2	Qubit de spin	14
2.1.3	Démonstrations modernes	15
2.1.4	Verrous technologiques	16
2.2	Réseaux de neurones artificiels	22
2.2.1	Concepts et définitions	22
2.2.2	Apprentissage profond	23
2.2.3	Topologies de réseaux	25
2.2.4	Accomplissements et limitations	28
2.3	Memristors	30
2.3.1	Concepts et définitions	30
2.3.2	Implémentations physiques	31
2.3.3	Calcul en mémoire	32
2.3.4	Imperfections et paramètres non idéaux	33
2.4	Réseaux de neurones sur memristors	39
2.4.1	Apprentissage sur puce ou déporté	40
2.4.2	Implémentations existantes	41
2.4.3	Effets des paramètres non idéaux et co-conception	42
3	Calibration automatique de boîtes quantiques	45
3.1	Avant-propos	45
3.1.1	Objectif de l'article	46

3.1.2	Résumé en français	46
3.2	Robust quantum dots charge autotuning using neural network uncertainty	48
3.2.1	Abstract	48
3.2.2	Introduction	48
3.2.3	Problem definition	50
3.2.4	Line detection	53
3.2.5	Autotuning	60
3.2.6	Discussion	66
3.2.7	Author contributions	67
3.2.8	Code and data availability	67
3.3	Bilan de l'article	69
4	Calibration automatique de boîtes quantiques expérimentale	71
4.1	Avant-propos	71
4.1.1	Objectif de l'article	72
4.1.2	Résumé en français	72
4.2	Experimental online quantum dots charge autotuning using neural networks	73
4.2.1	Abstract	73
4.2.2	Introduction	73
4.2.3	Methodology	75
4.2.4	Results	79
4.2.5	Discussion	81
4.2.6	Author contributions	82
4.2.7	Code and data availability	83
4.3	Bilan de l'article	84
5	Correction d'erreur quantique à base de memristors	85
5.1	Avant-propos	85
5.1.1	Objectif de l'article	86
5.1.2	Résumé en français	86
5.2	A cryogenic memristive neural decoder for quantum error correction	88
5.2.1	Abstract	88
5.2.2	Introduction	88
5.2.3	Problem statement	91
5.2.4	Electrical characterization	97
5.2.5	Results	98
5.2.6	Discussion	103
5.2.7	Author contributions	105
5.2.8	Code and data availability	106
5.3	Bilan de l'article	107
6	Conclusion	109
6.1	Synthèse des travaux	109
6.2	Perspectives	110
6.3	Contributions scientifiques	111

A Appendix: Robust quantum dots charge autotuning using neural network uncertainty	115
A.1 Datasets	115
A.1.1 Data selection	115
A.1.2 Data processing	116
A.1.3 Diagram annotation	116
A.1.4 Patch segmentation and automatic class labeling	116
A.1.5 Subset splitting	117
A.1.6 Stability diagram samples	117
A.2 Line detection methodology	121
A.2.1 Model training	122
A.2.2 Model uncertainty and confidence threshold calibration	123
A.2.3 Patch classification samples	127
A.3 Autotuning methodology	128
A.3.1 Exploration algorithm	128
A.3.2 Baselines	129
A.4 Results without cross-validation	130
B Appendix: Experimental online quantum dots charge autotuning using neural networks	133
B.1 Datasets	133
B.2 Model training	133
B.3 Confidence score	135
B.4 Run statistics	136
B.5 Patch measurement examples	136
C Appendix: A cryogenic memristive neural decoder for fault-tolerant quantum error correction	139
C.1 Simulation of quantum stabilizer circuits	139
C.2 Other hardware non-idealities	141
C.2.1 Analog-to-digital and digital-to-analog converter range and resolution	141
C.2.2 Reading variability	141
C.3 Experiments on TiO_x -based resistive memory devices	142
C.3.1 Conductance state programming	142
C.3.2 Programming variability characterization	143
C.3.3 Conductance state retention	143
C.4 Neural network training methods	144
C.4.1 Implementation of training and inference	144
C.4.2 Re-training methods	145
LISTE DES RÉFÉRENCES	149

LISTE DES FIGURES

1.1	Architectures de calcul	2
a	Architecture de von Neumann et son goulot d'étranglement	2
b	Architecture basée sur le calcul en mémoire	2
1.2	Nombre de transistors sur un circuit intégré en fonction du temps	3
1.3	Évolution du cout de calcul d'entraînement pour 426 modèles de la littérature	6
1.4	Représentation des différents concepts	7
a	De l'intelligence artificielle jusqu'à l'apprentissage profond	7
b	Domaines de recherche abordés	7
2.1	Photographie d'un calculateur quantique	14
2.2	Exemple de diagramme de stabilité	17
2.3	Représentation schématique de neurones artificiels et de deux exemples de leurs agencements en réseau	26
a	Neurone formel	26
b	Réseau de neurones à convolution	26
c	Perceptron multicouche	26
d	Réseau de neurones récurrents	26
e	Neurone d'un réseau bayésien	26
2.4	Principales catégories de mémoires à accès non séquentiel	32
2.5	Paramètres non idéaux des systèmes memristifs impactant la précision des réseaux de neurones	34
2.6	Comparaison entre un réseau de neurones formels et une implémentation physique basée sur un crossbar de memristors	40
3.1	Quantum dot devices and energy diagrams	52
a	Silicon split gate device	52
b	Gallium arsenide device	52
c	Silicon overlapping gate device	52
d	Silicon split gate energy diagram	52
e	Gallium arsenide energy diagram	52
f	Silicon overlapping energy diagram	52
3.2	Stability diagram examples	54
3.3	Transition line detection using a neural network	55
3.4	Example of transition line classification for a full stability diagram	61
3.5	Autotuning algorithm	63
4.1	Schematic representation of the experimental setup	76
4.2	Example of successful online experimental autotuning	77
4.3	Complete scan of the explored stability diagram	80
5.1	Decoding process	93
5.2	Surface code and stabilizer measurements	94

5.3	Recurrent neural network decoder architecture and corresponding memristive decoder circuit	95
5.4	Programming variability of TiO _x -based resistive memory and its impact on the neural decoder	99
5.5	Baselines and re-training methods for the memristive neural decoder	101
a	Schematic representation of the methods used to train the decoder .	101
b	Logical fault rate as a function of the physical fault rate	101
c	Logical fidelity as a function of the of stuck-at fault devices	101
d	Logical fidelity for the baselines and the re-trained decoders	101
A.1	Silicon split gate stability diagram samples	118
A.2	Gallium arsenide stability diagram samples	119
A.3	Silicon overlapping gate stability diagram samples	120
A.4	Diagram cross-validation methodology schematic	121
A.5	Training progress examples	123
a	Bayesian convolutional neural network	123
b	Convolutional neural network	123
c	Feed-forward neural network	123
A.6	Confidence thresholds calibration	125
a	Evolution of the threshold score	125
b	Histogram of the model classification error	125
A.7	Confusion matrices	126
a	Confusion matrix without using the confidence threshold	126
b	Confusion matrix using the confidence threshold	126
A.8	Patch samples	127
a	Patches correctly classified as “ <i>line</i> ”	127
b	Patches correctly classified as “ <i>no-line</i> ”	127
c	Patches incorrectly classified as “ <i>no-line</i> ” (labeled as “ <i>line</i> ”)	127
d	Patches incorrectly classified as “ <i>line</i> ” (labeled as “ <i>no-line</i> ”)	127
A.9	Detailed exploration strategy schematic	128
B.1	Example of convolutional neural networks training progress	135
B.2	Example of a confusion matrix for a trained convolutional neural networks	135
B.3	Patch samples for each class from the offline training set and the online experiment	137
a	Examples of offline patches labeled as “ <i>no-line</i> ”	137
b	Examples of offline patches labeled as “ <i>line</i> ”	137
c	Examples of online patches correctly classified as “ <i>no-line</i> ”	137
d	Examples of online patches correctly classified as “ <i>line</i> ”	137
C.1	Impact of the ADC and DAC resolution on the decoder’s test fidelity . . .	142
C.2	Impact of the output noise on the decoder’s test fidelity	143
C.3	TiO _x -based resistive memory device retention	144
C.4	Impact of the dropconnect re-training method for various stuck-at-fault rates	147

LISTE DES TABLEAUX

2.1	Travaux publiés sur la calibration automatique de boîtes quantiques pour la formation de qubits de spin	18
3.1	Transition line detection results	60
3.2	Autotuning results for each dataset and model	65
A.1	Summary of dataset specifications	115
A.2	Model architectures and training meta-parameters	121
A.3	Autotuning results using the baselines	130
A.4	Transition line detection results without cross-validation	130
A.5	Autotuning results for each dataset and model without cross-validation . .	131
B.1	Architecture of the convolutional neural networks used to detect transition lines during the experiment	134
B.2	Meta-parameters of the convolutional neural networks for training and inference	134
B.3	Experimental run results	136
C.1	Model and training metaparameters	144
C.2	Default simulated values of the hardware constraints and non-idealities . . .	146

LISTE DES ACRONYMES

- 1R** 1-résistance 31
1S1R 1-sélecteur-1-résistance 31
1TnR 1-transistor-n-résistance 31
3IT institut interdisciplinaire d’innovation technologique 12, 107
- ADC** analog-to-digital converter 96, 100, 103, 104, 141, 142, 145, 146
ASIC application-specific integrated circuit 4, 105
- BCNN** Bayesian convolutional neural network 55–57, 64, 121–123
BNN Bayesian neural network 27, 28, 109
- CMOS** complementary metal-oxide-semiconductor 15, 30, 39, 46, 48, 73, 90, 91, 104
CNN convolutional neural network 18, 19, 27, 28, 42, 53, 56–59, 61, 62, 64, 66, 69, 73, 75–79, 81, 109, 121, 123, 125–127, 133–136
CPU central processing unit 2–5
- DAC** digital-to-analog converter 96, 100, 103, 104, 141, 142, 145, 146
DNN deep neural network 25, 78
DS device-specific 91, 101–103, 141, 142, 144
- FF** feed-forward neural network 18, 27, 56, 57, 121, 123
FPGA field-programmable gate array 3, 81, 105
FTQC fault-tolerant quantum computation 88, 89, 92
- GaAs-QD** gallium arsenide 17, 51–53, 58–60, 63–65, 67, 115, 119, 128–131
GPU graphics processing unit 4, 6, 9
- HCS** highest conductance state 97–100, 143
HWA hardware-aware 88, 91, 101–103, 141, 142, 144, 145, 147
- IA** intelligence artificielle 7, 11, 22, 23, 28, 31, 111
IMC in-memory computation 88, 90, 91, 98, 103, 105
- KL** Kullback–Leibler 28
- LCS** lowest conductance state 97, 99, 100, 143
LFR logical fault rate 101
LLM large language model 22, 29
- MAC** multiply–accumulate 67, 97, 105
ML machine learning 23, 48, 49, 53, 66, 73–75, 81

- MLP** multilayer perceptron 27, 42
- MND** memristive neural decoder 89–91, 96–98, 100–105, 141, 142, 144, 145, 147
- MNIST** modified national institute of standards and technology 28, 39, 42
- MOS** metal-oxide-semiconductor 2, 9, 51
- MVM** matrix–vector multiplication 88, 90, 95–97, 100, 103, 141, 143, 146
- NN** artificial neural network 48–51, 53, 56–58, 60, 62, 64, 66, 67, 74, 90, 91, 96, 98, 102, 104, 105, 116, 122–124, 126, 133, 136
- PFR** physical fault rate 98, 99, 101, 103, 142, 143, 147
- QD** quantum dot 15, 48–53, 56, 59, 62, 64, 66, 67, 73–76, 78–82, 92, 115, 133, 136
- QEC** quantum error correction 88, 89, 91–93, 95, 96, 104, 105, 139, 140
- QPC** quantum point contact 17, 48, 51, 52, 115
- RAM** random access memory 31, 32
- ReLU** rectified linear unit 92, 95, 96, 103, 144
- ReRAM** redox-based RAM 32, 35
- RNA** réseau de neurones artificiels 7–12, 18, 22–25, 27, 29–31, 33, 34, 39–42, 46, 72, 86, 87, 107, 109–111
- RNN** recurrent neural network 21, 27, 89–93, 95–103, 109, 139–146
- SEM** scanning electron microscope 52, 76
- SET** single-electron transistor 48, 51–53, 60, 75, 76, 78, 80, 81, 115
- SFQ** since single-flux quantum 89
- Si-OG-QD** silicon overlapping gate 51–53, 58–61, 64, 65, 67, 83, 115, 120, 125–131
- Si-SG-QD** silicon split gate 51–53, 58–60, 62, 64–66, 115, 118, 123, 130, 131
- SRAM** static RAM 32, 90
- TIA** transimpedance amplifier 104
-

CHAPITRE 1

Introduction

1.1 Mise en contexte

“J’ai gagné à la loterie!” et “Il y a exactement un électron dans cette boîte quantique.” sont deux phrases qui contiennent de l’*information*. En 1948, lorsque Claude Shannon se demande s’il est possible de la quantifier, il arrive à la conclusion que la quantité d’information transmise par un message est inversement proportionnelle à la probabilité de l’événement décrit [1]. Concrètement, lorsque quelqu’un ayant joué à la loterie annonce avoir gagné, la quantité d’information est très grande, car ce résultat est inattendu. En considérant une chance de gagner sur 30 millions, l’annonce d’une victoire contient environ 25 bits d’information. En revanche, cette personne ne prendra certainement pas la peine de communiquer sur sa défaite, puisque ce résultat est tellement probable qu’il ne contient presque aucune information ($< 0,000\,000\,1$ bit). Lors de la conception d’un ordinateur quantique, le nombre d’électrons confinés est aussi une information que l’on souhaite obtenir. Cependant, comme nous le verrons dans les chapitres 3 et 4, accéder à cette information n’est pas aussi simple que de lire un ticket de loterie.

Cette définition précise et quantifiable de l’*information* a largement contribué à l’émergence de l’informatique moderne. Dans la même période, la machine de Turing [2] a décrit formellement comment un algorithme peut automatiquement traiter l’information, et l’architecture de von Neumann [3] a permis d’incarner cette machine sur un support physique, donnant ainsi naissance à l’ordinateur moderne. Depuis, l’informatique n’a cessé de progresser pour répondre aux besoins grandissants de la société, faisant face à des problèmes de plus en plus complexes [4]. De l’étude de la cosmologie, supportée par les gigaoctets (10^{11} bits) de données récoltées tous les jours par le télescope spatial James Webb [5], jusqu’au pétaoctet (10^{15} bits) mesuré quotidiennement au grand collisionneur de hadrons (LHC) [6, 7], sans oublier l’exaoctet (10^{18} bits) de vidéo consommé chaque jour sur Netflix¹, traiter efficacement l’information est devenue un enjeu crucial [8, 9]. Cette problématique est accentuée par les contraintes énergétiques [10, 11] et environnementales [12, 13], puisque le traitement de l’information réalisé dans les centres de données est responsable de 2 % de la consommation électrique mondiale [14].

1. Estimé à partir de about.netflix.com/news/what-we-watched-the-first-half-of-2024 et 3 Go/h.

La raison d'être de la recherche en informatique est donc de concevoir des systèmes capables de manipuler efficacement l'information, en prenant en compte les contraintes physiques. Pour cela, deux grandes approches ont historiquement contribué à faire avancer cette science : les innovations matérielles et logicielles.

1.1.1 Innovations matérielles

L'information ne pouvant exister sans un support physique [15], le premier enjeu est alors de trouver le support le plus adapté pour manipuler l'information de façon à résoudre les problèmes qui nous intéressent. Les premières tentatives pour concevoir un calculateur [16] sont passées par des systèmes mécaniques [17], électromécaniques [18] et analogiques [19]. Mais c'est l'invention du transistor en 1947 [20] qui a permis la conception de systèmes numériques, compacts, rapides et fiables. Puis, le passage du germanium au silicium en 1955 [21], menant ensuite à la technologie métal oxyde semi-conducteur, *metal-oxide-semiconductor (MOS)* en anglais, a propulsé la production industrielle des processeurs informatiques. Ces composants électroniques exploitent les propriétés électriques des semi-conducteurs pour manipuler l'information sous forme de courants électriques, et leur prédisposition à représenter des états binaires (0 ou 1) en fait le support idéal pour encoder des bits.

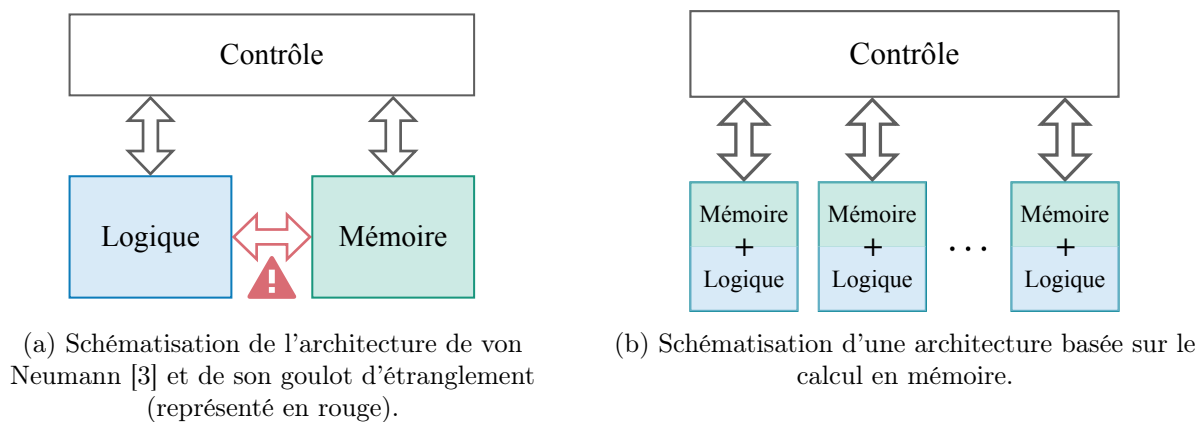


FIGURE 1.1 Architectures de calcul.

Avec plusieurs milliers de milliards (10^{12}) de transistors par habitant sur la planète², ces composants sont omniprésents dans nos vies et indissociables des sociétés modernes. En suivant les principes définis par l'architecture de von Neumann [3] (figure 1.1a), les transistors ont été regroupés en unités de calculs, appelées processeurs ou *central processing units (CPUs)* en anglais, qui exécutent des instructions séquentiellement pour traiter l'information. En 1965, Gordon Moore a conjecturé à partir d'observations empiriques que

2. Estimé à partir de : computerhistory.org/blog/13-sextillion-counting-the-long-winding-road-to-the-most-frequently-manufactured-human-artifact-in-history

le nombre de transistors sur un circuit intégré doublerait tous les deux ans [22]. Cette prédiction auto-réalisatrice a servi de feuille de route pour l'industrie, qui n'a cessé d'innover pour suivre cette tendance (figure 1.2). En 2024, l'industrie des semi-conducteurs est estimée à une valeur de 611 milliards de dollars [23], et la moindre perturbation dans cette production effrénée a des conséquences sur l'ensemble de l'économie [24–27].

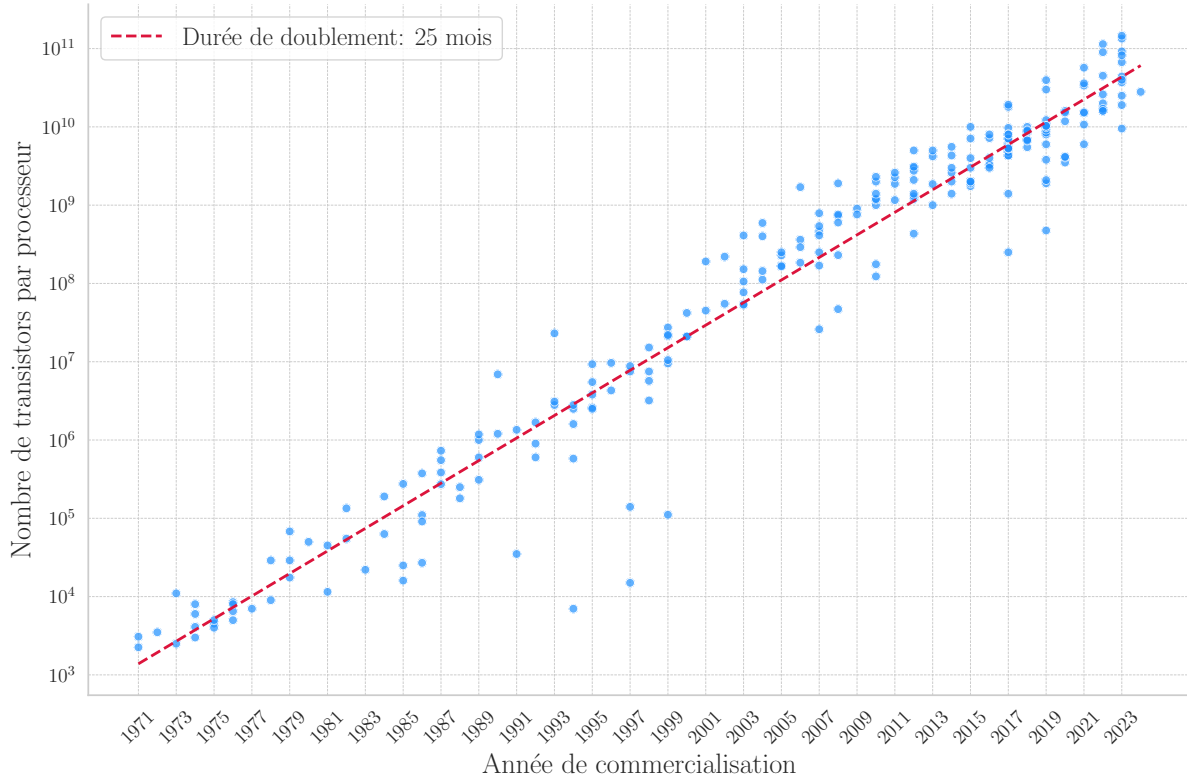


FIGURE 1.2 Évolution du nombre de transistors sur un circuit intégré en fonction du temps.

La fin du règne de la loi de Moore a été annoncée depuis longtemps [28–31], mais sa longévité a fait mentir les plus pessimistes. Il est cependant indéniable que cette tendance soit devenue plus difficile à suivre. Le temps où il “suffisait” de réduire la taille des transistors pour en mettre plus sur une puce est révolu [32, 33]. Des solutions alternatives ont donc été développées pour continuer à faire progresser nos capacités de calcul.

Une première solution est de compléter la flexibilité des CPU (capable d'exécuter n'importe quel programme³) avec des processeurs spécialisés, moins puissants, mais plus efficaces pour réaliser des tâches très précises [35]. Par exemple, les circuits logiques programmables, *field-programmable gate arrays (FPGAs)* en anglais, peuvent être reconfigurés pour réaliser des calculs spécifiques [36, 37]. Tandis que des circuits intégrés sur mesure,

3. Dans les limites définies par la thèse de Church-Turing [34].

application-specific integrated circuits (ASICs) en anglais, sont optimisés pour réaliser très efficacement une tâche unique, mais viennent avec un cout de fabrication plus élevé [38]. Ce sont cependant les processeurs graphiques, *graphics processing units (GPUs)* en anglais, qui ont été déployés à la plus grande échelle. Conçus pour générer des images 3D en temps réel, ils sont capables de réaliser des milliers de calculs très simples simultanément, là où un CPU ne peut en traiter qu'une dizaine en même temps. Initialement popularisés par les jeux vidéo, la capacité inédite de calculs parallèles à moindre cout des GPU a attiré l'attention des scientifiques et des ingénieurs [39–41]. Ainsi, leur usage a rapidement été détourné pour réaliser des calculs intensifs hautement parallélisables, tels que le minage de cryptomonnaies [42, 43], l'apprentissage profond (détaillé dans la section 2.2) et même la découverte de nombres premiers⁴.

Une autre façon de faire évoluer la loi de Moore est de repenser l'architecture traditionnelle de von Neumann [3], pour concevoir des systèmes plus efficaces [44]. Un ordinateur classique possède des composants discrets et modulables, pouvant facilement être remplacés ou mis à jour. C'est le cas par exemple des unités de mémoire (disque dur, mémoire vive, mémoire cache, etc., représentés en vert sur la figure 1.1a) et des unités de calcul (CPU, GPU, etc., représentés en bleu sur la figure 1.1a). Réorganiser les blocs de mémoire et de calculs peut drastiquement améliorer l'efficacité énergétique et la vitesse de traitement, mais encore une fois au détriment de la flexibilité. C'est par exemple la promesse des mémoires émergentes [45], telles que les memristors (détaillé dans la section 2.3), qui proposent de combiner les fonctions de mémoire et de calculs dans un même composant, pour économiser les coûteux transferts de données entre les deux unités, comme illustré sur la figure 1.1b.

Enfin, la solution la plus radicale pour passer un nouveau cap dans le traitement de l'information est de changer de paradigme de calcul. Tous les ordinateurs évoqués précédemment manipulent l'information en se basant sur la physique classique, notamment l'électromagnétisme pour représenter des bits logiques. Mais comme l'a fait remarquer Yuri Manin en 1980 [46] (puis Richard Feynman [47]), le monde quantique offre des propriétés uniques permettant de réaliser certains calculs beaucoup plus efficacement (détaillé dans la section 2.1). Un ordinateur quantique fonctionnel serait en effet capable de résoudre de nombreux problèmes concrets tels que la factorisation d'entiers [48], la découverte de nouveaux médicaments [49, 50] ou l'optimisation financière [51] en un temps polynomial, là où un ordinateur classique nécessiterait un temps exponentiel.

4. Le plus grand nombre premier connu à ce jour ($2^{136279841} - 1$) a été découvert grâce à un réseau de plusieurs milliers de GPU : mersenne.org/primes/press/M136279841.html.

1.1.2 Innovations logicielles

Quel que soit le support physique, l'information ne peut être traitée sans une surcouche logicielle pour abstraire les concepts matériels et permettre aux utilisateurs de manipuler les données de manière intuitive. Cette étape conditionne aussi grandement l'efficacité du traitement, puisque c'est à ce niveau que les algorithmes sont conçus pour résoudre des problèmes. Par exemple, multiplier naïvement deux matrices de 4096×4096 (contenant 10^9 bits d'information) avec le langage de programmation Python prend 7 heures, tandis que le même calcul implémenté en utilisant des instructions spécifiquement optimisées pour le CPU prend 0,41 seconde sur la même machine [52], soit un gain de vitesse d'un facteur 60 000 simplement en utilisant un algorithme plus adapté et en envoyant des instructions plus spécifiques à l'unité de calcul. Cet exemple est particulièrement pertinent dans le cas de l'apprentissage profond, où la majorité des opérations reposent sur des multiplications matricielles (sections 2.2 et 2.4).

Historiquement les programmes informatiques reposaient presque exclusivement sur des méthodes formelles, basées sur des concepts mathématiques rigoureux pour garantir la validité des résultats. On sait par exemple que l'algorithme *quicksort* [53] sera toujours capable de trier une liste de taille n en un maximum de $\mathcal{O}(n \log n)$ opérations, quel que soit le contenu de la liste. Cette approche est toujours largement préférable dans de nombreux domaines, mais elle est intrinsèquement incompatible avec la manipulation de notions floues, tels que la reconnaissance d'images ou que le traitement du langage naturel. Il est en pratique impossible de définir formellement ce qu'est un chat, ou de décrire sans ambiguïté la sémantique du mot "abracadabrantique". Ce problème est intéressant car il ne dépend pas uniquement du nombre d'opérations par seconde qu'un supercalculateur est capable de réaliser, mais principalement de notre capacité à décrire le problème dans un langage informatique. Conscient de cette limitation, Alan Turing a proposé dès 1950 de remplacer, dans ce contexte, les méthodes formelles par des méthodes empiriques d'apprentissage, s'inspirant du fonctionnement du cerveau humain [54] (section 2.2).

Bien que le concept d'apprentissage automatique soit presque aussi vieux que celui du transistor, il a fallu attendre les années 2010 pour que cette idée porte ses fruits. Il existe plusieurs raisons à ce délai : (i) il a fallu trouver une méthode d'apprentissage efficace pour les neurones artificiels [55], les réseaux de neurones [56, 57] et les réseaux profonds [58] ; (ii) il a ensuite fallu passer par beaucoup de déconvenues avant d'arriver à la conclusion qu'une approche formelle ne fonctionnerait pas [59] ; et (iii) il a finalement fallu que la puissance de calcul disponible soit suffisante pour entraîner des réseaux d'envergure intéressante. Pendant toute cette période de tâtonnement, de 1950 à 2010, les modèles

proposés nécessitaient une puissance de calcul modeste (figure 1.3), avec un doublement du cout d’entraînement tous les 20 mois. Mais après 2010, avec l’arrivée de l’apprentissage profond [58] et la démocratisation des GPU, la demande a explosée, avec un doublement de la puissance de calcul tous les 6 mois (figure 1.3). À partir de ce moment, même la cadence exponentielle de la loi de Moore n’a plus été suffisante pour suivre la demande de calcul, renvoyant ainsi la balle dans le camp des fabricants de matériel [60].

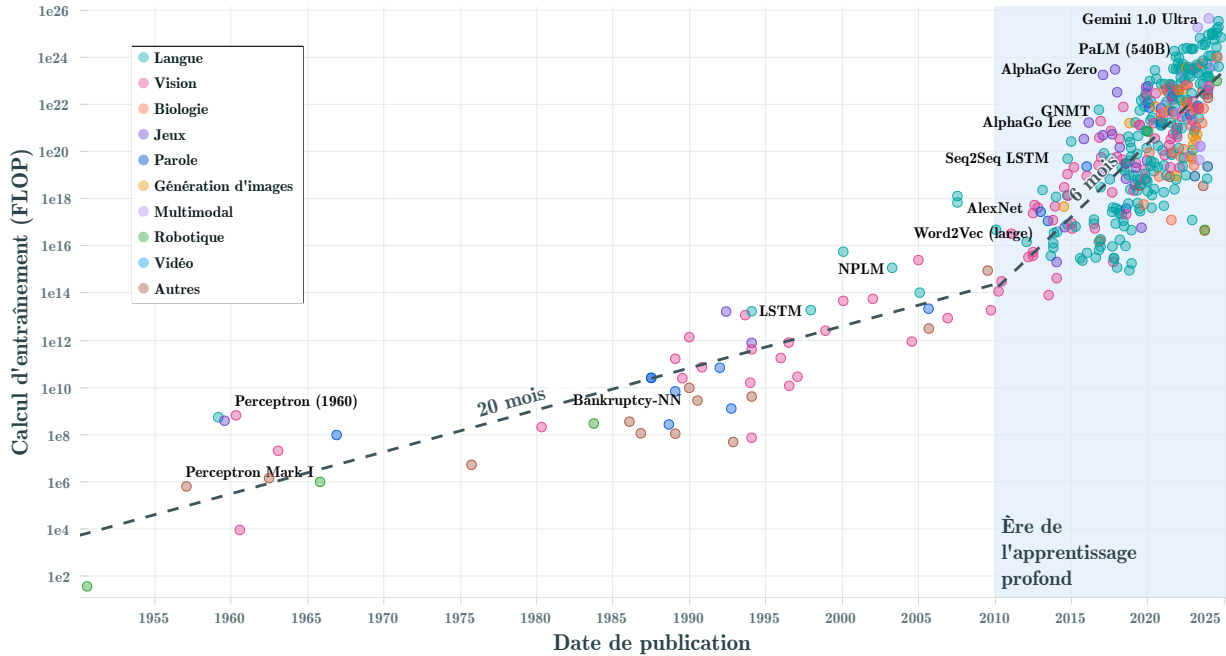


FIGURE 1.3 Évolution du cout de calcul d’entraînement pour 426 modèles de la littérature. L’axe des y représente la puissance de calcul estimée pour entraîner chaque modèle, en nombre d’opérations en virgule flottante par seconde (FLOPS). Les droites en pointillés représentent la durée de doublement de la puissance de calcul, avant et après 2010. L’ère de l’apprentissage profond se démarque par la démocratisation des processeurs graphiques [61] et la popularisation des réseaux de neurones profonds [58]. Figure adaptée de [62].

1.2 Problématiques

Il existe donc de nombreuses façons de faire progresser la science du traitement de l’information, et les différents axes de recherche s’entremêlent, se croisent et se soutiennent mutuellement. Dans cette thèse, nous allons porter notre attention sur les problématiques liées à la mise à l’échelle concrète des calculateurs quantiques. Et pour ce faire, nous allons nous appuyer sur deux innovations logicielles et matérielles récentes : les réseaux de neurones artificiels et les memristors (figure 1.4b).

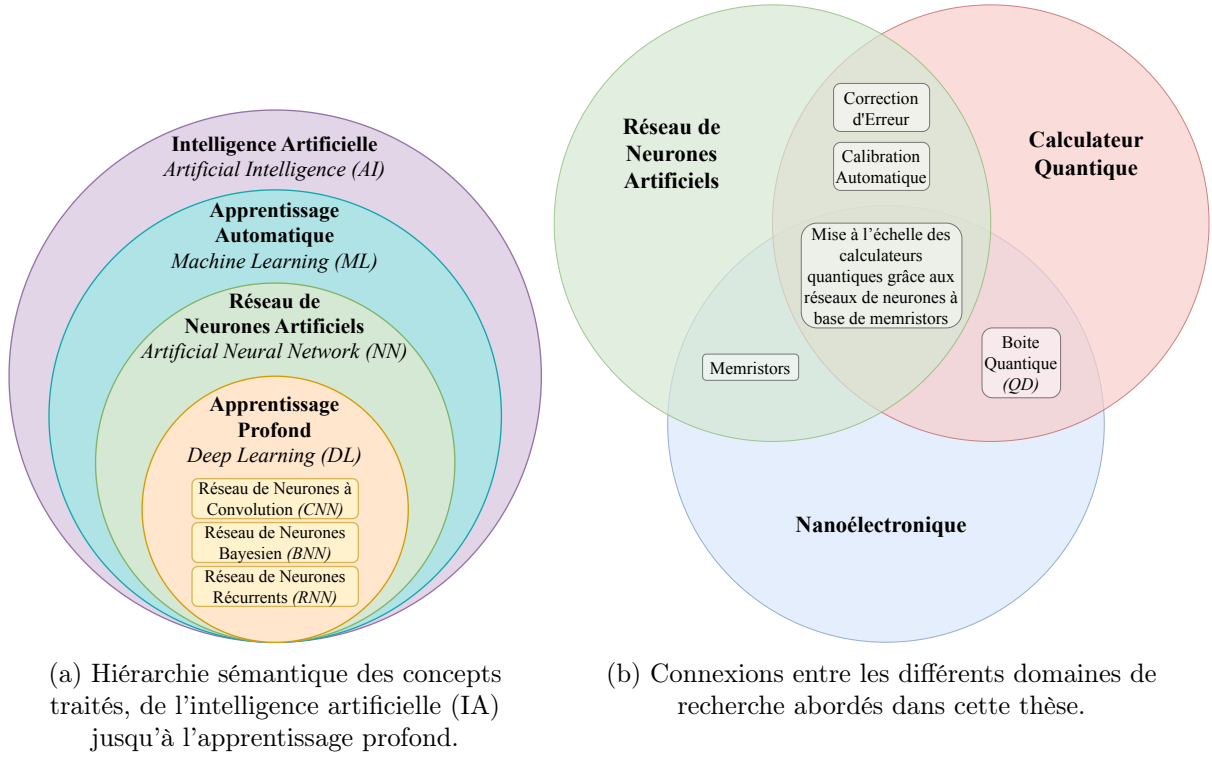


FIGURE 1.4 Représentation des différents concepts sous forme de diagrammes de Venn.

1.2.1 Mise à l'échelle des calculateurs quantiques à l'aide de réseaux de neurones

Un calculateur quantique fonctionne parfaitement sur le papier [63], mais son implémentation physique extrêmement contraignante rend son déploiement à grande échelle très difficile. Malgré la réalisation de nombreuses preuves de concepts [64–67], aucun ordinateur quantique réellement utile n'a encore vu le jour. Parmi les nombreux obstacles au déploiement de ces systèmes, nous allons nous intéresser à deux verrous technologiques qui peuvent être résolus en utilisant des réseaux de neurones artificiels (RNA) [68] : la calibration de boîte quantique et la correction d'erreur quantique. Dans les deux cas, ces problèmes doivent être traités dans l'environnement cryogénique, au plus proche des calculateurs quantiques, afin de contourner un troisième verrou technologique : le goulot d'étranglement des circuits de contrôle (section 2.1.4.3). Ce dernier impose des contraintes très strictes sur le budget thermique (et donc énergétique) des systèmes de contrôle.

1.2.1.1 Calibration de boîte quantique

Utiliser la mécanique quantique pour réaliser des calculs peut se faire par l'intermédiaire de différents supports physiques. Exploiter le spin d'un électron est l'une des méthodes des plus prometteuses (section 2.1.2), mais pour y parvenir, il faut être capable de piéger et contrôler finement un électron dans un espace de quelques nanomètres (figure 3.1). Il

est ensuite nécessaire de répéter cette opération des centaines, voire des milliers de fois, pour instancier suffisamment de portes logiques afin d'exécuter des programmes.

Aujourd'hui, les boîtes quantiques sont majoritairement calibrées par des chercheurs ou des ingénieurs qui ajustent manuellement les paramètres de contrôle en fonction des mesures effectuées [69]. Cette approche n'est pas compatible avec la mise à l'échelle de cette technologie pour plusieurs raisons : (i) il serait trop coûteux de faire appel à un expert chaque fois qu'un ordinateur quantique doit être initialisé ; (ii) le temps de calibration manuelle d'une seule boîte peut prendre plusieurs heures ; et (iii) cela implique de transférer les mesures hors de l'environnement cryogénique, vers des appareils de contrôle à température ambiante (figure 2.1).

Il n'est cependant pas trivial de remplacer l'humain par une machine pour réaliser cette tâche. Les très faibles signaux mesurés pour informer de l'état d'une boîte quantique doivent être amplifiés, induisant du bruit et rendant les approches formelles très difficiles à appliquer. De plus, les procédés de fabrications à l'échelle nanométrique sont sujets à des variations aléatoires, rendant chaque boîte quantique unique et imposant une calibration sur mesure.

1.2.1.2 Correction d'erreur quantique

Tout support physique contenant de l'information peut être corrompu. Cela peut venir d'une perturbation extérieure (bruit thermique, variation de voltage, rayonnement cosmique, etc.) ou d'une erreur interne (défaut de fabrication, usure, etc.). Les unités de calculs classiques bénéficient de procédures de correction d'erreur bien établies et relativement faciles à mettre en place [70–72]. Mais cette tâche est beaucoup moins évidente dans le cas des calculateurs quantiques, d'une part, car les états quantiques sont par nature très sensibles à l'environnement, et d'autre part parce que l'information qu'ils encodent ne peut être copiée ni mesurée sans être détruite [63].

Ces caractéristiques font de la correction d'erreur un défi majeur et une nécessité absolue pour la réalisation de calcul quantique à grande échelle. Une piste prometteuse pour y parvenir est d'utiliser des codes correcteurs d'erreur quantique [73–75], permettant de détecter et corriger les erreurs sans détruire l'information. Cette approche demande cependant de réaliser des opérations de mesure et de correction très précises et rapides [76], directement dans l'environnement cryogénique.

1.2.2 Inefficacité énergétique des réseaux de neurones artificiels

Les deux problèmes mentionnés précédemment peuvent être adressés avec des RNA, mais les contraintes thermiques imposées par l'environnement cryogénique semblent à première

vue incompatibles avec la consommation énergétique colossale de ces modèles. Un GPU *NVIDIA H100 SXM* utilisé dans les centres de calculs pour entraîner et exécuter de larges RNA, consomme jusqu'à 700 W pour réaliser 6×10^{16} opérations en virgule flottante par seconde⁵, alors que la puissance de refroidissement d'un réfrigérateur à dilution contenant un calculateur quantique est typiquement de l'ordre de quelques watts. Cette consommation énergétique est principalement due à deux facteurs : la taille des modèles (nombre de paramètres) et l'architecture des processeurs, qui n'est pas optimisée pour cette tâche.

La taille des RNA n'a cessé d'augmenter depuis leur conceptualisation en 1950 [62], pour une raison simple : plus de paramètres se traduit presque toujours par de meilleures performances [77–79]. Mais malheureusement, cette tendance implique aussi une augmentation du cout de calcul (figure 1.3), et donc de consommation énergétique.

Les GPU ont permis de faire passer les RNA dans une nouvelle ère en 2010 grâce à leur capacité de calcul parallèle [61]. Mais aussi puissantes soient-elles, ces unités de calculs reposent sur l'architecture de von Neumann [3], qui impose des transferts de données entre les unités de calcul et de mémoire. Cette limitation fondamentale, connue sous le nom de goulot d'étranglement de von Neumann (figure 1.1a), est particulièrement préjudiciable pour les RNA, dont les calculs nécessitent une communication constante et intensive entre ces deux composants. Concrètement, cela signifie que la majorité de la puissance consommée par un GPU est utilisée pour déplacer des données, et non pour réaliser des calculs [10, 80].

1.2.3 Les imperfections des memristors

Les memristors [81–83], ou mémoires résistives, sont des composants électroniques émergents particulièrement prometteurs pour la réalisation de calcul en mémoire (figure 1.1b), évitant ainsi le goulot d'étranglement de von Neumann. De plus, il est possible d'exploiter pleinement les bénéfices de ce paradigme de calcul lors de la réalisation des opérations matricielles nécessaires à l'utilisation de RNA (section 2.4). Cependant, le développement de ces composants n'ayant débuté qu'en 2008 [83], leur maturité technologique est loin d'égaler les composants MOS classiques, qui ont eux bénéficié de 70 ans de recherche et développement.

Par conséquent, les memristors sont sujets à de nombreuses imperfections et limitations affectant la précision des calculs (section 2.3.4). Une partie de ces problèmes trouve son origine dans des procédés de fabrication encore difficilement reproductibles, qui ne permettent pas de fabriquer des composants parfaitement identiques, induisant des variations de performances d'un composant à l'autre et un rendement de production plus faible. Le

5. Informations techniques de NVIDIA : nvidia.com/h100

volume de production est aussi très restreint, ce qui rend ces composants plus coûteux à produire et à intégrer sur des systèmes existants. De plus, il n'est pas encore possible de rassembler plus de quelques dizaines de milliers de memristors sur une même puce, limitant ainsi la taille des RNA pouvant être implémentés, loin des milliards de paramètres utilisés par les plus grands modèles actuels [84, 85].

1.2.4 Questions de recherche

Ces problématiques mènent aux deux principales questions abordées dans cette thèse de doctorat :

1. Comment les réseaux de neurones artificiels à base de memristors peuvent contribuer à l'automatisation du contrôle de boîtes quantiques et à la correction d'erreur quantique ?
2. Comment prendre en compte les imperfections des memristors dans les réseaux de neurones afin d'en atténuer les effets indésirables, sans compromettre leur avantage énergétique ?

1.3 Objectifs

Pour répondre à ces questions, nous allons commencer par établir des méthodes à base de RNA pour calibrer des boîtes quantiques et corriger les erreurs quantiques. Puis, nous allons étudier comment implémenter au mieux ces solutions en considérant les imperfections des memristors et les contraintes physiques imposées par l'environnement cryogénique. Au cours de ces différentes étapes, nous nous efforcerons d'utiliser des méthodes pouvant être mises à l'échelle et compatibles avec une production industrielle.

1.3.1 Implémentation de solutions à base de réseaux de neurones artificiels

La première partie des travaux consistera donc à développer des méthodes à base de RNA pour la calibration de boîtes quantiques et la correction d'erreur quantique. Pour chacune de ces deux applications, il sera nécessaire de :

1. **Concevoir une base de données d'entraînement.** La performance des RNA reposant en grande partie sur la qualité et la quantité de données d'entraînement [77–79], elles devront être suffisamment représentatives des situations réelles pour garantir la généralisation des modèles. L'absence de jeux de données standards pour les problématiques qui nous intéressent nous oblige à collecter et agréger nous même ces informations [86].

2. **Choisir une architecture de réseau de neurones.** Il faudra identifier et tester différentes topologies de RNA [87] pour trouver la plus adaptée à chaque problème, tout en prenant en compte les contraintes de dimensions imposées par les memristors.
3. **Entraîner, tester et optimiser les RNA.** Les modèles développés devront être entraînés sur les données collectées, puis évalués dans des conditions aussi proches que possible de celles rencontrées en situation réelle. Plusieurs itérations pourront être réalisées afin d’optimiser les méta-paramètres des modèles.

À la fin de ces étapes, nous disposeront d’une évaluation quantitative de la performance des RNA numériques pour chacune des tâches. Ces scores nous serviront de base de référence à comparer avec les performances des modèles analogiques implémentés par la suite.

1.3.2 Atténuation des effets indésirables des imperfections des memristors

La seconde partie consistera à étudier les imperfections des memristors affectant les performances des RNA, puis de développer des méthodes d’atténuation pour répondre aux problématiques de la mise à l’échelle des calculateurs quantiques.

1. **Caractérisation des imperfections des memristors.** Il sera nécessaire d’identifier les potentiels défauts de ces composants et collecter des données expérimentales sur leurs caractéristiques. Ces informations serviront à concevoir des modèles numériques simples pour simuler leurs comportements.
2. **Évaluation de l’impact des imperfections.** L’effet des imperfections matérielles sur les performances des RNA pourra être estimé par des simulations numériques, et les résultats pourront être comparés aux scores de référence définis précédemment.
3. **Développement de méthodes de mitigation.** Grâce aux simulations précédentes, il sera possible d’identifier les imperfections les plus préjudiciables pour les applications ciblées, et de concevoir des méthodes pour les atténuer. Les performances des RNA seront à nouveau comparées aux scores de référence pour évaluer l’efficacité de ces méthodes.

L’ensemble de ces travaux permettra de répondre aux questions de recherche définies préalablement, en proposant des solutions concrètes pour lever les verrous technologiques bloquant la mise à l’échelle des calculateurs quantiques.

1.4 Contexte du projet

Cette thèse de doctorat s’inscrit donc dans un cadre multidisciplinaire, à l’intersection de l’intelligence artificielle (IA), de l’informatique quantique et de la nanoélectronique (figure 1.4). L’Université de Sherbrooke a été le cadre idéal pour mener à bien ce projet de

recherche, grâce à son expertise reconnue dans ces domaines, et la présence de l’institut interdisciplinaire d’innovation technologique (3IT)⁶ et de l’Institut quantique (IQ)⁷. Les mémoires émergentes développées et caractérisées par le groupe de recherche du Professeur Dominique Drouin au 3IT ont été les fondations de ce projet, tandis que l’expertise du Professeur Michel Pioro-Ladrière et du Professeure Éva Dupont-Ferrier ainsi que leurs étudiants a rendu possible les expériences sur les qubits de spin. Enfin, le Professeur Roger Melko a apporté de précieuses connaissances en informatique quantique et en apprentissage automatique.

1.5 Plan du document

Les cinq prochains chapitres de ce document sont organisés de la manière suivante :

Dans le **chapitre 2**, nous allons définir les concepts abordés durant cette thèse et présenter l’état de l’art des domaines de recherche concernés. Ceci couvrira : le calcul quantique, les réseaux de neurones artificiels, les memristors, et pour finir l’intersection de ces deux derniers domaines.

Les chapitres 3 à 5 décriront en détail les travaux originaux réalisés pour répondre aux questions de recherche. Ces trois chapitres seront présentés sous la forme d’articles scientifiques. Le **chapitre 3** décrira les méthodes développées pour automatiser la calibration de boîtes quantiques en utilisant des RNA. Le **chapitre 4** présentera la mise en place expérimentale de ces méthodes de calibration. Et pour finir, le **chapitre 5** abordera la problématique de la correction d’erreur quantique ainsi que la méthode développée pour atténuer les effets indésirables des imperfections des memristors. La première partie des objectifs (section 1.3.1) de cette thèse sera donc principalement traitée dans le cadre de la calibration automatique, tandis que la seconde partie des objectifs (section 1.3.2) sera réalisée dans le contexte de la correction d’erreur quantique.

Nous terminerons ce document par une synthèse des travaux réalisés et des perspectives pour les travaux futurs dans le **chapitre 6**.

6. 3IT : usherbrooke.ca/3it

7. IQ : usherbrooke.ca/iq

CHAPITRE 2

État de l’art

2.1 Calcul quantique

2.1.1 Concepts et définitions

2.1.1.1 Qubit

Concevoir un ordinateur revient à choisir un support pour encoder l’information, de façon à pouvoir la manipuler efficacement pour résoudre les problèmes qui nous intéressent. Dans le cas du cerveau biologique, le support physique est le neurone, résultant de 500 millions d’années d’évolution [88]. Un ordinateur classique exploite le concept de bit [1], dont la valeur binaire peut facilement être représentée par l’état physique d’un transistor. L’avantage fondamental du ordinateur quantique réside dans l’utilisation de qubits [63] pour représenter l’information. Contrairement au bit classique, le qubit peut-être dans un état de superposition, ce qui lui permet de représenter plusieurs états simultanément. Aussi contre-intuitives soient-elles, les propriétés de la matière à l’échelle quantique ouvrent la voie à un nouveau paradigme de calcul, permettant de résoudre des problèmes hors de portée des ordinateurs reposant sur la physique classique.

Il est aussi important de distinguer le qubit physique, support matériel de l’information et donc sujet aux erreurs, du qubit logique [89], qui est une abstraction plus robuste résultant de la combinaison de plusieurs qubits physiques. Un qubit logique peut intégrer un protocole de correction d’erreurs (section 2.1.4.2) pour atteindre le seuil de fiabilité nécessaire à l’implémentation d’algorithmes quantiques.

2.1.1.2 Calculateur quantique

Le calcul quantique [63] est une branche de l’informatique qui étudie les systèmes de calculs basés sur les principes de la mécanique quantique. En utilisant les principes de superposition et d’intrication [90] sur des qubits, un ordinateur quantique peut très efficacement manipuler l’information, à condition d’être parfaitement isolé de l’environnement extérieur. Un tel système peut en théorie résoudre des problèmes complexes en un temps polynomial, là où un ordinateur classique nécessiterait un temps exponentiel. Ce qui permettrait de résoudre des problèmes importants tels que la simulation quantique [47, 91], l’exécution d’algorithmes de recherche rapide [92], ou la résolution de problèmes d’optimisation NP-difficiles [93, 94].

Les prototypes de calculateurs quantiques fabriqués aujourd'hui se présentent sous la forme de circuits électroniques refroidis à des températures proches du zéro absolu au sein de cryostats (figure 2.1).

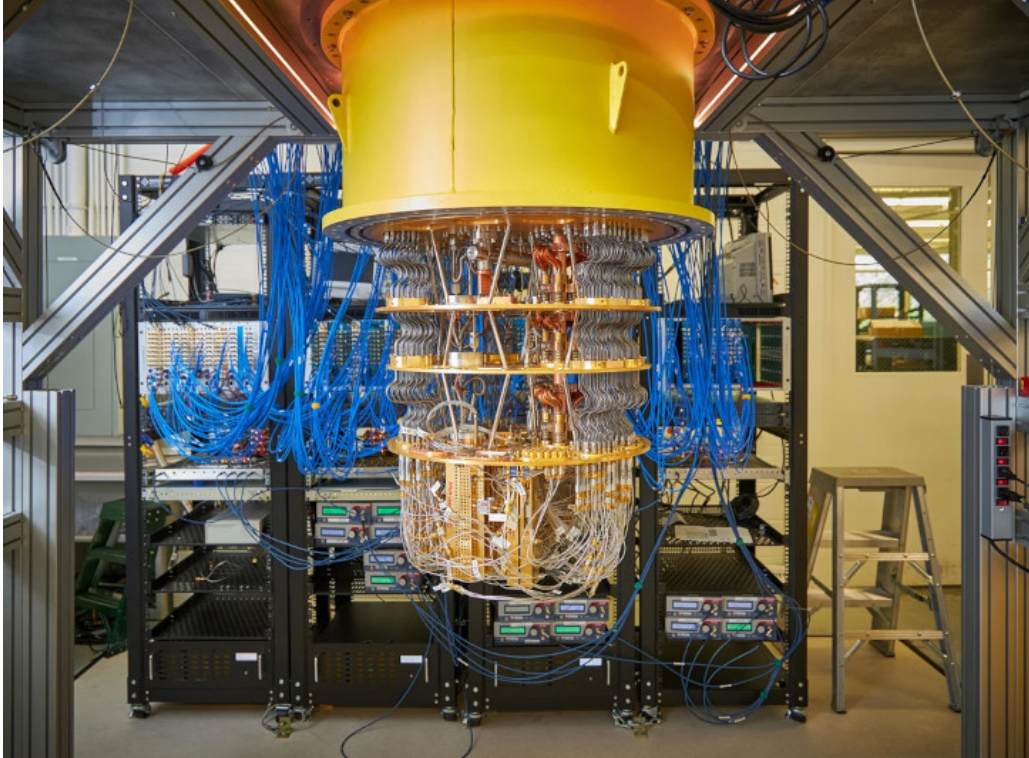


FIGURE 2.1 Photographie d'un ordinateur quantique, incluant un cryostat ouvert (cylindre jaune) et son électronique de contrôle (dans les racks noirs). Lors de son fonctionnement, le cryostat est intégralement fermé pour isoler les qubits de l'environnement extérieur. Les câbles bleus permettent de connecter les composants à l'intérieur du cryostat (refroidi à des températures proches du zéro absolu) avec l'électronique de contrôle (à température ambiante). Crédit : Rocco Ceselin [95].

2.1.2 Qubit de spin

Un qubit de spin permet d'encoder de l'information dans l'orientation du spin d'une particule, généralement un électron. Le spin est une propriété intrinsèque des particules quantiques qui peut être visualisée comme un moment angulaire. Dans ce contexte, les deux états de base du qubit, notés $|0\rangle$ et $|1\rangle$, correspondent aux deux orientations possibles du spin : “spin vers le haut” ($\uparrow$) et “spin vers le bas” ($\downarrow$). Cette propriété peut exister en superposition, permettant de représenter plusieurs états simultanément, et être intriquée avec d'autres qubits pour réaliser des opérations quantiques.

Il existe plusieurs méthodes pour former des qubits de spin [96], par exemple avec des nanotubes de carbone [97,98] ou donneurs de phosphore placés dans un réseau de silicium pur [99]. Mais dans le cadre de ce projet, nous allons spécifiquement nous intéresser à la

méthode proposée en 1997 par Daniel Loss et David DiVincenzo [100], consistant à former des qubits de spin en isolant des électrons à l’aide de grilles électrostatiques à la surface d’un semi-conducteur, dans des puits de potentiel électromagnétique, appelés boîtes quantiques [101, 102], *quantum dots (QDs)* en anglais. Cette méthode a la particularité d’être implémentable sur des substrats de silicium, permettant de bénéficier des infrastructures matures de nano-fabrication industrielle *complementary metal-oxide-semiconductor (CMOS)* [103–105]. De plus, il a été démontré que les qubits de spin ont des temps de cohérence relativement longs [106, 107], présentent une bonne robustesse thermique avec des températures de fonctionnement supérieures à 1 K [108], et permettent de concevoir des portes logiques ayant un haut taux de fidélité [109–114]. Tous ces avantages les rendent particulièrement prometteurs pour la réalisation de calculs quantiques à grande échelle.

2.1.3 Démonstrations modernes

Quatre décennies après la conceptualisation de l’informatique quantique [46, 47], la réalisation de calculs concrets et utiles reposant sur la mécanique quantique reste un véritable défi technique. Bien que des démonstrations aient permis de confirmer la validité des concepts théoriques, les calculateurs quantiques actuels (qualifiés de “bruités à tailles intermédiaires” [115, 116]) sont encore trop petits et rudimentaires pour résoudre le moindre problème utile [117].

En 2001, l’algorithme de Shor [48] a été implémenté expérimentalement pour la première fois sur un ordinateur quantique [64], permettant de factoriser le nombre 15 en ses facteurs premiers. Cette démonstration a permis de confirmer le fonctionnement pratique de l’algorithme, mais les sept qubits physiques utilisés étaient très loin d’être suffisants pour factoriser un nombre réellement intéressant. Peu de temps après, des portes logiques à deux qubits ont été implémentées avec succès [65], ouvrant la voie à des circuits plus complexes. À partir de 2015, des simulations de molécules ont été réalisées en utilisant des qubits supraconducteurs [66], promettant ainsi de nouvelles applications dans le domaine de la chimie. Une étape décisive est franchie en 2019 [67] lorsque Google a prétendu avoir atteint la “suprémie quantique” [118, 119] en exécutant un calcul en 200 secondes qui prendrait 10 000 ans à un superordinateur classique. Bien que cette affirmation ait été contestée [120], elle a marqué un tournant dans le développement de l’informatique quantique en faisant fonctionner avec succès un processeur programmable de 53 qubits supraconducteurs.

Néanmoins, une augmentation drastique de la taille et de la fiabilité des calculateurs quantiques est encore nécessaire pour résoudre des problèmes concrets. Par exemple, factoriser une clé RSA de 2 048 bits en 8 heures avec l’algorithme de Shor [48] nécessiterait plusieurs

millions de qubits physiques [121]. Et la simulation de molécules utiles pour la découverte de nouveaux médicaments [49, 50] nécessiterait au minimum une centaine de qubits logiques [117].

2.1.4 Verrous technologiques

Les ordinateurs conventionnels s'utilisent à température ambiante et s'accommodent des contraintes bien connues de la micro et nanoélectronique. Les circuits quantiques, en revanche, doivent être isolés de toutes perturbations extérieures, ce qui implique notamment d'être refroidis à des températures proches du zéro absolu. Ces contraintes extrêmes posent de nombreux défis techniques qu'il faut surmonter pour réaliser un ordinateur quantique pleinement fonctionnel. Nous allons ici nous attarder sur trois de ces défis : (i) la calibration de boîtes quantiques ; (ii) la correction d'erreur quantique ; et (iii) le goulot d'étranglement lié à l'électronique de contrôle.

2.1.4.1 Calibration de boîtes quantiques pour la formation de qubits de spin

L'utilisation du spin $1/2$ des électrons [123] en guise de qubit présente de nombreux avantages (section 2.1.2), mais isoler une unique particule dans une structure de quelques nanomètres de façon reproductible est une tâche particulièrement complexe. Avant d'être utilisées pour réaliser des calculs, les boîtes quantiques doivent donc être précisément calibrées, c'est-à-dire instanciées dans une configuration spécifique. Cette procédure est guidée par la détection de charge au cœur du dispositif, informant indirectement du contenu de la boîte quantique. Lorsque ces mesures sont présentées en deux dimensions, où chaque axe correspond à la variation de tension d'une grille, et les pixels l'intensité du courant mesuré, l'image résultante est appelée "diagramme de stabilité" (figure 2.2). Dans ces diagrammes, des lignes trahissent le mouvement de porteurs de charge entre la boîte quantique et un réservoir, permettant d'obtenir de précieuses informations sur l'état interne du système. La calibration est généralement divisée en cinq étapes séquentielles [69] :

1. **Initialisation** : Placer le système dans un état dans lequel il est possible de manipuler et mesurer l'état de la boîte quantique. Ce qui implique de refroidir le dispositif à quelques dizaines de millikelvins, d'initialiser les appareils de mesures et les circuits de contrôle, de calibrer le détecteur de charge, et de placer les grilles électrostatiques à des tensions compatibles avec l'état souhaité. Cette étape ne présente pas de défis particuliers et peut être réalisée de manière automatique.
2. **Calibration du régime** : Trouver les tensions de grille permettant de placer le système dans une topologie spécifique. C'est-à-dire, identifier le nombre de boîtes quantiques distinctes formées dans le dispositif (par exemple, régime simple ou double). Cette étape repose sur l'analyse des motifs visibles dans les diagrammes de stabilité.

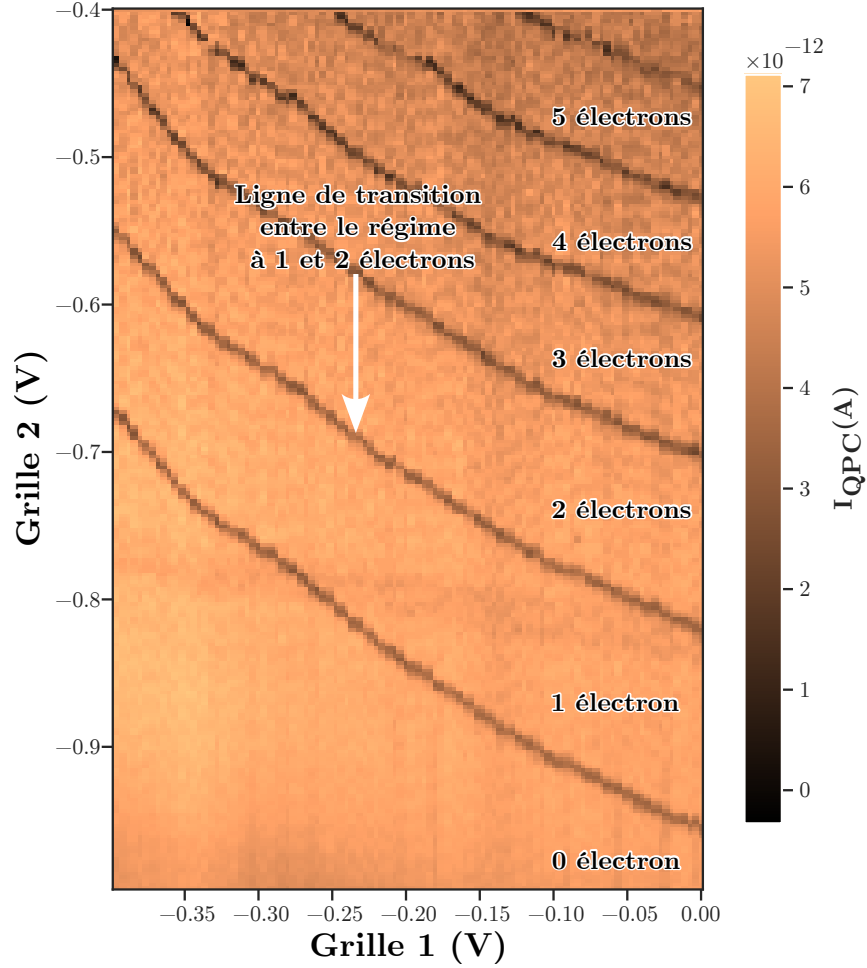


FIGURE 2.2 Exemple de diagramme de stabilité provenant du jeu de données GaAs-QD.

Cette image est obtenue en mesurant le courant électrique au niveau d'un contact ponctuel quantique (QPC), pour chaque combinaison de tension de grille, sur un dispositif à base d'arséniure de gallium [122]. Chaque pixel couvre une région de $2,5 \text{ mV}^2$. D'autres exemples de diagrammes sont accessibles dans les figures 3.2 et 4.3 et annexe A.1.6.

Automatiser cette étape est loin d'être trivial puisque l'espace des tensions de grille est vaste et les mesures sont bruitées.

3. **Établir la contrôlabilité** : Compenser les effets de couplage entre les grilles de contrôle. L'objectif est de configurer des grilles virtuelles permettant d'agir de manière indépendante sur la boîte quantique. Il est possible d'automatiser cette étape en utilisant des algorithmes de traitement d'image pour mesurer précisément l'angle des lignes de transition de charge et ainsi déterminer la correction à appliquer. Cette étape n'est cependant pas indispensable puisque les effets de couplage peuvent être pris en compte lors des étapes suivantes.
4. **Calibration de la charge** : Trouver les tensions de grille permettant de placer un nombre spécifique d'électrons dans chaque boîte quantique. Il est crucial de connaître

le nombre de charges présentes dans le dispositif pour mettre en place des portes quantiques. Tout comme la calibration du régime, cette étape nécessite d'explorer un diagramme de stabilité pour y trouver des indices sur l'état interne du système. La méthode la plus commune consiste à vider la boîte quantique, puis de compter le nombre de lignes de transition de charge jusqu'à atteindre le nombre d'électrons ciblés (généralement entre 1 et 3).

5. **Réglages fins** : Ajuster le couplage tunnel [124] entre les boîtes quantiques. Cette étape finale permet d'optimiser la fidélité des portes quantiques.

Aujourd'hui, ces étapes de calibration sont majoritairement réalisées manuellement par des techniciens. Cette approche chronophage n'est viable que dans le cas d'expérimentations à petite échelle [125], il n'est pas envisageable de procéder de la sorte avec plusieurs centaines de qubits. C'est donc avec cet objectif de mise à l'échelle que des méthodes de calibration automatique sont développées.

TABLEAU 2.1 Tableau récapitulatif des travaux publiés sur la calibration automatique de boîtes quantiques pour la formation de qubits de spin. Le numéro des étapes de calibration correspond à : 1) Initialisation, 2) Calibration du régime, 3) Établir la contrôlabilité, 4) Calibration de la charge, 5) Réglages fins. La lettre **D** indique que l'étape a été réalisée sur des doubles boîtes quantiques et **S** sur des simples boîtes. La couleur des cellules indique le contexte dans lequel l'étape a été réalisée : *marron* pour les simulations numériques, *bleu* pour les tests sur des diagrammes hors ligne et *vert* pour les démonstrations expérimentales en ligne. La colonne "Nb Exp." indique le nombre d'itérations de calibration automatique réalisées dans cette étude. La colonne "Durée" indique la durée moyenne de chaque itération de calibration en minutes pour les démonstrations expérimentales.

Article	Étapes					Méthodes	Nb Exp.	Durée (min)	Taux de succès
	1	2	3	4	5				
[126]		D				CNN	46	8	96 %
[127]		D				CNN	1 000	-	88 %
[128]		D				CNN	45	140	31 %
[129]		D				Pic détection	-	70	-
[130]		D				CNN	30	-	93 %
[131]		D				Random forest	3	44	-
[132]		D	D	D		CNN + Pic détection	23	-	90 %
[133]		S	S	S		CNN	3	-	-
[134]				D		CNN	160	-	57 %
[135]				S		Pic détection	-	-	-
[136]				S		FF	25	-	75 %
[137]						FF	-	-	99 %
[138]		D	D	D		Hybride	13	2 280	77 %

Les diagrammes de stabilité pouvant être interprétés comme des images, il est possible d'utiliser les méthodes de *vision artificielle* [139] pour détecter les lignes de transition de charge, en particulier pour les étapes 2 à 3. Les RNA sont particulièrement adaptés à

ce type de tâche (section 2.2), car ils sont capables d’identifier des formes dans des données bruitées avec une grande précision. Il est par exemple possible de classifier le régime d’une boîte quantique avec 90 % de précision sur des données synthétiques [127] et 77 % sur des données expérimentales [140] en utilisant un réseau de neurones à convolution, *convolutional neural network (CNN)* en anglais, ce qui surclasse les méthodes de vision artificielle traditionnelles [141–144]. En combinant cette méthode avec un algorithme de calibration automatique, il est possible d’atteindre le régime souhaité (étape 2) avec 85 % de réussite [128]. Cependant, chaque mesure effectuée sur la boîte quantique est couteuse en temps, il est donc important que la calibration puisse être réalisée avec une quantité minimale de données [145]. D’autres travaux cherchent à contrôler le nombre d’électrons présents dans le dispositif [135] afin d’en isoler un seul (étape 4). Dans le cas assez complexe d’un régime double, il est nécessaire de trouver la configuration possédant deux fois un électron, ce qui ne peut aujourd’hui être réalisé de manière autonome qu’avec 57 % de réussite [134]. La méthode la plus avancée de calibration automatique [138] démontrée expérimentalement à ce jour permet d’atteindre un taux de succès de 77 % et couvre l’ensemble des étapes de calibration. Bien que prometteuse, cette procédure est très complexe à mettre en place et prend plusieurs dizaines d’heures pour être complétée, et n’est pas facilement transférable vers d’autres dispositifs.

Le processus de calibration automatique de boîtes quantiques est encore au stade exploratoire, aucune méthode n’existe à ce jour pour réaliser l’ensemble des étapes de calibration de manière autonome et fiable (tableau 2.1). En revanche, nous pouvons noter que les méthodes basées sur l’apprentissage automatique ont montré des résultats prometteurs. La majorité des travaux publiés se concentre sur l’étape de calibration du régime (2), tandis que les étapes de contrôlabilité (3) et de calibration de la charge (4) sont moins étudiées. Dépendamment de l’environnement de test, plusieurs défis sont mentionnés dans la littérature : les modèles entraînés sur des données synthétiques sont difficiles à transférer vers des environnements expérimentaux, les méthodes développées à l’aide de diagrammes hors ligne souffrent du manque de données et de l’absence de jeux de données standardisés, et les démonstrations expérimentales en ligne sont limitées par le temps de mesure et la disponibilité des dispositifs.

2.1.4.2 Correction d’erreur quantique

L’état d’un qubit est par nature très sensible aux perturbations, ce qui rend les calculs quantiques particulièrement vulnérables aux erreurs [116, 146]. Malgré les efforts considérables mis en place pour isoler les systèmes quantiques de l’environnement extérieur, les erreurs restent inévitables. De plus, l’impossibilité de cloner un état quantique [147, 148] complexifie grandement la tâche de correction. Une erreur quantique peut affecter un qu-

bit de deux façons : erreur de *bit-flip*, lorsque $|0\rangle$ devient $|1\rangle$, et inversement (équivalent à une erreur de bit classique) ou erreur de *phase-flip*, lorsque le qubit subit une inversion de phase, passant par exemple de $\alpha|0\rangle + \beta|1\rangle$ à $\alpha|0\rangle - \beta|1\rangle$.

L'approche la plus simple pour corriger les erreurs quantiques est de répéter l'information, puis de comparer les différents états quantiques à la fin du calcul [149]. Par exemple, un qubit logique en état $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ peut être encodé par une superposition de trois qubits physiques : $|\psi\rangle = \alpha|000\rangle + \beta|111\rangle$. Si un seul qubit physique subit erreur de *bit-flip*, il sera alors possible de détecter et corriger le problème.

Le principal objectif d'un protocole de correction d'erreur [73–75] est donc de réduire le nombre d'erreurs logiques en dessous du nombre d'erreurs physiques, tout en minimisant le nombre de qubits physiques nécessaires. De manière générale, il se déroule en cinq temps :

1. **Encodage** : Le qubit logique est représenté par un ensemble de qubits physiques, qui doivent être préparé dans un état collectif spécifique.
2. **Opérations logiques** : Les opérations quantiques sont appliquées sur le qubit logique, en utilisant des portes logiques pour manipuler les qubits physiques.
3. **Collecte des syndromes** : Les qubits physiques sont mesurés afin d'obtenir de l'information sur leur état. Puisque la mesure est une opération destructive dans le monde quantique, l'information est obtenue par l'intermédiaire de qubits auxiliaires, permettant ainsi au qubit logique de rester dans un état de superposition. La mesure résultante est appelée “*syndrome*”.
4. **Décodage** : Les syndromes mesurés sont utilisés pour déterminer quelles sont les erreurs les plus probables subies par le qubit logique. Cette opération peut être plus ou moins complexe en fonction de l'encodage utilisé. Il est important de noter que la mesure des syndromes peut aussi contenir des erreurs, ce qui rend le décodage d'autant plus délicat.
5. **Correction** : En fonction des erreurs détectées, des opérations sont appliquées pour corriger l'état du qubit logique.

Il existe de nombreuses approches de correction d'erreurs [150, 151], dont certaines qui ont démontré expérimentalement leur efficacité à petite échelle [152–157]. Un candidat prometteur est le code de surface [158], qui permet de corriger efficacement des erreurs de *bit-flip* et de *phase-flip* en tirant profit de la configuration planaire et de mesures locales. Bien que cette solution permette de garder espoir dans la réalisation de calculateurs quantiques robustes à l'erreur, son implémentation concrète n'est pas triviale. Le principal défi réside dans la mise en place d'un système de décodage efficace. Une approche naïve basée sur une simple table de correspondance entre syndromes et erreurs est rapidement

inapplicable, car le nombre de syndromes possibles croît exponentiellement avec le nombre de qubits physiques. Par exemple, un code de surface de distance 7 peut générer $5,6 \times 10^{14}$ syndromes différents, nécessitant une table de correspondance de plus de 70 Tb de stockage classique. De plus, le fait que la lecture des syndromes peut elle-même contenir des erreurs rend les approches naïves d’autant moins efficaces.

Afin de rendre possible le décodage en temps réel pour un grand nombre de qubits, plusieurs approches ont été proposées pour compresser la table de correspondance entre syndromes et erreurs logiques. Des solutions basées sur des graphes [159–162] fonctionnent pour des codes de distance allant jusqu’à 51 [163], mais au prix d’une grande complexité et d’une consommation énergétique importante. Les méthodes basées sur l’apprentissage automatique [164] permettent de compresser la table de correspondance tout en conservant une grande précision. Cette approche a l’avantage de s’adapter aux erreurs provenant d’un système quantique spécifique, mais nécessite un grand nombre de données et consomme également beaucoup de ressources de calcul. Parmi les décodeurs basés sur l’apprentissage automatique, les réseaux de neurones récurrents, *recurrent neural networks (RNNs)* en anglais, sont particulièrement adaptés pour cette tâche, car leur capacité à traiter les données de façon séquentielle leur permet d’identifier des corrélations temporelles entre les syndromes [165, 166].

2.1.4.3 Goulot d’étranglement des circuits de contrôle

La photographie présentée en figure 2.1 illustre le dispositif expérimental supraconducteur utilisé par Google [67]. Les câbles bleus permettent de contrôler et mesurer les 54 qubits avec l’électronique situés à l’extérieur du cryostat. Cette approche fonctionne bien pour manipuler quelques dizaines de qubits, mais ne peut être mise à l’échelle pour un ordinateur quantique de plusieurs centaines de qubits, ce qui est le minimum requis pour résoudre des problèmes intéressants (section 2.1.3). Pour manipuler plusieurs milliers de qubits, la largeur cumulée des câbles serait tout simplement plus large que le cryostat lui-même. Cette limitation est un verrou technologique bien connu des ingénieurs du domaine, souvent qualifié de “goulot d’étranglement du câblage” [167–169]. De plus, de longs câbles induisent du bruit électromagnétique et représentent une source de chaleur additionnelle perturbant inévitablement les résultats. Ce problème est amplifié dans le cas des qubits de spin (section 2.1.2), qui nécessitent [105] : une tension continue réglable, des impulsions de tension indépendantes pour chaque grille, des champs électromagnétiques réglables pour chaque boîte quantique et un système rapide de mesure de l’état quantique.

Ce goulot d’étranglement est intimement lié aux deux verrous technologiques décrits précédemment, puisque la calibration automatique des boîtes quantiques nécessite de nom-

breuses itérations de mesure–calibration, et que la correction d’erreur quantique requiert de mesurer précisément et rapidement les syndromes.

La solution la plus évidente pour résoudre ce problème est d’intégrer l’électronique de contrôle directement dans le cryostat, à proximité des qubits [170]. Le nombre de câbles sortants ne serait ainsi plus proportionnel au nombre de qubits, ce qui faciliterait grandement la mise à l’échelle de calculateurs quantiques. Cependant, cette solution crée de nouvelles contraintes sur l’électronique, qui doit : (i) être capable de fonctionner à des températures cryogéniques ; (ii) émettre très peu de chaleur pour ne pas perturber les qubits (ce qui implique une consommation électrique très faible) ; et (iii) être suffisamment compacte pour prendre place dans un cryostat. Cette électronique froide est activement développée, mais est généralement fabriquée sur mesure, ce qui implique un coût de développement élevé [171].

2.2 Réseaux de neurones artificiels

2.2.1 Concepts et définitions

L’utilisation de RNA est une méthode parmi d’autres de l’apprentissage automatique, et l’apprentissage automatique est l’une des méthodes étudiées dans le domaine de l’IA (figure 1.4a). L’étude de ces méthodes, de la plus générique (IA) à la plus spécifique (apprentissage profond) a permis de résoudre toute une catégorie de problèmes qui étaient précédemment hors d’atteinte des méthodes formelles¹.

2.2.1.1 Intelligence artificielle

Est *artificiel* ce qui est produit par l’homme, par opposition à ce qui est naturel. Cette définition relativement claire permet de catégoriser sans trop de doute un ordinateur comme étant un objet artificiel. En revanche, l’*intelligence* est un concept flou qui crée de nombreux débats [172]. De plus, l’intelligence est souvent associée à d’autres concepts tout aussi abstraits, comme la conscience, la créativité, les émotions, la capacité à raisonner, à penser, etc. Pour faire avancer cette question, Alan Turing a proposé en 1950 [54] un test objectif et pragmatique pour déterminer si une machine peut penser. Le célèbre test de Turing consiste à faire dialoguer un humain avec un interlocuteur inconnu par l’intermédiaire de messages textuels dans l’objectif de déterminer si son correspondant est un humain ou une machine. En 2023, 1,5 million de personnes se sont prêtées au jeu en dialoguant avec des grands modèles linguistiques, *large language models (LLMs)* en anglais, et échouant

1. Définition des méthodes formelles : hemesh.larc.nasa.gov/fm/fm-what.html, en opposition aux méthodes d’apprentissage automatique.

40 % du temps à déterminer la nature de leur interlocuteur [173], passant ainsi le test tel qu’il avait été défini 73 ans plus tôt.

Depuis, de nouveaux tests plus exigeants ont été proposés [174, 175] pour continuer à faire avancer ce champ de recherche. Dans le cadre de cette thèse, nous qualifierons d’IA l’ensemble des méthodes ayant pour but de concevoir “un système artificiel capable de résoudre des problèmes complexes en ayant accès à une quantité restreinte d’information, d’énergie et de temps”. Cette définition est dans la continuité pragmatique de résolution de tâches qui a été le moteur du domaine depuis ses débuts.

2.2.1.2 Apprentissage automatique

L’apprentissage automatique, *machine learning (ML)* en anglais, aussi introduit par Alan Turing en 1950 [54], est basé sur un constat simple : écrire explicitement le code source d’un programme capable d’approcher les capacités d’un cerveau humain adulte est une tâche titanesque qui semble hors de portée de l’ingénierie humaine. La solution qu’il proposa est de concevoir un programme comparable au cerveau d’un nouveau-né : vierge de toute connaissance, mais capable d’apprendre. Cette tâche paraît plus facile à réaliser, mais nécessite néanmoins une importante quantité de données et une grande puissance de calcul afin de réaliser l’étape d’apprentissage. Ce qui a été rendu possible par les avancées technologiques de ces dernières décennies, notamment l’avènement d’internet et le développement des cartes graphiques [176].

2.2.1.3 Réseaux de neurones artificiels

Les RNA sont des modèles de calcul dont le fonctionnement est inspiré de leurs homologues biologiques. En 1943, Warren McCulloch et Walter Pitts démontrent pour la première fois la possibilité de réaliser des opérations logiques à l’aide de neurones formels [177, 178]. Frank Rosenblatt [55] a ensuite exploité le principe de l’apprentissage automatique pour résoudre des problèmes de classification. Bien que limités dans un premier temps, ces réseaux se sont avérés capables d’approximer toutes fonctions continues [179, 180], et ainsi contourner les problèmes d’explosion combinatoire souvent rencontrés avec les approches classiques.

2.2.2 Apprentissage profond

2.2.2.1 Le neurone formel

Le neurone formel est l’unité élémentaire de tout RNA, il a été conçu comme une représentation mathématique d’un neurone biologique. Concrètement, il peut être représenté comme une fonction ($f_{n.f.}$) prenant n variables en entrée (de x_1 à x_n) et retournant une sortie y (équation (2.1) et figure 2.3a).

$$f_{\text{n.f.}}(x_i, \dots, x_n) = \varphi \left(b + \sum_{i=1}^n w_i x_i \right) = y \quad (2.1)$$

Le neurone formel multiplie un poids w_i avec chaque variable d'entrée, puis la somme de ces éléments est passée à une fonction d'activation non linéaire φ . Un biais (ou valeur seuil) b est ajouté à la somme afin d'obtenir un degré de liberté supplémentaire.

Ce modèle permet de réaliser certaines opérations logiques (ET, OU, NON), à la condition d'utiliser une fonction d'activation appropriée. La première description des neurones formels [177] utilisait la fonction de *Heaviside* (fonction échelon), mais les implémentations modernes y préfèrent des alternatives continues telles que la fonction *sigmoïde* (équation (2.2)), pour faciliter l'apprentissage.

$$\varphi_{\text{sig}}(x) = \frac{1}{1 + e^{-x}} \quad (2.2)$$

Il est aussi courant d'utiliser la fonction *ReLU* (équation (2.3)) pour sa simplicité et sa capacité à limiter les problèmes de disparition de gradient [181].

$$\varphi_{\text{relu}}(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases} \quad (2.3)$$

Mais de nombreuses autres fonctions d'activation existent [182]. Sa seule propriété importante est d'être non linéaire, car un RNA composé uniquement de fonctions linéaires ne serait pas en mesure d'approximer une fonction non linéaire. La fonction d'activation est généralement fixe pour un neurone, il est cependant possible de faire varier ses poids w et son biais b afin d'obtenir le comportement souhaité. Ces derniers sont donc habituellement considérés comme étant ses *paramètres*.

2.2.2.2 Le perceptron

Si l'on souhaite résoudre un problème concret avec un neurone, il est nécessaire de calibrer correctement ses paramètres. Le perceptron [55] définit une règle d'apprentissage qui permet de trouver les paramètres optimaux d'un neurone formel de manière à séparer un jeu de données en deux classes. Chaque exemple du jeu de données est envoyé en entrée du neurone sous la forme d'un vecteur de taille n , puis chaque poids d'index i (de 0 à n) est ajusté en fonction de la sortie, d'après la règle suivante :

$$w'_i = w_i + \alpha(y_t - y)x_i \quad (2.4)$$

Où w'_i est le nouveau poids, w_i le poids actuel, y_t la sortie souhaitée (c'est-à-dire la classe de l'exemple), y la sortie actuelle, x_i la valeur de l'entrée correspondante au poids i , et α le taux d'apprentissage. Le biais est ici considéré comme étant le poids w_0 dont l'entrée est toujours égale à 1.

Cette méthode d'apprentissage automatique garantit de trouver une séparation entre deux classes si elles sont linéairement séparables.

2.2.2.3 Les réseaux de neurones profonds

Un neurone isolé ne peut réaliser que des opérations triviales. Pour résoudre des problèmes concrets, il est nécessaire d'agréger les neurones sous forme de réseaux, dans lesquels la sortie d'un neurone est l'entrée d'un autre (figure 2.3c). Mais configurer tous les paramètres d'un ensemble de neurones pour qu'ils réalisent une tâche spécifique est un problème qui requiert une stratégie d'apprentissage plus complexe que celle du perceptron. Cette méthode est connue sous le nom d'apprentissage profond [58], en référence aux nombreuses couches qui composent ces réseaux de neurones. Il s'agit donc d'une méthode d'apprentissage automatique spécifique aux réseaux de neurones profonds, *deep neural networks* (*DNNs*) en anglais.

De manière générale, l'utilisation d'un RNA se fait en deux étapes :

1. **Entraînement** : Étape de calcul intensif, pouvant durer de quelques minutes à plusieurs jours, pendant laquelle les paramètres du réseau sont continuellement modifiés en suivant une stratégie d'apprentissage dans le but de converger vers la résolution d'une tâche précise.
2. **Inférence** : Utilisation d'un réseau préalablement entraîné afin de réaliser des prédictions sur des données non présentes dans la phase d'apprentissage. Cette étape est moins gourmande en ressources puisque les paramètres ne sont plus modifiés. Mais si le réseau est déployé dans un environnement de production, l'inférence peut être répétée un grand nombre de fois, ce qui nécessite des ressources de calculs conséquentes.

2.2.3 Topologies de réseaux

En utilisant le neurone formel comme une brique élémentaire, il est possible de construire toute une variété de réseaux [87] qui peuvent être déployés dans différents contextes. Certains types de RNA sont adaptés pour réaliser des tâches spécifiques, par exemple,

on n'utilisera pas le même réseau pour classifier des images [183] ou pour traduire un texte [184]. Parmi toutes les topologies de réseaux étudiés à ce jour, nous allons brièvement passer en revue celles qui nous intéressent dans le cadre de cette thèse.

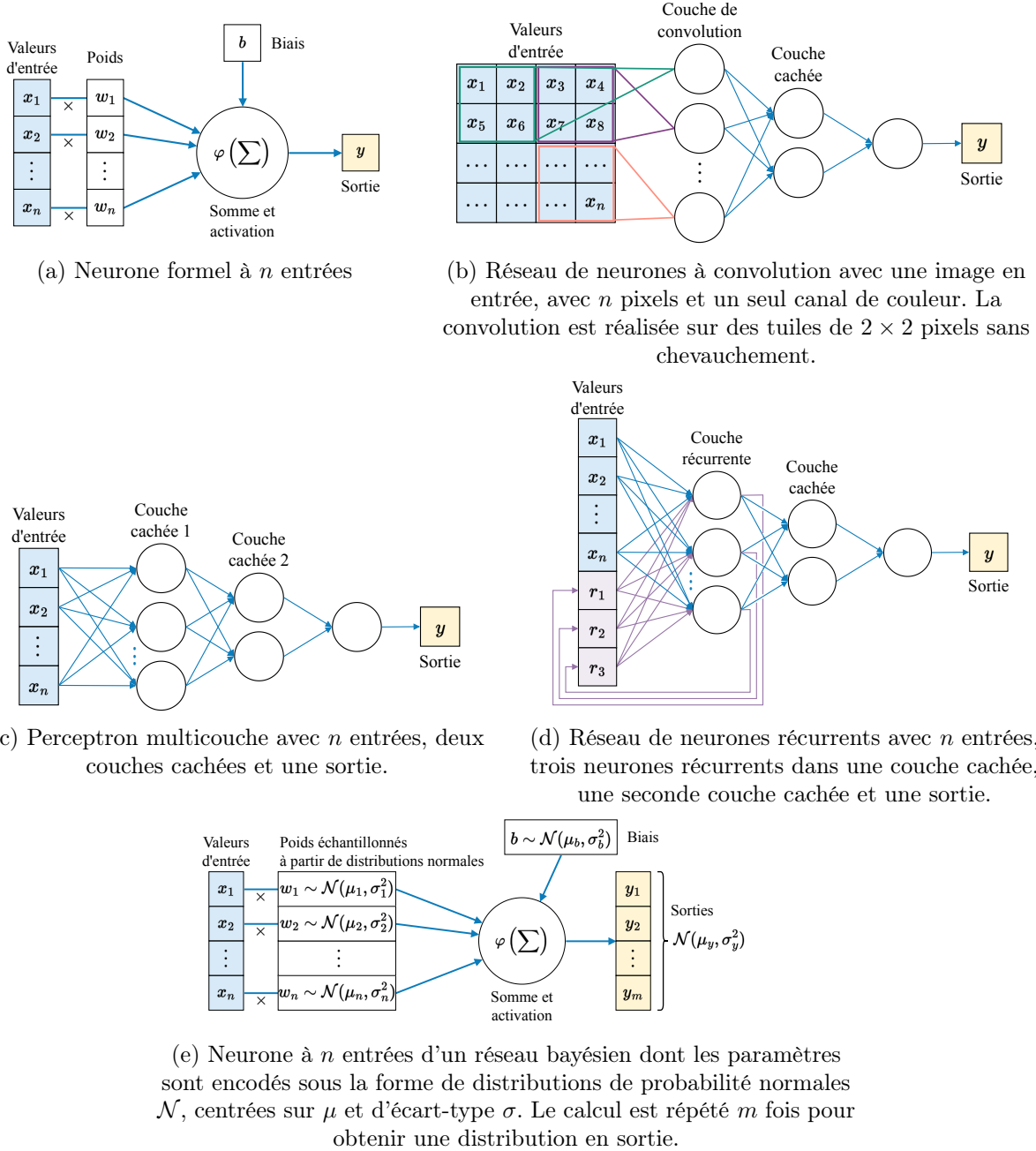


FIGURE 2.3 Représentation schématique de neurones artificiels et de deux exemples de leurs agencements en réseau. Les biais et les poids sont masqués dans les réseaux pour des raisons de lisibilité.

2.2.3.1 Perceptron multicouche

Le perceptron multicouche, *multilayer perceptron (MLP)* ou *feed-forward neural network (FF)* en anglais, est le réseau de neurones le plus basique puisqu'il est composé d'une entrée, d'une à plusieurs couches cachées complètement connectées et d'une sortie (figure 2.3c). Sa force réside donc dans sa simplicité d'implémentation. Il est en théorie capable d'approximer toute fonction continue [179, 180], c'est-à-dire qu'avec suffisamment de neurones cachés et une stratégie d'apprentissage adaptée, il peut en théorie toujours modéliser la relation entre un ensemble d'entrées et de sorties. En pratique, le nombre de neurones et le temps d'apprentissage est limités par les capacités de calcul de nos ordinateurs, ce qui restreint l'utilisation des MLP à des problèmes simples.

2.2.3.2 Réseau de neurones à convolution

Le réseau de neurones à convolution (CNN) a été conçu pour traiter les images de façon plus efficace qu'un réseau classique [56]. Là où le MLP associe un poids entre toutes les valeurs d'entrée et chaque neurone de la première couche (figure 2.3c), le CNN tire profit de la structure en plusieurs dimensions des données pour organiser les relations de façon locale. Chaque neurone de la première couche ne reçoit en entrée qu'une région de la matrice (figure 2.3b), ce qui favorise la détection de formes tout en limitant le nombre de paramètres, et par la même occasion les ressources nécessaires pour réaliser les calculs.

2.2.3.3 Réseau de neurones récurrents

Le réseau de neurones récurrents (RNN) est une topologie de RNA particulièrement adaptée pour traiter des données séquentielles [185–187]. Dans sa version la plus simple, il est composé d'une ou plusieurs couches cachées de neurones récurrents (figure 2.3d), dont la valeur de sortie est conservée en mémoire, de manière à être réinjectée en entrée du réseau lors de l'itération suivante. Cette capacité à conserver une information sur plusieurs itérations permet d'identifier des caractéristiques temporelles dans les données, comme des séquences de mots ou le bruit provenant d'un appareil lors de mesures répétées. De tels modèles ont notamment été utilisés pour réaliser des tâches de traduction automatique [184], de reconnaissance de la parole [188] et de génération de texte [189].

2.2.3.4 Réseau de neurones bayésien

Contrairement aux RNA évoqués précédemment, le réseau de neurones bayésien, *Bayesian neural network (BNN)* en anglais, ne se distingue pas par l'organisation de ses neurones, mais par son approche probabiliste du traitement de l'information. Reposant sur le cadre théorique défini par le théorème de Bayes [190], un BNN est en théorie capable de quantifier l'incertitude associée à chaque inférence [191, 192], ce qui en fait un outil particulièrement adapté pour les tâches de classification et de régression, où le niveau de confiance dans la prédiction est un facteur critique.

La difficulté d'implémentation d'un BNN réside dans le calcul de la probabilité à *posteriori*, qui ne peut être calculé en temps raisonnable pour tout problème de taille réaliste. Il existe cependant de nombreuses façons d'approximer cette probabilité [193], par exemple, en utilisant des méthodes d'échantillonnages [194–197] ou l'approximation de Laplace [142, 198, 199]. Mais la méthode la plus couramment utilisée est l'inférence variationnelle [200–205], où la distribution *posterior* des paramètres, souvent intractable, est approximée par une distribution plus simple (par exemple par une distribution normale pour chaque paramètre, tel qu'illustré en figure 2.3e). Durant l'entraînement, l'objectif est de minimiser la divergence de Kullback–Leibler (KL) [206] entre cette approximation et la *posterior* réelle en maximisant une borne inférieure. Pour ce faire, on échantillonne des poids depuis cette distribution pour calculer la vraisemblance et mettre à jour les paramètres (la moyenne μ et l'écart-type σ si une distribution normale est choisie). En phase d'inférence, plusieurs échantillonnages successifs des paramètres du BNN permettent en théorie de produire une distribution de sorties reflétant l'incertitude sur les prédictions en tenant compte de la variabilité apprise sur les données d'entraînement. Un exemple de CNN bayésien est représenté sur la figure 3.3c.

2.2.4 Accomplissements et limitations

2.2.4.1 Les réussites

Pour mesurer les avancées dans le domaine de l'IA des tâches précises sont définies et standardisées. Des scores et des taux d'erreurs sont ainsi des indicateurs impartiaux des progrès effectués. L'exemple historique est le jeu de données *modified national institute of standards and technology (MNIST)*² qui a permis de démontrer en 1998 la supériorité des CNN pour la reconnaissance de motifs visuels avec le réseau *LeNet-5* [56]. L'ensemble de données est constitué de 70 000 images de chiffres manuscrits (de 0 à 9), et la tâche consiste à identifier correctement 10 000 d'entre elles (le reste étant réservé à l'apprentissage). *LeNet-5* a réalisé cette tâche de classification avec 0,95 % d'erreur, après avoir été entraîné pendant 3 jours afin d'ajuster ses 60 000 paramètres. Aujourd'hui, le meilleur modèle fait sept fois moins d'erreurs de classification, avec seulement 0,13 % d'images mal classifiées [183], mais requiert 25 fois plus de paramètres. À titre de comparaison, la performance humaine est estimée à 0,2 % d'erreurs pour cette même tâche.

Une autre façon, plus frontale, de se comparer à la machine est de la défier au jeu. Cette approche a la particularité d'être spectaculaire pour le grand public et vexante pour les joueurs professionnels. À ce jour, les victoires les plus remarquables de l'IA face aux cerveaux biologiques ont eu lieu : aux échecs [207] en 1997, au go [208, 209] en 2016, au

2. *MNIST* documentation : yann.lecun.com/exdb/mnist

poker (variante *heads-up no-limit*) [210, 211] en 2017, puis en 2019 à StarCraft II [212] et à Dota 2 [213].

Depuis 2020, les RNA ont passé un nouveau cap en réalisant un grand nombre de tâches utiles dans un contexte industriel, forçant ainsi des secteurs entiers à s'adapter [214]. La publication de *GPT-3* [84, 85], par exemple, a été remarquée par le grand public pour sa capacité à générer du texte de qualité à partir d'une simple phrase d'entrée via une interface web publiquement accessible³. Le modèle est composé de 175 milliards de paramètres, soit près de 3 000 000 fois plus que *LeNet-5*, et a été entraîné sur 570 Go de texte. Depuis sa publication, *GPT-3* et ses versions suivantes [215] ont notamment influencé la santé publique [216, 217] et l'éducation [218]. De façon moins remarquée, mais tout aussi significative, des RNA ont aussi récemment contribué à la découverte de nouveaux algorithmes [219], la synthèse de molécules organiques [220], la prédiction météorologique [221, 222], la découverte de nouveaux matériaux [223] ou encore la génération d'images [224].

2.2.4.2 Les limitations

Malgré les prouesses incontestables des RNA profonds, il existe un certain nombre de limitations conceptuelles. À commencer par le manque de fondement théorique de ces méthodes. Bien que l'apprentissage automatique soit en partie décrit formellement [225, 226], le fait que l'efficacité prédite soit en dessous de celle observée en pratique [227] montre que le fonctionnement de ces modèles n'est pas entièrement compris. De plus, il n'existe pas de réelle garantie de convergence ou de généralisation pour les algorithmes d'apprentissage profond. Le manque de transparence [228] des RNA est également un problème majeur pour certains domaines d'application, comme la santé ou la justice, où il est nécessaire de comprendre les raisons derrière une décision. Pour des véhicules autonomes reposant sur des RNA [229], il est tout aussi critique de garantir l'intégrité du système face à des attaques malveillantes [230–232]. Enfin, la confidentialité des données d'entraînement est difficile à garantir avec les méthodes actuelles [233], ce qui peut poser des problèmes éthiques, notamment depuis la démocratisation des LLM [234]. Ces lacunes théoriques rendent la conception de réseaux hasardeuse : le choix des hyperparamètres est principalement basé sur des méthodes empiriques. Pour la même raison, il est aujourd'hui impossible de savoir a priori quelle quantité de données d'entraînement est optimale pour résoudre un problème donné, ni le nombre de paramètres nécessaires pour obtenir un résultat satisfaisant. L'abondance de données [235] et la sur-paramétrisation [236, 237] ont

3. *ChatGPT* : openai.com/index/chatgpt

été des stratégies efficaces pour entraîner des modèles performants, mais ces approches coûteuses ne sont pas toujours applicables en pratique.

En plus de ces considérations théoriques, les RNA sont également limités par des contraintes pratiques. Puisque l'augmentation du nombre de paramètres, de la quantité de données d'entraînement et de la durée de l'entraînement améliorent presque systématiquement leurs performances [77–79], les meilleurs modèles sont aussi généralement les plus onéreux [11]. Menant à une course à l'armement, où les plus grandes entreprises investissent des sommes colossales [214] pour obtenir un avantage compétitif. Impliquant par la même occasion une émission de CO₂ non négligeable [10, 238–240] et une consommation d'eau importante pour refroidir les centres de calculs [241]. Par exemple, *Chat-GPT* émet de l'ordre de 1 000 tonnes de CO₂ par mois [242, 243], soit l'équivalent de l'émission de 1 300 français au cours de la même période [244].

2.3 Memristors

2.3.1 Concepts et définitions

2.3.1.1 Memristor

Le memristor (ou mémoire résistive) est un composant électronique passif possédant une valeur de résistance qui dépend de son état interne. Cet état à la particularité d'être modulable par des stimulations externes telles que des impulsions de tension. Une fois modifié, l'état interne du memristor—donc sa valeur de résistance—est conservé même hors tension [82]. Il a été décrit pour la première fois en 1971 par Leon Chua [81] avant d'être implémenté physiquement en 2008 [83]⁴.

Pour certaines applications, les memristors remplacent des unités de calcul basées sur la technologie CMOS [245, 246], qui sont aujourd'hui dominantes dans l'industrie. Mais malgré leur arrivée tardive, l'approche des memristors présente de nombreux avantages techniques : un faible coût énergétique de permutation (~ 10 fJ [247]), une permutation rapide (85 ps [248]), une densité d'intégration élevée ($> 0,7$ Tb/cm² [249]) et une persistance de l'état.

Par leurs caractéristiques physiques uniques, les memristors suscitent un intérêt pour l'industrie des mémoires non volatiles afin de stocker des données [250, 251]. Dans une telle mémoire, un état de forte résistance est interprété comme la valeur binaire 1 et une faible résistance comme 0. Mais l'utilisation qui va nous intéresser ici est le calcul en mémoire

4. L'appellation de *memristor* pour les implémentations modernes est parfois contestée car ce sont des systèmes à plusieurs éléments et non des composants élémentaires. Toutefois, cette terminologie est communément admise, nous l'utiliserons donc dans sa définition large de *système à résistance modulable*.

(détaillé dans la section 2.3.3) car cela offre de nouvelles perspectives pour l'implémentation de RNA.

2.3.1.2 Crossbar

Un crossbar désigne un ensemble de composants électroniques disposés sous la forme d'une matrice et reliés par des lignes horizontales d'un côté et verticales de l'autre (figure 2.6b). Dans le cadre de ce projet, les composants aux intersections de ces lignes seront des memristors. D'un point de vue fonctionnel, ce type de dispositif peut être associé à une mémoire à accès non séquentiel, en anglais *random access memory (RAM)*.

2.3.2 Implémentations physiques

La technologie des memristors est très jeune, mais son potentiel disruptif dans le domaine de l'IA alimente son développement [60, 252, 253]. L'implémentation physique idéale qui répond aux contraintes industrielles et applicatives n'a pas encore été découverte, mais de nombreux candidats se disputent cette place. Ainsi, une grande variété de mécanismes physiques de variations d'état de résistance est envisagée [254–257] (figure 2.4). Pour chacune de ces catégories, différents matériaux peuvent être utilisés et de nombreuses méthodes de fabrications sont possibles. L'étude détaillée de toutes ces implémentations ne fait pas partie des objectifs de ce projet, il est cependant important de noter qu'elles influencent fortement les caractéristiques et imperfections du système final [258, 259] (section 2.3.4).

Il existe aussi une grande variété de crossbars qui répondent tous à des problématiques différentes. Les plus grands peuvent atteindre une taille de 512×512 memristors [260]. Mais l'augmentation de la taille implique une augmentation de la résistance des interconnexions et la probabilité de courant de fuite (section 2.3.4.1). Ce problème peut être contourné par une mise à l'échelle horizontale, c'est-à-dire une augmentation du nombre de crossbars de plus petites tailles fonctionnant conjointement [261]. Il est donc courant d'intégrer plusieurs crossbars sur un même circuit, par exemple, en interconnectant 64 crossbars de 256×256 memristors, ce qui permet d'y stocker 4 194 304 valeurs [262].

Après la taille, la seconde caractéristique importante d'un crossbar est la méthode utilisée pour sélectionner un memristor spécifique. Il est possible de distinguer au moins trois approches : (i) avec sélecteur actif, 1-transistor-n-résistance (1TnR), un transistor devant chaque memristor ou groupe de memristors ; (ii) avec sélecteur passif, 1-sélecteur-1-résistance (1S1R), qui est souvent une diode empilée verticalement avec le memristor ; et (iii) sans sélecteur, 1-résistance (1R). Chaque méthode possède des bénéfices et des désavantages [259, 263], par exemple 1TnR permet une grande précision [261], mais au prix d'une faible densité d'intégration, 1R rend la sélection complexe et 1S1R et un

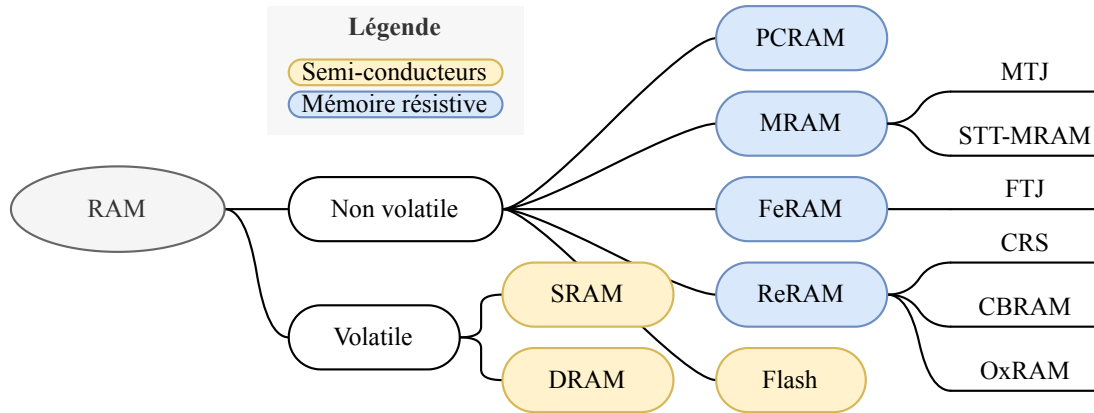


FIGURE 2.4 Principales catégories de mémoires à accès non séquentiel (RAM). Mémoires à base de semi-conducteurs : static RAM (SRAM), dynamic RAM (DRAM) et Flash. Mémoires résistives : phase change RAM (PCRAM), magnetoresistive RAM (MRAM), ferroelectric RAM (FeRAM), redox-based RAM (ReRAM), magnetic tunnel junction (MTJ), spin-transfer torque MRAM (STT-MRAM), ferroelectric tunnel junction (FTJ), complementary resistive switch (CBS), conductive-bridging RAM (CBRAM) et oxide-based RAM (OxRAM).

bon compromis entre les deux précédents, mais risque d'accentuer certains paramètres non idéaux.

Enfin, le crossbar a besoin d'un circuit périphérique pour assurer son fonctionnement : génération des impulsions de lecture et d'écriture, calcul des fonctions d'activation, conversion des signaux d'entrée numériques, etc. Tous ces composants additionnels vont grandement influencer les performances globales du système [258], de sa précision à sa consommation énergétique.

2.3.3 Calcul en mémoire

Afin d'entrevoir l'intérêt que présente les memristors, il faut comprendre comment les calculs sont réalisés sur un ordinateur classique. L'immense majorité des systèmes informatiques modernes est basée sur une architecture matérielle dite de von Neumann [3], du nom de son principal inventeur. Celle-ci peut être schématiquement divisée en trois parties (figure 1.1a) : (i) l'unité logique et arithmétique ; (ii) l'unité de contrôle ; et (iii) la mémoire (volatile et non-volatile). Cette architecture modulable a soutenu l'évolution de l'informatique depuis plus de 70 ans. Mais aujourd'hui, la consommation de données de plus en plus intensive a mis en lumière une limitation technique au niveau du transfert d'informations entre la mémoire et l'unité logique [10]. Ce problème, connu sous le nom de *goulot d'étranglement de von Neumann*, peut être atténué par diverses améliorations techniques [264], mais la seule façon de le régler définitivement est de repenser l'architecture dans son ensemble.

Le calcul en mémoire est une approche prometteuse pour répondre à ce problème de transfert de données. C'est un paradigme de calcul, radicalement différent de celui de von Neumann, qui propose de fusionner l'unité logique et une partie de la mémoire en une unité plus petite (figure 1.1b). Avec cette nouvelle architecture, l'augmentation de la puissance ne se fait plus en améliorant les modules, mais en augmentant leur nombre. Ainsi le canal de communication problématique n'est plus nécessaire, offrant un gain significatif de vitesse et d'énergie [265–267].

Les memristors rendent possible la mise en place d'un tel système, car ils sont capables de réaliser les deux opérations du module *mémoire + logique*. La valeur résistance R peut être modifiée, conservée puis lue, ce qui correspond bien au rôle d'une mémoire. Une opération de multiplication, entre la valeur stockée R et une valeur d'entrée x peut être effectuée grâce à la loi d'Ohm, où x est encodé dans une tension électrique. Et finalement, la structure du crossbar permet de calculer la somme des sorties en appliquant les lois de Kirchhoff. Bien que la diversité des opérations soit plus restreinte qu'avec une architecture de von Neumann, il est possible d'utiliser des crossbars de memristors pour optimiser certains calculs spécifiques. Par exemple, ils sont parfaitement adaptés pour implémenter les calculs fondamentaux des neurones formels, qui reposent sur le principe de multiplication et accumulation [258] (équation (2.1)).

2.3.4 Imperfections et paramètres non idéaux

La miniaturisation des composants électroniques permet de concevoir des appareils de plus en plus compacts, mais cette croisade vers l'infiniment petit vient avec de nombreux défis techniques. À l'échelle nanométrique, il est difficile de reproduire plusieurs fois exactement le même composant, ce qui induit fatalement une variabilité dans les performances finales. De plus, à ces dimensions, les lois de la physique contraignent fortement les procédés de fabrication.

Les crossbars et les memristors sont donc soumis à de nombreuses imperfections [256, 258, 259, 263, 268, 269], que l'on regroupera ici sous le nom de paramètres non idéaux ou “non-idéalités”, en opposition aux modèles théoriques parfaits qui n'existent que dans le monde des mathématiques. Dans le cadre de ce projet, nous les analyserons d'après leurs impacts sur les performances d'un RNA. Un résumé des non-idéalités abordées dans la suite de cette section est illustré par la figure 2.5.

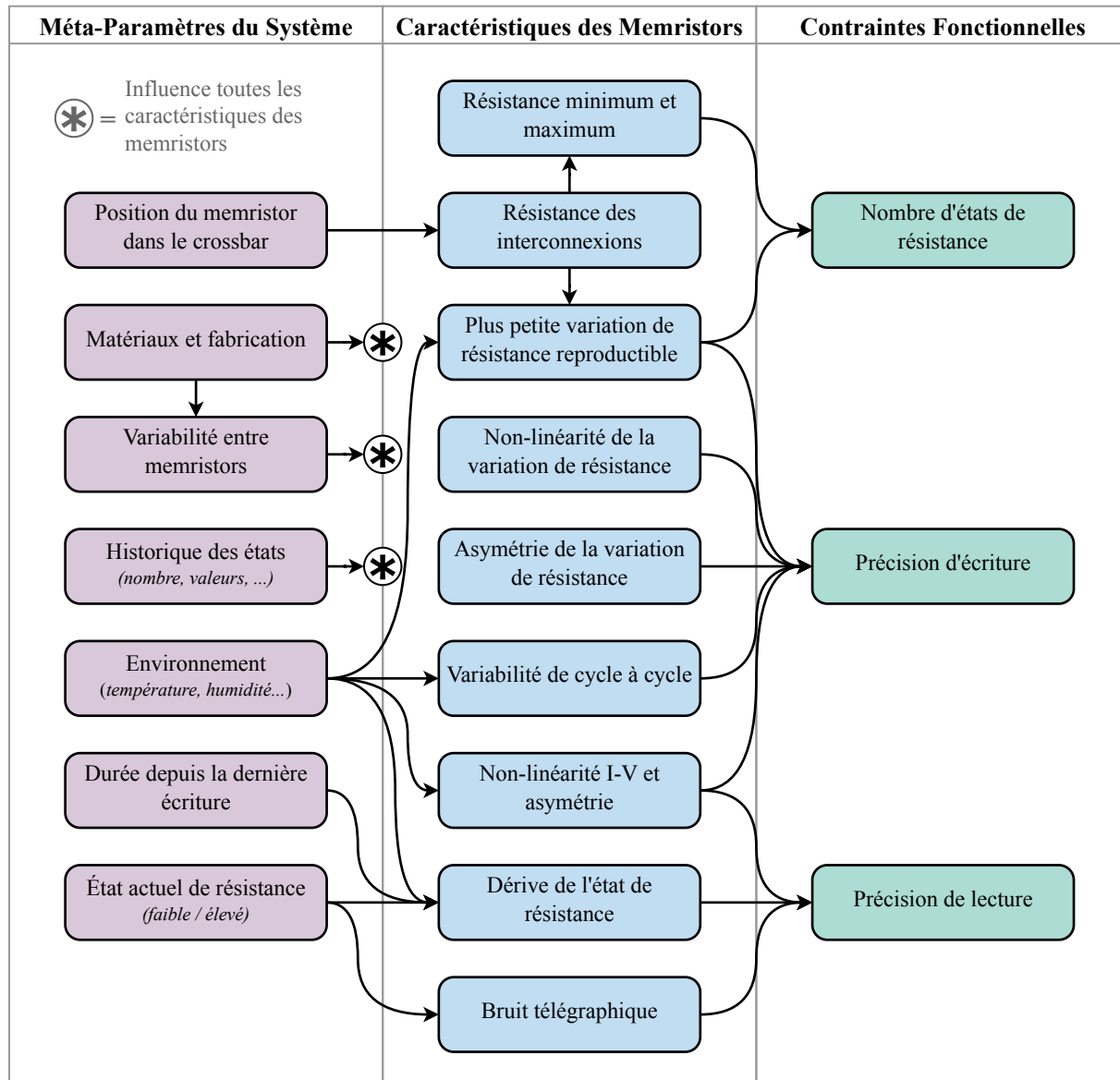


FIGURE 2.5 Classification schématique des paramètres non idéaux des systèmes memristifs d'après leurs impacts sur la précision des réseaux de neurones. Chaque connexion peut être lue comme “*peut avoir une influence significative sur*”, mais sans considération pour le niveau d'impact relatif. La première colonne *Méta-Paramètres du Système* regroupe les variables relatives au système memristif dans son ensemble qui impactent indirectement la précision des réseaux de neurones artificiels (RNA), par leurs influences sur les caractéristiques des memristors. La seconde colonne *Caractéristiques des Memristors* regroupe les paramètres qui impactent directement la précision des RNA. La troisième colonne *Contraintes Fonctionnelles* regroupe les mesures qui sont habituellement utilisées comme référence pour quantifier les performances des memristors.

2.3.4.1 Caractéristiques des memristors

Résistance minimum et maximum

Chaque memristor possède un état de résistance minimum ($R_{\min}$) et maximum ($R_{\max}$) qui sera en pratique contraint par la technologie utilisée [245]. Cependant, pour des applications binaires, il est important que $R_{\min}$ et $R_{\max}$ soient faciles à discriminer, c'est pourquoi il est courant de considérer uniquement leur ratio $R_{\text{ratio}} = R_{\min}/R_{\max}$. Si, $R_{\min} = R_{\max}$ l'état est considéré comme bloqué et le memristor n'est plus fonctionnel [270]. Pour notre application, où la résistance encode un poids synaptique, il est important que $R_{\min}$ et $R_{\max}$ soient au plus proche des valeurs extrêmes pour représenter efficacement les connexions entre les neurones. Dans le cas où deux memristors (m^+ et m^-) encodent un seul poids synaptique, on considérera que $R_{\min} = R_{\max}^- - R_{\min}^+$ et $R_{\max} = R_{\max}^+ - R_{\min}^-$.

Résistance des interconnexions

Dans un crossbar, les memristors sont reliés par des lignes de métal afin de répartir le courant d'entrée et d'accumuler le courant de sortie (figure 2.6b). Une partie du courant est cependant perdue sous forme de chaleur lorsque celui-ci circule dans ces lignes, ce qui se traduit par une résistance indésirable le long de ces lignes d'interconnexion⁵. Ainsi la résistance effective ($R_{\text{effective}}$) d'un memristor aux coordonnées i, j d'un crossbar sera la somme de : (i) la résistance d'accès (R_i) de la ligne l'entrée ; (ii) la résistance ($R_{m_{ij}}$) correspondante à l'état du memristor ; et (iii) la résistance d'accès (R_j) de la ligne de sortie. Ce qui implique qu'à état équivalent les memristors n'auront pas les mêmes valeurs $R_{\text{effective}}$ en fonction de leur localisation dans le crossbar, cette caractéristique limite inévitablement la taille de ce dernier.

Plus petite variation de résistance reproductible

La valeur de résistance d'un memristor n'évolue pas de façon parfaitement continue, mais par paliers plus ou moins espacés. Ce comportement est directement lié au changement discret de l'état interne du memristor, par exemple la construction ou la dissolution d'un filament dans le cas des redox-based RAM (ReRAM) [271, 272]. Si la variation minimale ($\Delta_{\min}$) est égale à l'amplitude maximale de variation de résistance ($R_{\max} - R_{\min}$), alors on parlera de variation binaire [251]. Cette valeur est principalement influencée par la variabilité de cycle à cycle et par les caractéristiques du signal électrique qu'il est possible d'adresser à ce memristor.

Non-linéarité de la variation de résistance

Idéalement, la valeur de résistance R d'un memristor devrait évoluer linéairement en réponse à un stimulus s qui lui est appliqué. Autrement dit, la nouvelle valeur serait définie

5. Le cas particulier de la supraconductivité n'est pas considéré ici [271]

par : $R_{t+1} = R_t + s \times \lambda$, où λ serait une valeur scalaire constante. Mais ce n'est pas ce que nous observons dans la majorité des cas [273–276], où λ varie en fonction de l'état actuel de résistance R . Par exemple, λ peut être beaucoup plus faible lorsque R est proche de $R_{\min}$ et $R_{\max}$, ce qui donne une courbure de programmation proche d'une fonction sigmoïde lorsque s est fixe.

Asymétrie de la variation de résistance

La résistance d'un memristor peut varier dans deux directions, elle peut augmenter (phase de potentialisation) ou baisser (phase de dépression), dépendamment de la nature du stimulus s qui lui est appliquée. La variation est qualifiée de symétrique si à niveau de résistance équivalent $\Delta_s = -\Delta_{-s}$. Mais cette propriété est rarement observée en pratique [245, 274–276], ce qui se traduit visuellement par une courbe de potentialisation asymétrique à celle de dépression d'après l'axe des ordonnées.

Variabilité de cycle à cycle

Nous avons vu que le changement de résistance Δ_s d'un memristor est influencé par : l'intensité du stimulus s , le signe du stimulus (asymétrie), la valeur actuelle de résistance (non-linéarité) et les valeurs limites $R_{\min}$ et $R_{\max}$. Mais même en connaissant parfaitement tous ces paramètres pour un composant donné, il est impossible de prédire exactement la valeur de Δ_s avant d'appliquer s [268, 274, 275]. Cette part d'incertitude est vraisemblablement imputable à l'environnement local et à l'état interne du memristor à l'échelle atomique. Ces facteurs, trop complexes pour être modélisés, sont regroupés au sein d'une même variable ϵ considérée comme complètement stochastique. Dans ce contexte, un cycle fait référence à une permutation complète de l'état de résistance, c'est-à-dire un retour à R_0 après être passé par $R_{\min}$ et $R_{\max}$.

I-V non-linéarité

Afin de lire la valeur de résistance d'un memristor, il est possible d'utiliser la loi d'Ohm, en fixant la tension électrique puis en mesurant l'intensité. Cependant, cette loi présuppose une évolution linéaire de l'intensité par rapport à la tension, ce qui n'est pas toujours observé en pratique [277–279].

Dérive de l'état de résistance

Les memristors font théoriquement partie de la famille des mémoires non volatiles, mais en pratique leurs états de résistance peuvent changer au cours du temps [280]. La période de rétention est très variable en fonction de la technologie utilisée et de la température. Pour être officiellement considérée comme non volatile, une mémoire doit être capable de conserver une valeur pendant 10 ans.

Bruit télégraphique

Le bruit télégraphique est un phénomène électrique aléatoire bien connu dans le monde des semi-conducteurs [281, 282]. Il est caractérisé par de brusques créneaux de courant (à tension constante), dont l'origine provient de défauts de fabrication, notamment à l'interface des matériaux. Les memristors ne sont pas épargnés par cette perturbation [283], qui est d'autant plus forte que le niveau de résistance est élevé [284]. Ce bruit affecte donc directement la précision de lecture du courant de sortie.

2.3.4.2 Méta paramètres du système

Position du memristor dans le crossbar

Dans un crossbar, tous les memristors ne sont pas logés à la même enseigne puisque certains sont placés en fin de ligne et d'autres au début. Cette différence de position peut rendre plus ou moins difficile l'accès à chaque memristor et influencer la valeur de résistance des interconnexions [271].

Matériaux et fabrication

Le choix des matériaux, les procédés de fabrications et la stratégie d'intégration sont cruciaux pour déterminer les caractéristiques du système final. Ce qui inclut entre autres : le mécanisme de permutation, le système de sélection (actif ou passif), la méthode d'écriture et de lecture. Il serait donc possible de fractionner ce paramètre, mais nous conserverons ici ce niveau de granularité pour des raisons de simplicité. Ce paramètre est probablement l'un des plus impactants, car il conditionne presque toutes les caractéristiques du système [245, 259, 263, 275].

Variabilité entre memristors

Même lorsqu'ils sont fabriqués avec le même procédé, chaque memristor est unique à l'échelle nanométrique et peut donc exhiber des caractéristiques différentes. Dans la littérature, ce paramètre est particulièrement mis en avant pour son influence sur la différence de dynamique d'évolution de la résistance d'un memristor à l'autre [274, 275]. Mais il est important de considérer que sa portée ne se limite pas à cela puisqu'il influence presque toutes les caractéristiques des memristors.

Historique des états

Le comportement à l'instant t d'un memristor ne dépend pas uniquement de son état R_t , mais aussi des différentes opérations d'écriture qui lui ont été appliquées précédemment. Le mécanisme de variation de la résistance peut effectivement s'altérer après un certain nombre de permutations. Le nombre et l'intensité des changements de résistances qui ont été appliqués à un memristor peuvent donc affecter ses performances futures, jusqu'à une

éventuelle fin de vie. Le nombre de permutations qu'il est possible de réaliser avant d'observer une altération significative des caractéristiques du système est de manière générale assez élevé, cette limite peut atteindre les 10^{10} itérations [285].

Environnement

L'environnement local du système, tel que la température [286] et l'humidité, peut perturber significativement ses performances [287]. À noter que le système lui-même peut altérer son environnement par émission de chaleur.

Durée depuis la dernière écriture

Par son impact direct sur la dérive de l'état de résistance [280], il peut être important de considérer le temps écoulé depuis la dernière opération d'écriture sur chaque memristor.

État actuel de résistance

L'état interne d'un memristor a pour fonction première de faire varier la valeur de résistance, mais d'autres caractéristiques sont affectées par effet de bord. C'est par exemple le cas du bruit télégraphique et de la précision d'écriture.

2.3.4.3 Contraintes fonctionnelles

Nombre d'états de résistance

En manipulant des signaux analogiques, il peut sembler paradoxal de parler d'un nombre fini d'états de résistance. En théorie, un memristor possède effectivement un nombre presque infini d'états, puisqu'ils correspondent aux différentes configurations possibles des atomes constituant la partie résistive du composant. Cependant, en pratique [245, 275] le nombre d'états de résistance N est limité par les bornes maximum $R_{\max}$, minimum $R_{\min}$ et par la plus petite variation de résistance que le système est capable d'adresser de façon reproductible $\Delta_{\min}$. Formellement cette valeur est donc définie par $N = (R_{\max} - R_{\min}) / \Delta_{\min}$.

Afin de comparer les systèmes memristifs avec les systèmes binaires traditionnels, il est courant de convertir ce nombre d'états en un nombre de bits. Ainsi, les memristors les plus performants peuvent atteindre 300 états [288], ce qui équivaut à un stockage d'information de 8,2 bits. Mais il est important de noter que la totalité est rarement exploitable en pratique car la faible précision d'écriture et de lecture empêche la discrimination de deux états proches.

Précision d'écriture

La précision d'écriture est influencée par différentes caractéristiques du dispositif, certains effets peuvent être anticipés (asymétrie, non-linéarité, etc.) et d'autres sont imprédictibles (variabilité de cycle à cycle). Cette valeur définit donc la marge d'erreur moyenne ε_w avec laquelle il est possible d'atteindre un état de résistance donné. L'opération d'écriture

d'un état R_n peut ainsi être simplifiée de cette façon : $\text{écrire}(R_n) = R_n \pm \varepsilon_w$. Il est possible d'utiliser des algorithmes itératifs de *lecture-écriture* qui vont permettre de réduire ε_w [261, 273, 275, 276]. Mais l'utilisation de telles méthodes a le désavantage d'augmenter grandement le temps et la consommation énergétique de chaque opération d'écriture. Les algorithmes itératifs sont donc plus adaptés pour les entraînements déportés (section 2.4.1.2), qui ne nécessitent qu'une seule étape d'écriture. À noter que l'erreur ε_w peut être supérieure à l'écart minimal de résistance entre deux états ($\Delta_{\min}$) et que l'erreur s'accumule lorsque plusieurs opérations d'écriture sont effectuées sans lecture.

Précision de lecture

La valeur de résistance d'un memristor est déduite en mesurant l'intensité du courant puis en appliquant la loi d'Ohm, ce qui équivaut à une opération de lecture, mais aussi à une multiplication (section 2.3.3). Tout comme l'écriture, cette opération est perturbée par différentes caractéristiques du dispositif qui peuvent être regroupées sous un facteur d'erreur ε_r tel que $\text{lecture}(R_n) = R_n \pm \varepsilon_r$.

2.3.4.4 Bilan

Les memristors étant encore activement étudiés, nous pouvons espérer que le perfectionnement des procédés de fabrication [289] et l'exploration de nouveaux matériaux permettent de corriger ces défauts. Mais certains sont difficiles, voire impossibles à corriger, il est donc nécessaire d'envisager des solutions alternatives. Des stratégies de correction d'erreurs sont abondamment utilisées avec la technologie CMOS pour faire face à ce type de problème [70–72]. Mais il est malheureusement difficile d'utiliser des méthodes de correction similaires sans nuire à l'efficacité énergétique et au parallélisme des calculs. Si l'on souhaite conserver les avantages des crossbars, il est donc nécessaire de prendre en considération leurs paramètres non idéaux au sein de l'application finale.

2.4 Réseaux de neurones sur memristors

Nous avons vu dans la partie précédente que les systèmes de calcul basés sur les memristors sont une alternative attrayante pour répondre aux limitations de l'architecture des ordinateurs modernes. Mais le prix à payer pour ce gain de performance est aussi une perte de polyvalence, seules certaines applications permettent de tirer pleinement profit du calcul en mémoire.

Les RNA ont à la fois besoin d'une évolution matérielle pour continuer à progresser [290] et peuvent bénéficier de ce nouveau paradigme de calcul [256, 259, 265]. En effet, dans un RNA, deux catégories de données sont manipulées : (i) les valeurs d'entrées (x) qui n'ont pas besoin d'être conservées après l'inférence (images de chiffres dans le cas de MNIST) ;

et (ii) les valeurs des paramètres du réseau, qui doivent être modifiées pendant l'étape d'entraînement puis conservées sur le long terme. Ainsi un RNA peut être implémenté sur un crossbar de memristors, où les valeurs d'entrées sont encodées dans un potentiel électrique et les paramètres du réseau dans la résistance des memristors (figure 2.6).

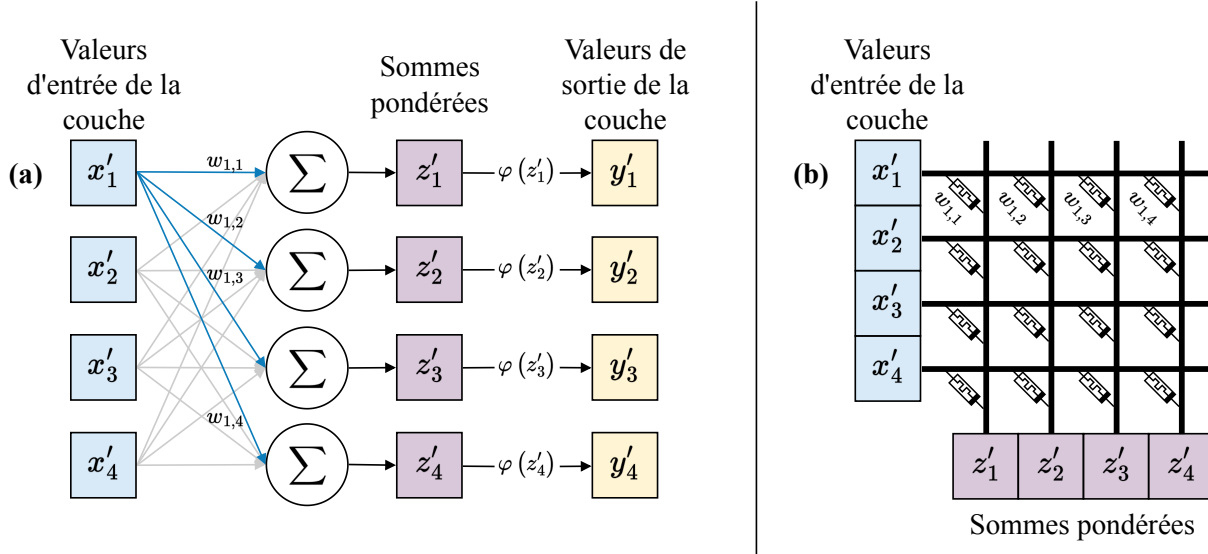


FIGURE 2.6 Comparaison entre (a) un réseau de neurones formels et (b) une implémentation physique basée sur un crossbar de memristors. Pour des raisons de lisibilité, seuls les poids de la première valeur d'entrée sont représentés ($w_{1,1-4}$). Une seule couche à 4 neurones d'un réseau est représentée, les valeurs de sortie y' peuvent donc être les entrées de la couche suivante. Le crossbar de memristors (b) permet de réaliser la première étape du calcul des neurones, qui est la somme pondérée, mais la fonction d'activation φ doit être prise en charge par son circuit périphérique.

2.4.1 Apprentissage sur puce ou déporté

Dans le contexte de l'apprentissage automatique à base de memristors, il existe deux approches pour réaliser la phase d'entraînement [273] : directement sur le crossbar ou déportée sur un ordinateur classique. Elles possèdent chacune leur utilité en fonction du scénario d'utilisation. Ce constat a poussé au développement de méthodes hybrides cherchant à combiner les avantages de ces deux approches.

2.4.1.1 Sur puce

Un apprentissage sur puce, ou *in situ*, consiste à entraîner un RNA sur le crossbar qui le représente. C'est-à-dire, de mettre à jour les paramètres en modifiant directement les valeurs de résistance des memristors correspondants.

Cette approche met à profit le parallélisme et l'efficacité du calcul en mémoire afin d'économiser du temps et de l'énergie pendant l'étape d'entraînement du réseau [266, 267]. En plus de cela, l'apprentissage sur puce permet d'atténuer certaines imperfections ma-

térielles [270, 271, 275, 276], notamment celles qui limitent le nombre d'états de résistance accessibles sur certains memristors du crossbar. Si par exemple un défaut bloque un paramètre du réseau à sa valeur maximale ou minimale, les autres paramètres pourront s'adapter pour compenser et ainsi conserver la performance globale du système.

2.4.1.2 Déporté

Un apprentissage déporté, ou *ex situ*, se déroule en deux étapes. La première consiste à entraîner le RNA de façon numérique, à partir d'un ordinateur conventionnel. Puis une fois l'entraînement terminé et tous les paramètres fixés, ces derniers sont copiés sur les memristors d'un crossbar.

Avec cette méthode, il n'est pas possible de bénéficier des avantages de vitesse et d'énergie des memristors pendant l'entraînement, puisque la majorité des calculs sont effectués sur un ordinateur de von Neumann. Mais cela permet en revanche d'ajuster plus précisément les paramètres puisque ces ordinateurs peuvent atteindre une précision très élevée et la copie une à une de ces valeurs permet d'appliquer des méthodes d'atténuation des erreurs d'écriture. Par exemple, avec plusieurs itérations de *lecture-écriture* [261, 273, 275, 291] et une anticipation des effets de non-linéarité et d'asymétrie. Enfin, l'apprentissage déporté facilite la production à grande échelle d'une solution de réseaux de neurones sur memristors, car un unique entraînement pour une tâche précise peut être copié sur un grand nombre d'appareils.

Pour exploiter à la fois les avantages des méthodes d'apprentissage sur puce et déportées, des méthodes hybrides ont été proposées [261]. Celles-ci commencent par un apprentissage déporté (entraînement numérique et copie), suivi d'une étape de réglages de précision qui consiste à ré-entraîner les paramètres de la dernière couche sur puce.

Cette approche ne règle pas les problèmes d'efficacité énergétique ni de vitesse d'apprentissage, mais elle facilite la mise à l'échelle tout en garantissant une certaine robustesse face aux états bloqués et aux erreurs d'écriture.

2.4.2 Implémentations existantes

Des implémentations fonctionnelles de réseaux de neurones basées sur memristors ont été démontrées à plusieurs reprises. Mais pour des raisons pratiques, beaucoup de travaux exploratoires ont recours à des simulations numériques.

2.4.2.1 Simulations

Les simulations systèmes sont souvent utilisées pour évaluer les effets des paramètres non idéaux, car cela permet d'isoler facilement chaque paramètre et ainsi mesurer indépendamment leurs impacts sur les performances de l'application ciblée. De cette façon, il a été

possible d'estimer une précision de 94 % sur MNIST avec un MLP à 3 couches [271]. En raison de leurs nombreuses applications industrielles, les CNN ont profité d'une attention particulière, ce qui a permis de mettre en évidence leur sensibilité à certains paramètres, tels que la variabilité de cycle à cycle et le nombre d'états de résistance [275, 276].

2.4.2.2 Expérimentales

Des tâches de classification de motifs ont été testées expérimentalement, d'abord pour un problème simple grâce à une implémentation minimaliste d'un réseau de perceptrons à une seule couche [273]. Puis un MLP à deux couches a permis d'atteindre 91 % de précision sur une version basse résolution de MNIST, malgré un taux de défaillance de 11 % des memristors du crossbar [270]. Et plus récemment, un CNN à 5 couches a été reproduit à partir de 8 crossbars de 2048 memristors chacun [261]. Ce qui a permis d'atteindre 96 % de précision sur le jeu de données MNIST, avec une efficacité énergétique plus de 100 fois supérieure à celle observée avec une architecture de von Neumann.

2.4.3 Effets des paramètres non idéaux et co-conception

La potentielle commercialisation d'appareils intégrant des RNA basés sur memristors dépend principalement de la gestion des paramètres non idéaux. Une co-conception entre le matériel et le logiciel est donc souvent mise en place pour optimiser les performances [259, 265, 271, 292].

2.4.3.1 Règle d'apprentissage

Il semble que le choix de la règle d'apprentissage soit un facteur clé pour s'accommoder des contraintes matérielles. La rétropropagation du gradient sur plusieurs couches risque de provoquer rapidement une accumulation des erreurs due aux imprécisions de lecture [274]. Cependant, des méthodes légèrement différentes permettent d'atténuer ce problème, l'ajout d'un *momentum* par exemple peut aider à converger vers une solution dans un contexte bruité [291]. Une option plus radicale serait de privilégier les règles d'apprentissages locales, telles que la plasticité en fonction du temps d'occurrence des impulsions [293], utilisées pour entraîner des réseaux de neurones à décharges [294].

Afin de s'accommoder des contraintes induites par les dispositifs physiques, il est envisageable de remplacer le calcul de la dérivée de la fonction d'activation par une fonction de substitution lors de la rétropropagation de l'erreur [295]. Cette méthode ne garantit pas une réduction de l'erreur à chaque itération, mais elle permet de s'affranchir de la non-dérivabilité (pour les neurones à activation binaire et les réseaux à décharges).

2.4.3.2 Codage des paramètres

Dans un RNA classique, les paramètres peuvent aussi bien être positifs que négatifs, mais la valeur de résistance d'un memristor est évidemment toujours supérieure à zéro. Il est donc

nécessaire de trouver une solution pour résoudre ce paradoxe. Une solution logicielle serait d'adapter les calculs pour éviter l'utilisation de nombres négatifs [296]. Et une solution matérielle serait d'utiliser deux memristors pour représenter chaque paramètre [273, 274], un positif et un négatif. Cette dernière méthode a en plus l'avantage de lisser la variabilité et la non-linéarité tout en augmentant le nombre d'états de résistance effectif pour chaque paramètre [275]. Si le nombre de memristors n'est pas un problème, la solution peut être étendue à plusieurs paires par paramètre pour accentuer ces bénéfices [276].

La non-linéarité de l'évolution de la résistance rend certains états plus difficiles à atteindre, il peut donc être préférable d'éviter ces valeurs pendant l'entraînement du réseau. Enfin, si la précision d'écriture est limitée à un certain nombre de bits, il peut être préférable d'arrondir la valeur de résistance souhaitée de façon stochastique [276].

CHAPITRE 3

Calibration automatique de boîtes quantiques

3.1 Avant-propos

Titre français :

Calibration automatique de boîtes quantiques en utilisant l'incertitude des réseaux de neurones

Auteurs et affiliations :

Victor Yon^{1,2,3}, Bastien Galaup^{1,2,3}, Claude Rohrbacher^{2,3,4}, Joffrey Rivard^{2,3,4},
Clément Godfrin⁵, Ruoyu Li⁵, Stefan Kubicek⁵, Kristiaan De Greve⁵,
Louis Gaudreau⁶, Eva Dupont-Ferrier^{2,3,4}, Yann Beilliard^{1,2,3}, Roger G. Melko^{7,8} et
Dominique Drouin^{1,2,3}

¹ Institut Interdisciplinaire d'Innovation Technologique (3IT), Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 0A5

² Laboratoire Nanotechnologies Nanosystèmes (LN2) — CNRS 3463, Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 0A5

³ Institut quantique (IQ), Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 2R1

⁴ Département de physique, Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 2R1

⁵ IMEC, Kapeldreef 75, 3001 Leuven, Belgium

⁶ National Research Council Canada, Quantum and Nanotechnologies Research Center, ON, Ottawa, Canada, K1A 0R6

⁷ Department of Physics and Astronomy, University of Waterloo, Waterloo, ON, Canada, N2L 3G1

⁸ Perimeter Institute for Theoretical Physics, Waterloo, ON, Canada, N2L 2Y5

État de publication : Publié (18 octobre 2024)

Journal : Machine Learning: Science and Technology

Référence : [297]

3.1.1 Objectif de l'article

La technologie de qubits de spin présente de nombreux avantages pour réaliser des calculs quantiques (section 2.1.2). Notamment une très bonne compatibilité avec les procédés existants de fabrication CMOS, permettant la mise en place rapide de production à grande échelle. Cet avantage a mené, par exemple, à la fabrication de puces en Ge/SiGe pouvant contenir 16 boîtes quantiques avec des grilles partagées [125]. Mais en pratique, initialiser un grand nombre de boîtes quantiques en un temps raisonnable reste un problème technique non résolu (section 2.1.4.1).

Pour adresser ce verrou technologique, il est nécessaire d'établir une méthode de calibration automatique des boîtes quantiques. Les méthodes les plus efficaces reposent sur des RNA pour détecter indirectement l'état des boîtes quantiques à partir des caractéristiques visuelles présentes dans les diagrammes de stabilité. Cependant, aucune méthode précédemment développée ne permet d'obtenir un taux de succès satisfaisant en présence des perturbations attendues dans un environnement expérimental.

L'objectif de cette première étude est de développer une méthode robuste de calibration automatique de charge de boîtes quantiques. Puis d'évaluer hors ligne la fiabilité de la méthode proposée de façon rigoureuse et quantitative. Travailler avec des diagrammes de stabilité statiques (préalablement mesurés) offre la flexibilité nécessaire pour tester rapidement de nombreuses méthodes dans une large variété de situations. Nous avons ainsi pu simuler 81 000 calibrations en quelques heures sur 27 diagrammes de stabilité différents.

Cette étude définit une référence en termes de performance pour la conception future d'appareils de calibration intégrés au sein du cryostat. L'architecture du RNA est maintenue simple dans l'objectif de faciliter l'intégration future sur un crossbar de memristors (section 2.4).

3.1.2 Résumé en français

Cette étude présente une procédure autonome pour confiner des porteurs de charge à l'intérieur de boîtes quantiques, à l'aide de méthodes d'apprentissage automatiques. La procédure de calibration minimise l'intervention humaine, répondant ainsi à l'un des défis majeurs de la mise à l'échelle des technologies de qubits de spin. Cette méthode exploite les RNA pour identifier les lignes de transition bruitées dans les diagrammes de stabilité, guidant ainsi une stratégie d'exploration robuste grâce aux estimations d'incertitude du modèle. Testée sur trois ensembles de données expérimentaux représentant différentes technologies de boîtes quantiques distinctes, cette approche atteint un taux de réussite

supérieur à 99 % dans les cas optimaux, où plus de 10 % de ce succès est directement attribuable à l'exploitation de l'incertitude. Les contraintes exigeantes des petits ensembles de données contenant une grande variabilité d'un diagramme à l'autre, nous ont permis d'évaluer les capacités et les limites de la procédure proposée.

3.2 Robust quantum dots charge autotuning using neural network uncertainty

3.2.1 Abstract

This study presents a machine learning-based procedure to automate the charge tuning of semiconductor spin qubits with minimal human intervention, addressing one of the significant challenges in scaling up quantum dot technologies. This method exploits artificial neural networks to identify noisy transition lines in stability diagrams, guiding a robust exploration strategy leveraging neural network uncertainty estimations. Tested across three distinct offline experimental datasets representing different single-quantum-dot technologies, this approach achieves a tuning success rate of over 99% in optimal cases, where more than 10% of the success is directly attributable to uncertainty exploitation. The challenging constraints of small training sets containing high diagram-to-diagram variability allowed us to evaluate the capabilities and limits of the proposed procedure.

3.2.2 Introduction

Multiple quantum computing technologies compete to outperform classical computers for practical applications, yet the journey toward achieving this objective is fraught with considerable technical challenges. This article specifically studies quantum systems based on semiconductor spin qubits [96, 100–102] formed by electrostatically confining electrons in quantum dots (QDs). This technology presents several advantages, such as high gate fidelities [109–113], long coherence times [106, 107], thermal robustness at temperatures greater than one kelvin [108], and compatibility with well-established complementary metal-oxide-semiconductor (CMOS) technologies [103, 104, 298]. This compatibility further amplifies scalability potential and enables co-integration with industrial electronics [299, 300]. However, QD technology is still in an early stage of development, and the best materials [301, 302], fabrication techniques, use cases, and control procedures are yet to be discovered.

One bottleneck in the large-scale implementation of quantum computers based on QDs is the complexity of charge tuning: one of the preliminary tasks required to set up a qubit. This tuning task confines a specific number of charge carriers in one or multiple QDs using the information provided by indirect measurements, which is typically obtained by tuning the voltages at two control gates (G1 and G2 in Figure 3.1). The results of voltage sweeping can be visualized as two-dimensional images termed stability diagrams (examples in Figure 3.2a,b,c), where pixels encode the current measured using a single-electron transistor (SET) [303] or a quantum point contact (QPC) [304]. One could then

identify the transition lines (highlighted with green lines in Figure 3.2d,e,f) that represent a change in the number of charge carriers inside the QD. Given the knowledge that electron-based QDs are empty at low gate voltages, it is possible to deduce the number of charges for each area in the voltage space (blue areas in Figure 3.2d,e,f).

However, the task is challenging to automate because the stability diagram parameter space is vast, noisy, and device-dependent. For this reason, the number of charges inside the QD is usually tuned manually by experimentalists based on informed guesses and human heuristics. While this approach is sufficient for small proof-of-concept studies in research laboratories, it is incompatible with large-scale industrial applications. Moreover, measuring a large area of a stability diagram could take hours, which significantly slows down QD initialization, especially since it often requires iterative measurements. One could attempt to automate the tuning procedure using classical algorithms and human expertise [305], but this approach still requires human involvement and is tailored to specific hardware. Thus, it does not satisfy the requirements for scaling up the technology or accelerating the development of new designs.

Recent studies [134,136] have suggested automating the tuning procedure using a machine learning (ML) approach with artificial deep artificial neural networks (NNs) [58]. These methods have shown promising results but are not yet reliable enough to consider fully autonomous large-scale QD control. The relatively high failure rate of these ML approaches (25 % [136] for single-QD and 43 % [134] for double-QD charge tuning) could be attributed to *(i)* the overconfidence of the model while facing ambiguous, out-of-distribution, too small, or noisy measurements and *(ii)* the low error tolerance of the exploration strategy. For example, one transition line misdetection will often lead to the failure of the entire charge tuning procedure. In the context of coarse tuning, [306] addressed a similar issue by first training an NN to evaluate the measurement quality before sending it to a second NN to infer the class. [132] combined this method with custom peak-finding techniques to tune offline double QDs into a specific charge regime with an accuracy of 89.7 %. Here, we chose a more versatile approach by training a single NN to detect the transition lines and providing a classification confidence score at the same time. One could also improve the performance of ML models by using more parameters and a larger dataset, but a very large model could be challenging to integrate near the tuned devices, and the high cost of collecting more diagrams is prohibitive. Even so, ML models never provide the guarantee of perfect accuracy.

This study introduces a hardware-agnostic autotuning procedure that leverages NN uncertainty to significantly enhance the robustness of semiconductor spin qubit charge tuning

with minimal human intervention. This is achieved by training an NN to identify transition lines in a stability diagram using supervised learning, coupled with an exploration strategy incorporating the NN’s predictions and confidence score (measure of uncertainty). We considered two methods to estimate the confidence score of the NNs: one with a heuristic from classical NNs and one using the Bayesian framework applied to NNs [204, 205, 307]. These two methods have been evaluated based on their concrete ability to reduce the critical failure rate during the autotuning procedure across three different experimental offline datasets measured using different single-QD hardware.

3.2.3 Problem definition

3.2.3.1 Project scope

Before using QDs as qubits to perform operations, each QD must be tuned to a specific charge state by applying appropriate gate voltages. The complete calibration procedure can be divided into five distinct steps: *(i) bootstrapping*: cooling the device and bringing its regime into the appropriate parameter range; *(ii) coarse tuning* [127–129, 131, 132, 140, 306, 308]: tuning the QDs to a specific topology (e.g., single QD, double QD); *(iii) establishing controllability* [132, 308–310]: setting up virtual gates that compensate for capacitive cross-talk; *(iv) charge state tuning* [132, 134–136]: tuning the QDs to a specific charge configuration (number of electrons in our case); *(v) fine-tuning* [308, 311]: adjusting the inter-dot tunnel coupling. For a more detailed description of these steps, refer to [69]. Performing these tasks quickly and reliably is one of the technical challenges preventing the scale-up of the number of qubits in QD-based quantum computers.

This work focuses on automating the task of charge state tuning *(iv)*. We assume the system is bootstrapped *(i)* and the voltage range contains a single QD *(ii)*. However, our method does not require setting up virtual gates *(iii)*, since we opted for a flexible approach by detecting the slope of the transition lines during the procedure.

3.2.3.2 Problem frame and constraints

We framed the problem of charge state tuning as an exploration task, where each step consists of detecting charge transition lines in the measurement of a voltage space subsection. The ultimate goal is to reach a specific charge regime while minimizing the exploration cost. In this case, the cost is the measurement time, which is directly affected by the number of steps and the size of the subsection measured. This approach allows us to break down the problem into two distinct tasks: transition line detection (Section 3.2.4) and the diagram exploration strategy (Section 3.2.5).

To uphold the robustness and adaptability of our methodology, we consciously minimized the utilization of preprocessing techniques while maintaining a relatively simple NN model. This decision was grounded in the ultimate objective of implementing this solution in custom electronics inside the cryogenic environment of a dilution refrigerator to perform in situ online tuning. The computational constraints and the need for a streamlined process inherent to this environment necessitate a careful balance between model complexity and operational energy efficiency. To evaluate the efficacy of our approach across various QD designs, we tested our method on three distinct datasets, each obtained from a different hardware and research group. This ensures that our solution is not tailored specifically to one quantum device but can be applied to a wide range of QD-based spin qubit hardware.

3.2.3.3 Datasets

We tested our approach on offline experimental measurements obtained from three different QD device architectures across distinct experimental setups. The first dataset, referred to as silicon split gate (Si-SG-QD) [312], contains 17 stability diagrams (Figure 3.2a) measured from devices fabricated with a metal-oxide-semiconductor (MOS) single-layer architecture (Figure 3.1a). The second dataset, referred to as gallium arsenide (GaAs-QD) [122], is composed of 9 stability diagrams (Figure 3.2b) obtained by measuring the current of a device fabricated with a GaAs/AlGaAs heterostructure (Figure 3.1b). The third and last dataset, referred to as silicon overlapping gate (Si-OG-QD) [298], includes 12 stability diagrams (Figure 3.2c) measured from devices fabricated with a MOS stacked-layer architecture (Figure 3.1c). The diagrams for the first and third datasets are measured using SET, while the ones for the second dataset are measured using a QPC. The measurements are two-dimensional stability diagrams represented as images, where the x - and y -axes correspond to the voltages applied to gates G1 and G2, respectively. If more than two gates are available in the experimental setup, the other ones are fixed to a voltage compatible with a single-QD regime.

Using simulated stability diagrams could increase the size of the training set [140, 313], but at the cost of a distribution shift between the training and testing sets, which could harm the quality of uncertainty quantification. We therefore chose to rely exclusively on experimental data. We also minimized measurement preprocessing to ensure compatibility with future in situ online implementation of this autotuning method. Only input normalization was applied to the measured patch before being sent to the classification model. See Appendix A.1 for detailed information regarding dataset preparation and diagram selection.

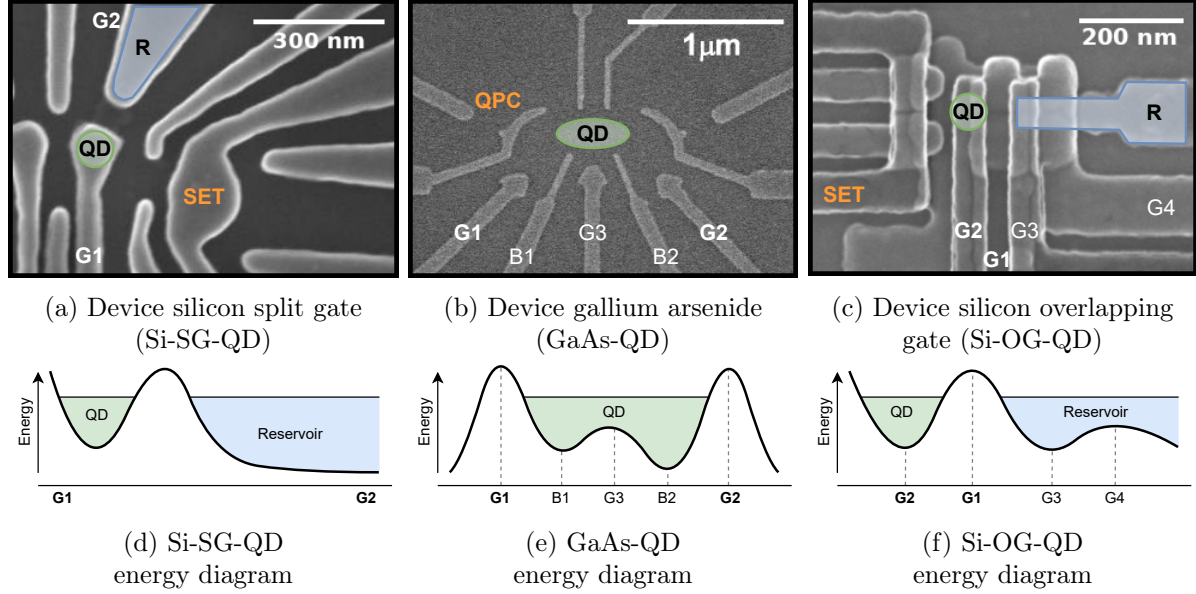


FIGURE 3.1 Quantum dot (QD) devices used to measure the stability diagrams. **(a,b,c)** scanning electron microscope (SEM) images of experimental devices. A single QD can be created by tuning electrostatic gates 1 (G1) and 2 (G2) to appropriate voltages. **(a)** G2 controls the electronic density of an electron reservoir (R), and the single-electron transistor (SET) is used as a charge sensor. See [312] for experimental details. **(b)** The tunnel barriers labeled B1 and B2 are set to a fixed voltage, as well as gate 3 (G3), which is required for more than one QD only. The quantum point contact (QPC) is used as a charge sensor. See [122] for experimental details. **(c)** G1 and G4 are tunnel barriers. The voltages at G3, G4, and R are fixed to allow for single-QD formation under G2 and an extended reservoir. The SET is used as a charge sensor. See [298] for experimental details. **(d,e,f)** Energy diagrams representing the formation of a single QD for each device.

Each dataset features different defects, noise, and transition line patterns, creating a large variety of challenges for the autotuning procedure. The diagrams from silicon split gate (Si-SG-QD) present strong oscillating background noise caused by the cross-capacitance effects of G1 and G2 acting on the SET, which can be hard to differentiate from the transition lines (see diagonal parasitic oscillations in Figure 3.2a). Therefore, the most challenging part of tuning such devices is reliably detecting the lines, especially in low- and high-current areas. The challenge is entirely different for diagrams from the gallium arsenide (GaAs-QD) dataset, where the transition lines are usually easy to identify but can fade (see first line in Figure 3.2b) and curve unexpectedly. This specificity moves the challenge to the exploration task, where missing a fading line can lead to a wrong number of charges in the QD. Finally, the diagrams from silicon overlapping gate (Si-OG-QD) present a peculiar hysteresis noise when the reservoir tunneling rate is slower than the voltage sweeping speed (see top left in Figure 3.2c), which must be correctly classified by the model and taken into account by the exploration strategy. Appendix A.1.6 presents a diagram sample of each dataset. All measurement files used to generate the diagrams in the present study can be downloaded from [314].

3.2.4 Line detection

3.2.4.1 Methods

The first part of the procedure consists of automatically detecting charge transition lines in subsections of offline experimental stability diagrams (Figure 3.3). An image segmentation model [315, 316] could identify lines, but it would require scanning large diagram sections, which is expected to slow down the autotuning process. To relax the input size requirement and reduce the model’s complexity, we address this problem as a supervised binary classification task (“*line*” or “*no-line*”). The choice of an NN rather than alternative statistical methods [141–144] is firstly driven by the goal of designing a hardware-agnostic method that can adapt to the rapidly evolving QD research field. ML algorithms can automatically adjust to new data features, such as a new artifact caused by a novel measurement setup or new transition line patterns induced by a change in the QD hardware design. NN methods also have the potential to employ transfer learning methods [317] to facilitate swift adaptation of the model to newly introduced devices. Furthermore, NNs have demonstrated outstanding performance in detecting patterns within images and filtering various types of noise, especially convolutional neural networks (CNNs) [318–321]. Therefore, they are well suited for detecting lines within images, such as transition lines in two-dimensional stability diagrams. Finally, NNs are known to maintain good scalabil-

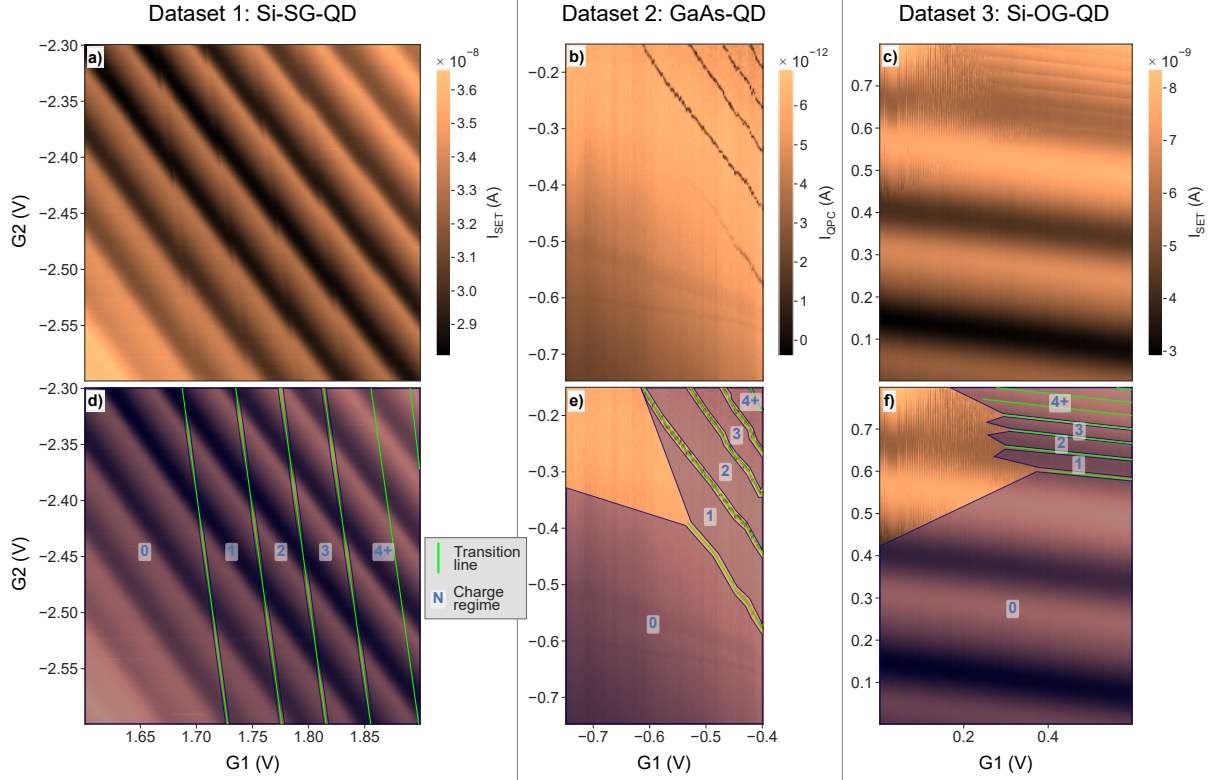


FIGURE 3.2 Example of one stability diagram from each dataset. (a,b,c) Representation of the stability diagrams as images, where pixels encode the measured current for given voltage values at the gates (G_1 and G_2). (d,e,f) Same diagrams with manual annotations of transition lines in green and charge regime areas in blue. The regions with more than 3 charges are grouped under the annotation “4+”. A voltage area not annotated with a charge regime due to fading lines or ambiguous boundaries is considered an “unknown charge regime”. More examples can be found in Appendix A.1.6.

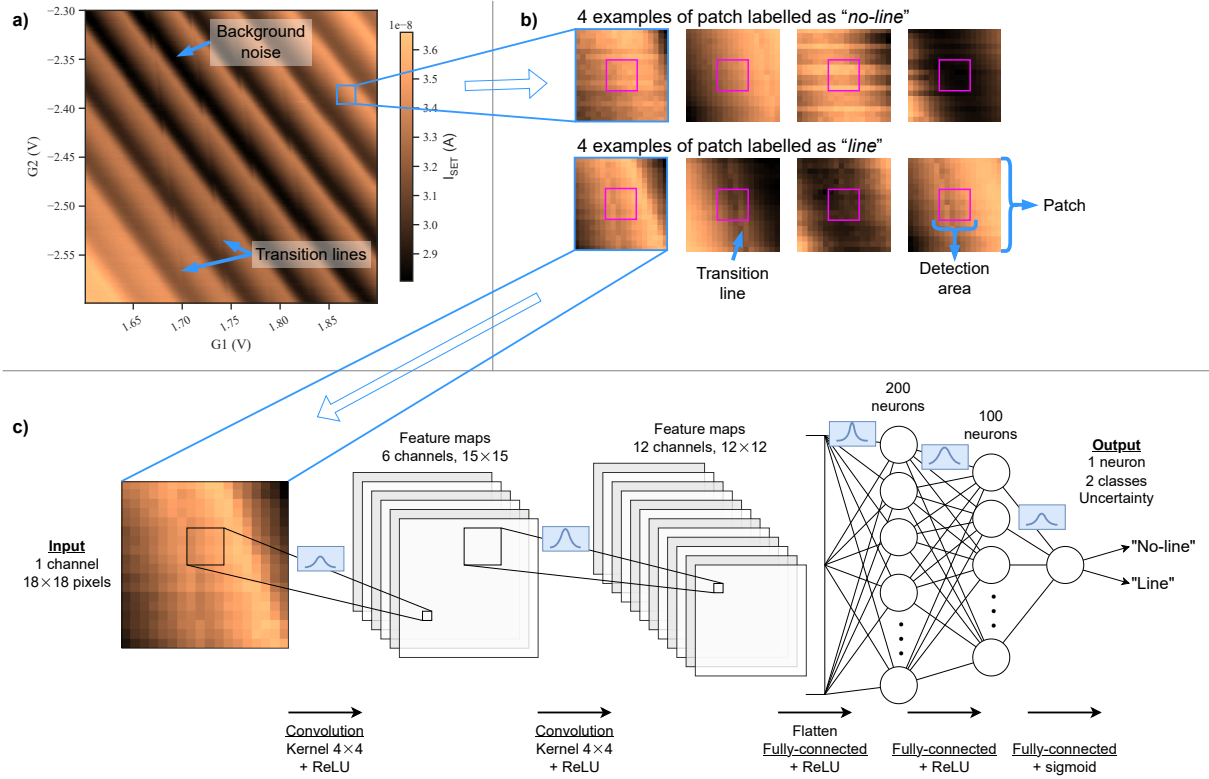


FIGURE 3.3 Automatic transition line detection using a Bayesian convolutional neural network (BCNN) in a stability diagram. (a) A subsection (patch) of the voltage space is measured. This diagram presents strong oscillating background noise that should not be misinterpreted as a transition line. (b) To train the model, each patch is categorized as “line” if a transition line annotation intersects with the detection area (in pink) in the center of the patch. Otherwise, the patch is categorized as “no-line”. (c) The patch is then sent as input to a model, where a forward pass propagates the information through convolutional and fully connected layers. In this example, a BCNN is represented, where each parameter is encoded as a Gaussian distribution.

ity [322] with respect to problem size and complexity; a crucial characteristic as we aim to develop a solution that is compatible with large arrays of QDs.

For this classification problem, we evaluated the performance of three NN architectures: CNNs, Bayesian convolutional neural networks (BCNNs), and feed-forward neural networks (FFs). CNNs are known for their effectiveness in image classification tasks [320], and they provide a good tradeoff between complexity and performance. BCNNs are expected to offer robust uncertainty estimations [191, 192], which can be beneficial for improving the reliability of automatic charge-tuning procedures. BCNNs implement the variational inference [200, 202] method with the Bayes-by-Backprop [203] learning rule. Meanwhile, the simpler FF is a reference for comparison with the more complex CNN and BCNN models.

All NNs presented in this article are fed with the same inputs and have comparable binary outputs. The model input is a small subsection of the voltage space (also referred to as a patch), where pixels represent the measured current values. The patch size is fixed to 18×18 data points as an empirical tradeoff between measurement speed and model accuracy. The patches are generated by splitting the diagrams into evenly spaced squares and then distributing them into training, validation, and test sets, where the test patches are extracted from a unique diagram excluded from the training set, and the other patches are randomly distributed between the training (90 %) and validation (10 %) sets. Each patch is automatically classified as “*line*” if an annotation of the transition line intersects with the detection area in its center (Figure 3.3b). This labeling approach provides more context to the NN while keeping the classification window narrow enough to fit between two transition lines.

The most basic NN used here is the FF, which propagates the input through two fully connected layers. The CNN extends this model by adding two convolution layers before the fully connected layers. The BCNN architecture is identical to the CNN, with each free parameter (weights and biases) encoded as a Gaussian distribution, defined by a mean and a variance (Figure 3.3c). Each model is trained to infer the patch class as a binary output and express the uncertainty of this classification as a confidence percentage. More technical details about data processing, manual diagram annotation, automatic patch labeling, and NN meta-parameters can be found in Appendices A.1 and A.2.

Introducing a confidence score into our model provides an additional layer of information about the model’s predictions. We leverage this information at the exploration level to significantly reduce the impact of misclassifications, which are usually responsible for

disrupting the autotuning procedure. We distinguish between three types of uncertainties [193, 323]:

1. **Model uncertainty** (also known as epistemic or systemic uncertainty) arises from a lack of knowledge due to imperfect training. In the context of transition line detection, this could be due to non-optimal meta-parameters or a low number of training diagrams. This type of uncertainty can be reduced as the model is exposed to more data, highlighting the importance of a comprehensive training process. In practice, the size of the training set is restrained by the acquisition cost of the experimental stability diagrams.
2. **Data uncertainty** (also known as aleatoric or statistical uncertainty), on the other hand, is associated with the inherently stochastic nature of the tested data. For the line classification task, this could result from variability in measurement tools, fabrication imperfections, or cross-talk, among other factors. These factors are often out of the experimentalist's control.
3. Finally, **distributional uncertainty** arises from an inadequate representation of the test set within the training set. In the context of line classification, this can occur when the stability diagram used for testing presents noise or features that are not present in the training set. Not using synthetic data helps us mitigate this type of uncertainty, but diagram-to-diagram variability is a source of distributional uncertainty that can never be entirely avoided.

The computation of the confidence score is handled differently between standard (CNN or FF) and Bayesian NNs. For a standard NN, we employ a simple heuristic score (Equation (3.1)) to estimate uncertainty based on the distance between the output y of the NN and the closest class [324, 325]. This estimation is based on the probabilistic interpretation [326] of the cross-entropy function [327] used to optimize the model. In the case of BCNNs, we use the variational inference [193, 202] approach by computing the normalized standard deviation σ based on N repeated inferences with newly sampled parameters (Equation (3.2)). Both formulas are trivial in the case of binary classification with one output neuron, but they can be generalized to multi-class problems [324].

$$\text{Heuristic confidence score} = |0.5 - y| \times 2 \quad (3.1)$$

$$\text{Bayesian confidence score} = 1 - \sigma(y_1, y_2, \dots, y_N) \times 2 \quad (3.2)$$

In addition, we performed a calibration step to optimize the exploration–exploitation tradeoff [328, 329]. NNs are typically calibrated using regularization methods during training or by empirically scaling individual bins of the reliability diagram after training [193, 330–332] to ensure that the confidence score is a good approximation of the actual probability of correctness. For example, among all classifications rated with 80 % confidence, 20 % of them are expected to be incorrect. In our case, the validation set does not consistently cover the range of confidence scores due to the low number of classification errors in some cases (e.g., the middle panel of Figure A.6b). This constraint makes the computation of calibration metrics unreliable, specifically for bin-based approaches. We worked around this problem by calibrating the confidence thresholds instead of the model confidence scores. The threshold defines the confidence value under which the classification will be considered too uncertain to be trusted during stability diagram exploration. After each training, the threshold value is optimized by minimizing the score defined in Equation (3.3) through a grid search on the validation set. The goal is to find an optimal tradeoff between the number of errors above the threshold (Err) and the total number of samples under the threshold (UT), where τ is a meta-parameter that defines the target ratio (fixed to 0.2 in this study). This approach relaxes the requirement regarding the number of calibration samples while maintaining the practical functionality of the confidence score. More details about the calibration process can be found in Appendix A.2.2.

$$\text{Threshold score} = \text{Err} + \text{UT} \times \tau \quad (3.3)$$

3.2.4.2 Results

With appropriate meta-parameters (Table A.2), the line classification task reached more than 90 % accuracy on every dataset and model combination (Table 3.1). The most significant performance difference between the NN architectures is visible for the Si-SG-QD dataset, where convolution layers are necessary to filter the oscillating parasitic background. For the GaAs-QD and Si-OG-QD datasets, the benefits of the convolution are less evident, since the measurement noise is lower and the transition lines are often clearly visible. For all the datasets, the classification performance of the Bayesian CNN is close to that of their classical counterparts, but the additional complexity of the Bayesian layers slows down the training by a factor of 4 on average. No overfitting was observed during training (Figure A.5), suggesting that the dropout of the classical layers and the Bayesian layers both act as efficient regularization methods.

A qualitative analysis of the misclassified patch samples (Figure 3.4 and Appendix A.2.3) suggests that we approach an optimal classification rate for each dataset, even if the model

accuracy never reaches 100 %. The remaining errors are often caused by ambiguous inputs that would be hard to classify at the patch scale, even by a human expert. The most common causes of misclassification are as follows: (i) ambiguous labeling (a fading line or a line annotation near the detection area), (ii) annotation errors (position inaccuracy or human error), and (iii) strong noise at the patch location. The proportion of ambiguous patches varies between the datasets, which explains why the average line classification accuracy differs. The transition lines from GaAs-QD and Si-OG-QD can be irregular and fading (94.4 % and 92.5 % accuracy with the best models, respectively), while the Si-SG-QD lines are straight and more predictable, simplifying patch classification (96.9 % accuracy with the CNN). The most challenging errors to address are the ones related to out-of-distribution issues caused by features in the test diagram that were missing from the training set. In an online scenario, this issue could occur if the device we are trying to tune is too different or if there are physical defects that did not exist in the previous experiments. This problem could be mitigated by a more diversified training set, a higher QD fabrication quality with less variability, or a reliable confidence score that correctly expresses the distributional uncertainty.

One can improve the effective model accuracy by excluding model predictions that are below the confidence threshold. The model outputs can then be interpreted as three-class inferences: “*line*” (output = 1 and confidence $\geq$ threshold), “*no-line*” (output = 0 and confidence $\geq$ threshold), and “*unknown*” (confidence $<$ threshold). On average, across all the datasets and models, this approach decreased the number of errors by 70 %, at the price of 11 % of the patches being classified as “*unknown*” (Table 3.1 and Supplementary Figure A.7). This method pushes the CNN classification accuracy above 97 % for all the datasets, which allows for robust autotuning procedures. However, we did not observe any benefits of using the Bayesian confidence score compared to the classical heuristic.

TABLE 3.1 Line detection results for each dataset and model. The performances are averaged over 10 runs using different random seeds. The standard deviation of the run performances represents the variability of the methods. Each run is a cross-validation across every diagram of the dataset. The best test accuracy scores of each dataset are highlighted in bold. The equivalent table without cross-validation is available in Table A.4.

Dataset	Model	Accuracy	Accuracy above threshold	Error reduction using threshold	Rate below threshold
Si-SG-QD	BCNN	96.9% ± 0.1	99.3% ± 0.1	78.8% ± 3.0	4.8% ± 0.4
	CNN	96.9% ± 0.1	99.4% ± 0.1	82.4% ± 2.7	4.9% ± 0.5
	FF	90.4% ± 1.1	99.3% ± 0.1	90.8% ± 1.3	22.3% ± 1.3
GaAs-QD	BCNN	93.3% ± 0.5	96.2% ± 0.4	48.5% ± 6.8	9.5% ± 1.1
	CNN	94.6% ± 0.2	97.4% ± 0.2	59.6% ± 4.3	8.4% ± 0.8
	FF	93.1% ± 0.2	96.5% ± 0.4	57.5% ± 5.9	9.4% ± 1.6
Si-OG-QD	BCNN	92.5% ± 0.4	96.9% ± 0.3	56.8% ± 3.2	10.6% ± 0.7
	CNN	90.7% ± 0.3	98.1% ± 0.2	77.8% ± 1.9	16.2% ± 0.5
	FF	90.8% ± 0.2	97.9% ± 0.1	77.8% ± 1.0	17.0% ± 0.5

3.2.5 Autotuning

3.2.5.1 Methods

Now that we have established a high-accuracy line detection method using NNs, we need to define an exploration strategy that uses model classification and uncertainty to efficiently explore the stability diagram space until the region of interest can be located. One step of this exploration is a cycle of the following: *(i)* patch measurement from a charge sensor on the device (simulated by extracting data from offline diagrams in this study), *(ii)* line detection using the trained NN, and *(iii)* deciding the following patch coordinates based on the exploration policy. The borders of the diagrams constrain the exploration to safe gate voltage ranges. The number of steps can be seen as a proxy for the relative tuning time, since the measurement is the longest part of the tuning process by orders of magnitude. The actual duration depends on the number of data points, the sensor type, and the electronic performance. For example, obtaining an 18×18 patch of Si-SG-QD takes approximately 2 min by measuring the SET current for the 324 points over a range of 36×36 mV using a *Keysight 34465A* multimeter. Although the measurement time can be shortened using more advanced and integrated equipment, it is likely to remain the limiting factor. Therefore, our exploration strategy aims to minimize the number of steps while maximizing the tuning success rate.

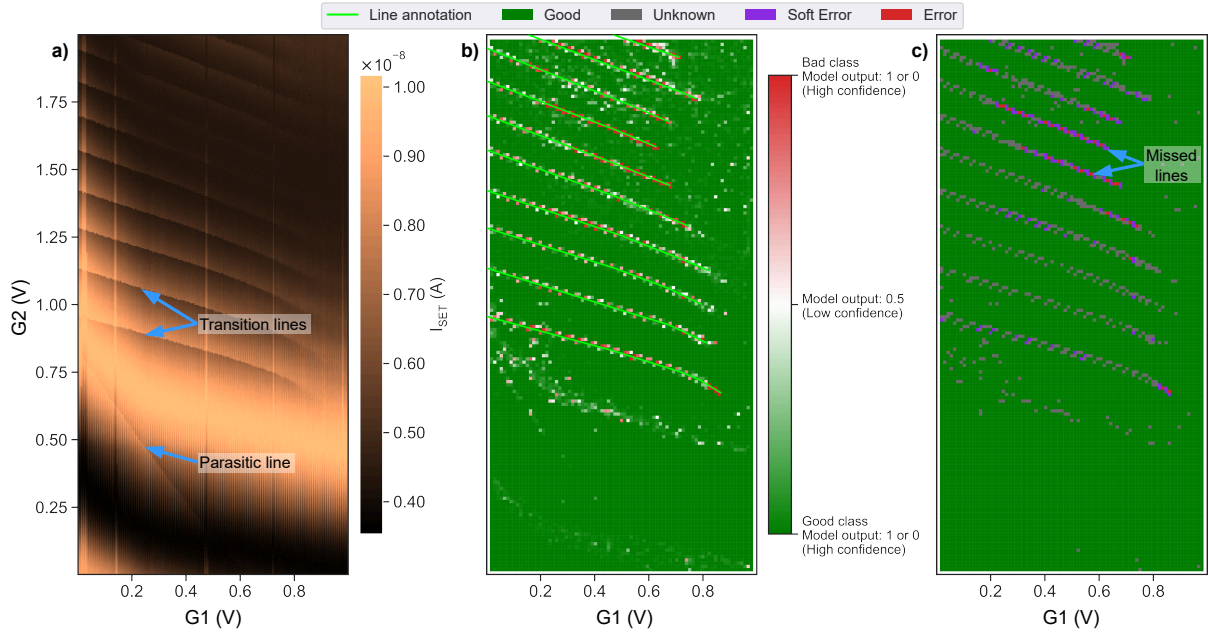


FIGURE 3.4 (a) Example of a stability diagram from the silicon overlapping gate (Si-OG-QD) dataset. This specific diagram presents a parasitic line in the low-voltage area.

(b) The same stability diagram is divided into patches classified using a convolutional neural network (CNN) trained using cross-validation [333] (Figure A.4). The color gradient of the patches represents the model uncertainty, where 0.5 is the lowest confidence score according to Equation (3.1). The confidence score is lower in low-contrast areas, around the parasitic line, and near transition lines (due to possible intersection ambiguity between the line labels and the detection area of the patch). (c) The same stability diagram after the application of the confidence threshold. Most errors are avoided, except for the end of a few fading lines and some line sections in the low-current area at the top of the diagram. The “soft errors” represent misclassifications near a line, which are not expected to induce tuning errors.

The typical strategy [132, 134–136] is to first search for the zero-electron regime, which is characterized by the absence of transition lines in a large area (bottom-left corners in the diagrams shown in Figure 3.2). Then, we count the number of transition lines until we reach the desired regime. In the case of one-electron tuning, the target location will be between the first and second lines. However, this simple exploration strategy is very susceptible to misclassification or hardware changes.

To make the exploration robust and compatible with most QD technologies, we adapt this exploration strategy, as illustrated in Figure 3.5 and a video¹. This approach can adapt on the fly to different line slopes (step 2) and spacings (step 3) while checking for any fading lines (step 4). When a patch is classified with a confidence score below the threshold, we explore the space surrounding it to reduce the risk of critical failure due to NN misclassification. We validate or refute the presence of a line by taking additional steps in its supposed direction (see the purple arrow in Figure 3.5b, step 4) until a patch is classified with high confidence.

We evaluated this exploration strategy for each diagram that contained the one-electron regime within the measured voltage range (nine for each dataset). We used a k-fold cross-validation method [333] (Figure A.4) to test our approach on every valid diagram while preventing the inclusion of testing patches in the training set. This testing method should be close to an online autotuning situation, where we want to tune a new device based on previous experiments. The tuning success represents the proportion of final voltage coordinates inside the one-electron regime area out of 50 random starting points for every diagram in the dataset. The entire experiment is reproduced 10 times using different random seeds that affect the NN parameter initialization, the training process, and the random starting points. In total, 1.1×10^7 steps were evaluated in 81,000 offline autotuning simulations using 810 independently trained NNs.

3.2.5.2 Results

The results for each possible combination of dataset and NN are summarized in Table 3.2, showing the autotuning success rate with and without leveraging the NN’s confidence score (uncertainty-based). Using the model uncertainty information consistently reduces the number of tuning failures (−53 % on average) at the price of a few additional steps (+22 % on average). The best improvement is observed for the CNN on Si-SG-QD, where the tuning success rate increased from 88.8 % to 99.5 % when the autotuning procedure exploited the confidence score (−95 % tuning failures and +8.5 % steps).

1. Method presentation and animated autotuning examples: youtu.be/9pPrgrIx9O0

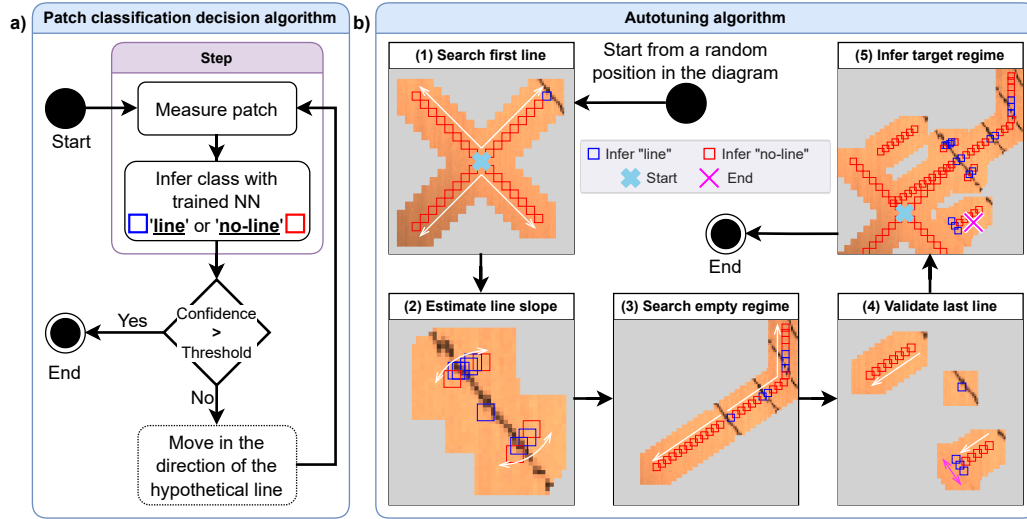


FIGURE 3.5 (a) Schematic representation of the patch classification algorithm. The purple box encloses the logic of one exploration step. (b) Step-by-step example of the autotuning algorithm using the gallium arsenide (GaAs-QD) dataset (complete diagram in Figure 3.2b). The arrows represent the direction of the exploration, and the gray area is unmeasured space during the current step. (1) Search the first line by exploring the voltage space in 4 directions. (2) Estimate the line slope by scanning two sequences of patches in circular arcs around the first patch with a line. (3) Explore the space perpendicularly to the first line to gather information about the distance between lines. Stop after reaching 3 times the average distance without detecting a new line at lower voltages. (4) Search for possible missed lines under the first line by scanning multiple sections where we would expect to find a new line according to the average line distance and slope estimation. In this example, a fading line is correctly detected at the bottom of the image, but the low confidence of the model triggers a validation procedure. More scans are then processed on the hypothetical line direction (purple arrow) until a higher confidence inference validates or invalidates the line's existence. (5) Deduce the one-electron regime location based on the leftmost line position, the slope, and the average space between lines (all scans from previous steps are represented here). The algorithmic details of each step are available in Appendix A.3.1.

Surprisingly, the confidence score provided by the Bayesian version of the CNN does not provide a significant improvement over the simple heuristic obtained using the standard CNN. The average line detection accuracy is similar for the CNN and BCNN on the Si-SG-QD dataset, and the line detection performance is slightly better for the BCNN on Si-OG-QD. However, on every dataset, the CNN reaches a higher tuning success by using the confidence heuristic score (highlighted in bold in Table 3.2). Even in the case where the BCNN line detection accuracy is higher (Si-OG-QD), the tuning success benefits more from the CNN confidence score (+16.4 % tuning success) than the one from the BCNN (+8.9 % tuning success). This result suggests that the number of critical misclassifications above the confidence threshold is more frequent with the Bayesian NN’s uncertainty compared to the classical confidence heuristic score.

The line detection accuracy can provide valuable indications regarding the NN’s performance, but it is not entirely correlated to the tuning success rate, since some misclassifications are much more harmful than others during diagram exploration. If a classification error occurs near a line (soft errors in Figure 3.4c), it will likely be corrected at the next step. However, an error on a line, or far from it, will likely affect the final tuning result. For example, if a noisy patch is wrongly classified as a line in the empty regime area, the number of charge carriers inside the QD will be overestimated by one.

The uncertainty-based autotuning method with convolution models allows for a nearly perfect tuning success rate (>99 %) on the Si-SG-QD dataset. This high performance is attributable to (i) the convolution layers’ ability to efficiently filter the oscillating background noise, (ii) the exploitation of the model confidence score to prevent most critical errors, and (iii) the straight transition lines that simplify the exploration. The line detection task also benefits from the large training set (72,000 patches) and the low variability between diagram features.

The GaAs-QD and Si-OG-QD datasets present more feature diversity in their diagrams with a smaller training set, which makes the classification task more challenging. The shape of the transition lines is also less predictable than the Si-SG-QD dataset, which increases the risk of missing a line or failing to detect its slope and spacing. The significantly higher tuning performance without cross-validation (Table A.5) suggests that the training sets do not cover the diversity of the stability diagrams. This is well illustrated by the CNNs on GaAs-QD, where the tuning success rate is improved by 11.6 % without cross-validation (92.2 % versus 80.6 %). This hypothesis is confirmed by the high variability of the tuning success rates between the diagrams in these two datasets. The results are very polarized between the diagrams for which the one-electron regime is found nearly

100 % of the time, while some are more complex to tune and exhibit a very low success rate. A larger or more consistent dataset could reduce the tuning success loss observed with cross-validation.

Overall, noise and hardware imperfections are relatively easy to manage for the line detection task but more challenging to deal with at the diagram scale during exploration. Therefore, the reliability of the autotuning procedure is mainly affected by the shape and consistency of the transition lines.

TABLE 3.2 Autotuning results for each dataset and model, with and without using the model uncertainty information provided by the confidence score. The line detection accuracy and tuning success variability are computed over 10 runs using different random seeds. Each run is a cross-validation over every diagram in the dataset. The best tuning success rates for each dataset are highlighted in bold. The tuning success rates can be compared to the baselines presented in Table A.3. The equivalent table without cross-validation is available in Table A.5.

Dataset	Model	Line detection accuracy	Uncertainty-based tuning	Average step number	Tuning success
Si-SG-QD	BCNN	96.9% ± 0.1	Yes	164	99.2% ± 0.7
			No	148	88.2% ± 2.3
	CNN	96.9% ± 0.1	Yes	165	99.5% ± 0.7
			No	152	88.8% ± 2.4
	FF	90.4% ± 1.1	Yes	194	72.4% ± 6.3
			No	122	23.9% ± 5.4
GaAs-QD	BCNN	93.3% ± 0.5	Yes	104	75.3% ± 4.2
			No	92	55.9% ± 3.8
	CNN	94.6% ± 0.2	Yes	103	80.6% ± 3.9
			No	93	72.4% ± 2.9
	FF	93.1% ± 0.2	Yes	105	72.8% ± 4.5
			No	92	58.4% ± 3.4
Si-OG-QD	BCNN	92.5% ± 0.4	Yes	185	75.2% ± 2.5
			No	155	66.3% ± 3.4
	CNN	90.7% ± 0.3	Yes	193	78.1% ± 1.7
			No	150	61.7% ± 3.0
	FF	90.8% ± 0.2	Yes	200	78.0% ± 2.5
			No	151	59.3% ± 2.4

3.2.6 Discussion

The proposed method can be used to automate the charge tuning of any spin-based semiconductor single QD using less than 10 annotated stability diagrams for training and some basic prior knowledge of the device characteristics (expected working range voltage, approximate transition lines slope, and spacing). The autotuning procedure is resilient to noise and physical imperfections but sensitive to high device-to-device variability and training dataset quality (number of diagrams and feature coverage). This is well illustrated by the Si-SG-QD dataset, which features strong noise but consistent diagrams, allowing for a robust autotuning procedure with only 23 tuning failures over 4500 CNN uncertainty-based tuning simulations. Therefore, this method, associated with a good fabrication yield, has the potential to enable parallel charge tuning of large QD arrays, which is currently not practical using a manual approach.

This study demonstrates the benefits of using NN uncertainty to automate the tuning of QDs based on only partial measurements of the stability diagrams. The confidence score provides valuable information regarding the model prediction, which can be used to design robust autotuning procedures. However, we did not see a clear benefit of using a Bayesian NN over a standard one, even though Bayesian models are specifically designed to provide uncertainty measurements. This behavior could be explained by the distributional uncertainty being poorly captured by the Bayesian NNs [334], which could be the predominant source of uncertainty when using the cross-validation testing method. Moreover, many approximations [202, 307] are necessary to keep the Bayesian NN training and inference tractable, which could be detrimental to the overall quality of the model and its coherence with the initial Bayesian framework. Thus, the additional complexity and computing cost induced by the Bayesian NN is hard to justify when a simpler NN can provide a more reliable confidence score. In future work, other methods of uncertainty quantification [335] (e.g., dropout as Bayesian approximation [204] or ensemble learning [336]) and prior selection [337, 338] could be evaluated to improve the reliability of the confidence score. Bayesian optimization associated with classical ML [339, 340] also appears to be a promising avenue to harness the complexity of quantum engineering.

While applying quantum computing gates requires more than one QD, we developed and tested our approach on single-QD datasets as a first step toward robust double- and triple-QD autotuning. Since NNs are known for their good scalability, we expect this method to be expandable to larger input dimensions [129, 341] and greater numbers of classes. When the problem’s complexity increases, the benefits of the confidence score could become even more important.

To avoid any wiring bottleneck [167] between the fridge and external electronics, this calibration method could be integrated near the QDs in the 4 K stage of the cryogenic environment. One way to meet this requirement would be to transfer the trained NN to low-power and cryo-compatible hardware, such as circuits based on arrays of memristive devices [342–344]. This specialized hardware takes advantage of in-memory computing [252, 291] to efficiently perform the multiply–accumulate (MAC) operations [258] required for NN inference. Therefore, memristor-based systems represent promising candidates to realize scalable autotuning from inside the fridge with close-loop measurements and minimal disturbance over the QDs. Future studies should be performed to simulate these hardware implementations and demonstrate real-time online autotuning using the proposed method.

3.2.7 Author contributions

All authors contributed to this article and approved of the submitted version.

Victor Yon: methodology, datasets aggregation and processing, software implementation, running experiments, results analysis and visualization, writing—original draft preparation.

Bastien Galaup: contributed to software implementation, running experiments, review and editing.

Claude Rohrbacher and Joffrey Rivard: provided data for the Si-OG-QD dataset, provided expertise and technical assistance with QD technology, review and editing.

Clément Godfrin, Ruoyu Li, Stefan Kubicek, and Kristiaan De Greve: manufactured the devices used to obtain the Si-OG-QD dataset.

Louis Gaudreau: provided data for the GaAs-QD dataset, provided expertise and technical assistance on QD technology, review and editing.

Yann Beilliard: methodology, supervision, funding acquisition, review and editing.

Eva Dupont-Ferrier and Roger Melko: supervision, review and editing.

Dominique Drouin: methodology, supervision, funding acquisition, review and editing.

3.2.8 Code and data availability

The Python source code used to aggregate and build the QD stability diagram dataset is publicly accessible on *GitHub*: github.com/3it-inpaqt/qdsd-dataset.

The Python source code used to run all the experiments presented in this article is publicly accessible on *GitHub*: github.com/3it-inpaqt/dot-calibration-v2/tree/offline-article.

The raw and processed stability diagram measurements used to train and test the model presented in this article are publicly available for download from [314].

All simulation outputs and results used to generate the tables and figures presented in this article are publicly available for download from [345].

3.3 Bilan de l'article

Cette étude a permis de valider qu'un CNN associé à une stratégie d'exploration est une solution viable pour la calibration automatique de boîtes quantiques. L'exploitation de l'incertitude du modèle nous a permis d'augmenter significativement le taux de succès de la procédure de calibration, de façon à surpasser les méthodes existantes pour la calibration de charge d'une boîte quantique simple.

La mesure et l'utilisation d'un score d'incertitude dans le cadre de l'apprentissage automatique est en soit une question de recherche importante pour de nombreuses applications où une erreur peut avoir des conséquences vitales (la santé, les véhicules autonomes, etc.). La pertinence de l'utilisation des réseaux de neurones bayésiens est notamment une question ouverte, et cette expérience présente un cas d'utilisation concret où cette méthode ne présente aucun bénéfice notable. Les résultats de cette étude peuvent donc être utiles à des applications autres que la calibration de boîtes quantiques.

CHAPITRE 4

Calibration automatique de boîtes quantiques expérimentale

4.1 Avant-propos

Titre français :

Calibration automatique de boîtes quantiques expérimentale en temps réel à l’aide de réseaux de neurones

Auteurs et affiliations :

Victor Yon^{1,2,3}, Bastien Galaup^{1,2,3}, Claude Rohrbacher^{2,3,4}, Joffrey Rivard^{2,3,4}, Alexis Morel^{2,3,4}, Dominic Leclerc^{2,3,4}, Clément Godfrin⁵, Ruoyu Li⁵, Stefan Kubicek⁵, Kristiaan De Greve⁵, Eva Dupont-Ferrier^{2,3,4}, Yann Beilliard^{1,2,3}, Roger G. Melko^{6,7} et Dominique Drouin^{1,2,3}

¹ Institut Interdisciplinaire d’Innovation Technologique (3IT), Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 0A5

² Laboratoire Nanotechnologies Nanosystèmes (LN2) — CNRS 3463, Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 0A5

³ Institut quantique (IQ), Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 2R1

⁴ Département de physique, Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 2R1

⁵ IMEC, Kapeldreef 75, 3001 Leuven, Belgium

⁶ Department of Physics and Astronomy, University of Waterloo, Waterloo, ON, Canada, N2L 3G1

⁷ Perimeter Institute for Theoretical Physics, Waterloo, ON, Canada, N2L 2Y5

État de publication : Soumis (2 octobre 2024)

Journal : Nano Letters

Référence : [346]

4.1.1 Objectif de l'article

L'étude précédente (chapitre 3) a permis d'établir une procédure de calibration automatique grâce à l'évaluation de nombreux paramètres (architecture de réseau de neurones artificiels (RNA), méthode d'entraînement, utilisation de l'incertitude, etc.). L'optimisation de tous les méta-paramètres n'aurait pas été possible sans la rapidité et la flexibilité offerte par les simulations hors ligne. Il est cependant indispensable de se confronter à la réalité expérimentale pour valider la viabilité de la méthode proposée.

L'objectif de l'étude suivante a donc été de connecter le code Python avec les instruments de mesure et de contrôle de tension des grilles, pour piloter en temps réel le processus de calibration automatique. Nous avons ainsi pu évaluer l'efficacité d'un modèle entraîné sur des diagrammes de stabilités hors ligne lorsqu'il est transféré à un environnement expérimental. Contrairement à la plupart des expérimentations de la littérature, nous avons une fois de plus privilégié une évaluation quantitative, en mesurant le taux de succès de la procédure après plusieurs itérations randomisées.

4.1.2 Résumé en français

Les qubits de spin promettent une mise à l'échelle efficace des calculateurs quantiques, mais ils ont cependant besoin de procédures fiables de calibration automatique. Cette étude présente une calibration expérimentale du nombre d'électrons isolés dans une boîte quantique, à l'aide d'un réseau de neurones convolutif intégré dans un système de contrôle en boucle fermée. L'algorithme de calibration explore en temps réel l'espace des tensions pour localiser les lignes de transition de charge, permettant ainsi d'isoler le régime à un seul électron sans intervention humaine. La méthode proposée a permis de localiser le régime ciblé 95 % du temps au cours de 20 expériences itératives sur un dispositif refroidi à 25 mK. Ces résultats confirment la robustesse de la procédure face au bruit de mesure, et aucune perte de performance n'a été observée en comparaison aux tests précédemment réalisés sur un jeu de données hors ligne. Chaque cycle de calibration a duré en moyenne 2 heures et 9 minutes, principalement en raison de la vitesse limitée de la mesure de courant. Ce travail valide la faisabilité de la calibration automatique de charge de boîte quantique en temps réel piloté par un RNA, faisant ainsi progresser le développement vers le contrôle de matrice de qubits à grande échelle.

4.2 Experimental online quantum dots charge autotuning using neural networks

4.2.1 Abstract

Spin-based semiconductor qubits hold promise for scalable quantum computing, yet they require reliable autonomous calibration procedures. This study presents an experimental demonstration of online single-dot charge autotuning using a convolutional neural network integrated into a closed-loop calibration system. The autotuning algorithm explores the gates' voltage space to localize charge transition lines, thereby isolating the one-electron regime without human intervention. This exploration leverages the model's uncertainty estimation to find the appropriate gate configuration with minimal measurements while reducing the risk of failures. In 20 experimental runs, our method achieved a success rate of 95 % in locating the target electron regime, highlighting the robustness of this approach against noise and distribution shifts from the offline training set. Each tuning run lasted an average of 2 hours and 9 minutes, primarily due to the limited speed of the current measurement. This work validates the feasibility of machine learning-driven real-time charge autotuning for quantum dot devices, advancing the development toward the control of large qubit arrays.

4.2.2 Introduction

Semiconductor spin qubits [96, 100–102, 347] can encode quantum information using the spin- $1/2$ of a charge carrier, which can be manipulated using external magnetic fields to perform quantum computing based on the principles of superposition and entanglement. This technology stands out due to its high gate fidelity [109–113, 348, 349], long coherence times [106, 107], thermal robustness [108, 350, 351], and compatibility with existing complementary metal-oxide-semiconductor (CMOS) technologies [103, 104, 300, 312, 352]. These characteristics make spin qubits excellent candidates for building scalable quantum computers using already existing industrial fabrication methods [299, 300, 353, 354]. However, significant engineering challenges remain, such as improving device fabrication quality [302, 355–357] and developing autonomous calibration procedures for a large number of quantum dots (QDs) [125, 358].

A spin qubit is formed by trapping a specific number of charge carriers within an isolated island (QD) using, for example, the electrostatic confinement gates of a nanoscale device (as shown in Figure 4.1a). Calibrating the QD device to achieve a specific physical state is a complex task that is generally approached in a series of sequential steps [69]. Initially, the device is cooled, local charge sensing systems are activated, and the voltages of the con-

finement gates are adjusted to operate within appropriate ranges for data collection [126]. Following this, the gate voltage ranges corresponding to a known global structure (such as single- or double-QD configurations) are calibrated. This step is usually referred to as coarse tuning [127–129, 131, 132, 140, 306, 308]. Next, optional virtual gates could be established to compensate for capacitive crosstalk, ensuring precise control of the individual QDs without unwanted interference [132, 308–310, 359, 360]. Subsequently, the device gate voltages are precisely tuned to achieve a specific charge carrier count in each QD, which in our experiment refers to the number of electrons in a single QD. This step is usually referred to as charge state tuning [130, 132, 134–136, 297, 305, 361]. Finally, the system’s physical parameters are refined in preparation for quantum computations [137, 308, 311, 362].

Performing efficient and accurate calibration every time the system is cooled down is critical to deploying large-scale QD-based systems. However, automatizing this process is challenging due to the sensitivity of the operational parameters, where each variable can affect others nonlinearly, exponentially increasing the complexity as the number of tuned devices grows. Variability in device fabrication adds another layer of difficulty, as each QD can behave differently, requiring customized tuning approaches. These devices and the measurement method are also susceptible to environmental conditions (e.g., thermal noise and electromagnetic interference), adding stochasticity to the measurements.

Recent progress in producing larger arrays of QDs [125, 353] has accelerated the need for robust control procedures. This challenge has been partially addressed by leveraging machine learning (ML) models [129], which can be used to tune quantum devices by automatically navigating through the large and noisy parameter space. However, ML methods, especially artificial neural networks (NNs), are sensitive to the distribution shift [193] between the training and testing data and are well known for unexpected failures [363]. Therefore, it is necessary to validate any NN-based autotuning method in a real-world experimental environment [126]. However, due to the scarcity and expense of the hardware required to run such experiments, most autotuning demonstrations are performed offline (i.e., using static pre-recorded data). A few studies have demonstrated online experimental procedures for the first steps of the autotuning process: bootstrapping [126], coarse tuning [128, 131], and creating virtual gates [359, 360]. [305] showed an early demonstration of charge state autotuning using Gabor filters [364] and classical optimization algorithms, resulting in a single successful run after 3 hours. Only [138] ran and benchmarked online experiments that covered the full tuning procedure. Their method—based on a combination of tree search, Bayesian optimization, segmentation algorithms, and NNs—allowed them to reach 77 % success over 13 runs for complete QD tuning. However, each run took

an average of 38 hours to complete due to the time required to measure large stability diagrams and the repeated calibration necessary after a stage failure.

The autotuning algorithm used during the experiment has been developed and tested offline in a previous study [297]. It relies on a convolutional neural network (CNN) [318,319,365] trained to detect charge transition lines in a small section of the stability diagram. The training is performed in a supervised manner on a dataset [314] composed of static measurements made on similar silicon QD devices. Then, a closed-loop calibration procedure allows us to find the one-electron regime of a single QD by following an autonomous exploration strategy that leverages the CNN inference and uncertainty score. Exploiting the model’s uncertainty improves the robustness of the autotuning by reducing the risk of critical failures caused by potential misclassifications [297] compared to classical ML methods [69,136].

In this experiment, we successfully transfer the above ML-based charge autotuning method—developed using offline measurements—to real-time charge tuning on an experimental setup. The results confirm that the device-to-device variability and the resolution shift between the online and offline data do not negatively affect the line detection performance. We were also able to measure valuable information regarding the autotuning duration and identify the current measurement as the time bottleneck.

4.2.3 Methodology

The charge tuning process is typically guided by a stability diagram (two-dimensional current–voltage scan presented in Figure 4.3a), which is generated based on indirect QD measurements from a single-electron transistor (SET) for charge detection while sweeping the gate voltages. The transition lines (highlighted in green in Figure 4.3c) inform us of an electron movement between the reservoir and the QD (represented in Figure 4.1a,b). Given the knowledge that the QD is empty when no lines are visible at low gate voltages, it is possible to deduce the number of charges for a given position in the stability diagram (blue areas in Figure 4.3c). Although experts can perform manual tuning in small-scale experiments [361], this is too slow and labor-intensive for large-scale industrial applications utilizing multiple QDs. Automating this task is necessary but challenging due to the noise induced by the environment and the measurement electronics, the device-to-device variability, and the variety of existing hardware implementations.

The autotuning is approached as an exploration problem, where we start from a random unknown position in the voltage space, gather information about the surroundings by performing local measurements, and search for the targeted charge regime. The transition

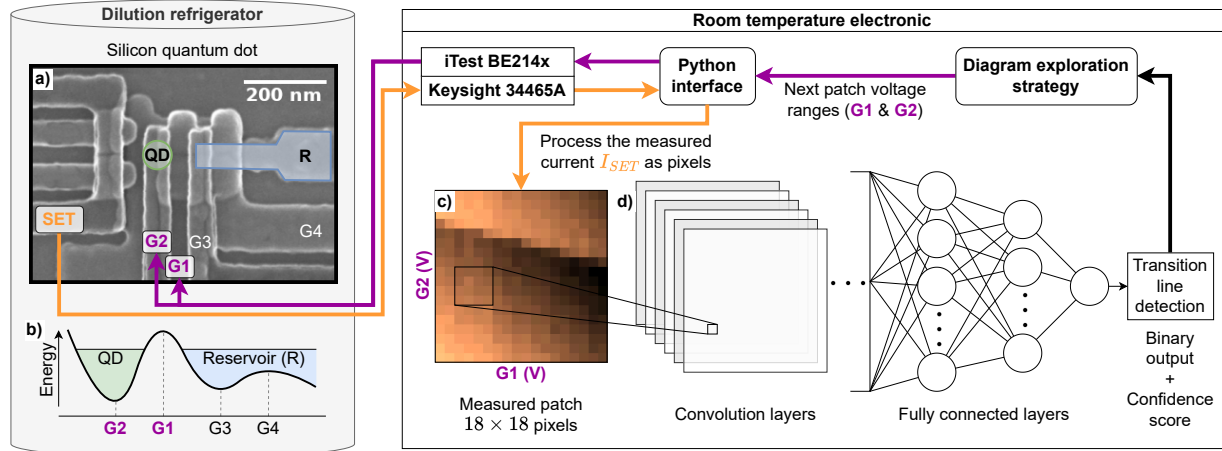


FIGURE 4.1 Schematic representation of the experimental setup. A silicon quantum dot (QD) device is cooled in a dilution refrigerator with a base temperature of 20 mK. The voltages at gates 3 (G3) and 4 (G4) are fixed, while a *iTest BE214x* controls the voltages at gates 1 (G1) and 2 (G2) through a Python interface. The scan coordinates are defined by an autonomous closed-loop procedure using a trained convolutional neural network (CNN) and an exploration strategy algorithm. **a)** scanning electron microscope (SEM) image of a silicon QD device with overlapping gates, similar to the one used during this experiment. The electrons flow from the reservoir (R) to the QD. See [298] for fabrication details.

b) Energy diagram representing the formation of a single QD in this device. **c)** A subsection of the voltage space (referred to as “patch”) was scanned by sweeping the voltages at gates G1 and G2 and measuring the current using the single-electron transistor (SET) connected to a *Keysight 34465A* multimeter. **d)** Trained CNN that processed the measured patch through two convolutional layers and two fully connected layers to infer the presence of a transition line as a binary output.

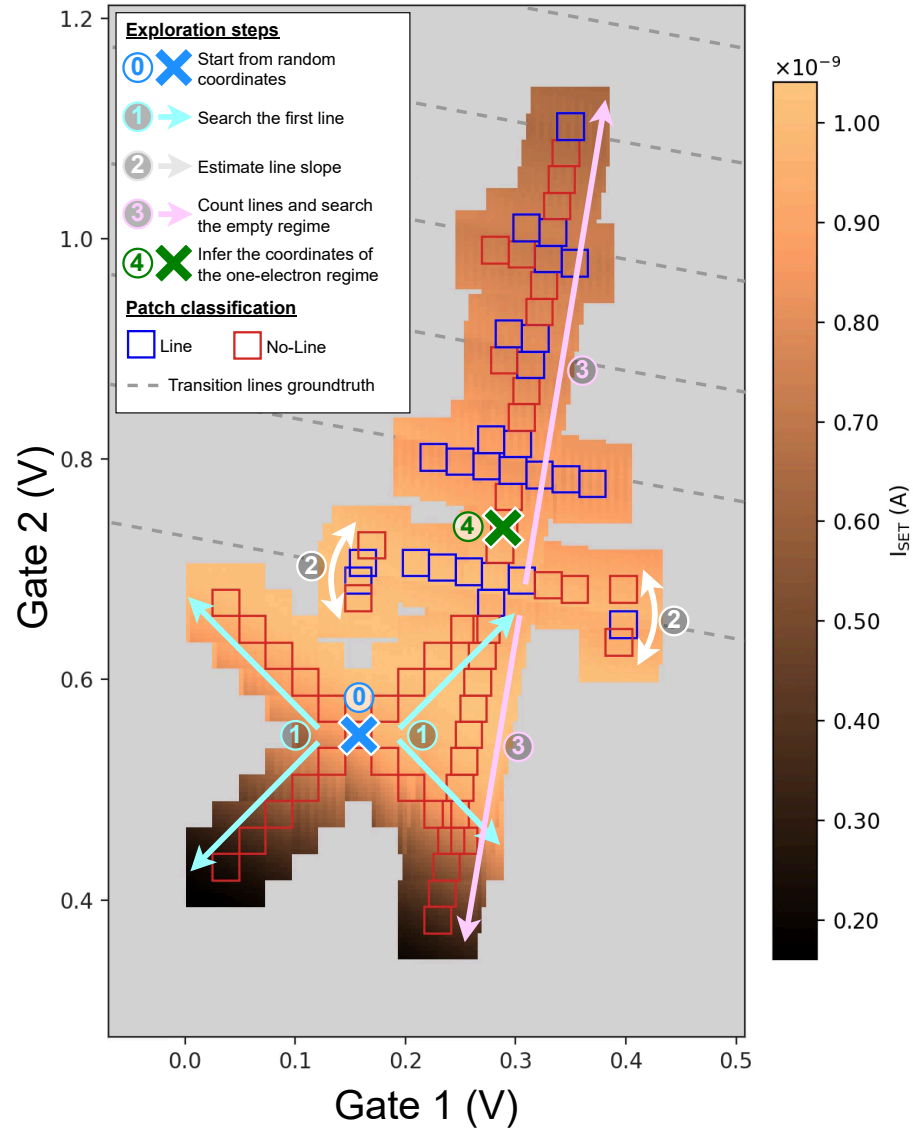


FIGURE 4.2 Example of successful online experimental autotuning. The arrows represent the direction of the exploration, and the gray area represents the unmeasured voltage space for this run. The blue and red squares represent the classification of the patch by the convolutional neural network (CNN) (“line” and “no-line”, respectively). The cyan arrows (1) represent the exploration in four directions to search for the first transition line. The white arrows (2) represent the slope estimation step, performed by scanning two sections of the first detected line. Finally, the pink arrows (3) perpendicular to the estimated line slope represent the empty regime search (*down*) and the line count (*up*) procedure. A complete scan of this diagram is given in Figure 4.3a. See [297] for more detail regarding this tuning method.

lines guide the exploration by providing valuable information regarding the number of electrons in the QD. One exploration step consists of scanning a subsection of the voltage space (referred to as “patch” and represented in Figure 4.1c), sending it to the input of a CNN-based line detection model and deciding the next area to explore based on an exploration strategy (an autotuning example is presented in Figure 4.2). The patch size is fixed to 18×18 data points as a tradeoff between the measurement time (a smaller area is faster to scan) and the line detection accuracy (a larger area simplifies the line detection). Due to the noisy nature of the measurements and the small size of the patch, classical signal processing and pattern detection methods [141–144] are not robust enough to reliably detect transition lines [132, 135, 136, 305]. We opted for a supervised classification approach using a deep neural network (DNN) [58], which is known to be the best-performing method for pattern detection in noisy images [139, 318, 319, 365, 366].

The line detection and exploration strategy are based on the autotuning method described in detail by [297]. Nine stability diagrams, measured during previous experiments on similar QD devices and manually annotated, are used to generate a training set of 33,429 patches. Each of these patches is categorized as “*line*” or “*no-line*” depending on whether an annotation of a transition line intersects its center. More information on the diagram annotation and patch labeling is available in Appendix B.1. A CNN (represented in Figure 4.1d) is then optimized using gradient backpropagation [367] to classify patches in the training set (refer to Appendix B.2 for details on the training methodology).

The trained CNN is transferred to a computer connected to a *iTest BE214x* that is capable of controlling the voltage applied to each gate of the QD device and measuring the SET current using a *Keysight 34465A* multimeter, as illustrated in Figure 4.1. The autotuning algorithm, implemented in Python¹, plays a central role at each step of the exploration by: (i) determining the next patch to measure based on the exploration strategy, (ii) transferring the voltage sweeping instructions to the multimeter, (iii) processing the measured current as a normalized image, and (iv) detecting a transition line by feeding the measured patch to the CNN input. To reduce the risk of tuning failures induced by potential patch misclassifications, each inference is associated with a confidence score that estimates the model’s uncertainty for a given input. This score is calculated using a simple distance-based heuristic [324, 325] (more information in Appendix B.3). When a patch is classified with a confidence score under the threshold, a verification procedure is automatically triggered to validate or refute the presence of a line in the surrounding area. This uncertainty-based exploration has been demonstrated to significantly improve the tuning

1. Python source code: github.com/3it-inpaqt/dot-calibration-v2

success rate in offline experiments [297]. It is therefore expected to improve the robustness of autotuning in the case of unexpected perturbations related to the experimental context.

A silicon QD device with overlapping gates [298] was installed and cooled down in a dilution refrigerator with a base temperature of 20 mK. Once the device was cooled, we ran 20 consecutive and independent autotuning procedures.

We did not set virtual gates, as the tuning algorithm was designed to adapt to the capacitive crosstalk. Each run started at a random location (blue crosses in Figure 4.3b) in a voltage range of $[-0.25 \text{ V}, 0.5 \text{ V}]$ for gate 1 (G1) (expected working range for the barrier gate between the QD and the reservoir) and $[0 \text{ V}, 2 \text{ V}]$ for gate 2 (G2) (expected range for a single QD formed during previous experiments using similar devices). The voltage space was then partially explored with iterative patch measurements within the starting voltage ranges, increasing by 0.1 V in every direction. It was necessary to define boundaries to maintain the hardware’s integrity and ensure that no time was lost exploring areas that were incompatible with a single-QD configuration. At the end of the experimental runs, we performed a complete scan of the explored area. The resulting stability diagram (shown in Figure 4.3a) was annotated by experts to identify the transition lines and the charge areas (shown in Figure 4.3c). By placing the final coordinates of the 20 runs in this figure, we were able to count the number of times we reached the targeted one-electron regime.

4.2.4 Results

Positioning the final voltage coordinates in a complete scan of the explored stability diagram (Figure 4.3a) allows us to evaluate which autotuning runs reach the targeted charge regime. Among the 20 experimental tunings performed, 19 (95 %) successfully located the one-electron regime (green crosses in Figure 4.3b), while only 1 (5 %) autotuning run failed (red cross in Figure 4.3b). An analysis of this failure revealed that it was not caused by patch misclassification but by a problem in the exploration logic related to the voltage boundaries. This issue has been fixed in the last version of the Python implementation. A visual animation of each run is available in a video ².

Each autotuning run took an average of 2 hours and 9 minutes (standard deviation: 46 min) and 110 steps (standard deviation: 38) to complete. For comparison, scanning the full stability diagram presented in Figure 4.3a took approximately 7 hours. Each step required 324 current measurements to obtain one patch (for 18×18 pixels), taking an average of 67 seconds using a *Keysight 34465A* multimeter, representing 96 % of the process duration. Thus, data transfer and processing (including the CNN inference time) represent only a

2. Online autotuning experiment video: youtu.be/zGIQWEZex0s

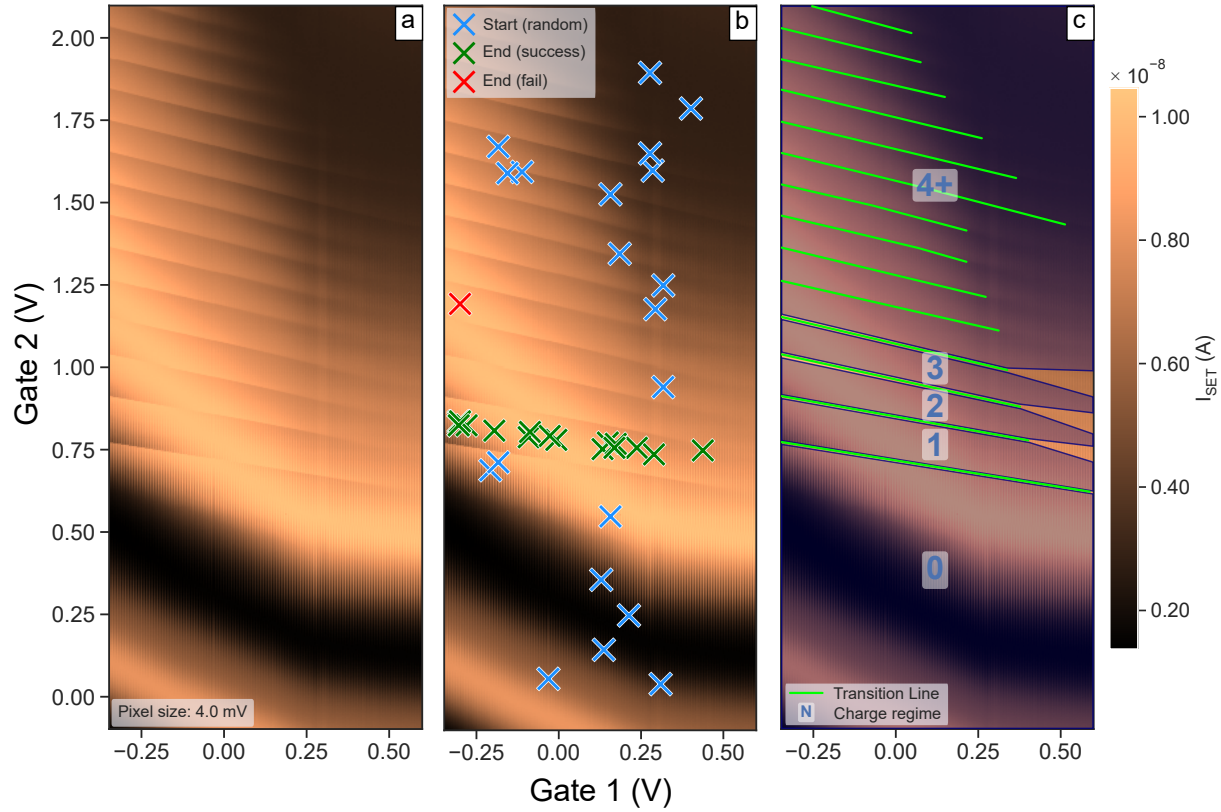


FIGURE 4.3 Complete scan of the explored stability diagram, processed after the experiment. **a)** Representation of a stability diagram as an image, where pixel values encode the current measured using the single-electron transistor (SET) as a function of the gate voltages applied to the quantum dot (QD)(see Figure 4.1). **b)** Start and end coordinates of the 20 online experimental autotuning runs. The blue crosses represent the random starting point of the runs. The green crosses represent the final coordinates of the successful autotuning runs (when the coordinates are inside the area annotated as a one-electron regime). The red cross represents the end coordinates of the run that did not reach the target regime. **c)** Same diagram with manual annotations of transition lines in green and charge regime areas in blue. The region with four or more charges is annotated as “4+”. The voltage areas not covered by a charge annotation (due to fading lines) are considered “*unknown charge regimes*”.

fraction of the tuning time. The high variability between run durations is attributable to the variable distance from the random starting point to the one-electron regime. The individual run statistics are available in Table B.3.

Prior offline benchmarks obtained by applying this autotuning method to similar devices led to a lower tuning success rate (78 %) [297] despite the comparable line detection accuracy of the CNNs. The higher success rate obtained during this online experiment can be explained by several factors: *(i)* Online tuning allows us to precisely select the patch resolution, while offline diagrams need to be interpolated to compensate for the non-homogeneous step size of the voltage sweeping between measurements (see comparison examples in Appendix B.5). This data processing is detrimental to the measurement quality and could explain some line detection failures during offline tests. *(ii)* The fixed voltage range covered by offline diagrams often artificially increases the tuning difficulty by allowing for the exploration of irrelevant areas (e.g., beyond the barrier threshold). *(iii)* Offline autotuning experiments were performed on data measured from multiple devices, covering an extensive range of physical defects and measurement noise. The unique device used in this online experiment appears to be of a good quality (e.g., low noise, no parasitic dots), providing optimal autotuning conditions.

4.2.5 Discussion

This online experiment demonstrates the feasibility of autonomous real-time tuning based on ML. It validates that the expected distributional shift between the offline training data and the online measurements is not detrimental to the procedure’s performance. On the contrary, qualitative observations suggest that the absence of preprocessing improved the robustness of the autotuning. We also confirmed that the line detection model and the exploration strategy were versatile enough to successfully tune a QD device that was not in the training set.

The relatively slow tuning time (>2 hours) does not satisfy the requirement for practical tuning of a large QD array. However, the SET measurement time, identified as the main bottleneck, could be drastically improved by optimizing the sensing method and hardware. For example, one could speed up the measurement time by a factor of 10 by implementing the measurement sequence on a dedicated processor [368] (e.g., a field-programmable gate array (FPGA)). This approach would cut the communication time between the instruments and the control computer, allowing for faster sweep rates. The radio-frequency reflectometry technique [126, 233] could also be used to speed up the charge sensing by measuring the impedance of the SET sensor at a fixed frequency, enabling single-shot readouts with only several microseconds of integration time [369]. This time can even be

reduced to 400 ns with a Josephson parametric amplifier that also provides a high signal-to-noise ratio [368]. However, the whole measurement pipeline should be optimized to take advantage of this hardware acceleration, including the data transfer, latency, and processing time. On the other hand, it is possible to limit the effect of this measurement bottleneck with a software approach; for example, by optimizing the meta-parameters (Tables B.1 and B.2) in a way that reduces the number of measured points (e.g., smaller patches and a larger voltage distance between pixels) and reducing the number of steps (e.g., higher confidence threshold and smaller voltage boundaries). Finally, improving the fabrication processes of the QD chip is likely to reduce device-to-device variability, which will directly improve the autotuning robustness and reduce the number of steps by narrowing the voltage ranges to explore.

Further optimization can be achieved by moving the control electronics (represented in the right panel of Figure 4.1) to the cryogenic environment inside the dilution refrigerator. Bringing the control electronics closer to the tuned device, either by co-integrating them with the QD chip [300, 354] or with additional nearby electronics, can reduce the parasitic capacitance added to the current measurements and allow for the control of large-scale arrays of QDs by circumventing the wiring bottleneck [167] between the QD devices and the control electronics. However, this *in situ* autotuning scheme requires the development of cryo-compatible and low-power custom electronics [170, 297, 342–344].

4.2.6 Author contributions

All authors contributed to this article and approved of the submitted version.

Victor Yon: methodology, software implementation, experiments, results analysis and visualization, writing—original draft preparation.

Bastien Galaup: software implementation, experiments, manuscript review.

Claude Rohrbacher, Joffrey Rivard, Alexis Morel, and Dominic Leclerc: initialize and configure the experimental setup, methodology, manuscript review.

Clement Godfrin, Ruoyu Li, Stefan Kubicek, and Kristiaan De Greve: manufactured the devices, manuscript review.

Roger Melko: supervision, manuscript review.

Eva Dupont-Ferrier, Yann Beilliard, and Dominique Drouin: methodology, supervision, funding acquisition, manuscript review, editing.

4.2.7 Code and data availability

The Python source code used to aggregate and build the dataset is publicly accessible on *GitHub*: github.com/3it-inpaqt/qdsd-dataset.

The Python source code used to run all the experiments presented in this article is publicly accessible on *GitHub*: github.com/3it-inpaqt/dot-calibration-v2.

The raw and processed stability diagram measurements used to train the line detection model for this experiment are publicly available for download from [314]. The subset used to train the models in this paper is referred to as silicon overlapping gate (Si-OG-QD).

All trained models, experimental measurements, and results presented in this article are publicly available for download from [370].

4.3 Bilan de l'article

Cette validation expérimentale confirme que la méthode est applicable dans un scénario réaliste de calibration automatique. Associé aux nombreuses simulations hors lignes présentées dans l'étude précédente (chapitre 3), l'ensemble de ces résultats nous permet d'être confiant dans la fiabilité de cette solution et pose des fondations solides pour la mise en place de systèmes de contrôle à plus grande échelle.

La solution proposée permet dès aujourd'hui d'accélérer la mise en place d'expériences en laboratoire. Mais plusieurs défis restent à relever avant de déployer cette méthode pour calibrer de façon autonome plusieurs milliers de boîtes quantiques. Les étapes nécessaires pour atteindre cet objectif sont les suivantes : *(i)* réduire le nombre de paramètres du modèle afin qu'il soit facilement transférable sur un crossbar de memristors ; *(ii)* évaluer l'impact des non-idéalités des memristors sur la précision du modèle ; *(iii)* concevoir un circuit compatible avec des températures cryogéniques ; et *(iv)* intégrer ce circuit dans un système de contrôle en boucle fermée proche des boîtes quantiques, au sein d'un cryostat. Toutes ces étapes requièrent des études supplémentaires et devraient grandement bénéficier des développements futurs dans le domaine des réseaux de neurones à base de memristors.

CHAPITRE 5

Correction d'erreur quantique à base de memristors

5.1 Avant-propos

Titre français :

Décodeur neuronal cryogénique à base de memristors pour la correction d'erreur quantique

Auteurs et affiliations :

Victor Yon^{*,1,2,3,4}, Frédéric Marcotte^{*,2,3,4,5}, Pierre-Antoine Mouny^{1,2,3,4},
Gebremedhin A. Dagnew⁵, Bohdan Kulchytsky⁵, Sophie Rochette¹,
Yann Beilliard^{2,3,4,5}, Dominique Drouin^{1,2,3,4} et Pooya Ronagh^{1,6,7,8}

* Ces auteurs ont contribué de manière égale à ce travail.

¹ Irréversible Inc., Sherbrooke, QC, Canada, J1K 1B7

² Institut Interdisciplinaire d'Innovation Technologique (3IT), Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 0A5

³ Laboratoire Nanotechnologies Nanosystèmes (LN2) — CNRS 3463, Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 0A5

⁴ Institut quantique (IQ), Université de Sherbrooke, Sherbrooke, QC, Canada, J1K 2R1

⁵ 1QB Information Technologies (1QBit), Vancouver, British Columbia, Canada, V6E 4B1

⁶ Institute for Quantum Computing, University of Waterloo, Waterloo, ON, Canada, N2L 3G1

⁷ Department of Physics and Astronomy, University of Waterloo, Waterloo, ON, Canada, N2L 3G1

⁸ Perimeter Institute for Theoretical Physics, Waterloo, ON, Canada, N2L 2Y5

État de publication : Soumis (23 octobre 2024)

Journal : Quantum Science and Technology

Référence : [343]

5.1.1 Objectif de l'article

Les deux premières études (chapitres 3 et 4) ont permis de développer et évaluer une méthode de calibration automatique de boîtes quantiques en utilisant un réseau de neurones artificiels (RNA). L'étape suivante consiste à intégrer ce RNA sur un circuit à base de memristors compatible avec un environnement cryogénique. Les recherches préalablement effectuées dans le groupe ont démontré le bon fonctionnement de memristor à basse température [171, 342, 358, 371–374]. Mais l'impact des non-idéalités des memristors (section 2.3.2) sur la précision des calculs reste à être évalué. Dans le cas où les imperfections matérielles réduiraient significativement les performances du RNA, il serait nécessaire de mettre en place des méthodes de mitigations durant l'entraînement.

Dans ce chapitre, nous évaluons cette problématique dans le contexte de la correction d'erreur quantique (section 2.1.4.2). Cette application possède de nombreux points communs avec la calibration automatique des boîtes quantiques, notamment la nécessité d'opérer au plus proche des qubits, à température cryogénique, avec un RNA. En plus de la contrainte d'efficacité énergétique imposée par cet environnement, la correction d'erreur quantique requiert de décoder très rapidement les syndromes (de l'ordre de la centaine de nanosecondes par cycle [375]). Par rapport à la calibration automatique, cette application a l'avantage d'être implémentable sur un RNA plus petit, ce qui ouvre la porte à des premières démonstrations plus rapides et moins coûteuses.

Cette étude s'est inscrite dans une collaboration avec les entreprises 1Qbit¹ et Irréversible², qui ont fourni l'expertise en correction d'erreur quantique et les données nécessaires à la mise en place de l'expérience.

5.1.2 Résumé en français

Les décodeurs neuronaux reposent sur des RNA afin de classifier les syndromes extraits des codes de correction d'erreur quantique, permettant ainsi de déterminer les opérateurs de récupération appropriés pour maintenir l'intégrité de l'information durant les calculs. Ce type de décodeur se démarque par sa précision de décodage élevée, issue de sa capacité à s'adapter au bruit spécifique du matériel. Dans le cas d'une intégration au sein d'un schéma de décodage multi-étages, il est nécessaire de minimiser le temps de décodage pour correspondre à la fréquence des mesures de stabilisation, et d'établir une co-intégration étroite avec les processeurs quantiques. La réalisation de décodeurs neuronaux cryogéniques peut non seulement améliorer les performances des décodeurs de niveaux supérieurs, mais aussi minimiser les délais de communication et éviter les goulots d'étranglement du câblage.

1. 1Qbit : 1qbit.com

2. Irréversible : irreversible.tech

Dans cette étude, nous proposons un décodeur neuronal basé sur des crossbar de memristors, utilisés à la fois pour stocker les paramètres du RNA et effectuer des multiplications matrice-vecteur dans le domaine analogique. À travers des simulations soutenues par des mesures expérimentales, nous étudions l'impact des non-idéalités des memristors à base de TiO_x sur la fidélité du décodage. Nous développons des méthodes de ré-entraînement adaptées aux imperfections matérielles pour atténuer la perte de fidélité, restaurant les performances initiales du décodeur pour le code de surface de distance 3. Ce travail ouvre la voie à la conception de circuits cryogéniques à base de memristors, rapides et à faible consommation énergétique, pour une correction d'erreur quantique intégrée dans le cryostat et tolérante au bruit.

5.2 A cryogenic memristive neural decoder for fault-tolerant quantum error correction

5.2.1 Abstract

Neural decoders for quantum error correction (QEC) rely on neural networks to classify syndromes extracted from error correction codes and find appropriate recovery operators to protect logical information against errors. Its ability to adapt to hardware noise and long-term drifts make neural decoders a promising candidate for inclusion in a fault-tolerant quantum architecture. However, given their limited scalability, it is prudent that small-scale (local) neural decoders are treated as first stages of multi-stage decoding schemes for fault-tolerant quantum computers with millions of qubits. In this case, minimizing the decoding time to match the stabilization measurements frequency and a tight co-integration with the QPUs is highly desired. Cryogenic realizations of neural decoders can not only improve the performance of higher stage decoders, but they can minimize communication delays, and alleviate wiring bottlenecks. In this work, we design and analyze a neural decoder based on an in-memory computation (IMC) architecture, where crossbar arrays of resistive memory devices are employed to both store the synaptic weights of the neural decoder and perform analog matrix–vector multiplication. In simulations supported by experimental measurements, we investigate the impact of TiO_x -based memristive devices' non-idealities on decoding fidelity. We develop hardware-aware re-training methods to mitigate the fidelity loss, restoring the ideal decoder's pseudo-threshold for the distance-3 surface code. This work provides a pathway to scalable, fast, and low-power cryogenic IMC hardware for integrated fault-tolerant QEC.

5.2.2 Introduction

Fault-tolerant quantum computation (FTQC) holds the promise of solving extremely difficult problems with efficient time and space complexity [48, 121]. However, this efficiency is at the cost of resource-intensive classical procedures that are required for protecting the logical quantum state of the quantum processor against noise [376]. This includes, (a) physically protecting the quantum state by cooling and isolating the quantum processor, and (b) performing quantum control and QEC protocols. The former is the reason many quantum technologies operate at cryogenic temperatures. But, the latter requires classical computing circuits that have historically been designed and manufactured to operate at room temperature and their high heat dissipations hinders their use inside cryostats.

The transfer and processing of the data generated by QEC protocols present numerous challenges: (i) With the anticipated tens of millions of physical qubits needed for FTQC, the repeated microsecond-long error correction cycles would produce terabytes of syndrome data per second to be processed for days- or even months-long computations [121, 377, 378]. Transferring this amount of data from the cryogenic environment to classical electronics located at room temperature quickly leads to wiring bottlenecks [167]. (ii) Transferring small signals from measurement at the quantum processor level through multiple temperature stages to be processed by room temperature instrumentation requires careful amplification to mitigate the increased thermal noise at each stage. Finally, (iii), an FTQC implementation of non-Clifford gates requires active error correction, that is, real-time processing of the syndrome data by a decoder module, and application of the resulting recovery operations at a time scale comparable to the coherence time of the quantum system [75, 76]. Classical processors capable of operating accurately at cryogenic temperatures can mitigate these challenges. This requires computing with extremely low heat dissipation, as the cooling power of dilution refrigerators is limited (1 to 2 W at the 4 K stage).

Since single-flux quantum (SFQ) digital electronics are natively compatible with cryogenic temperatures, recent works [379–381] propose SFQ-based architectures for decoding using various heuristic approximations to graph-matching algorithms [160–162], while [381] proposes a binarized SFQ-based neural decoders. Those systems are highly complex and consume a lot of energy, casting doubts on the ability to fabricate them reliably and on their potential benefits. The main challenge in building scalable cryogenic fault-tolerant decoders is the need for high-density cryogenic memory blocks for storing at least two types of information:

- (a) **The program data:** Even the simplest decoding heuristics require complex computations that rely on large look-up tables to store the logic and execute the algorithmic steps of the decoder [382, 383].
- (b) **The input data:** Fault-tolerant error correction requires processing multiple rounds of imperfect stabilizer measurements (typically as a graph-based algorithm on a 3-dimensional lattice) at once. Moreover, current decoding logic depends on historical progression of the algorithm [384–386]. Therefore, some type of memory must store and recall a historical state of computation.

In this paper, we introduce a memristive neural decoder (MND) which combines the advantages of recurrent neural network (RNN) and the low-power consumption of resistive

memory arrays to eliminate the issues related to storage and recall for both types of memory blocks. In neural decoders, the program data (item (i) above) is the weights and biases of an artificial neural network (NN) [165]. This information is stored in the conductance states of the resistive memory devices which are physically tuned, thereby providing a realization of IMC [387]. In addition, an RNN comprises an *internal state* which is a real-valued vector (or a higher-dimensional tensor) that encodes a latent representation of its prior inference (the h wires in Figure 5.1). The RNN will therefore not need to receive the syndrome data of multiple rounds of error correction at once, and instead stream their processing one at a time as they get generated, while updating its internal state in each iteration (from h_i to h_{i+1} in Figure 5.1).

More specifically, we investigate TiO_x -based analog resistive memory devices using TiN electrodes [388]. A crossbar arrangement of these memory cells enables the matrix-vector multiplication (MVM) operations at the heart of NN algorithms to be performed natively by relying on Ohm's law and Kirchoff's circuit law, thus removing the time- and energy-intensive process of moving data from memory to processing units [389]. Such non-volatile memristive devices have small footprints [81–83, 390], and benefit from complementary metal-oxide-semiconductor (CMOS) compatible fabrication processes [388], data retention time of up to 10 years [391], and analog switching dynamics [392], making them promising candidates for efficient MVM in terms of processing time, energy dissipation, and scalability [258, 393, 394]. Furthermore, they operate very well at cryogenic temperatures [342, 371, 372, 374], are robust to temperature variations, and can be calibrated to adjust to long-term drifts in the input signal or environmental noise.

Moreover, the MND processes neural activations (the feature tensors propagating forward along the network) as inherently analog signals which are never converted to digital data in binary representation. This allows for a much better footprints compared to the alternative CMOS-based digital (binary) memory technologies such as static RAM (SRAM) and resistive random access memory which were previously considered for low-power digital neural decoding [395, 396]. We note that the mixed-signal nature of MND makes it attractive for soft decoding [397] by directly interfacing the readout resonator and eliminating long and error-prone amplification and measurement of syndrome bits. Since it is difficult to scale neural decoders up for large surface code patches, we envision that such a tightly integrated decoder-QPU hybrid system may be augmented with further (more global) decoding stages such as large scale Union-Find or collision clustering decoders [163, 398].

Our early MND prototype is fabricated in CMOS 180 nm and demonstrates cryogenic compatibility down to 35 K [399]. It exhibits a decoder delay of 1 μs per stabilizer mea-

surement round, which is similar to Collision Clustering decoders able to perform real-time QEC by demonstrating a 2×2 stability experiment [400]. Our decoder delay is currently bottlenecked by the pulse width used at cryogenic temperature and could be reduced to 200 ns with minimal CMOS design optimization. It is expected that our processing delay will not increase significantly with larger QEC codes as analog RNN allows for parallel computation. Additionally, the MND prototype consumes 3.4 mW at 35 K to perform the RNN computation and 10.9 mW for the auxiliary electronics used to interface with the memristors. It is anticipated that the distance-3 MND will consume ~ 50 mW for the RNN computation and roughly 120 mW for the memristor-CMOS interfacing electronics in CMOS 180 nm. The power consumption can be significantly reduced by using smaller CMOS nodes, e.g., 22 nm FDSOI exhibits power consumption up to 70 times smaller than CMOS 180 nm [401], yielding to a 2.5 mW power consumption per distance-3 MND.

However, non-idealities [263, 269, 402–404] of memristor arrays are known to deteriorate NNs’ performance, our study is focused on the impact of key TiO_x -based devices’ non-idealities on the accuracy of MNDs. Stuck-at fault devices have the greatest impact on the decoder’s performance. Therefore, we propose and implement two techniques for mitigating their detrimental effect. The hardware-aware (HWA) method consists of improving NNs’ robustness by re-training it while taking into account typical hardware constraints. In contrast, the device-specific (DS) re-training method uses the exact location of stuck-at fault devices to specifically adapt to the imperfections of a given device. We show that the latter approach allows for high-fidelity decoding of the distance-3 surface code.

Our paper is structured as follows. First, we provide a formal definition of the decoding problem using a distance-3 surface code, we describe the NN architecture of our decoder, and present the corresponding memristive decoder circuit. We then experimentally characterize key non-idealities of the TiO_x -based resistive memory devices (e.g., programming variability, stuck-at fault rate, retention time). We then assess the impact of these non-idealities in simulations, and demonstrate that the HWA and DS re-training methods recover most of the ideal neural decoder’s fidelity. Finally, we discuss the engineering advantages of leveraging analog IMC hardware for QEC decoding, and provide some future perspectives.

5.2.3 Problem statement

5.2.3.1 Decoding the surface code

In Figure 5.1, the process of active error correction via stabilizer measurements and decoding is schematized. Here, successful error correction amounts to matching the decoder-proposed recovery operator (r_X, r_Z) with the logical error (ϵ_X, ϵ_Z) afflicting the logical

state $|\psi\rangle_L$ encoded by the error correcting code. The performance of the decoder is dependent on its speed due to idling errors associated with the decoding delay. Therefore, minimizing the decoding time as much as possible is desirable.

We consider the distance-3 rotated surface code, which can be realized with 17 physical qubits [405–407]. As shown in Figure 5.2(a), the qubits are arranged on a square lattice, comprising data qubits (circles filled in white) and ancilla, or syndrome, qubits (circles filled in black). Data and syndrome qubits differ only in terms of their function within the code, and they can be implemented using physical systems such as superconducting circuits, trapped ions, quantum dots (QDs), or topological qubits. Each qubit interacts with its neighbours in a specified manner. The order and mechanism of interaction is determined by the stabilizers being measured, as shown in Figure 5.2(b) and (c) for the stabilizers $X_e X_f X_g X_h$ and $Z_a Z_b Z_c Z_d$, respectively.

5.2.3.2 Neural decoder architecture

We consider an RNN decoder module similar to the ones described in Refs. [165] and [166]. It may be difficult to train neural decoders for arbitrarily large topological codes; however, we note that the largest topological patch that must be actively decoded during FTQC depends on the largest entangling gate between a logical magic state and other logical qubits [408]. Additionally, neural decoders are great candidates for inclusion in distributed [409] and hierarchical decoding schemes [410] in order to create larger-scale decoding systems. We restrict our benchmarking to the X syndromes because the performance would be the same for the Z syndromes. Appendix C.1 describes the model used to simulate the quantum circuit and obtain the syndromes datasets and labels for training the RNN decoder. The generated syndromes dataset [411] is used to train and test the RNN. Our RNN architecture is illustrated in Figure 5.3(b). It consists of a fully connected recurrent layer and a fully connected output evaluation layer. There are 4 input nodes for receiving the syndrome data of an error correction round and 32 internal state nodes to store the hidden state of the previous round via recurrent connections. Assuming similar error rates for physical gates and measurements, the distance-3 code requires three QEC cycles to be fault tolerant [165, 166, 412]. The outputs of the recurrent layer, after application of the activation function, a rectified linear unit (ReLU) function in this case, are fed back to the input of the internal state nodes when the next error correction round's syndrome data (s^X) arrive to the input nodes, and the process is repeated for a total of at least 3 cycles, as illustrated in Figure 5.3(a). In order to evaluate the logical error rate of the scheme, we measure the data qubits in the final cycle (see Appendix C.1).

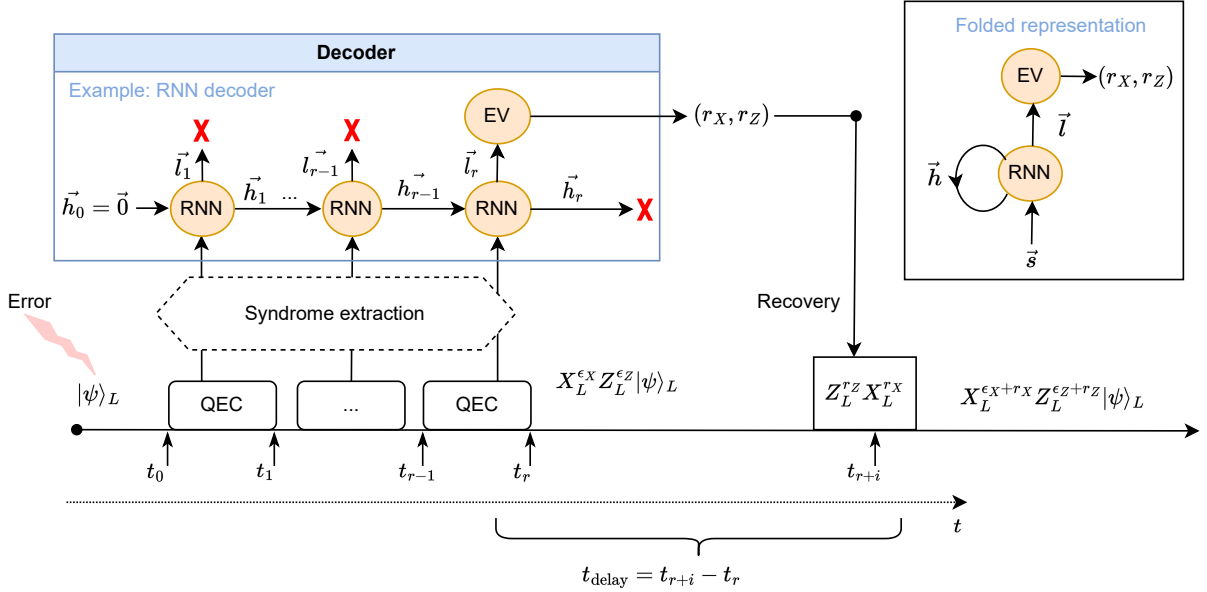


FIGURE 5.1 Decoding process. In quantum error correction (QEC), syndrome extraction rounds are performed cyclically to protect an encoded quantum state $|\psi\rangle_L$ from logical errors. Typically, the X and Z stabilizers' syndromes are processed separately but simultaneously by two independent decoding modules. The syndrome measurement outcomes are fed to the decoder modules, which analyze them to produce a decision output (r_X, r_Z) . This decision output is used to construct the recovery operation $Z_L^{r_Z} X_L^{r_X}$. Between the end of the last QEC round at t_r and the application of the recovery operation at t_{r+i} , idling errors can affect the qubits, and they will not be taken into account by the decoding algorithm. The encoded state just prior to the application of recovery operations is therefore $X_L^{\epsilon_X} Z_L^{\epsilon_Z} |\psi\rangle_L$, where $\epsilon_{x,z}$ represents the cumulative effect of the errors that occur during the QEC rounds and the idling time t_{delay} . Following the application of the recovery operation (here assumed to be ideal for simplicity), the final encoded logical quantum state is $X_L^{\epsilon_X+r_X} Z_L^{\epsilon_Z+r_Z} |\psi\rangle_L$. Here, the example decoder is based on a recurrent neural network (RNN). Each new syndrome is used as the input to the RNN and the hidden state $\vec{h}_i$ is passed to the subsequent recurrence. At the end of the QEC process, the output state $\vec{l}_r$ is passed to an evaluation module (EV) (the fully connected output layer in the case of an RNN) to provide the final binary output of the decoder. On the right-hand side of the figure, the standard folded representation of this RNN is illustrated. A more detailed schematic architecture of this RNN decoder is depicted in Figure 5.3.

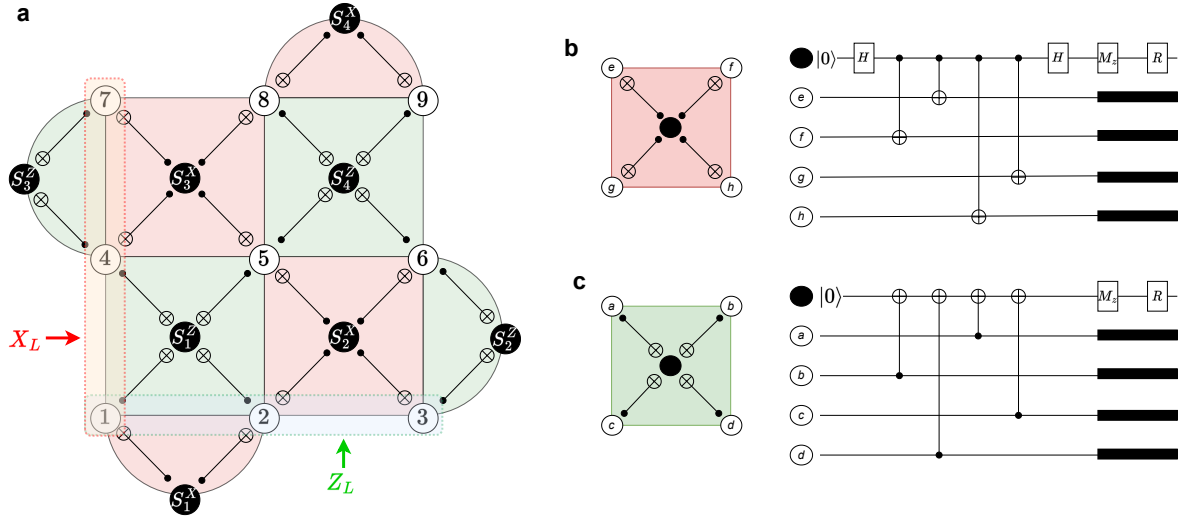


FIGURE 5.2 Surface code and stabilizer measurements. **a)** Distance-3 rotated surface code, also called surface-17. Data qubits 1 to 9 are identified by circles filled in white, and syndrome qubits are identified by circles filled in black. Each syndrome qubit resides on a “plaquette” corresponding to a specific stabilizer measurement S . The sequence of operations realizing an X stabilizer (pink plaquette) and a Z stabilizer (green plaquette) are shown in (b) and (c). In this example, a logical operator X_L (Z_L) can be realized by applying a chain of physical X (Z) operators on data qubits between the top (left) and bottom (right) edge of the grid. **b)** Sequence of circuit operations between a syndrome qubit (circle filled in black, the top qubit in the circuit) and its neighbouring data qubits, e, f, g , and h , realizing the stabilizer measurement $X_e X_f X_g X_h$. It consists of syndrome qubit initialization, Hadamard gates (H), CNOT gates, a projective measurement along the z -axis (M_Z), and a reset (R) to the $|0\rangle$ state, regardless of the measured outcome. **c)** Sequence of circuit operations between a syndrome qubit and its neighbouring data qubits, a, b, c , and d , realizing the stabilizer measurement $Z_a Z_b Z_c Z_d$. It consists of syndrome qubit initialization, CNOT gates, a projective measurement along the z -axis (M_Z), and a reset (R) to the $|0\rangle$ state, regardless of the measured outcome. The black rectangles represent idling of the data qubits, which is important for simulation of the circuits. More details on the exact procedure used for the simulation of these parity-check circuits is provided in Appendix C.1.

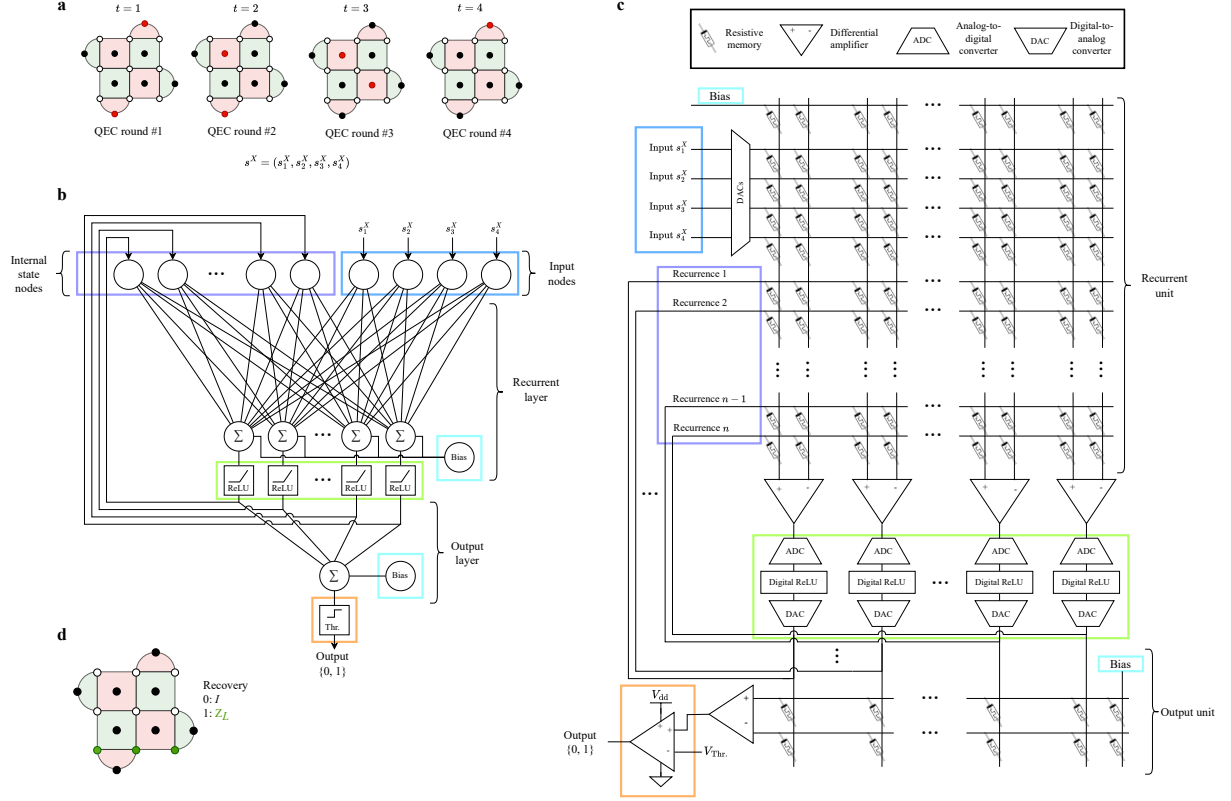


FIGURE 5.3 Recurrent neural network (RNN) decoder architecture and corresponding memristive decoder circuit. Quantum error correction (QEC) is handled as a sequential classification problem. **a)** Error correction rounds performed by measuring stabilizer operators on the lattice of physical qubits. Ancilla qubits (the circles filled in red or black representing an error or no error being detected, respectively) measure the *bit-flip* X (*phase-flip* Z) stabilizers $\hat{S}_{\{1-4\}}^X$ ($\hat{S}_{\{1-4\}}^Z$). Measured eigenvalues of ancillas are mapped to classical bits to build the syndromes s^X and s^Z , which serve as inputs to the neural decoder. Here, we consider only the s^X syndromes, obtained at times $t = 1, 2, 3, 4$. **b)** RNN decoder architecture for the distance-3 surface code. It comprises a recurrent layer and an output layer. Bias is also applied to both layers. The syndrome at $t = 1$ is the initial input along with the initialized hidden state h_0 . The output of the recurrent layer is routed back to be used as new input to the internal state nodes, as a new syndrome ($t = 2$) is provided to the input nodes. This is repeated until the last QEC round at time $t = n$ (here, $n = 4$), when the output of the recurrent layer is forwarded to the output layer, where the value of the single output neuron is compared to a threshold value to produce a binary classification. **c)** Memristive decoder circuit. It is composed of two arrays of memristors. The first is applied recursively (recurrent array) until $t = n$, and the second acts as the classifier (output array). Input syndromes are converted from the digital domain to the analog domain by digital-to-analog converters so that the memristive crossbar can perform the analog matrix-vector multiplication (MVM). The memristor conductances are programmed according to the digitally trained weights of the equivalent RNN decoder. Differential amplifiers subtract the outputs from each pair of memristor columns to obtain the resulting node's value. This signal is forwarded to the activation function array (green rectangle), where an analog-to-digital conversion is performed to apply a digital rectified linear unit (ReLU) activation function. The signal is converted back to the analog domain before being routed back to the recurrence input ports or sent to the output array. Finally, the value of the output array is passed through a comparator to produce a binary classification. An additional memristor row in each array allows the application of a bias signal. **d)** Recovery operation. To conclude the QEC process, a recovery operation is applied to the data qubits according to the classification provided by the decoder. The identity is applied in the case where no logical error has been detected (output 0), and the Z_L operator is applied (circles filled in green) if a logical error has been detected (output 1).

After the fourth round has been provided to the input nodes, the output of the recurrent layer is forwarded to the output layer and passed through a threshold function. Subsequently, the neural decoder outputs a single binary result, 0 or 1, indicating whether a logical error has occurred at the end of the QEC rounds (see Figure 5.3(d)).

5.2.3.3 Memristive neural decoder

We present a memristive electronic circuit to implement the neural decoder architecture discussed above, where the parameters of the RNN are stored in crossbar arrays of TiO_x resistive memory. The MND architecture is shown in Figure 5.3(c). It comprises two distinct memristor arrays: the recurrent array, which maps the weights corresponding to the recurrent layer, and the output array, which maps the weights corresponding to the output layer [413], in accordance with the architecture shown in Figure 5.3(b). Input ports receive data from syndrome measurements in the form of voltage pulses. Between the two crossbars of memristive devices, we perform an analog-to-digital conversion to apply a digital ReLU activation function. The output is then converted back into an analog signal in the form of a voltage pulse. Performing the activation function in the digital domain simplifies the circuit design by removing concerns about analog signal deterioration, as it offers full control over the shape of the signal sent into the second layer. The range and resolution of analog-to-digital converter (ADC) and digital-to-analog converter (DAC) is discussed in Appendix C.2. Finally, the output port provides the binary classification result of the decoder, which determines the recovery operation to be applied to the surface code.

The memristor crossbar arrays needed to perform analog MVM consist of two TiN electrodes separated by a TiO_x -based switching layer [388]. Following an initial non-reversible electroforming process, a conductive filament containing oxygen vacancies is created within the oxide layer [414], which can be subsequently at least partially dissolved and re-established through voltage pulses applied on the electrodes, leading to programmable, non-volatile conductance states for the memristive device [415].

It is therefore possible to map the weight matrix of an NN layer into the conductance states $G_{jk}^\pm$ of the memristors inside a crossbar array. To implement both positive and negative weights, a differential pair of memristive devices is used (see Figure 5.3(b)). The weight-to-conductance mapping procedure is given by

$$G_{jk}^\pm = |w_{jk}| \frac{G_{\text{HCS}} - G_{\text{LCS}}}{w_{\text{max}}} + G_{\text{LCS}}, \quad (5.1)$$

where $w_{\max}$ is the absolute maximum weight of a given layer, and G_{HCS} and G_{LCS} are the highest conductance states (HCSs) and lowest conductance states (LCSs), respectively. In other words, they are the maximum and minimum values that can be programmed on TiO_x -based memristive devices. If $w_{jk} > 0$, G_{jk}^+ is programmed with respect to Equation (5.1) and G_{jk}^- is set to G_{LCS} . If $w_{jk} < 0$, G_{jk}^- is programmed with respect to Equation (5.1) and G_{jk}^+ is set to G_{LCS} . After the RNN training has completed, the weights can be mapped to conductance states.

A crossbar configuration allows memristive devices to realize MVM natively, by relying on Ohm’s law and Kirchhoff’s current law. In Figure 5.3(c), the current output of each column in the crossbar array is the sum of the input rows’ voltages multiplied by the effective conductance values of the differential pairs of memristors. From the circuit laws, we have

$$i_k = \sum_j (G_{jk}^+ - G_{jk}^-) v_j, \quad (5.2)$$

where i and v denote output current and input voltage, respectively. For each differential amplifier, the symbols “+” and “−” set in superscript form denote the polarity of the pins wired to the memristors. From Equation (5.2), each column implements naturally the multiply–accumulate (MAC) operation [416].

5.2.4 Electrical characterization

In this section, we describe and experimentally characterize various hardware non-idealities of TiO_x -based resistive memory devices. Due to the variability of the fabrication process and switching mechanisms, these devices exhibit multiple non-idealities [263, 268, 269, 417], such as read variability, programming variability, and stuck-at fault malfunction (that is, the memory devices become stuck in either HCS or LCS after electroforming or shortly after a conductance programming attempt [418]). These non-idealities are expected to decrease the fidelity of an MND. Other non-idealities such as random telegraphic noise and $1/f$ noise are also commonly reported for oxide-based devices [269]; however, we do not investigate their impact, as it is expected to be insignificant for high-speed MNDs operating with input voltage pulses in the nanosecond range. Regarding conductance state retention, Supplementary Information Figure C.3 shows no noticeable change in the conductance state after 8 hours, confirming the stability of the memory state of these devices, acceptable for target applications. Furthermore, we have recently demonstrated data retention at 4.2 K for over 15 minutes [374], suggesting that TiO_x -based resistive memory devices are good candidates for a cryogenic MND.

Programming variability, also called cycle-to-cycle variability, is responsible for the inaccurate mapping of trained weights to the conductance states of memristors. Figure 5.4(a) shows the programming procedure of 11 conductance states on our fabricated memristors using a closed-loop read–write–verify algorithm [415] (see Appendix C.3). It can be seen that the target values given by Equation (5.1) cannot be reached exactly due to the stochastic nature of the resistive switching process. Therefore, a programming variability model that accounts for cycle-to-cycle variability and device-to-device variability has been obtained from experimental characterizations of TiO_x -based resistive memory devices [374]. The characterization process is detailed in Appendix C.3. In this model, the actual programmed conductance values are expressed as

$$G_{jk}^{\pm} \leftarrow G_{jk}^{\pm} + \mathcal{N}(0, \sigma_{\text{prog}}(G_{jk}^{\pm})), \quad (5.3)$$

where $\sigma_{\text{prog}}(G_{jk}^{\pm})$ follows the polynomial fit of Figure 5.4(b). The median value of the relative variations, that is, $\sigma_{\text{prog}}(G_{jk}^{\pm})/G_{jk}^{\pm}$, is 0.8 % for TiO_x -based resistive memory devices in the conductance range of [60, 200] μS .

The device yield of a chip is usually slightly under 100 % due to variations and imperfections in the nanofabrication process. This translates to a non-zero probability of stuck-at fault devices. In this case, the device cannot be programmed to the desired conductance for mapping a given weight of the NN. Recent work on passive crossbars of memristors has shown a probability of stuck-at fault devices on the order of ~ 1 % [419–421]. In the case of our TiO_x -based resistive memory devices, this probability can reach ~ 10 % and the devices are usually stuck in their HCS.

5.2.5 Results

Due to the hardware non-idealities characterized in the previous section, it is expected that the transfer of a digitally trained neural decoder to the equivalent memristor-based implementation would decrease its performance. We simulate the impact of these key non-idealities on the fidelity of an MND implemented in a mixed-signal IMC architecture. We then benchmark mitigation strategies using re-training allowing for almost complete compensation for the negative impact of the non-idealities of memristors.

We first evaluate the impact of the programming variability observed in TiO_x -based memristive devices by investigating their effects on the decoder performance. Figure 5.4(c) shows the evolution of the decoding fidelity of a distance-3 RNN for a typical physical fault rate (PFR) of 10^{-2} when the programming noise is increased. The experiment is repeated 10 times to obtain a mean fidelity value (shown using a blue curve). Note that

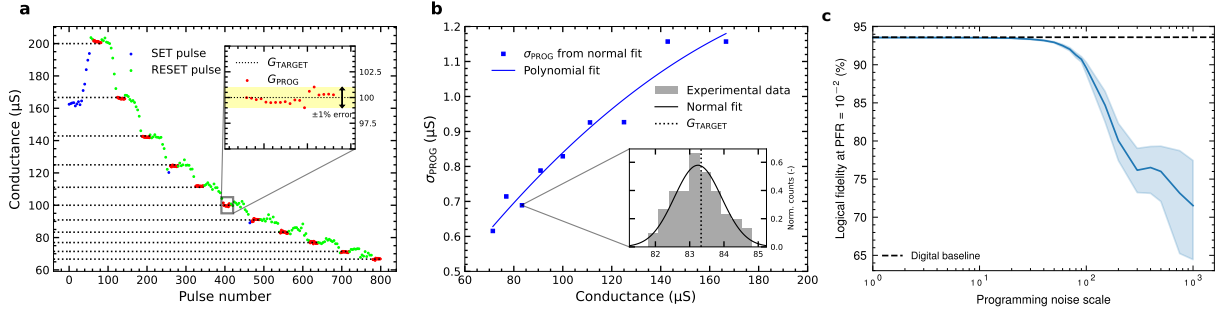


FIGURE 5.4 Programming variability characterizations of TiO_x -based resistive memory and its impact on the neural decoder. **a)** Pulse programming of 11 conductance states of a TiO_x -based resistive memory device. Positive voltage pulses induce a conductance increase (labeled “SET pulse”) due to the growth of the conductive filament inside the TiO_x layer between the electrodes. Negative voltage pulses are employed to decrease the conductance (“RESET pulse”) due to the rupture of the conductive filament. The dotted black lines denote the target conductance (“ G_{TARGET} ”), whereas the red dots denote the programmed conductance (“ G_{PROG} ”) values upon convergence of the closed-loop read–write–verify algorithm. The inset is a zoom-in on the readout of the 100 μS state within the allowed error of 1 %. **b)** Programming variability model based on the programming standard deviations of multiple conductance states. The multilevel programming cycle shown in (a) is conducted 10 times in a double sweep for 10 devices. For each conductance state, the standard deviation of the distribution of programmed values is extracted to fit a programming variability model, except for the highest conductance state (HCS) and the lowest conductance state (LCS). **c)** Decoding fidelity of a distance-3 surface code for a typical physical fault rate (PFR) of 10^{-2} using a trained recurrent neural network (RNN), as a function programming variability of the weights, represented as a factor of the polynomial fit used in (b). The error bands represent the 95 % confidence interval over 10 random seeds. The performance of the RNN remains close to the digital baseline before the noise standard deviation reaches a factor of ~ 10 times higher than the experimental value, after which the fidelity drops significantly.

no noticeable fidelity decrease is observed below about 10 times the experimental noise value, therefore, the programming variability of TiO_x -based resistive memory devices does not represent a roadblock for the realization of the MND. Evaluating the impact of the resolution of the DACs and ADCs (see Figure C.1) leads to the same conclusion, as an 8-bit discretization of the input and output values does not present any significant impact on the performance of the decoding. Lastly, the neural decoder appears to be robust against the reading variability (up to 1 %) induced by the analog electronics during an MVM operation (see Figure C.2). The methodology used to evaluate the impact of the DACs and ADCs, and the reading variability is described in Appendix C.2.

We then study the impact of stuck-at fault devices on the fidelity of the neural decoder, which is expected to reach to up to 10 % for the evaluated memristor technology. To emulate the impact of this non-ideality, a random subset of network parameters is set to zero when testing the decoder. This situation corresponds to the case where one of the memristor of a differential pair is stuck in HCS or LCS and the other is either also stuck in the same state or purposefully programmed in order to bring the logical analog weight to zero. The results presented in Figure 5.5(c) (green curve) show that, without any re-training designed to tackle this non-ideality, the stuck-at fault rate significantly reduces the performance of the decoder. A decrease of more than 17 % in fidelity is observed when the percentage of stuck-at fault devices during inference reaches 8 % (this is equivalent to 15 % of the model parameters, since a parameter is encoded via a pair of devices).

Therefore, we explore two re-training methods to attempt to restore the initial fidelity of the MND (see Figure 5.5a), which we compare to two baselines:

- **Digital baseline:** an ideal version of the neural decoder that can perform syndrome processing at room temperature on classical hardware. The parameters are trained using a deep learning method [58] (see Table C.1) and are represented as 32-bit floating-point variables that do not include hardware non-idealities. The RNN architecture is based on Ref. [165]. The digital baseline performance is represented in black in Figure 5.5, which we use as an upper bound and a reference against which to compare the other methods.
- **MND baseline:** a naive implementation of an analog memristor-based neural decoder that could be integrated in a cryogenic environment. The RNN's architecture is identical to the digital baseline, but the parameters are converted into the equivalent memristor conductance values after digital training. Hardware constraints and memristor non-idealities are simulated based on experimental characterizations (see Table C.2), but no re-training is used. The MND baseline performance is repre-

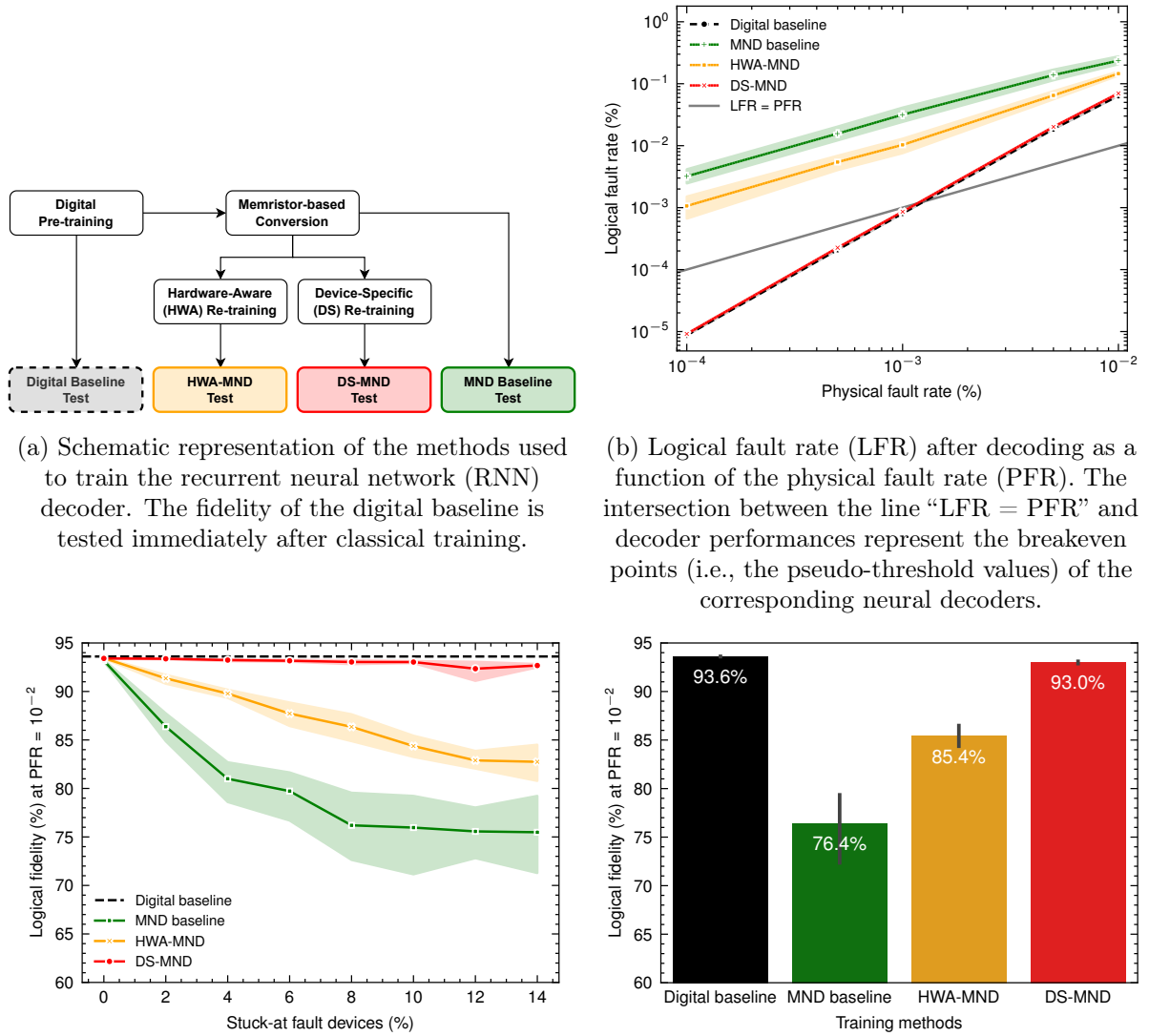


FIGURE 5.5 Baselines and re-training methods for the memristive neural decoder (MND). The hardware-aware (HWA) and device-specific (DS) MND methods benefit from a re-training step after being converted to a memristor-based model with the *IBM Analog Hardware Acceleration Kit* [422], whereas the MND baseline training method is converted and tested without any re-training. For the results presented in (b) and (d), the rate of stuck-at fault devices is set at the expected fabrication yield of 10 %. For the results presented in (b), (c), and (d), the digital baseline fidelity is obtained with an equivalent neural decoder. The error bars and shaded regions represent the 95 % confidence interval over 10 random seeds.

sented in green in Figure 5.5, which is expected to be a lower bound for other MND methods.

- **Hardware-aware memristive neural decoder (HWA-MND):** the MND baseline augmented with a re-training post-processing. The re-training consists of one additional training epoch during which different random weights are set to 0 at each forward pass. The dropconnect method [423] is typically used for regularization, but in the context of HWA re-training it is incorporated to improve the robustness of the RNN against stuck-at fault devices. For this reason, we set the dropconnect rate to match the expected number of weights blocked due to stuck-at fault devices in the characterized hardware (see Figure C.4 for the different stuck-at fault rates). At the testing stage, the hardware non-idealities are simulated (see Table C.2). The performance of HWA-MND is represented in yellow in Figure 5.5.
- **Device-specific memristive neural decoder (DS-MND):** a variation of the HWA-MND, where the dropconnect is not random but follows a specific device. During the entire re-training epoch, the weights corresponding to the stuck-at fault memristors in a given crossbar are fixed at 0. This approach forces the RNN to precisely adapt its parameters to this hardware limitation. The performance of DS-MND is represented in red in Figure 5.5.

Appendix C.4 provides details about the implementation of training and inference in our experiments, and we have made the simulation data available in Ref. [424].

Dropping connections randomly during the HWA-MND re-training provides a generic mitigation strategy applicable to any crossbar of memristors given that the average number of stuck-at fault devices is known prior to re-training. However, it does not allow for a full recovery of the digital baseline performance even for low rates of stuck-at fault devices. DS-MND addresses this shortcoming. This re-training of DS-MND completely avoids updating weights that cannot be reliably programmed in the crossbar. The idea of this DS re-training is to leverage precise knowledge of the crossbars' electrical characterizations to find the best NN parameters for that particular memristive circuit. It has the advantage of converging towards a nearly optimal solution, at the cost of having to individually characterize each crossbar of the MND to localize stuck-at fault devices, a task which can be performed through methods such as march tests [425, 426]. The simulation results, with the expected 10 % rate of stuck-at fault devices (see Figure 5.5d), show that only DS-MND reaches within 1 % of the digital baseline's fidelity, whereas the HWA-MND recovers only about half of the performance loss due to hardware non-idealities.

Figure 5.5b represents the decoding performance of the MNDs studied in the case of different PFRs for a fixed percentage of stuck-at fault devices of 10 % to match the typical fabrication yield. Only the DS re-training method maintains the pseudo-threshold of the memristive decoder near the pseudo-threshold of the baseline decoder. Therefore, based on the characterized non-idealities of TiO_x -based resistive memory devices, it is a necessity to introduce specific knowledge of the chip during re-training to achieve the highest decoding performance in the case of a distance-3 code based on the RNN we have studied. The general solution provided by HWA-MND provides robustness against device-specific hardware issues, but remains insufficient.

5.2.6 Discussion

In this section, we discuss our novel hardware approach for implementing an RNN decoder relying on the IMC paradigm to perform MVM on passive crossbars of memristors. We implemented a simulation based on the *IBM Analog Hardware Acceleration Kit* [422] that accounts for the experimentally characterized non-idealities of TiO_x -based resistive memory devices and measured their impact on our neural decoder’s performance. By applying computational methods to mitigate key hardware non-idealities, we improved the robustness of the neural decoder. In particular, we found that using the dropconnect method during re-training can greatly improve the fidelity of an MND that whose performance is significantly reduced by stuck-at fault devices. Moreover, we have seen that localizing stuck-at fault memory devices in the crossbars and using that knowledge to disable the corresponding connections during re-training can lead to a fidelity score comparable to that of the digital baseline (i.e., having a $<1\%$ difference). Furthermore, the MND exhibits only a small drop in the pseudo-threshold for the distance-3 surface code in numerical simulations (see Figure 5.5(b)). Therefore, we can conclude that our results support the effectiveness of specialized training methods for MNDs to achieve near-optimal performance.

Although an experimental proof of concept has not yet been performed on fully integrated hardware, the results we have presented offer a promising pathway to realizing high-fidelity neural decoders using IMC and analog memristive devices. One interesting avenue for future research is the development of a fully analog version of the memristive decoder circuit presented in this paper. Such an implementation would bring further benefits in terms of the decoding time and energy efficiency. Analog activation functions have been reported in the literature [427]. For instance, the ReLU function can be applied in the analog domain using multimodal transistors [428]. It is therefore possible to envision MNDs using an analog activation function after the first layer instead of implementing an ADC followed by a digital activation function followed by a DAC, as presented in

this paper. A practical circuit would also necessitate many more electronic components to realistically perform the decoding task, some of which we did not consider here. For example, transimpedance amplifiers (TIAs) would be needed to convert a current signal to a voltage signal at the output of each memristor row. Also, an analog memory unit might be necessary to store and transmit the hidden states' signals back to the recurrence input ports.

In our work, we made the assumption that imperfections arising from analog or digital CMOS components (e.g., noise introduced by differential amplifiers, TIAs, ADCs and DACs; divergence of the analog activation function from its digital counterpart in the case of a fully analog implementation; and signal distortion arising from multiple recurrences) are much less critical in terms of the fidelity of the MND in comparison to the non-idealities exhibited by the memristors. However, future work should include a more exhaustive analysis of the impact of circuit-level imperfections (e.g., using training methodologies that have been introduced [429, 430]).

Scalability is a well-known issue for QEC decoders: as the error correction code distance increases, the syndrome space grows exponentially. Neural decoders thus require an exponentially larger dataset to learn about correlations between syndromes and the occurrences of logical errors. Our decoder cannot overcome the issue of scalability. However, it can be integrated in a multi-stage or hierarchical decoding scheme [409, 410, 431]. Multi-stage or hierarchical approaches have been proposed to improve the efficiency of NN-based decoders by using a combination of two decoding modules; in some instances [432], the first module is a simple classical decoder, and the second is an NN-based decoder. The role of the NN-based module is to act as a supervisor, identifying when the correction suggested by the simple decoder will lead to a logical error. This approach results in a constant execution time once the NN has been trained, regardless of the physical error rate, and scales linearly with the number of qubits in the code. A more recent study [433] demonstrates that NNs can be used as local decoders to remove an initial set of errors, thereby reducing the syndrome space and enabling the fast execution of a global decoder (minimum-weight perfect matching in the study) to correct the remaining errors. In this sense, a fully analog version of our MND could be integrated in a hierarchical decoding strategy and act as a local decoder to feed inputs to a global decoder.

Another challenge related to the scalability of decoders is the cryogenic compatibility of the chosen hardware. Indeed, as the number of physical qubits increases, to avoid a wiring bottleneck between the control electronics at room temperature and the quantum processor in a cryogenic environment, it is highly beneficial to integrate the decoder hardware

directly within the cryostat [167]. Within this scope, the energy consumption and thus the heat dissipation of the decoder should be minimized to avoid perturbations in the quantum system. The MND presented in this paper shows promise in terms of cryogenic compatibility as the MAC operations rely on an energy-efficient memristive IMC architecture instead of digital circuit blocks such as multipliers and adders. Even if current hardware implementations of NNs employing application-specific integrated circuits (ASICs) and field-programmable gate arrays (FPGAs) are not optimized in comparison to CPU-based approaches, they continue to suffer from the delays and energy expenditure associated with digital MAC operations. Furthermore, in recently proposed approaches introducing the idea of quantized IMC for QEC [395,396], the implementations still require digital multipliers and adders. From their fast inference time and energy efficiency, MNDs are a promising technology for direct integration in a dilution refrigerator. However, the cryogenic compatibility of a memristor-based fully analog integrated circuit for QEC and the detailed characterization of its decoding time and power dissipation remains to be investigated.

5.2.7 Author contributions

All authors contributed to this article and approved of the submitted version.

Victor Yon: methodology, software implementation, run simulations, results analysis and visualization, writing—original draft preparation.

Frédéric Marcotte: methodology, software implementation, run simulations, hardware characterization, writing—original draft preparation.

Pierre-Antoine Mouny: methodology, software implementation, hardware characterization, manuscript editing.

Gebremedhin A. Dagnew: methodology, syndrome dataset generation, manuscript editing.

Bohdan Kulchytskyy: syndrome dataset generation, manuscript review, supervision.

Sophie Rochette: methodology, manuscript editing, supervision.

Yann Beilliard: methodology, manuscript editing, supervision.

Dominique Drouin: manuscript review, supervision.

Pooya Ronagh: methodology, manuscript editing, supervision.

5.2.8 Code and data availability

The code from our study is available from the corresponding author upon reasonable request. The syndromes dataset has been made publicly available [411] as well as the model training and simulation output data [424].

5.3 Bilan de l'article

Cette étude a permis de démontrer qu'il était possible d'intégrer un réseau de neurones artificiels (RNA) sur un circuit à base de memristors pour une application cryogénique, et ce même en présence d'importants défauts de fabrication. La méthode de ré-entraînement développée pour ce décodeur neuronal n'est pas dépendant du type de RNA, ni de la tâche réalisée. Cette approche peut donc facilement être transférée à d'autres applications similaires, telles que la calibration automatique de boîtes quantiques présentées dans les chapitres précédents. De plus, les simulations étant réalisées à partir de données expérimentales provenant de prototypes développés à l'institut interdisciplinaire d'innovation technologique (3IT), il est envisageable de réaliser une preuve de concept expérimentale dans un futur proche.

CHAPITRE 6

Conclusion

6.1 Synthèse des travaux

Le calculateur quantique, grâce à sa capacité sans précédent de traitement de l'information, a le potentiel de révolutionner de nombreux secteurs d'activité [434] qui sont actuellement limités par la puissance des ordinateurs classiques. Cependant, la mise en place de systèmes quantiques opérationnels à grande échelle est encore entravée par de nombreux défis techniques d'une complexité inédite. Dans ce manuscrit, nous avons proposé des solutions aux problématiques de calibration des qubits de spin et de correction d'erreur quantique, en exploitant la puissance des réseaux de neurones artificiels (RNA), associée à l'efficacité des circuits à base de memristors.

Dans les chapitres 3 et 4, nous avons défini une méthode de calibration automatique de boîte quantique, que nous avons évalué sur trois technologies différentes. Cette approche tire profit de la capacité de reconnaissance de motifs des réseaux de neurones à convolutions (CNN) afin d'ajuster les paramètres de contrôle sans intervention humaine, même en présence de mesures fortement bruitées. Nous avons montré que l'évaluation du niveau d'incertitude des RNA permet une réduction de plus de 10 % des erreurs lors de la procédure de calibration, menant à un taux de succès de plus de 99 % sur un jeu de données expérimentales hors ligne (tableau 3.2) et de 95 % lors d'une démonstration en temps réel (figure 4.3). Ces résultats ont été obtenus à partir d'une heuristique d'incertitude basée sur de simples CNN, surpassant en performance les réseaux de neurones bayésiens (BNN) pourtant plus complexes. Cette méthode établit ainsi une base solide pour le développement de systèmes de contrôle de qubits de spin à grande échelle et une démonstration concrète de l'importance de l'incertitude dans les applications d'apprentissage automatique.

Dans le chapitre 5, nous avons utilisé un réseau de neurones récurrents (RNN) pour décoder efficacement les syndromes générés par un code de correction d'erreur quantique. Nous avons montré qu'une implémentation matérielle à base de memristors permettrait de respecter les contraintes de temps et d'énergie imposées par les systèmes quantiques, avec une vitesse d'inférence estimée à ~ 200 ns et une consommation énergétique de $\sim 2,5$ mW [399], mais au détriment de la performance du décodeur. Ce qui nous a amené à proposer une

méthode de ré-entraînement de RNA, prenant en compte les imperfections de memristors, pour rétablir les performances initiales du décodeur (figure 5.5), tout en maintenant les avantages de l’implémentation matérielle. Le décodeur ainsi obtenu permet d’atteindre 93,0 % de fidélité sur un jeu de données synthétiques simulant un code de surface de distance 3, soit une perte de performance de seulement 0,6 % par rapport à un décodeur idéal. Cette méthode d’entraînement, associée au circuit à base de memristors, ouvre la voie à une intégration dans un système de décodage hiérarchique à plusieurs étages [410, 431] pour la manipulation de qubits logiques robustes aux erreurs physiques.

De ces travaux découle une méthode généralisable à de nombreuses applications pour lesquelles la consommation énergétique des RNA est un enjeu important [10]. Les principales étapes étant de : (i) définir une base de référence à partir d’un RNA classique (collecte des données d’entraînement, définition de la structure du réseau et optimisation des meta-paramètres) ; (ii) estimer les performances du RNA sur l’architecture matérielle ciblée (caractérisation expérimentale du matériel, modélisation des imperfections et simulation de l’implémentation) ; et (iii) ré-entraîner le RNA pour compenser les imperfections matérielles (identifier les imperfections critiques, développement d’une méthode de ré-entraînement et comparer les performances avec la base de référence).

6.2 Perspectives

La méthode de calibration automatique de boîte quantique a été développée de façon à être compatible avec une grande variété d’installations expérimentales, indépendamment de la technologie utilisée, et le code source Python, en source ouverte, est conçu pour être facilement modifiable et extensible. Ce qui permet déjà au groupe de recherche de la professeure Eva Dupont-Ferrier de l’utiliser pour leurs propres travaux en collaboration avec l’IMEC¹. De plus, cette méthode continue d’être développée dans le cadre de l’Alliance Quantique² impliquant le groupe du professeur Dominique Drouin et Irréversible. Les travaux en cours visent à étendre la méthode actuelle pour supporter la calibration de double ou triple boîtes quantiques. Ce qui peut être réalisé soit en adaptant le modèle de détection binaire (“aucune ligne” ou “ligne”) en un modèle capable d’identifier plus finement les données d’entrée (“aucune ligne”, “horizontale”, “verticale” ou “diagonale”), soit en développant un second modèle capable d’estimer l’angle d’une ligne de transition arbitraire. Ensuite, le modèle, qu’il soit pour une ou plusieurs boîtes, devra être intégré sur un circuit à basse consommation, connecté au système de contrôle au sein de l’environnement

1. IMEC : imec-int.com

2. Subventions en quantique du programme Alliance : nserc-crsng.gc.ca/Innovate-Innover/alliance_-quantum-alliance_quantique

cryogénique. Enfin, il sera aussi possible d'améliorer la stratégie d'exploration (décrite dans la section 3.2.5.1), soit en optimisant les hyper-paramètres et la logique interne, soit en définissant une stratégie radicalement différente. Bien que l'approche bayésienne n'ait pas été efficace pour estimer l'incertitude des RNA, son utilisation dans le cadre de l'exploration des diagrammes de stabilité semble plus prometteuse. Il serait en effet possible d'exploiter les connaissances à priori sur la géométrie de l'espace pour explorer en priorité les régions dont le niveau d'incertitude est le plus élevé, dans le but de limiter le nombre de mesures et ainsi réduire le temps de calibration.

La confirmation de la viabilité d'un décodeur cryogénique à base de memristors a mené au dépôt d'un brevet [435] et à la fabrication d'un prototype par Irréversible [399]. Il reste cependant à étendre ce concept à un plus grand nombre de qubits physiques, à fabriquer un prototype à taille réelle et à proposer une architecture complète de décodage hiérarchique à plusieurs étages [410]. Bien que l'architecture proposée soit compatible avec des codes de surface de grandes tailles, la fabrication de matrices de memristors de taille suffisante et la conception de circuits compatibles avec un environnement cryogénique reste un défi technique important. Pour le relevé, il sera nécessaire de tester (en simulation ou via des prototypes) de nombreuses implémentations basées sur diverses technologies et composants.

Ainsi, ce projet de recherche a non seulement permis de proposer des solutions concrètes à des problématiques actuelles de l'informatique quantique, mais a également mis en lumière le potentiel des memristors pour concevoir des applications d'intelligence artificielle (IA) à basse énergie.

6.3 Contributions scientifiques

Ce projet a donné lieu à plusieurs contributions scientifiques sous forme d'articles et de conférences, listées ici par catégorie et ordre chronologique.

Contributions principales

Victor Yon, Amirali Amirsoleimani, Fabien Alibart, Roger G. Melko, Dominique Drouin et Yann Beilliard.

“Exploiting non-idealities of resistive switching memories for efficient machine learning”. *Frontiers in Electronics* (25 mars 2022).

doi.org/10.3389/felec.2022.825077 — Référence [269]

Victor Yon, Bastien Galaup, Claude Rohrbacher, Joffrey Rivard, Clément Godfrin, Ruoyu Li, Stefan Kubicek, Kristiaan DeGreve, Louis Gaudreau et Eva Dupont-Ferrier.

“Robust quantum dots charge autotuning using neural network uncertainty”. *Machine Learning : Science and Technology* (18 octobre 2024).

doi.org/10.1088/2632-2153/ad88d5 — Référence [297]

Victor Yon, Bastien Galaup, Claude Rohrbacher, Joffrey Rivard, Alexis Morel, Dominic Leclerc, Clement Godfrin, Ruoyu Li, Stefan Kubicek, Kristiaan De Greve, Eva Dupont-Ferrier, Yann Beilliard, Roger G. Melko et Dominique Drouin.

“Experimental online quantum dots charge autotuning using neural networks”. *arXiv preprint* (30 septembre 2024).

doi.org/10.48550/arxiv.2409.20320 — Référence [346]

Victor Yon, Frédéric Marcotte, Pierre-Antoine Mouny, Gebremedhin A. Dagnew, Bohdan Kulchytskyy, Sophie Rochette, Yann Beilliard, Dominique Drouin et Pooya Ronagh.

“A cryogenic memristive neural decoder for fault-tolerant quantum error correction”. *arXiv preprint* (23 octobre 2024).

doi.org/10.48550/arxiv.2307.09463 — Référence [343]

Conférences

Victor Yon, Stefanie Czischek, Yann Beilliard, Michel Pioro-Ladrière, Gaudreau Louis, Roger G. Melko et Dominique Drouin. “Quantum dot charge autotuning with bayesian neural networks for spin qubit applications”.

Silicon Quantum Electronic Workshop (2–5 octobre 2022).

hal.science/hal-03822883

Victor Yon, Stefanie Czischek, Yann Beilliard, Michel Pioro-Ladrière, Gaudreau Louis, Roger G. Melko et Dominique Drouin. “Robust quantum dot charge autotuning with bayesian neural networks”.

Machine Learning in Natural Sciences : from Quantum Physics to Nanoscience and Structural Biology (19–22 septembre 2022).

hal.science/hal-03717569

Contributions comme co-auteur

Amirali Amirsoleimani, Fabien Alibart, **Victor Yon**, Jianxiong Xu, M. Reza Pazhouhandeh, Serge Ecoffey, Yann Beilliard, Roman Genov et Dominique Drouin.

“In-memory vector-matrix multiplication in monolithic complementary metal-oxide-semiconductor-memristor integrated circuits : Design choices, challenges, and perspectives”. *Advanced Intelligent Systems* (23 août 2020).

doi.org/10.1002/aisy.202000115 — Référence [258]

Stefanie Czischek, **Victor Yon**, Marc-Antoine Genest, Marc-Antoine Roux, Sophie Rochette, Julien Camirand Lemyre, Mathieu Moras, Michel Pioro-Ladrière, Dominique Drouin et Yann Beilliard.

“Miniaturizing neural networks for charge state autotuning in quantum dots”. *Machine Learning : Science and Technology* (24 novembre 2021).

doi.org/10.1088/2632-2153/ac34db — Référence [136]

Mathieu Moras, Marc-Antoine Roux, Larissa Njejjimana, Stefanie Czischek, **Victor Yon**, Marc-André Tétrault et Michel Pioro-Ladrière.

“Adaptive filtering and classification for automated tuning of quantum dots into the single electron regime”. *APS March Meeting* (2023).

ui.adsabs.harvard.edu/abs/2023APS..MARM74010M/abstract

Philippe Drolet, Raphaël Dawant, **Victor Yon**, Pierre-Antoine Mouny, Matthieu Valdenaire, Javier Arias Zapata, Pierre Gliech, Sean Wood, Serge Ecoffey, Fabien Alibart, Yann Beilliard et Dominique Drouin.

“Hardware-aware training techniques for improving robustness of ex-situ neural network transfer onto passive TiO_2 rram crossbars”. *arXiv preprint* (29 mai 2023).

doi.org/10.48550/arxiv.2305.18495 — Référence [292]

Pierre-Antoine Mouny, Maher Benhouria, **Victor Yon**, Patrick Dufour, Linxiang Huang, Yann Beilliard, Sophie Rochette, Dominique Drouin et Pooya Ronagh.

“Towards a cryogenic CMOS-memristor neural decoder for quantum error correction”. *IEEE International Conference on Quantum Computing and Engineering (QCE)* (15–20 septembre 2024).

10.1109/QCE60285.2024.00149 — Référence [399]

Pierre-Antoine Mouny, **Victor Yon**, Sophie Rochette, Pooya Ronagh, Dominique Drouin, Yann Beilliard et Frédéric Marcotte.

“A memristor-based circuit for implementing a quantum error correction decoder and methods for training and inference of a memristor-based quantum error correction decoder neural network”. *Brevet PCT/CA2024/050673* (novembre 2024).

patentscope.wipo.int/search/en/detail.jsf?docId=WO2024234109 — Référence [435]

CHAPTER A

Appendix: Robust quantum dots charge autotuning using neural network uncertainty

A.1 Datasets

TABLE A.1 Summary of dataset specifications. SET and QPC stand for single-electron transistor and quantum point contact, respectively.

Dataset short name	Si-SG-QD	GaAs-QD	Si-OG-QD
Dataset full name	Silicon split gate quantum dot	Gallium arsenide quantum dot	Silicon overlapping gate quantum dot
Current measurement method	SET	QPC	SET
Pixel size	1 mV	2.5 mV	2 mV
Detection area offset	6 pixels	7 pixels	6 pixels
Prior line distance	30 mV	16 mV	30 mV
Prior line slope (0° = horizontal, 90° = vertical)	75°	45°	−10°
Use last-line validation	No (not necessary)	Yes	Yes
Number of diagrams with transition line annotations (used for line detection training and test)	17	9	12
Number of diagrams with transition line and charge area annotations (used for autotuning test)	9	9	9
Number of patches (18 × 18 pixels each)	72,182	4,349	48,081
Patch class ratio (no-line / line)	33.6	3.3	8.2
References	[312]	[122]	[298]

A.1.1 Data selection

The stability diagrams used in this article are the result of experimental measurements performed in prior studies on quantum dot (QD) devices [122, 312]. Thus, most of the measurements were not suitable for this work, and a selection was made. Only stability diagrams meeting the following criteria were included in the final datasets:

General criteria:

- All diagrams of the same dataset are measured using similar devices. Hardware differences are acceptable between datasets, but consistency is required for each independent training.
- A similar measurement step size is required between G1 and G2. A step size mismatch between the two measured gates makes pixel interpolation less reliable.
- Double-QD diagrams are excluded.

Transition line detection patch datasets:

- At least one transition line must be visible or partially visible in the diagram.

Autotuning datasets:

- At least three transition lines must be visible or partially visible in the diagram.
- A clearly identifiable empty regime must be in the measured range (no line for at least two times the average space between visible lines).

A.1.2 Data processing

The long-term goal of this project is to implement an automated control loop as close as possible to the quantum device. Thus, we want to minimize the preprocessing of the input data to avoid any computation overhead inside the cryogenic environment. This is why we do not compute the derivative on the patches or perform complex data processing, such as the method proposed by [136]. Furthermore, most statistical methods require a large sample size to be reliable, which is incompatible with our small patch measurement strategy.

To improve the convergence of the models, we normalized the input model to a range between 0 and 1 based on the smallest and largest current value of each patch.

A.1.3 Diagram annotation

We manually annotated the transition lines and the charge regime areas using *Labelbox* [436] at the diagram level. When the transition lines were mixed with background noise, we used the derivative of the images to place more accurate annotations. We only annotated the lines that we could visually identify. We never filled the gap between two sections of a line, even when it was clear that they were a continuation of the same transition.

The voltage space was segmented into charge regime areas annotated from 0 to 3 electrons, then the voltage space corresponding to more than 3 electrons was annotated as “4+”. The location of each area was deduced from the position of the transition lines and the knowledge that the empty regime was located at the bottom left of the images. When the regime of a section was ambiguous or difficult to identify (too noisy, fading line, few pixels around a line), no annotations were set, and the regime of this voltage space was considered as “*unknown*” during the experiment (examples in Figure 3.2e,f). When an autotuning ended in an “*unknown*” regime, it was always considered as a failure.

Annotating the transition lines and charge regimes of one stability diagram took between 5 and 30 min, depending on their complexity. It took approximately 10 h to annotate the 38 stability diagrams used in this study.

A.1.4 Patch segmentation and automatic class labeling

The patch size was fixed at 18×18 pixels as a tradeoff between measurement time and classifier performance. With a smaller patch size, it becomes difficult to distinguish a transition line from noise. A larger patch would directly increase the time required to measure it, reducing the exploration efficiency.

To train and test the artificial neural networks (NNs), we split the diagrams into patches by sliding an 18×18 window with 10 overlapping pixels between each step. We then automatically assigned a class to each patch using the manual transition line annotations set previously (see Appendix A.1.3). If at least one line intersected with the detection

area (Figure 3.3b), the patch was categorized as “*line*”; otherwise, it was categorized as “*no-line*”.

A.1.5 Subset splitting

Each patch dataset was split into three mutually exclusive subsets: training, validation, and test.

Each subset was used for a specific purpose:

Training set: used to train the line detection model.

Validation set: used after training to select the step corresponding to the best model (green stars in Figure A.5) and to calibrate the confidence thresholds (Figure A.6).

Test set: used to evaluate the line classification accuracy and the autotuning success rate after training and calibration.

In the case of training with cross-validation (Figure A.4), the subsets were split as follows:

Training set: composed of 90 % of the patches, randomly selected among the training diagrams.

Validation set: composed of 10 % of the patches, randomly selected among the training diagrams.

Test set: composed of every patch of the testing diagram.

In the case of training without cross-validation, the subsets were split as follows:

Training set: composed of 70 % of the patches, randomly selected among every diagram.

Validation set: composed of 10 % of the patches, randomly selected among every diagram.

Test set: composed of 20 % of the patches, randomly selected among every diagram.

A.1.6 Stability diagram samples

All of stability diagrams used in this study can be downloaded from [314]. To illustrate the transition line annotation methodology and demonstrate the diagram diversity of each dataset, two diagrams of each dataset are presented in Figures A.1 to A.3.

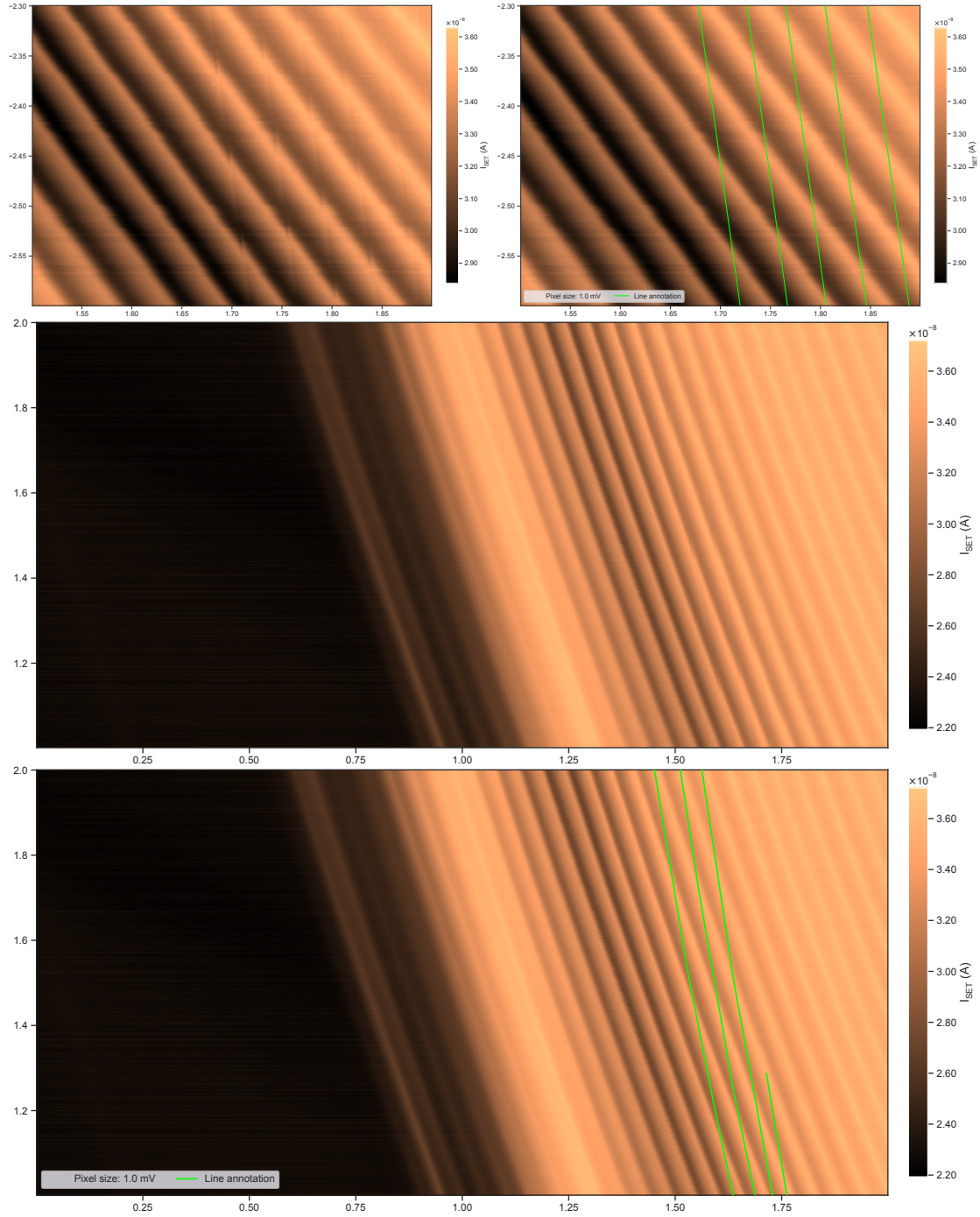


FIGURE A.1 Two silicon split gate (Si-SG-QD) stability diagram examples, without and with transition line annotations. The x -axes correspond to the G1 gate voltage, and the y -axes correspond to the G2 gate voltage.

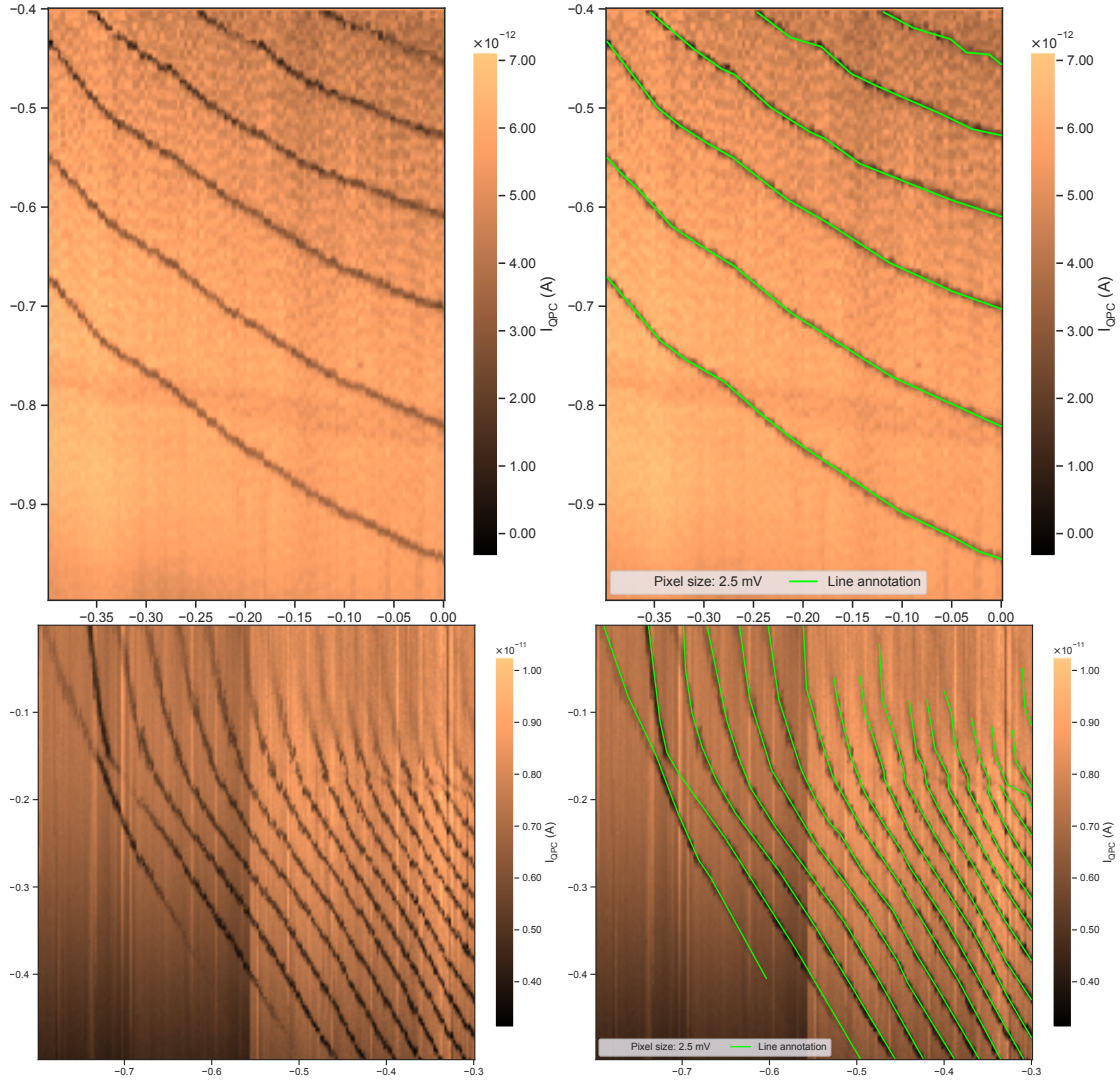


FIGURE A.2 Two gallium arsenide (GaAs-QD) stability diagram examples, without and with transition line annotations. The x -axes correspond to the G1 gate voltage, and the y -axes correspond to the G2 gate voltage.

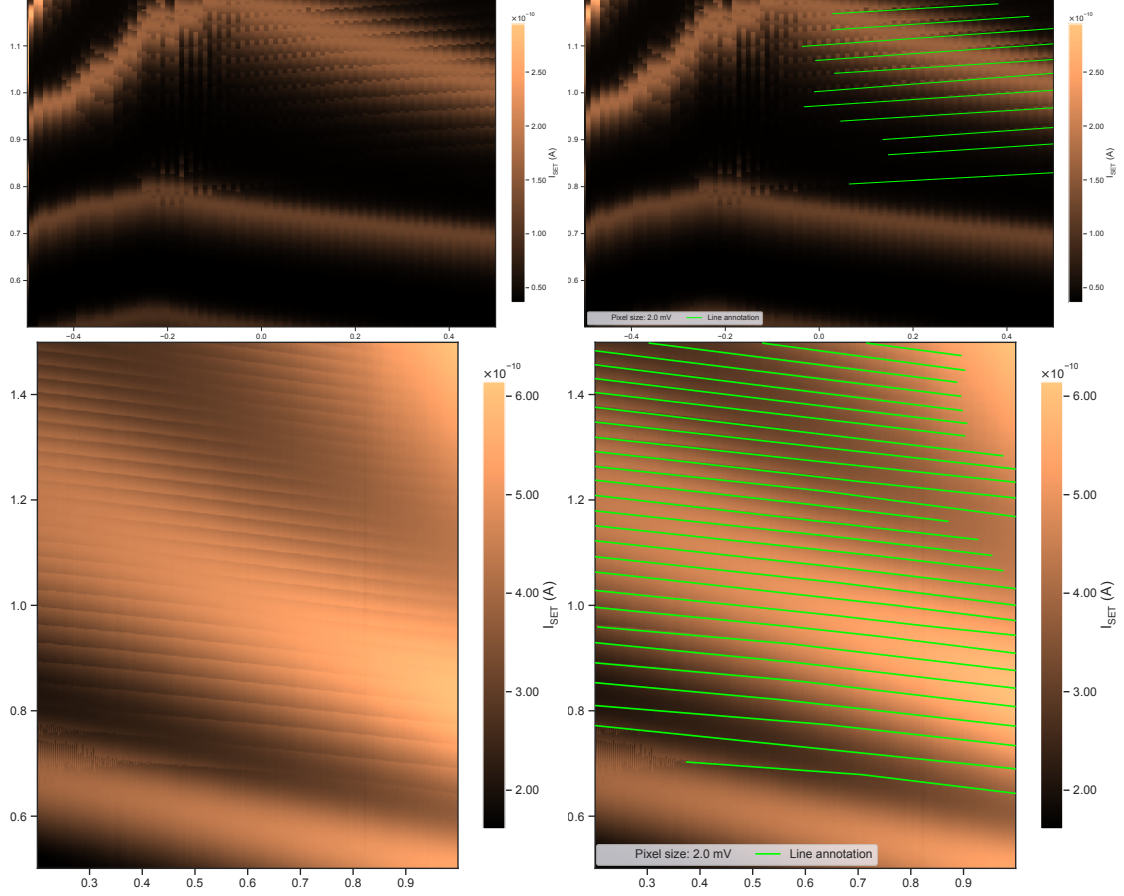


FIGURE A.3 Two silicon overlapping gate (Si-OG-QD) stability diagram examples, without and with transition line annotations. The x -axes correspond to the G1 gate voltage, and the y -axes correspond to the G2 gate voltage.

A.2 Line detection methodology

TABLE A.2 Model architectures and training meta-parameters used in this study.

Meta-parameters	FF	CNN	BCNN
Number of training updates	15,000	30,000	
Layers description	Fully connected: - 400 - 100	Convolutions: - 4×4 kernel, 12 channels - 4×4 kernel, 24 channels Fully connected: - 200 - 100	
Number of free parameters	170,201	367,267	734,534
Loss function	Binary cross-entropy		Binary cross-entropy + complexity cost [203]
Optimizer	Adam [437]		
Learning rate	0.0005	0.001	
Dropout rate	60 %		0 %
Batch size	512		
Number of inferences per patch (N in Equation (3.2))	1		10

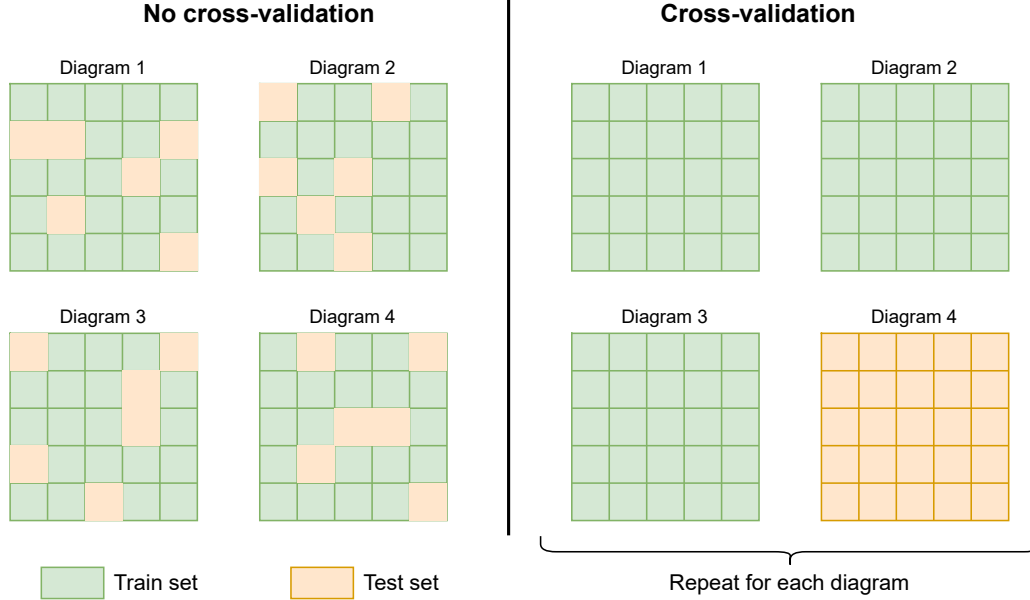


FIGURE A.4 Diagram cross-validation methodology schematic. The left panel represents the “No cross-validation” method, where every diagram is split into patches, then a random set of patches is used for training, and the rest are used for testing. The right panel represents the “Cross-validation” method [333], where every diagram is split into patches, then the patches of one diagram are used for testing, while the patches of the others are used for training. This process is repeated for every diagram, and the performance is averaged.

A.2.1 Model training

Each NN was trained using PyTorch [438] v2.1 in a supervised manner. We also used the *BLiTZ* library [439] to create the Bayesian layers, sample the parameters, and compute the parameter updates using *Bayes-by-Backprop* [203] for the Bayesian convolutional neural networks (BCNNs). The training parameters and the NN architectures are described in Table A.2. The source code used to obtain the results presented in this article is available on GitHub¹, and the output files (including the trained model parameters) can be downloaded from [345].

The imbalanced class ratio (“*line*” / “*no-line*”) caused convergence issues, since the loss could be easily reduced by predicting only the most frequent class (“*no-line*”). We worked around this problem by re-balancing the patch dataset using weighted sampling for each batch.

The meta-parameters presented in Table A.2 were selected based on a grid search and informed guesses. It is likely that these meta-parameters could be optimized and fine-tuned to improve the accuracy, accelerate the training, and reduce the number of free parameters. However, the scope of this study is to demonstrate the feasibility of the autotuning procedure and the benefits of using model uncertainty, not to achieve the best possible performance.

1. Git repository: github.com/3it-inpaqt/dot-calibration-v2/tree/offline-article

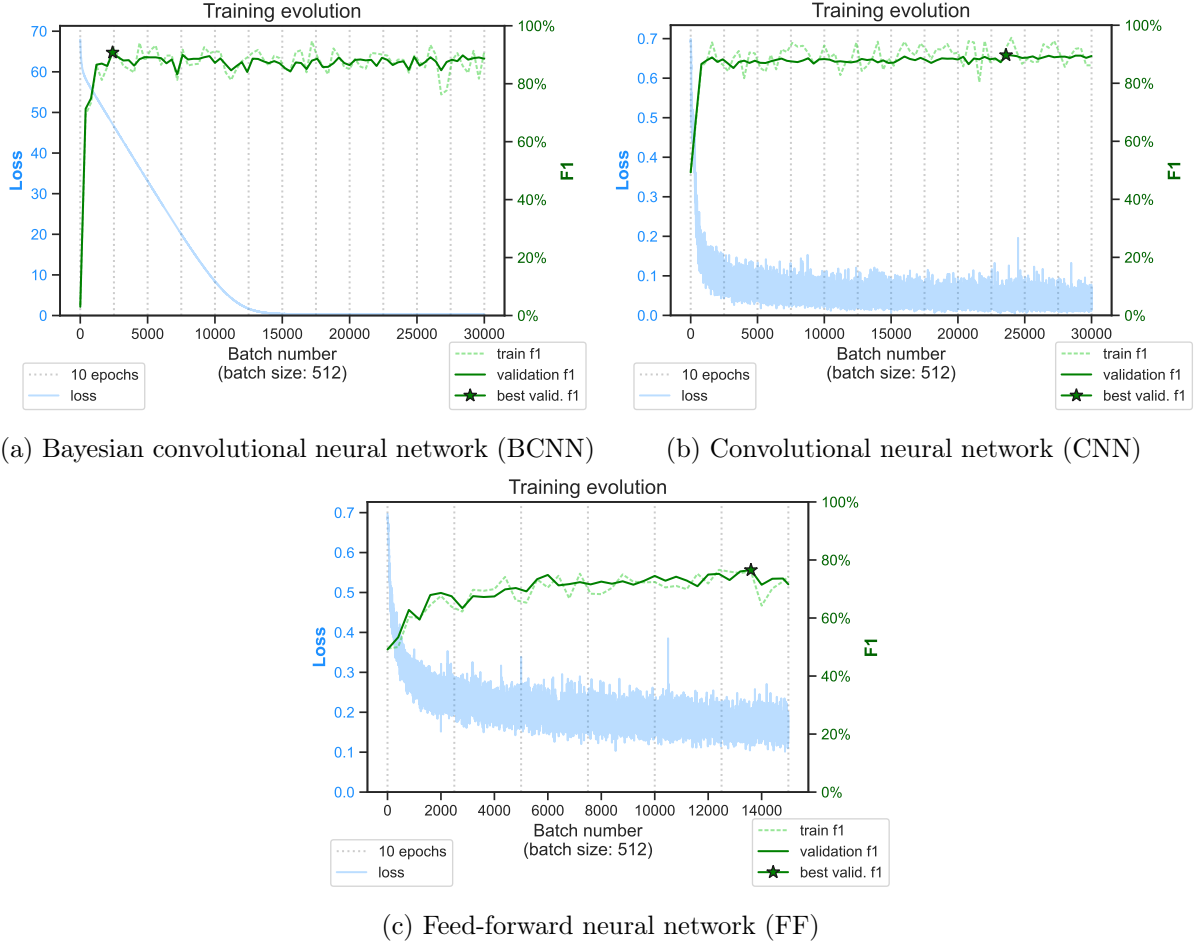


FIGURE A.5 Examples of training progress for the silicon split gate (Si-SG-QD) dataset using the different models and the cross-validation method. The F1 score is the harmonic mean of the precision and recall.

A.2.2 Model uncertainty and confidence threshold calibration

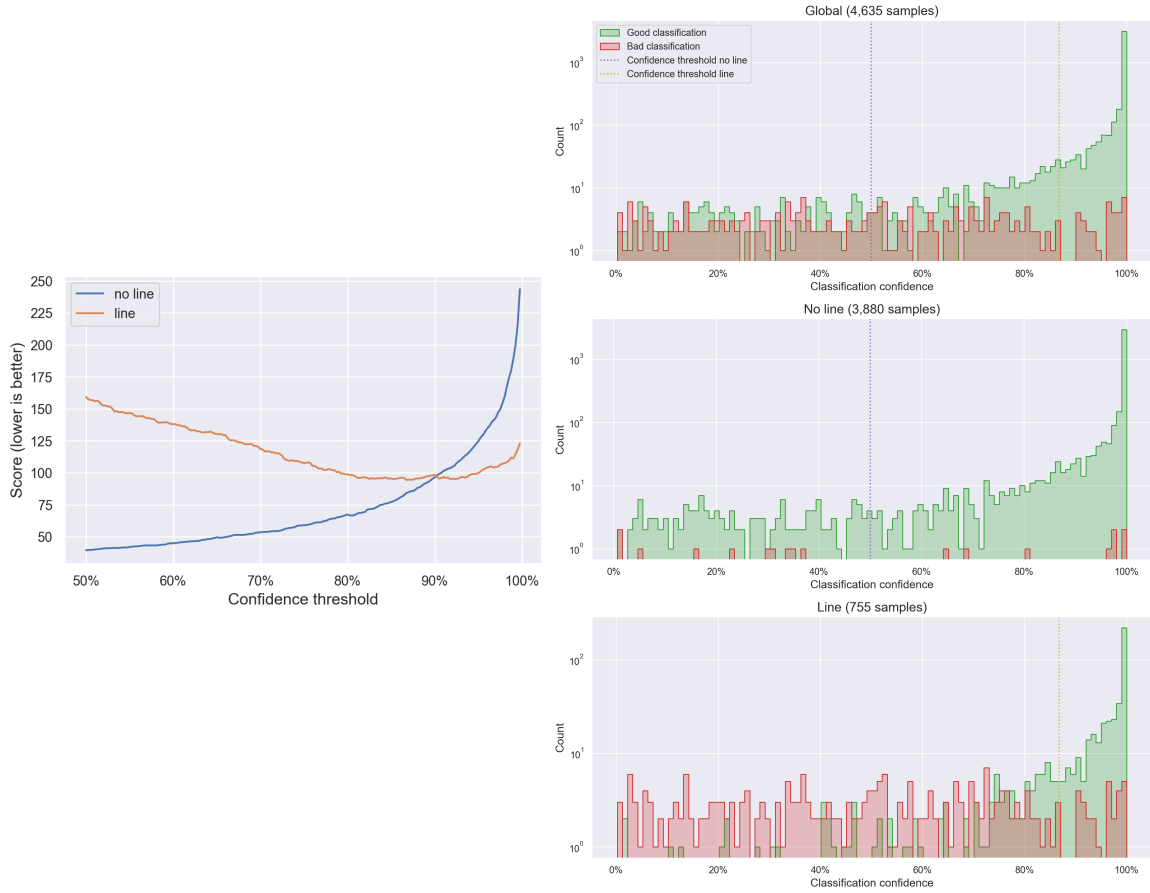
To estimate and use NN uncertainty, we made three important decisions: what model to use, what confidence metric to compute, and how to define the confidence threshold under which the inference should not be considered reliable.

We evaluated a standard convolutional neural network (CNN) and feed-forward neural network (FF) as references, since these are the most simple and well-established NNs for image pattern recognition. Such models are not specifically designed or trained to estimate uncertainty. Several studies [440–443] have suggested that classical NNs are generally overconfident and provide badly calibrated confidence scores. For the sake of simplicity, we defined the confidence score as the distance between the output and the closest class (Equation (3.1)). This score is normalized to a range between 0 and 1 to be easily interpretable and comparable as a confidence percentage.

We chose to implement a BCNN using variational inference [200,202] and trained it using the Bayes-by-Backprop [203] method. This choice was motivated by previously promising

results [191,192], and the library [439] was compatible with PyTorch [438]. The confidence score was computed as a normalized standard deviation (Equation (3.2)) from 10 inferences of the same input with newly sampled parameters.

A model is well calibrated if the confidence score accurately expresses the probability of making an error; for example, 20 % of the classifications with 80 % confidence should be wrong. However, a trained NN is never perfectly calibrated, and the calibration task is challenging for several reasons. First, NN training involves some stochastic processes (parameter initialization, stochastic gradient descent, etc.) that lead to different results. Therefore, two identical models trained separately could have very different and unpredictable miscalibration issues. Secondly, within one model, each predicted class could necessitate independent calibration (Figure A.6). We also suspect that the unbalanced class distribution of the dataset could amplify the problem. Finally, calibrating the confidence score using the training set induces a bias, since the out-of-distribution examples are (by definition) not represented. Correctly expressing the distributional uncertainty in the model score is critical, especially for the cross-validation testing method.



(a) Evolution of the threshold score (Equation (3.3)) when the threshold is varied. The threshold with the lowest score value is selected as the optimal threshold for each class.

(b) Histogram of the model classification error as a function of the confidence score. The confidence thresholds obtained from (a) are represented by the vertical dotted lines. The top panel shows the distribution for both classes combined, while the bottom panels show the distribution for each class independently.

FIGURE A.6 Example of confidence thresholds calibration for the silicon overlapping gate (Si-OG-QD) dataset using a convolutional neural network (CNN) and the cross-validation method. The thresholds are calibrated after training, on 4635 patches from the validation set. The confidence threshold of each class is calibrated independently. The small number of classification errors for the “no-line” class in the validation set makes the calibration challenging. Therefore, a value of 50 % is used as the default minimal threshold.

Model calibration methods require enough samples to cover the entire confidence spectrum, which might be challenging with a limited dataset. For example, a model with a high accuracy will correctly classify most of the samples with a high confidence score, implying that the samples with low confidence intervals will be very sparse (e.g., middle panel of Figure A.6b). Therefore, it is impossible to tell whether the model is under-confident or over-confident in these sparse intervals. Fortunately, the presented uncertainty-based exploration strategy does not require the exact probability of correctness for all the confidence ranges. The purpose of the confidence score in the context of this exploration task

is to optimize the exploration–exploitation tradeoff [328, 329]. When classification is performed with a confidence score under the threshold, adjacent patches will be explored. If the algorithm explores too much, it may waste time on unnecessary measurements, leading to inefficiency. Conversely, if it exploits too soon, it risks making incorrect tuning decisions based on incomplete or inaccurate information. In other words, we want to reduce the number of errors (impacting the tuning success rate) relative to the number of patches scanned (impacting the tuning time). Calibrating the threshold value by minimizing the score defined by Equation (3.3) is a simple way to account for the exploration–exploitation tradeoff. Fixing the meta-parameter τ to 0.2 is equivalent to saying that we prefer to have up to five more patches scanned rather than having a classification error.

Other types of models, meta-parameter values, training methods, confidence scores, and calibration methods could be evaluated to address this autotuning problem. However, an extensive benchmark of NN uncertainty is out of the scope of this paper.

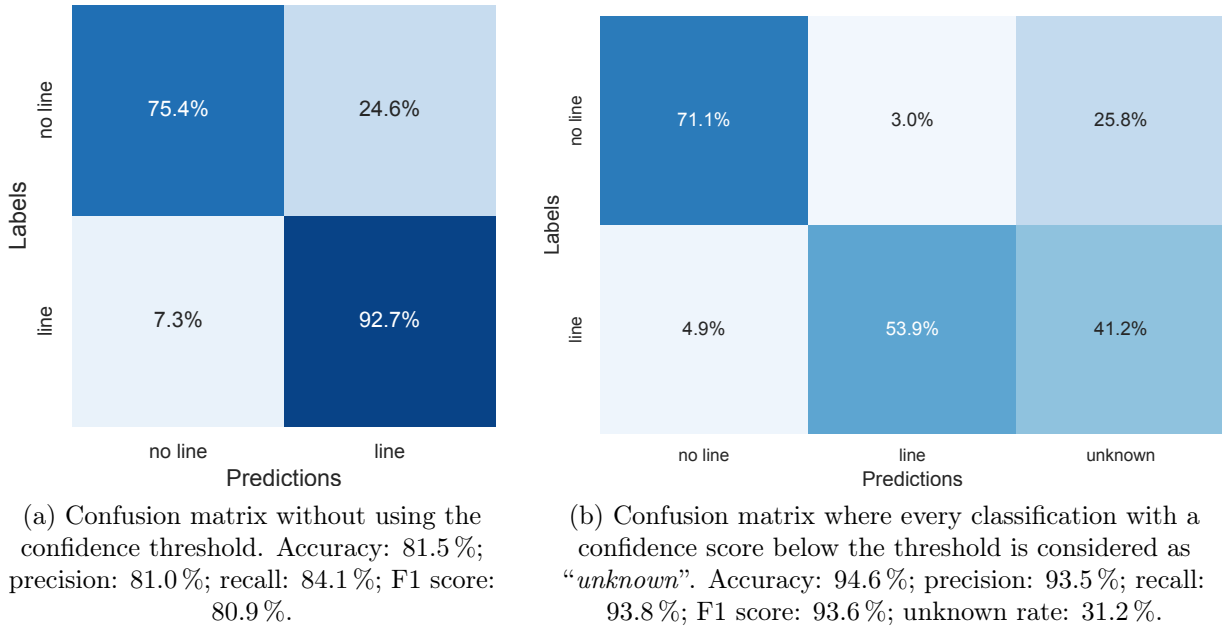


FIGURE A.7 Example of a confusion matrix for the silicon overlapping gate (Si-OG-QD) dataset using a convolutional neural network (CNN) and the cross-validation training method. Using the confidence threshold greatly improves the classification performance at the price of 31.2% of the patches being labeled as “*unknown*”.

A.2.3 Patch classification samples

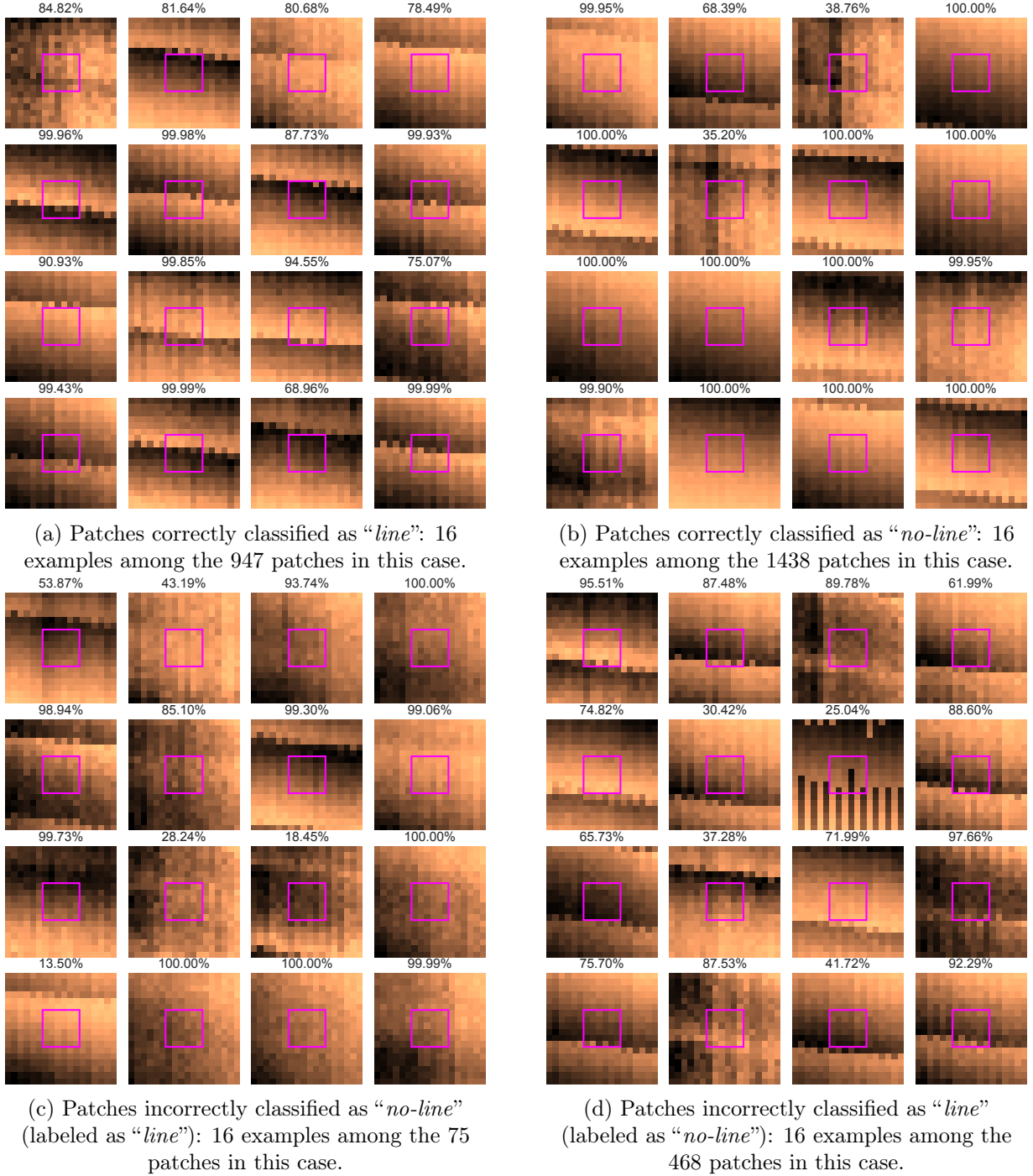


FIGURE A.8 Examples of patch classifications for the silicon overlapping gate (Si-OG-QD) dataset using a convolutional neural network (CNN) and the cross-validation training method. The patches are randomly selected among the test set. The pink squares in the center of the patches correspond to the detection area. A patch is labeled as “*line*” only if a transition line annotation crosses this square. The percentages at the top of each patch correspond to the confidence scores of the CNN computed using Equation (3.1).

Figure A.8 presents examples of the classification results to illustrate the line detection performance. We can observe that the classification errors (Figure A.8c,d) are often due to the presence of a transition line close to the detection area or annotation mistakes. In general, the confidence scores are lower when the patch is misclassified. More samples can be found in the simulation result files available for download from [345].

A.3 Autotuning methodology

A.3.1 Exploration algorithm

The autotuning exploration algorithm is illustrated as a block diagram in Figure A.9.

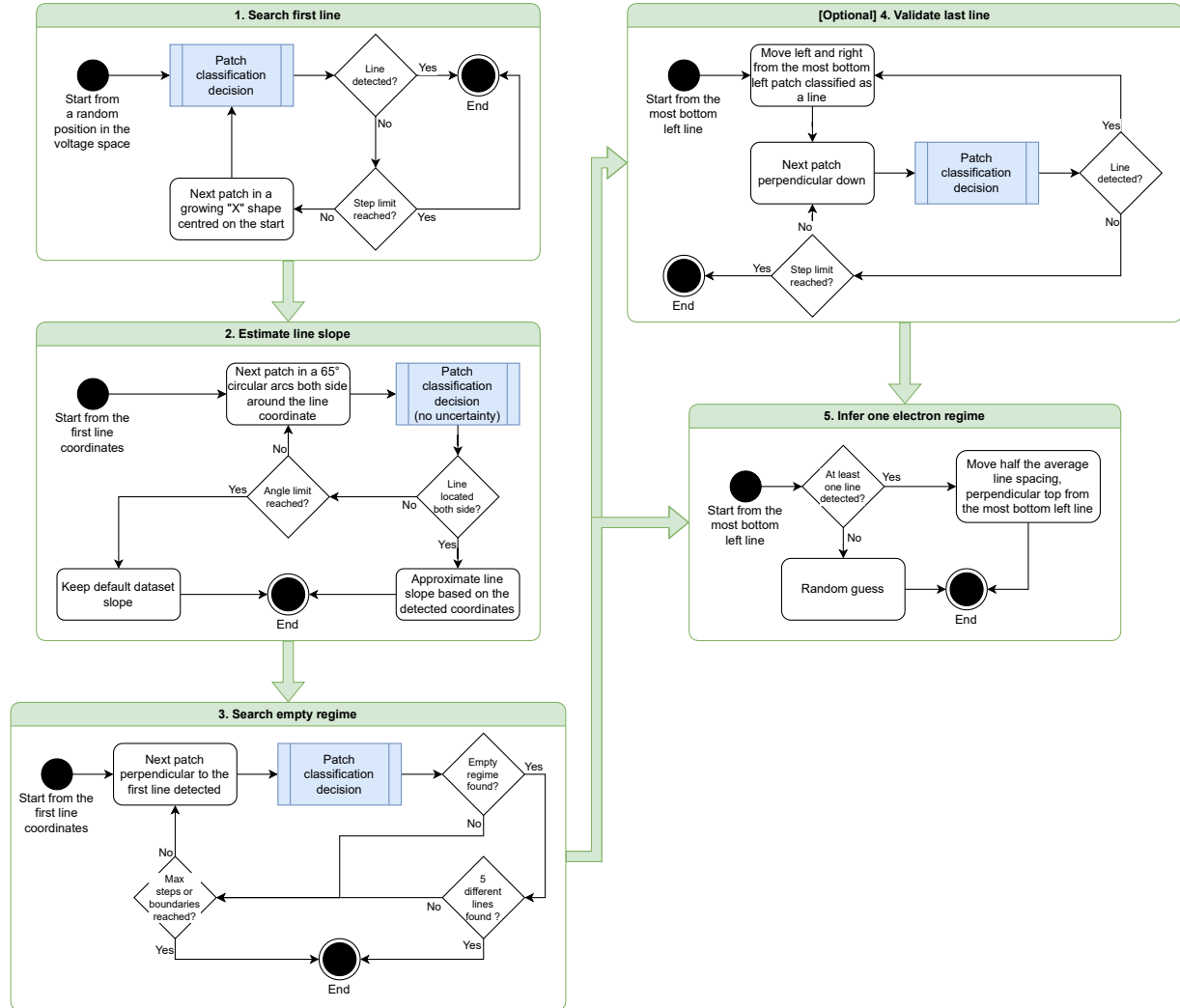


FIGURE A.9 Detailed exploration strategy schematic representing each step of the autotuning procedure. The optional *step 4* is enabled for the GaAs-QD and Si-OG-QD datasets to improve the tuning success rate when the first transition line is fading (see examples in Figure A.3).

Notes about the estimation of the transition line slope (step 2)

After finding the first patch containing a transition line, we scanned and classified a sequence of patches in circular arcs (up to 65°) on both sides surrounding this location. This scan sequence is illustrated in step 2 of Figure 3.5 (the white arrows emphasize the arc directions).

We consider an arc to be valid if we detect a sequence of patches matching the following pattern:

$$m \times \text{no-line}, n \times \text{line}, p \times \text{no-line}$$

where m , n , and p are positive integers representing the number of consecutive patches classified in the given label.

If both arcs are valid, we can estimate the coordinates of two points in the line by taking the position between the first and last patch coordinates in the n -th sequence of each arc. Using these two points and the *2-argument arctangent* function, we can extract the estimated slope value.

If at least one of the arcs is invalid (e.g., the line fades or a patch is misclassified), we keep the default slope value until the end of the run.

A.3.2 Baselines

Two simple baselines are used as references to interpret and analyze the results of the autotuning simulations. The benchmark methodology is identical to the other autotuning procedures, including the repetition over 10 random seeds and 50 random starting points for each stability diagram.

The “*oracle*” baseline uses the exact same exploration logic as the one presented in this article, but the line detection model is replaced by an “*oracle*” that has access to the ground truth labels. Therefore, every patch classification is 100 % accurate, with a maximal confidence score. This baseline allows us to isolate the tuning errors related to the exploration strategy from the ones caused by model misclassification. The results of this baseline corroborate the conclusion that the gallium arsenide (GaAs-QD) and silicon overlapping gate (Si-OG-QD) datasets are more challenging to explore, possibly due to the transition line irregularities and unpredictable shapes.

The “*random*” baseline represents the worst case of exploration, where a pair of coordinates is randomly selected inside the voltage space of the tuned stability diagram. The success rate of this baseline can be interpreted as the average surface area of the one-electron regime in the dataset’s diagrams. A larger surface is expected to be easier to find using an autotuning procedure.

Therefore, these two baselines defined the minimal and maximal tuning success rate and average number of steps expected for each dataset.

TABLE A.3 Autotuning results using “*oracle*” and “*random*” baselines. Variability over 10 random seeds.

Dataset	Baseline	Average step number	Tuning success
Si-SG-QD	Oracle	151	100.0% ± 0.1
	Random	0	11.8% ± 1.5
GaAs-QD	Oracle	94	96.7% ± 0.6
	Random	0	10.1% ± 1.4
Si-OG-QD	Oracle	164	92.0% ± 0.9
	Random	0	5.3% ± 1.0

A.4 Results without cross-validation

The performance loss observed when using the cross-validation method (illustrated in Figure A.4) gives us insight into the effects of the distributional uncertainty on the autotuning success rate. The results of the line detection model trained without cross-validation, presented in Table A.4, show high accuracy for every dataset (above 98 % when using the uncertainty threshold) and a consistently lower rate of instances below the threshold (number of patches classified as “*unknown*”) compared to the model trained with cross-validation (presented in Table 3.1). As shown in Table A.5, the autotuning success rates directly benefit from the model performance gain. This gap is even greater when the dataset contains very different diagrams, such as in the GaAs-QD dataset. Therefore, reducing the distributional uncertainty seems to be a promising way to improve the autotuning success rate, which could be achieved either by covering a boarder range of diagrams in the training set or by reducing the device fabrication variability.

TABLE A.4 Line detection results without cross-validation for each dataset and model. The performances are averaged over 10 runs with different random seeds. The standard deviation of the run performances represents the variability of the methods. Each run covers every diagram of the dataset. The best test accuracy scores of each dataset are highlighted in bold.

Dataset	Model	Accuracy	Accuracy above threshold	Error reduction using threshold	Rate below threshold
Si-SG-QD	BCNN	98.8% ± 0.1	99.8% ± 0.1	79.9% ± 5.1	2.0% ± 0.3
	CNN	98.7% ± 0.1	99.8% ± 0.1	86.4% ± 3.6	2.0% ± 0.2
	FF	96.5% ± 0.7	99.8% ± 0.1	92.9% ± 3.1	10.7% ± 2.3
GaAs-QD	BCNN	94.3% ± 0.9	96.9% ± 1.1	46.5% ± 15.9	8.0% ± 3.1
	CNN	95.2% ± 0.7	98.0% ± 0.9	59.7% ± 14.3	7.5% ± 2.6
	FF	93.9% ± 1.1	97.6% ± 0.8	60.9% ± 10.0	10.5% ± 2.4
Si-OG-QD	BCNN	95.3% ± 0.3	98.4% ± 0.2	65.8% ± 5.8	7.4% ± 1.4
	CNN	93.6% ± 0.4	99.1% ± 0.2	85.2% ± 2.3	11.4% ± 0.9
	FF	94.1% ± 0.3	98.9% ± 0.2	81.0% ± 3.0	11.1% ± 0.7

TABLE A.5 Autotuning results for each dataset and model without cross-validation, with and without using the model uncertainty information provided by the confidence score. The line detection accuracy and tuning success variability are computed over 10 runs with different random seeds. Each run covers every diagram of the dataset. The best tuning success rates of each dataset are highlighted in bold.

Dataset	Model	Line detection accuracy	Uncertainty-based tuning	Average step number	Tuning success
Si-SG-QD	BCNN	98.8% ± 0.1	Yes	166	99.5% ± 0.3
			No	151	92.2% ± 2.7
	CNN	98.7% ± 0.1	Yes	166	99.9% ± 0.2
			No	152	91.2% ± 3.5
	FF	96.5% ± 0.7	Yes	199	71.2% ± 11.7
			No	128	30.7% ± 15.0
GaAs-QD	BCNN	94.3% ± 0.9	Yes	101	85.4% ± 9.9
			No	93	66.2% ± 6.8
	CNN	95.2% ± 0.7	Yes	100	92.2% ± 2.5
			No	95	85.8% ± 5.5
	FF	93.9% ± 1.1	Yes	104	78.0% ± 5.6
			No	92	63.0% ± 3.8
Si-OG-QD	BCNN	95.3% ± 0.3	Yes	184	81.8% ± 2.3
			No	160	70.2% ± 3.5
	CNN	93.6% ± 0.4	Yes	189	84.3% ± 3.0
			No	153	61.7% ± 4.3
	FF	94.1% ± 0.3	Yes	194	85.0% ± 1.1
			No	154	65.5% ± 2.4

CHAPTER B

Appendix: Experimental online quantum dots charge autotuning using neural networks

B.1 Datasets

The dataset [314] is built from nine offline stability diagrams measured during previous experiments on similar QD devices. Each of them was manually annotated by experts with transition lines and charge regime areas (example in Figure 4.1c). The stability diagrams were decomposed into 33,429 patches of 18×18 pixels, then split into three subsets:

- 80 % training (26,743 patches): used to train the CNN to detect transition lines.
- 10 % validation (3343 patches): used after training to select the step corresponding to the best model (shown as a green star in Figure B.1).
- 10 % testing (3343 patches): used to evaluate the line classification accuracy after training.

Each patch is categorized as “*line*” if at least one transition line annotation intersects the 6×6 square at its center (pink squares in Figure B.3a,b). This labeling approach provides more context to the NN while keeping the classification window narrow enough to fit between two transition lines. Since the majority of the surface does not contain a line, only 8.6 % (3887) of the patches are categorized as “*line*”, while the others are classified as “*no-line*”.

B.2 Model training

The detection of transition lines is accomplished using CNNs containing 367,267 free parameters (weights and bias), structured as presented in Table B.1 and trained with the meta-parameters presented in Table B.2. We did not observe overfitting during training, as shown in Figure B.1. Once trained, the model reached satisfactory performance on the test set (example in Figure B.2), where most of the errors were related to ambiguous labels or noise patterns. The Python source code used for training and inference is available on *GitHub*¹ and relies on *PyTorch* [438].

The training of an NN is partially stochastic due to the random initialization of the parameter values and mini-batch sampling during training. To avoid selection bias, in this study, we trained a new CNN using a different random seed for each autotuning run. In a production context, it is possible to train and use only one CNN to tune any device for a given QD technology.

1. Python source code: github.com/3it-inpaqt/dot-calibration-v2

TABLE B.1 Architecture of the convolutional neural networks (CNNs) used to detect transition lines during the experiment. A simplified schematic of the layer structure is represented in Figure 4.1d.

Layer	Configuration
Input	18×18 pixels, 1 channel
Convolution 1	4×4 kernel, 6 channel
Convolution 2	4×4 kernel, 12 channel
Fully connected 1	200 neurons
Fully connected 2	100 neurons
	Binary classification
Output	(“ <i>line</i> ” or “ <i>no-line</i> ”) + uncertainty score

TABLE B.2 Meta-parameters of the convolutional neural networks (CNNs) used for training and inference.

Meta-Parameter	Value
Number of train updates	30,000
Loss function	Binary cross-entropy
Optimizer	Adam [437]
Learning rate	0.001
Dropout	60 %
Batch size	512
Pixel size	4 mV
Confidence threshold	90 %

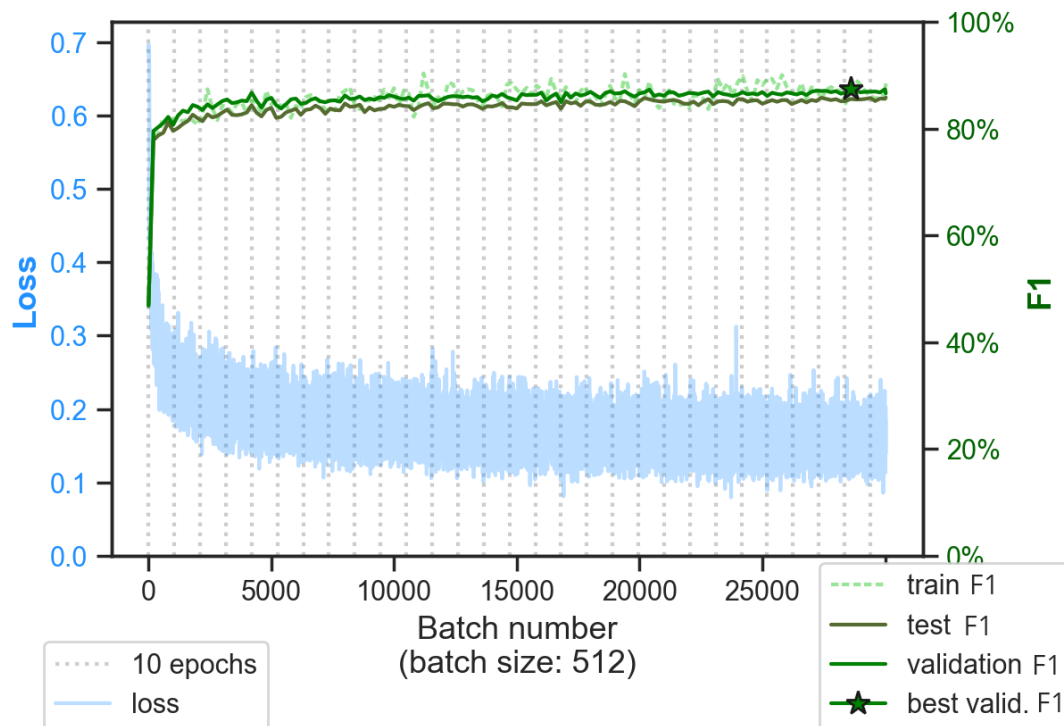


FIGURE B.1 Example of convolutional neural network (CNN) training progress. The F1 score is the harmonic mean of precision and recall. The model performance quickly converges to a maximum, and no overfitting is visible.

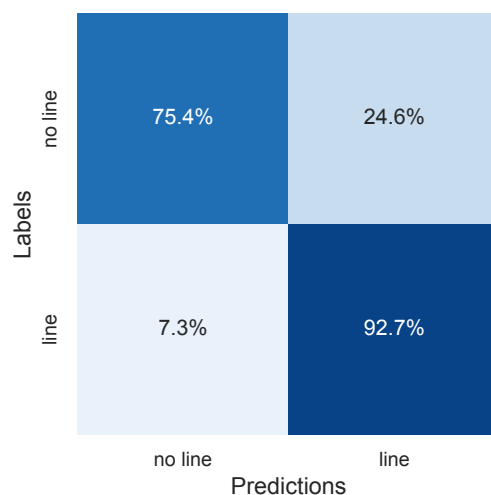


FIGURE B.2 Example of a confusion matrix for a trained convolutional neural network (CNN). On the test set, this model obtained the following scores: accuracy, 93.09 %; precision, 80.88 %; recall, 93.52 %; F1 score, 85.56 %.

B.3 Confidence score

The confidence of model inference can be evaluated using several different methods [191, 193, 307, 323, 330, 331]. However, a previous study [297] has shown that, in the context

of QD autotuning, a simple heuristic score based on the distance [324, 325] between the NN output (denoted as y) and the inferred class value (see Equation (B.1)) is an efficient approximation of the model’s uncertainty. This confidence score is leveraged to increase the tuning success rate with minimal overhead time by optimizing the exploration–exploitation tradeoff [328, 329].

$$\text{Confidence score} = |0.5 - y| \times 2 \quad (\text{B.1})$$

Based on offline tests, we empirically found that triggering the validation sequence if the confidence score was below 90 % provided a good tradeoff between the success rate and exploration time. Therefore, we used this value as a confidence threshold for the 20 online runs. This value could be optimized during future experiments.

B.4 Run statistics

TABLE B.3 Statistics and results of the 20 experimental runs.

Run ID	#Steps	Measurement Time (s)	Processing Time (s)	Total Time (s)	Final Coord. (V, V)	Charge Regime
01	96	6,363	259	6,623	(−0.298, 0.840)	1
02	71	4,717	188	4,905	(0.135, 0.756)	1
03	88	5,860	234	6,094	(0.171, 0.760)	1
04	151	10,421	400	10,822	(−0.278, 0.828)	1
05	112	7,460	295	7,755	(−0.025, 0.796)	1
06	167	11,462	445	11,907	(−0.302, 0.840)	1
07	72	4,786	191	4,977	(−0.298, 1.196)	4+
08	69	4,670	182	4,852	(0.239, 0.760)	1
09	165	11,475	445	11,920	(−0.302, 0.828)	1
10	117	7,792	309	8,101	(0.175, 0.772)	1
11	76	5,090	211	5,301	(0.291, 0.740)	1
12	158	10,751	436	11,188	(0.440, 0.752)	1
13	116	7,694	305	7,998	(0.151, 0.776)	1
14	160	11,138	429	11,567	(−0.302, 0.836)	1
15	80	5,299	211	5,510	(−0.298, 0.840)	1
16	89	5,917	236	6,153	(−0.089, 0.792)	1
17	116	7,790	311	8,101	(−0.085, 0.808)	1
18	62	4,127	166	4,293	(−0.005, 0.784)	1
19	75	5,063	200	5,263	(−0.194, 0.812)	1
20	169	11,737	457	12,195	(−0.302, 0.828)	1

B.5 Patch measurement examples

It was necessary to interpolate the images of the offline stability diagrams before using them to train the CNNs due to the variable sweeping step size used during the original experiments. This preprocessing slightly affected the quality of the offline training by duplicating some pixels (see patch examples in Figure B.3a,b). On the other hand, the patches measured during the online experiment did not suffer from this limitation, as it was possible to define the desired step size in the measurement interface. This led to a better resolution of the patch, with clearer lines (see patch examples in Figure B.3c,d).

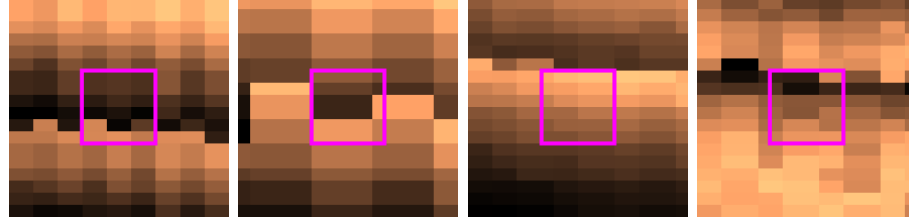
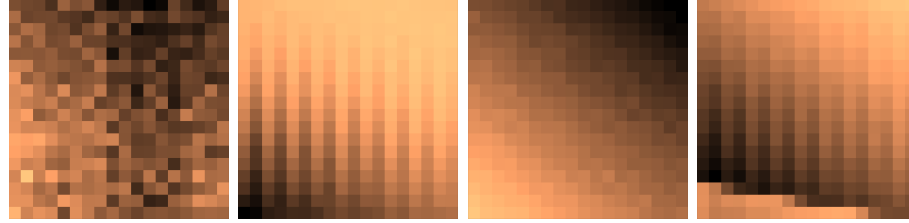
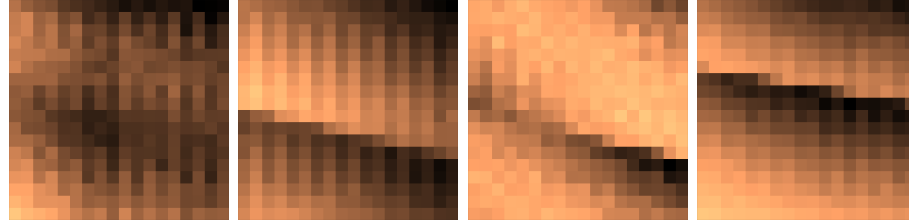
(a) Four examples of offline patches labeled as “*no-line*”.(b) Four examples of offline patches labeled as “*line*”.(c) Four examples of online patches correctly classified as “*no-line*”.(d) Four examples of online patches correctly classified as “*line*”.

FIGURE B.3 Patch samples for each class from the offline training set and the online experiment. **a,b)** The patches are measured using variable step sizes depending on the stability diagram, then interpolated at 4 mV per pixel. A patch is labeled as “*line*” only if a transition line annotation intersects with the detection area (highlighted in pink) at its center. **c,d)** The online patches are measured using a step size of 4 mV per pixel.

CHAPTER C

Appendix: A cryogenic memristive neural decoder for fault-tolerant quantum error correction

C.1 Simulation of quantum stabilizer circuits

Here we provide some background on the simulation of surface codes and the error model used to generate the dataset for the neural decoder’s training and testing (the dataset is available for download [411]). Our experiment is based on the distance-3 rotated surface code. This surface code corresponds to a 17-qubit system in which there are 9 data qubits (circles filled in white in Figure 5.2(a) of the main manuscript) and 8 ancilla qubits (circles filled in black) corresponding to the 8 stabilizer generators $S_1^X, S_2^X, S_3^X, S_4^X, S_1^Z, S_2^Z, S_3^Z$, and S_4^Z .

In our study, we rely on *Stim* [121] to generate the circuits shown in Figure 5.2 required to simulate the rotated surface code, *PyMatching* [444] for decoding with minimum-weight perfect matching in order to benchmark the relative performance of the recurrent neural network (RNN) decoder, and a custom-built glue code to integrate *Stim* and *PyMatching*. While writing this paper, the need for the glue code was eliminated due to recent updates in *PyMatching2*.

We simulate the memory- X rotated surface code, where preparation and final measurements are done in the X -basis. In addition, we conduct this simulation under circuit level noise, which means the following:

- with a noise probability of p , each two-qubit gate is followed by a two-qubit Pauli error drawn uniformly and independently from $\{I, X, Y, Z\}^{\otimes 2} \setminus \{I \otimes I\}$;
- with a probability of $2p/3$, the preparation of the $|0\rangle$ state is replaced by the state $|1\rangle = X|0\rangle$. Similarly, with a probability of $2p/3$, the preparation of the $|+\rangle$ state is replaced by the state $|-\rangle = Z|+\rangle$;
- with a probability of $2p/3$, any single-qubit measurement has its outcome flipped; and
- with a probability of p , each idling-qubit location is followed by a Pauli error drawn uniformly and independently from the stabilizer set $\{X, Y, Z\}$.

A *Stim* circuit is first initialized with a probability p for a given code distance and a number of rounds. This circuit is then repeatedly called to generate samples. The steps taken during circuit generation for performing the parity-check rounds shown in Figure 5.2(b) and (c) of the main manuscript are detailed as follows:

1. At the onset of the quantum error correction (QEC) cycle, the data qubits are initialized in the $|+\rangle$ state (for memory- X experiments), while ancilla qubits are set

- to the $|0\rangle$ state. These are replaced with the states $|+\rangle$ and $|1\rangle$, respectively, with a probability of $2p/3$, accounting for preparation errors.
2. Before the beginning of every parity check round, each data qubit is depolarized with a probability of p due to idling.
 3. To perform the parity-check circuits depicted in Figure 5.2(b) and (c), Hadamard gates are applied to the X -syndrome qubits, converting the $|0\rangle$ states to $|+\rangle$ states and the $|1\rangle$ states to $|-\rangle$ states, followed by single-qubit depolarizing noise with a probability of p .
 4. Next, four CNOT cycles are executed, and each cycle is succeeded by two-qubit depolarizing noise with a probability of p , following the configuration outlined in Figure 5.2(b) and (c).
 5. Hadamard gates are applied to the X -syndrome ancilla qubits, followed by depolarizing noise with a probability of p .
 6. With a measurement error probability of $2p/3$, all ancilla qubits are measured in the Z -basis, after which they are reset to the $|0\rangle$ state.
 7. To simulate preparation errors for the next QEC round, bit flips are applied with a probability of $2p/3$ on the ancilla qubits, along with idling, and steps 2 to 7 are repeated to conduct repeated syndrome measurement cycles.
 8. Finally, the data qubits are measured in the X -basis with a measurement error a probability of $2p/3$. The data qubit measured implicitly define one final round of computational-basis-type stabilizers (in this case X stabilizers) inferred via the corresponding parity-check matrix. Since this is a classical process and is not subject to the circuit-level noise defined during a typical stabilizer round, the syndromes extracted in this round are also referred to as “perfect syndromes”. The final state of the encoded logical qubit is then determined.

The first stabilizer measurement round is the encoding round, which creates an entangled logical qubit. In the absence of an error, the encoding results in a logical $|+\rangle$ state since the data qubits are initialized in a logical $|+\rangle$ state. Measuring a $|-\rangle$ logical state in the end thus indicates a logical error, and the goal of a decoder is to predict such events.

To generate samples for training the RNN, we first separate the X and Z syndromes, focusing only on the X syndromes. Next, we divide the X -syndrome measurements into separate time steps, corresponding to each syndrome extraction round. The training label represents the logical *bit-flip* value, where 0 indicates that the logical state corresponding to the given syndromes has remained unchanged, and 1 indicates that the logical state has flipped. The training, validation, and testing samples are generated using noise probabilities between $p = 0.00001$ and $p = 0.01$, along with 3 rounds of stabilizer measurement and then a reset.

A key preprocessing step before feeding measurements to the decoder for analysis is to take the differences between consecutive measurement outcomes. Syndromes indicate the presence or absence, as well as the type, of errors that have occurred to the physical qubits. The RNN outputs a prediction of 0 or 1, representing the absence or presence of a logical

error, respectively. The corresponding recovery operator is thus given by $(Z_L)^y$, where y is the binary output of the RNN.

C.2 Other hardware non-idealities

C.2.1 Analog-to-digital and digital-to-analog converter range and resolution

To simulate the behaviour of analog-to-digital converters (ADCs) and digital-to-analog converters (DACs) during inference on the crossbar arrays of memristors, a rounding function is applied,

$$f(x) = \begin{cases} -b & x < -b \\ \text{round}\left(\frac{x}{2b}n\right) \frac{2b}{n} & -b \leq x \leq b \\ b & x > b \end{cases}, \quad (\text{C.1})$$

where x represents the ADC and DAC input, its range is given by $[-b, b]$, and n is the number of quantization steps, for example, 256 for 8-bit resolution. The round function rounds the floating-point value to the closest integer. Based on the typical matrix-vector multiplication (MVM) output values, we fix the ADC range to $b = 6$. The DAC range is fixed to $b = 1$ to match with the sigmoid output range. In ADCs and DACs, there is a trade-off between resolution and acquisition frequency, that is, a lower resolution yields a higher acquisition frequency. An application where inference time needs to be minimized, such as with the neural decoder, requires a high acquisition frequency. From Figure C.1, we can see that an 8-bit resolution is appropriate to reach the highest acquisition frequency without substantially degrading the RNN decoder’s fidelity. Therefore, we simulate ADC and DAC discretization during the testing of a memristive neural decoder (MND), but no re-training strategy seems necessary to maintain the decoder’s fidelity.

C.2.2 Reading variability

Every analog electronic component induces a small reading variability. ADCs and DACs [445]; differential amplifiers; and memristors [446] involved in MVM operations are no exception. Based on experimental characterizations [374], we estimate that the dominant source of reading variability should come from memristors. To reduce the computational complexity, we simplify the simulation by considering every source of reading variability as one Gaussian random value added at the output of each row of memristors. As a conservative estimate, we chose a Gaussian distribution with a standard deviation equal to 1 % of the maximum logical value (defined as b in the previous section).

The effect of different reading noise on the neural decoder’s fidelity is illustrated in Figure C.2. This simulation result suggests that a 1 % reading noise is not detrimental to our decoding application. However the performance drops quickly when the value increases above this limit. We can observe that the random dropconnect applied with our hardware-aware memristive neural decoder (hardware-aware (HWA)-MND) re-training method also increases the robustness of the model against high reading variability. The device-specific memristive neural decoder (device-specific (DS)-MND) re-training method is more sensitive to perturbations that are not related to the stuck-at fault devices; this is visible by the faster fidelity drop in Figure C.2 when the output noise is increased in comparison to the

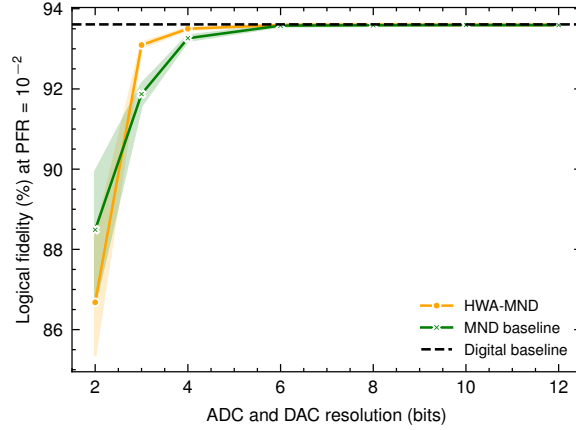


FIGURE C.1 Impact of the analog-to-digital converter (ADC) and digital-to-analog converter (DAC) resolution on the recurrent neural network (RNN) decoder’s test fidelity. The physical fault rate (PFR) is set to 10^{-2} . The test fidelity is obtained without including reading variability or defective devices, in order to assess the impact of resolution alone. At an 8-bit resolution, that is, 256 quantization steps between the boundary values -6 and 6 , the fidelity approaches the digital baseline fidelity (using a 32-bit floating point accuracy). The error range is obtained based on 10 independent training rounds.

HWA-MND method. This reading variability study shows that realistic noise values do not impact the performance enough to require a re-training method addressing specifically this issue. However, it is worth noting that a dropconnect strategy, applied in addition to the DS-MND method, could mitigate the negative effect of reading variability.

C.3 Experiments on TiO_x -based resistive memory devices

The studied TiO_x -based resistive memory devices are characterized by a *Keysight Technologies B1500A* semiconductor device parameter analyzer that has a 200 million samples/second waveform generator / fast measurement unit (WGFMU) using a *Lake Shore CPX-VF* probe station at room temperature. For all measurements, the electrodes at the bottom are grounded and the signals are applied to the electrodes at the top.

C.3.1 Conductance state programming

In our study, the resistance of memristors is programmed using a closed-loop read–write–verify algorithm [374, 415]. This algorithm allows the programming of a memristor to a target resistance by applying successive read and write pulses. Initially, a device’s resistance is read using a read pulse (having a 0.2 V amplitude and a 1 μs pulse width). If this measured resistance is larger than the target within a 1 % tolerance, a positive write pulse is applied (~ 1 V amplitude and 200 ns pulse width); if it is lower, a negative pulse is applied. A read pulse is then applied to check the new resistance state, which starts a new cycle. The pulse amplitude is increased after each consecutive pulse of the same polarity by a constant step in order to converge to the target resistance. Using this read–

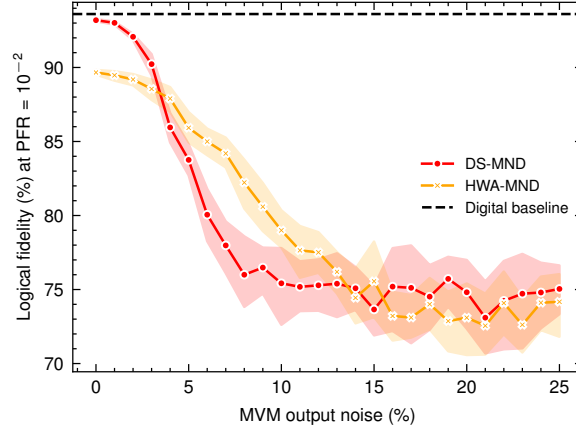


FIGURE C.2 Impact of the matrix–vector multiplication (MVM) output noise on the recurrent neural network (RNN) decoder’s test fidelity. The physical fault rate (PFR) is set to 10^{-2} . A random Gaussian value is added to the output of each memristor row to account for the expected reading noise produced by the electronic components and the memristors. The percentage is relative to the maximum logical output value (set to 6 in our simulations). All other non-idealities are kept as defined in Table C.2.

write–verify algorithm for various target levels of resistance, we can perform multilevel programming as shown in Figure 5.4(a) in the main manuscript.

C.3.2 Programming variability characterization

The programming variability model is based on experimental measurement conducted on TiO_x -based resistive memory devices. As shown in Figure 5.4(a) in the main manuscript, multilevel programming is performed from the highest conductance states (HCSs) to the lowest conductance states (LCSs). To assess the cycle-to-cycle programming variability (shown in Figure 5.4(b)), we successively perform multilevel programming from the LCSs to the HCSs 10 times. For each state and at each multilevel programming pass, the conductance is read 20 times and the mean value is kept. This process is repeated for 10 memristors to include device-to-device variability in the programming error model. The histogram of the mean programmed values for a conductance state is fitted by a Gaussian distribution and its standard deviation is extracted (also depicted in Figure 5.4(b)). The fit of the standard deviation σ corresponds to the programming variability which depends on the conductance of the programmed state.

C.3.3 Conductance state retention

The retention measurements presented in Figure C.3 are performed at room temperature for 8 hours for 5 conductance states. After the programming of a conductance state, a 1 ms-wide pulse with a 200 mV amplitude is applied every 5 minutes to measure the retention. The retention of the conductance states is measured successively. The 5 states demonstrate non-volatility over the eight-hour retention test. The conductance drift after 8 hours with respect to the initial conductance state can be fitted by a second-order polynomial function. Considering the small drift for this memristor technology, we have not considered this behaviour in the simulations presented in this paper.

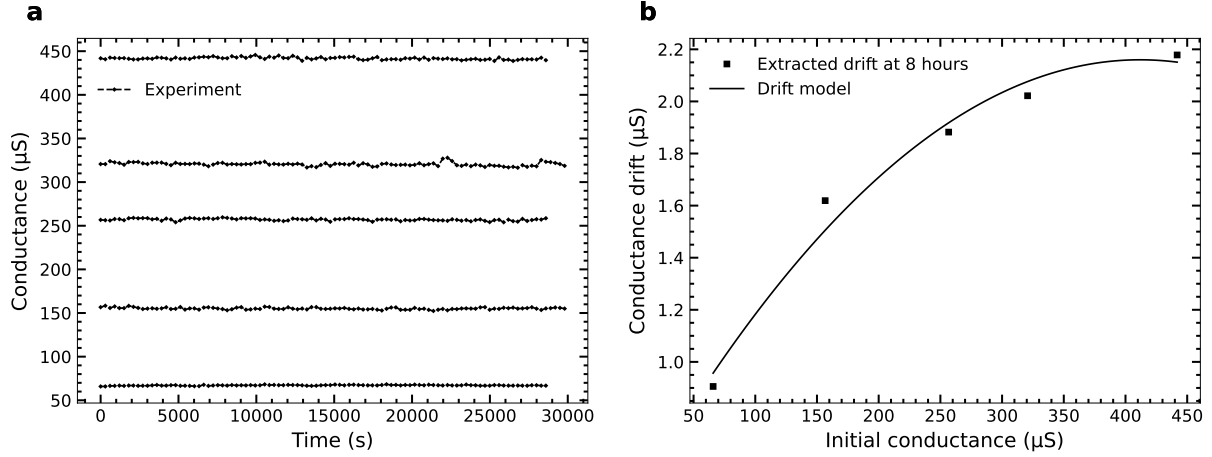


FIGURE C.3 TiO_x -based resistive memory device retention study. **a)** Retention of 5 conductance states between $65\ \mu\text{S}$ and $440\ \mu\text{S}$. No conductance drift is noticeable. **b)** Conductance drift of the 5 conductance states after eight-hour-retention measurements. The drift with respect to the initial conductance is fitted by a second-order polynomial function.

C.4 Neural network training methods

C.4.1 Implementation of training and inference

The training of the RNN always begins with a classical deep learning [58] optimization process, using *PyTorch* [438] with no consideration for hardware non-idealities (see Figure 5.5(a) in the main manuscript). The metaparameters used during this step are given in Table C.1. In the case of hardware-aware memristive neural decoder (HWA-MND) and device-specific memristive neural decoder (DS-MND) re-training, the RNN parameters (weights and biases) are loaded from an earlier digital training, and one additional training epoch is run to adapt the model to newly introduced hardware constraints. Every other metaparameter remains unchanged during re-training.

Metaparameters	Values
Input size	4 rounds of 4 binary syndromes
Hidden layer size	32
Activation function	ReLU
Batch size	16
Loss	Cross-entropy
Optimizer	Adam [437]
Learning rate	0.001
Training epochs	4 (+1 for HWA and DS methods)

TABLE C.1 Model and training metaparameters.

The RNN re-training and inference simulations are performed using the *IBM Analog Hardware Acceleration Kit* [422] v0.9.0. This Python library is used to simulate the computations executed on a crossbar array of memristors and the relevant peripheral circuit at

the behavioural level. We extend the original toolkit to consider the specific TiO_x -based resistive memory devices used in our work. The non-idealities we discuss in the hardware characterization section of the main manuscript are implemented in the simulator. For instance, the statistical programming variability is calibrated on our TiO_x -based resistive memory devices.

C.4.2 Re-training methods

For HWA re-training, the main manuscript focuses on introducing a dropconnect training method during the re-training of the MND to mitigate the effect of defective devices in the chip. However, there are other training methods that could improve performance. One such method consists in the addition of noise on weights during forward passes of training steps to account for the programming variability of the characterized TiO_x -based resistive memory devices. It has been shown that setting the training weight variability approximately equal to the inference weight variability is optimal [402]. However, in our case, we observed training instability issues with this method, which led to an unexpected gradient explosion in a fraction of the trained RNNs. Therefore, this method did not benefit the overall performance.

The study of various non-idealities highlighted that only stuck-at fault devices have a critical effect on the performance of decoding. We attribute this to the fact that TiO_x -based resistive memory devices exhibit a low programming variability (in comparison to phase-change memory devices reported in [402]); therefore, injecting noise during training does not significantly improve fidelity. Also, an ADC and DAC of 8-bit resolution is sufficient for maintaining a fidelity close to the digital baseline, as explained in Appendix C.2. Thus, the input and output quantization during training does not allow for greater fidelity. Moreover, it is likely that the binary output of the model helps mitigate most noise-related issues since only a perturbation near the threshold value would impact the final binary output of RNNs.

Another limitation of memristors is their limited resistance range, which constrains the parameters' values. This limitation can easily be tackled to improve test accuracy by using a method called weight clipping to trim the parameters' values in a specific range during re-training [402]. It allows a certain degree of control over the weight distribution to make the programming of weights to hardware easier. It is performed by clipping the weights of a layer, after each backward pass in the training steps, within the range $[-\alpha\sigma; \alpha\sigma]$, where α represents the scale of the clipping, and σ is the standard deviation of the weight distribution of that layer. Based on our empirical tests, the exact value of σ does not appear to strongly affect the decoder's performance. Thus, we decided to fix $\sigma = 2.5$ for all MND test simulations.

The decoding fidelity globally improves when the dropconnect method is introduced during re-training compared to the case where no dropconnect is employed (shown using a blue curve with the label "0%" in Figure C.4). It also appears that an RNN decoder trained with a high dropconnect rate will have a lower fidelity if no device is stuck, but its performance will decrease more slowly when the rate of stuck-at fault devices increases. Therefore, a good trade-off between fidelity and robustness seems to be a dropconnect rate equal to the number of blocked parameters (19%) given the expected stuck-at fault

Hardware constraints and non-idealities	Simulated values
Weight clipping	2.5σ of the Gaussian distribution of each layer's weights' values
DAC resolution	8 bits
ADC resolution	8 bits
DAC max logical value	1
ADC max logical value	6
MVM output noise	Gaussian noise added with a standard deviation equal to 1 % of the maximum logical value (6)
Max memristor resistance	$15,000 \Omega$
Min memristor resistance	$5,000 \Omega$
Memristor stuck-at-fault rate	10 %
Programming noise's polynomial fit	$y = -x^3(2.40\text{e-}5) + x^2(1.15\text{e-}2) - x(7.07\text{e-}2)$

TABLE C.2 Default simulated values of the hardware constraints and non-idealities in our study. These values are used to simulate the memristor-based recurrent neural network (RNN) with the *IBM Analog Hardware Acceleration Kit* [422].

device percentage (10 %). Since a parameter is encoded by two devices (for positive and negative values), the probability of a parameter being blocked (i.e., set to 0 %) is equal to the probability of at least one of the pair of devices being stuck.

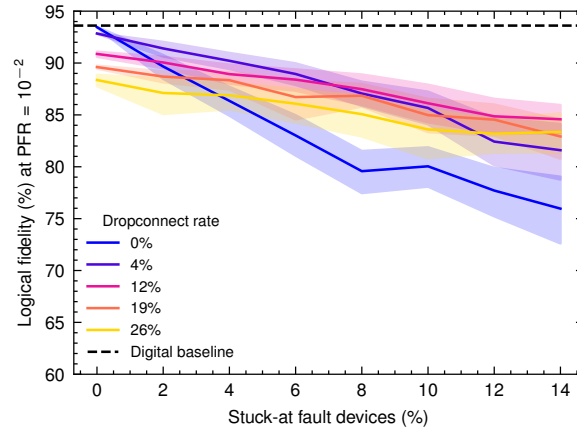


FIGURE C.4 Impact of the dropconnect re-training method for various stuck-at-fault rates. Decoding fidelity at a physical fault rate (PFR) of 10^{-2} , for different dropconnect probabilities during hardware-aware (HWA) memristive neural decoder (MND) re-training, as a function of the percentage of stuck-at fault devices during inference. The shaded error regions represent the 95 % confidence interval over 10 random seeds.

LISTE DES RÉFÉRENCES

- [1] C. E. Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27(3) :379–423, 1948. URL:<https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>.
- [2] A. M. Turing. On computable numbers, with an application to the entscheidungs problem. *Proceedings of the London Mathematical Society Series/2 (42)*, p. 230–42, 1936. DOI:10.1112/plms/s2-42.1.230.
- [3] J. von Neumann. First draft of a report on the EDVAC. *IEEE Annals of the History of Computing*, 15(4):27–75, 1993. DOI:10.1109/85.238389.
- [4] A. N. Kolmogorov. On tables of random numbers. *Theoretical Computer Science*, 207(2):387–395, nov. 1998. DOI:10.1016/s0304-3975(98)00075-9.
- [5] P. J. Baldwin, W. C. Evans, D. L. Guevara, J. B. Berner, E. L. Weir et A. C. McCarthy. NASA’s evolving Ka-band network capabilities to meet mission demand. Conference Paper 20220013867, NASA, 27th Ka and Broadband Communications Conference (Ka), sept. 2022. URL:<https://ntrs.nasa.gov/citations/20220013867>.
- [6] A. Alekseev, A. Kiryanov, A. Klimentov, T. Korchuganova, V. Mitsyn, D. Oley-nik, A. Smirnov, S. Smirnov et A. Zarochentsev. Scientific data lake for high luminosity LHC project and other data-intensive particle and astro-particle physics experiments. *Journal of Physics: Conference Series*, 1690(1):012166, déc. 2020. DOI:10.1088/1742-6596/1690/1/012166.
- [7] CERN. Storage - What data to record? URL:<https://home.cern/science/computing/storage>. Visité le 20-01-2025.
- [8] M. Naeem, T. Jamal, J. Diaz-Martinez, S. A. Butt, N. Montesano, M. I. Tariq, E. De-la Hoz-Franco et E. De-La-Hoz-Valdiris. *Trends and Future Perspective Challenges in Big Data*, p. 309–325. Springer Singapore, nov. 2021. DOI:10.1007/978-981-16-5036-9_30.
- [9] A. Adadi. A survey on data-efficient algorithms in big data era. *Journal of Big Data*, 8(1), jan. 2021. DOI:10.1186/s40537-021-00419-9.
- [10] K. Bourzac. Fixing AI’s energy crisis. *Nature*, oct. 2024. DOI:10.1038/d41586-024-03408-z.
- [11] A. de Vries. The growing energy footprint of artificial intelligence. *Joule*, 7(10):2191–2194, oct. 2023. DOI:10.1016/j.joule.2023.09.004.
- [12] K. Calvin, D. Dasgupta, G. Krinner, A. Mukherji, P. W. Thorne, C. Trisos, J. Romero, P. Aldunce, K. Barrett, G. Blanco, W. W. Cheung, S. Connors, F. Denton, A. Diongue-Niang, D. Dodman, M. Garschagen, O. Geden, B. Hayward, C. Jones, F. Jotzo, T. Krug, R. Lasco, Y.-Y. Lee, V. Masson-Delmotte, M. Meinshausen *et al.* *IPCC, 2023: Climate Change 2023: Synthesis Report*. Intergovernmental Panel on Climate Change, juil. 2023. DOI:10.59327/ipcc/ar6-9789291691647.
- [13] C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. Aga, J. Huang, C. Bai, M. Gschwind, A. Gupta, M. Ott, A. Melnikov, S. Candido, D. Brooks, G. Chauhan, B. Lee, H.-H. Lee, B. Akyildiz, M. Balandat,

- J. Spisak, R. Jain, M. Rabbat et K. Hazelwood. Sustainable AI: Environmental implications, challenges and opportunities. In D. Marculescu, Y. Chi et C. Wu, éd., *Proceedings of Machine Learning and Systems*, vol. 4, p. 795–813, 2022. URL:<https://proceedings.mlsys.org/paper%5Ffiles/paper/2022/file/462211f67c7d858f663355eff93b745e-Paper.pdf>.
- [14] E. Çam, Z. Hungerford, N. Schoch, F. P. Miranda et C. D. Y. nez de León. World energy outlook 2024. Rapport technique, IEA, Paris, oct. 2024. URL:<https://www.iea.org/reports/world-energy-outlook-2024>.
- [15] R. Landauer. The physical nature of information. *Physics Letters A*, 217(4-5):188–193, juil. 1996. DOI:10.1016/0375-9601(96)00453-7.
- [16] B. J. Copeland. The modern history of computing. In E. N. Zalta, éd., *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Winter 2020 édn, 2020. URL:<https://plato.stanford.edu/entries/computing-history/>.
- [17] D. Swade. *The difference engine*. Penguin Books, oct. 2002. DOI:10.5555/940732.
- [18] B. Randell. From analytical engine to electronic digital computer: The contributions of Ludgate, Torres, and Bush. *IEEE Annals of the History of Computing*, 4(4):327–341, oct. 1982. DOI:10.1109/mahc.1982.10042.
- [19] J. Small. General-purpose electronic analog computing: 1945-1965. *IEEE Annals of the History of Computing*, 15(2):8–18, 1993. DOI:10.1109/85.207740.
- [20] NobelPrize.org. The Nobel prize in physics 1956, 1956. URL:<https://www.nobelprize.org/prizes/physics/1956/summary/>. Visité le 20-01-2025.
- [21] M. Tanenbaum, L. B. Valdes, E. Buehler et N. B. Hannay. Silicon n-p-n grown junction transistors. *Journal of Applied Physics*, 26(6):686–692, juin 1955. DOI:10.1063/1.1722071.
- [22] G. E. Moore. Cramming more components onto integrated circuits. *Proceedings of the IEEE*, 86(1):82–85, avr. 1965. DOI:10.1109/n-ssc.2006.4785860.
- [23] T. Pröttel. Semiconductor market forecast spring 2024, juin 2024. URL:<https://www.wsts.org/esraCMS/extension/media/f/WST/6558/WSTS%5FFC-Release-2024%5F05.pdf>.
- [24] X. Zhang, Y. Zhang et G. Li. Strategic inventory in semi-conductor supply chains under industrial disruption. *International Journal of Production Economics*, 272:109254, juin 2024. DOI:10.1016/j.ijpe.2024.109254.
- [25] R. Beibit, O. Fatahi Valilai et H. Wicaksono. Estimating the COVID-19 impact on the semiconductor shortage in the european automotive industry using supervised machine learning. In *Proceedings of the 2023 10th International Conference on Industrial Engineering and Applications*, vol. 4 de *Icica-eu 2023*, p. 302–308. ACM, jan. 2023. DOI:10.1145/3587889.3588215.
- [26] J. Voas, N. Kshetri et J. F. DeFranco. Scarcity and global insecurity: The semiconductor shortage. *IT Professional*, 23(5):78–82, sept. 2021. DOI:10.1109/mitp.2021.3105248.
- [27] W. Mohammad, A. Elomri et L. Kerbache. The global semiconductor chip shortage: Causes, implications, and potential remedies. *IFAC-PapersOnLine*, 55(10):476–483, 2022. DOI:10.1016/j.ifacol.2022.09.439.

- [28] L. B. Kish. End of Moore's law: Thermal (noise) death of integration in micro and nano electronics. *Physics Letters A*, 305(3–4):144–149, déc. 2002. DOI:10.1016/s0375-9601(02)01365-8.
 - [29] I. Tuomi. The lives and death of Moore's law. *First Monday*, nov. 2002. DOI:10.5210/fm.v7i11.1000.
 - [30] J. R. Powell. The quantum limit to Moore's law. *Proceedings of the IEEE*, 96(8):1247–1248, août 2008. DOI:10.1109/jproc.2008.925411.
 - [31] K. Rupp et S. Selberherr. The economic limit to Moore's law. *IEEE Transactions on Semiconductor Manufacturing*, 24(1):1–4, fév. 2011. DOI:10.1109/tsm.2010.2089811.
 - [32] N. Zhang. Moore's Law is dead, long live Moore's Law! *ArXiv*, 2022. DOI:10.48550/arxiv.2205.15011. Preprint.
 - [33] K. S. Kim, J. Kwon, H. Ryu, C. Kim, H. Kim, E.-K. Lee, D. Lee, S. Seo, N. M. Han, J. M. Suh, J. Kim, M.-K. Song, S. Lee, M. Seol et J. Kim. The future of two-dimensional semiconductors beyond Moore's law. *Nature Nanotechnology*, 19(7):895–906, juil. 2024. DOI:10.1038/s41565-024-01695-1.
 - [34] M. Pégny. Les deux formes de la thèse de Church-Turing et l'épistémologie du calcul. *Philosophia Scientiae*, 16–3:39–67, nov. 2012. DOI:10.4000/philosophiascientiae.769.
 - [35] Y. Hu, Y. Liu et Z. Liu. A survey on convolutional neural network accelerators: GPU, FPGA and ASIC. In *2022 14th International Conference on Computer Research and Development (ICCRD)*. IEEE, jan. 2022. DOI:10.1109/iccrd54409.2022.9730377.
 - [36] C. Bobda, J. M. Mbongue, P. Chow, M. Ewais, N. Tarafdar, J. C. Vega, K. Eguro, D. Koch, S. Handagala, M. Leeser, M. Herbordt, H. Shahzad, P. Hofste, B. Ringlein, J. Szefer, A. Sanaullah et R. Tessier. The future of FPGA acceleration in datacenters and the cloud. *ACM Transactions on Reconfigurable Technology and Systems*, 15(3):1–42, fév. 2022. DOI:10.1145/3506713.
 - [37] K. Vipin et S. A. Fahmy. FPGA dynamic and partial reconfiguration: A survey of architectures, methods, and applications. *ACM Computing Surveys*, 51(4):1–39, juil. 2018. DOI:10.1145/3193827.
 - [38] E. Monmasson et M. Cirstea. FPGA design methodology for industrial control systems – A review. *IEEE Transactions on Industrial Electronics*, 54(4):1824–1842, août 2007. DOI:10.1109/tie.2007.898281.
 - [39] M. Taufer, N. Ganesan et S. Patel. GPU-enabled macromolecular simulation: Challenges and opportunities. *Computing in Science & Engineering*, 15(1):56–65, jan. 2013. DOI:10.1109/mcse.2012.42.
 - [40] T. Xue, S. Liao, Z. Gan, C. Park, X. Xie, W. K. Liu et J. Cao. JAX-FEM: A differentiable GPU-accelerated 3D finite element solver for automatic inverse design and mechanistic data science. *Computer Physics Communications*, 291:108802, oct. 2023. DOI:10.1016/j.cpc.2023.108802.
 - [41] D. Yuen, L. Wang, X. Chi, L. Johnsson, W. Ge et Y. Shi. *GPU Solutions to Multi-scale Problems in Science and Engineering*. Lecture Notes in Earth System Sciences. Springer Berlin Heidelberg, 2013. URL:<https://books.google.ca/books?id=aT8%5FAAAAQBAJ>.
-

-
- [42] S. B. Pandya, H. A. Sanghvi, R. H. Patel et A. S. Pandya. GPU and FPGA based deployment of blockchain for cryptocurrency – A systematic review. In *2022 International Conference on Computational Intelligence and Sustainable Engineering Solutions (CISES)*, vol. 95, p. 18–25. IEEE, mai 2022. DOI:10.1109/cises54857.2022.9844407.
 - [43] S. G. Iyer et A. Dipakumar Pawar. GPU and CPU accelerated mining of cryptocurrencies and their financial analysis. In *2018 2nd International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)*, 2018 2nd International Conference on. IEEE, août 2018. DOI:10.1109/i-smac.2018.8653733.
 - [44] X. Zou, S. Xu, X. Chen, L. Yan et Y. Han. Breaking the von Neumann bottleneck: Architecture-level processing-in-memory technology. *Science China Information Sciences*, 64(6), avr. 2021. DOI:10.1007/s11432-020-3227-1.
 - [45] A. Lu, J. Lee, T.-H. Kim, M. A. U. Karim, R. S. Park, H. Simka et S. Yu. High-speed emerging memories for AI hardware accelerators. *Nature Reviews Electrical Engineering*, 1(1):24–34, jan. 2024. DOI:10.1038/s44287-023-00002-9.
 - [46] Y. I. Manin. *Vychislimoe i nevychislimoe (Computable and Noncomputable)*. Radio, 1980.
 - [47] R. P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6–7):467–488, juin 1982. DOI:10.1007/bf02650179.
 - [48] P. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, SFCS-94. IEEE Comput. Soc. Press, 1994. DOI:10.1109/sfcs.1994.365700.
 - [49] A. K. Fedorov et M. S. Gelfand. Towards practical applications in quantum computational biology. *Nature Computational Science*, 1(2):114–119, fév. 2021. DOI:10.1038/s43588-021-00024-z.
 - [50] Y. Cao, J. Romero et A. Aspuru-Guzik. Potential of quantum computing for drug discovery. *IBM Journal of Research and Development*, 62(6):6:1–6:20, nov. 2018. DOI:10.1147/jrd.2018.2888987.
 - [51] R. Orús, S. Mugel et E. Lizaso. Quantum computing for finance: Overview and prospects. *Reviews in Physics*, 4:100028, nov. 2019. DOI:10.1016/j.revip.2019.100028.
 - [52] C. E. Leiserson, N. C. Thompson, J. S. Emer, B. C. Kuszmaul, B. W. Lampson, D. Sanchez et T. B. Schardl. There’s plenty of room at the Top: What will drive computer performance after Moore’s law? *Science*, 368(6495), juin 2020. DOI:10.1126/science.aam9744.
 - [53] C. A. R. Hoare. Algorithm 63, Partition; Algorithm 64, Quicksort; Algorithm 65, Find. *Communications of the ACM*, 4(7):321, juil. 1961. DOI:10.1145/366622.366642.
 - [54] A. M. Turing. Computing machinery and intelligence. *Mind*, LIX(236):433–460, oct. 1950. DOI:10.1093/mind/lix.236.433.
 - [55] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. DOI:10.1037/h0042519.
-

- [56] Y. LeCun, L. Bottou, Y. Bengio et P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. DOI:10.1109/5.726791.
 - [57] S.-i. Amari. Backpropagation and stochastic gradient descent method. *Neurocomputing*, 5(4–5):185–196, juin 1993. DOI:10.1016/0925-2312(93)90006-o.
 - [58] Y. LeCun, Y. Bengio et G. Hinton. Deep learning. *Nature*, 521(7553):436–444, mai 2015. DOI:10.1038/nature14539.
 - [59] J. S. Dramschi. *70 years of machine learning in geoscience in review*, p. 1–55. Elsevier, 2020. DOI:10.1016/bs.agph.2020.08.002.
 - [60] A. Mehonic et A. J. Kenyon. Brain-inspired computing needs a master plan. *Nature*, 604(7905):255–260, avr. 2022. DOI:10.1038/s41586-021-04362-w.
 - [61] S. Mittal et S. Vaishay. A survey of techniques for optimizing deep learning on GPUs. *Journal of Systems Architecture*, 99:101635, oct. 2019. DOI:10.1016/j.sysarc.2019.101635.
 - [62] E. AI. Data on notable AI models, 2024. URL:<https://epochai.org/data/notable-ai-models>. Visité le 20-01-2025.
 - [63] M. A. Nielsen et I. L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, juin 2012. DOI:10.1017/cbo9780511976667.
 - [64] L. M. K. Vandersypen, M. Steffen, G. Breyta, C. S. Yannoni, M. H. Sherwood et I. L. Chuang. Experimental realization of Shor’s quantum factoring algorithm using nuclear magnetic resonance. *Nature*, 414(6866):883–887, déc. 2001. DOI:10.1038/414883a.
 - [65] Y. A. Pashkin, T. Yamamoto, O. Astafiev, Y. Nakamura, D. V. Averin et J. S. Tsai. Quantum oscillations in two coupled charge qubits. *Nature*, 421(6925):823–826, fév. 2003. DOI:10.1038/nature01365.
 - [66] P. J. J. O’Malley, R. Babbush, I. D. Kivlichan, J. Romero, J. R. McClean, R. Barrens, J. Kelly, P. Roushan, A. Tranter, N. Ding, B. Campbell, Y. Chen, Z. Chen, B. Chiaro, A. Dunsworth, A. G. Fowler, E. Jeffrey, E. Lucero, A. Megrant, J. Y. Mutus, M. Neeley, C. Neill, C. Quintana, D. Sank, A. Vainsencher *et al.* Scalable quantum simulation of molecular energies. *Physical Review X*, 6(3), juil. 2016. DOI:10.1103/physrevx.6.031007.
 - [67] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barrens, R. Biswas, S. Boixo, F. G. S. L. Brandao, D. A. Buell, B. Burkett, Y. Chen, Z. Chen, B. Chiaro, R. Collins, W. Courtney, A. Dunsworth, E. Farhi, B. Foxen, A. Fowler, C. Gidney, M. Giustina, R. Graff, K. Guerin, S. Habegger *et al.* Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779):505–510, oct. 2019. DOI:10.1038/s41586-019-1666-5.
 - [68] Y. Alexeev, M. H. Farag, T. L. Patti, M. E. Wolf, N. Ares, A. Aspuru-Guzik, S. C. Benjamin, Z. Cai, Z. Chandani, F. Fedele, N. Harrigan, J.-S. Kim, E. Kyoseva, J. G. Lietz, T. Lubowe, A. McCaskey, R. G. Melko, K. Nakaji, A. Peruzzo, S. Stanwyck, N. M. Tubman, H. Wang et T. Costa. Artificial intelligence for quantum computing. *ArXiv*, 2024. DOI:10.48550/arxiv.2411.09131. Preprint.
-

- [69] J. P. Zwolak et J. M. Taylor. Colloquium : Advances in automation of quantum dot devices control. *Reviews of Modern Physics*, 95(1), fév. 2023. DOI:10.1103/revmodphys.95.011006.
- [70] R. W. Hamming. Error detecting and error correcting codes. *Bell System Technical Journal*, 29(2):147–160, avr. 1950. DOI:10.1002/j.1538-7305.1950.tb00463.x.
- [71] D. Bull, S. Das, K. Shivashankar, G. S. Dasika, K. Flautner et D. Blaauw. A power-efficient 32 bit ARM processor using timing-error detection and correction for transient-error tolerance and adaptation to PVT variation. *IEEE Journal of Solid-State Circuits*, 46(1):18–31, jan. 2011. DOI:10.1109/jssc.2010.2079410.
- [72] A. Malkov, D. Vasiounin et O. Semenov. A review of PVT compensation circuits for advanced CMOS technologies. *Circuits and Systems*, 02(03):162–169, 2011. DOI:10.4236/cs.2011.23024.
- [73] R. A. Wolf. Quantum error correction for kids. *ArXiv*, 2024. DOI:10.48550/arxiv.2405.06795. Preprint.
- [74] S. J. Devitt, W. J. Munro et K. Nemoto. Quantum error correction for beginners. *Reports on Progress in Physics*, 76(7):076001, juin 2013. DOI:10.1088/0034-4885/76/7/076001.
- [75] J. Roffe. Quantum error correction: An introductory guide. *Contemporary Physics*, 60(3):226–245, juil. 2019. DOI:10.1080/00107514.2019.1667078.
- [76] F. Battistel, C. Chamberland, K. Johar, R. W. J. Overwater, F. Sebastiano, L. Skoric, Y. Ueno et M. Usman. Real-time decoding for fault-tolerant quantum computing: Progress, challenges and outlook. *Nano Futures*, 7(3):032003, août 2023. DOI:10.1088/2399-1984/aceba6.
- [77] D. Ellis et N. Morgan. Size matters: An empirical study of neural network training for large vocabulary continuous speech recognition. In *1999 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings. ICASSP99 (Cat. No.99CH36258)*, p. 1013–1016 vol.2. IEEE, 1999. DOI:10.1109/icassp.1999.759875.
- [78] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu et D. Amodei. Scaling laws for neural language models. *ArXiv*, 2020. DOI:10.48550/arxiv.2001.08361. Preprint.
- [79] Y. Bahri, E. Dyer, J. Kaplan, J. Lee et U. Sharma. Explaining neural scaling laws. *Proceedings of the National Academy of Sciences*, 121(27), juin 2024. DOI:10.1073/pnas.2311878121.
- [80] M. Horowitz. 1.1 Computing’s energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. IEEE, fév. 2014. DOI:10.1109/isscc.2014.6757323.
- [81] L. Chua. Memristor-the missing circuit element. *IEEE Transactions on Circuit Theory*, 18(5):507–519, sept. 1971. DOI:10.1109/tct.1971.1083337.
- [82] L. Chua. Resistance switching memories are memristors. *Applied Physics A*, 102(4):765–783, jan. 2011. DOI:10.1007/s00339-011-6264-9.
- [83] D. B. Strukov, G. S. Snider, D. R. Stewart et R. S. Williams. The missing memristor found. *Nature*, 453(7191):80–83, mai 2008. DOI:10.1038/nature06932.

- [84] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess *et al.* Language models are few-shot learners. *ArXiv*, 2020. DOI:10.48550/arxiv.2005.14165. Preprint.
 - [85] R. Mao, G. Chen, X. Zhang, F. Guerin et E. Cambria. GPTEval: A survey on assessments of ChatGPT and GPT-4. *ArXiv*, 2023. DOI:10.48550/arxiv.2308.12488. Preprint.
 - [86] J. P. Zwolak, J. M. Taylor, R. W. Andrews, J. Benson, G. W. Bryant, D. Buterakos, A. Chatterjee, S. Das Sarma, M. A. Eriksson, E. Greplová, M. J. Gullans, F. Hader, T. J. Kovach, P. S. Mundada, M. Ramsey, T. Rasmussen, B. Severin, A. Sigillito, B. Undseth et B. Weber. Data needs and challenges for quantum dot devices automation. *NPJ Quantum Information*, 10(1), oct. 2024. DOI:10.1038/s41534-024-00878-x.
 - [87] S. Leijnen et F. v. Veen. The neural network zoo. In *IS4SI 2019 Summit*, vol. 65 de *Is4si 2019*, p. 9. MDPI, mai 2020. DOI:10.3390/proceedings2020047009.
 - [88] T.-Y. S. Park, J.-H. Kihm, J. Woo, C. Park, W. Y. Lee, M. P. Smith, D. A. T. Harper, F. Young, A. T. Nielsen et J. Vinther. Brain and eyes of kerygmachela reveal protocerebral ancestry of the panarthropod head. *Nature Communications*, 9(1), mars 2018. DOI:10.1038/s41467-018-03464-w.
 - [89] J. M. Gambetta, J. M. Chow et M. Steffen. Building logical qubits in a superconducting quantum computing system. *NPJ Quantum Information*, 3(1), jan. 2017. DOI:10.1038/s41534-016-0004-0.
 - [90] K. S. Lee, Y. P. Tan, L. H. Nguyen, R. P. Budoyo, K. H. Park, C. Hufnagel, Y. S. Yap, N. Møbjerg, V. Vedral, T. Paterek et R. Dumke. Entanglement in a qubit-qubit-tardigrade system. *New Journal of Physics*, 24(12):123024, déc. 2022. DOI:10.1088/1367-2630/aca81f.
 - [91] A. J. Daley, I. Bloch, C. Kokail, S. Flannigan, N. Pearson, M. Troyer et P. Zoller. Practical quantum advantage in quantum simulation. *Nature*, 607(7920):667–676, juil. 2022. DOI:10.1038/s41586-022-04940-6.
 - [92] L. K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing - STOC '96*, Stoc '96. ACM Press, 1996. DOI:10.1145/237814.237866.
 - [93] R. Au-Yeung, N. Chancellor et P. Halffmann. NP-hard but no longer hard to solve? Using quantum computing to tackle optimization problems. *Frontiers in Quantum Science and Technology*, 2, fév. 2023. DOI:10.3389/frqst.2023.1128576.
 - [94] L. Bittel et M. Kliesch. Training variational quantum algorithms is NP-hard. *Physical Review Letters*, 127(12), sept. 2021. DOI:10.1103/physrevlett.127.120502.
 - [95] D. Castelvecchi. The AI–quantum computing mash-up: Will it revolutionize science? *Nature*, jan. 2024. DOI:10.1038/d41586-023-04007-0.
 - [96] G. Burkard, T. D. Ladd, A. Pan, J. M. Nichol et J. R. Petta. Semiconductor spin qubits. *Reviews of Modern Physics*, 95(2), juin 2023. DOI:10.1103/revmodphys.95.025003.
-

-
- [97] S. Sapmaz, P. Jarillo-Herrero, L. P. Kouwenhoven et H. S. J. v. d. Zant. Quantum dots in carbon nanotubes. *Semiconductor Science and Technology*, 21(11):S52–s63, oct. 2006. DOI:10.1088/0268-1242/21/11/s08.
- [98] F. Kuemmeth, S. Ilani, D. C. Ralph et P. L. McEuen. Coupling of spin and orbital motion of electrons in carbon nanotubes. *Nature*, 452(7186):448–452, mars 2008. DOI:10.1038/nature06822.
- [99] B. E. Kane. A silicon-based nuclear spin quantum computer. *Nature*, 393(6681):133–137, mai 1998. DOI:10.1038/30156.
- [100] D. Loss et D. P. DiVincenzo. Quantum computation with quantum dots. *Physical Review A*, 57(1):120–126, jan. 1998. DOI:10.1103/physreva.57.120.
- [101] M. Veldhorst, C. H. Yang, J. C. C. Hwang, W. Huang, J. P. Dehollain, J. T. Muhonen, S. Simmons, A. Laucht, F. E. Hudson, K. M. Itoh, A. Morello et A. S. Dzurak. A two-qubit logic gate in silicon. *Nature*, 526(7573):410–414, oct. 2015. DOI:10.1038/nature15263.
- [102] T. F. Watson, S. G. J. Philips, E. Kawakami, D. R. Ward, P. Scarlino, M. Veldhorst, D. E. Savage, M. G. Lagally, M. Friesen, S. N. Coppersmith, M. A. Eriksson et L. M. K. Vandersypen. A programmable two-qubit quantum processor in silicon. *Nature*, 555(7698):633–637, fév. 2018. DOI:10.1038/nature25766.
- [103] R. Maurand, X. Jehl, D. Kotekar-Patil, A. Corna, H. Bohuslavskyi, R. Laviéville, L. Hutin, S. Barraud, M. Vinet, M. Sanquer et S. De Franceschi. A CMOS silicon spin qubit. *Nature Communications*, 7(1), nov. 2016. DOI:10.1038/ncomms13575.
- [104] N. I. D. Stuyck, R. Li, C. Godfrin, A. Elsayed, S. Kubicek, J. Jussot, B. T. Chan, F. A. Mohiyaddin, M. Shehata, G. Simion, Y. Canvel, L. Goux, M. Heyns, B. Goveoreanu et I. P. Radu. Uniform spin qubit devices with tunable coupling in an all-silicon 300 mm integrated process. In *2021 Symposium on VLSI Circuits*. IEEE, juin 2021. DOI:10.23919/vlsicircuits52068.2021.9492427.
- [105] L. M. K. Vandersypen, H. Bluhm, J. S. Clarke, A. S. Dzurak, R. Ishihara, A. Morello, D. J. Reilly, L. R. Schreiber et M. Veldhorst. Interfacing spin qubits in quantum dots and donors—hot, dense, and coherent. *NPJ Quantum Information*, 3(1), sept. 2017. DOI:10.1038/s41534-017-0038-y.
- [106] A. M. Tyryshkin, S. Tojo, J. J. L. Morton, H. Riemann, N. V. Abrosimov, P. Becker, H.-J. Pohl, T. Schenkel, M. L. W. Thewalt, K. M. Itoh et S. A. Lyon. Electron spin coherence exceeding seconds in high-purity silicon. *Nature Materials*, 11(2):143–147, déc. 2011. DOI:10.1038/nmat3182.
- [107] M. Veldhorst, J. C. C. Hwang, C. H. Yang, A. W. Leenstra, B. de Ronde, J. P. Dehollain, J. T. Muhonen, F. E. Hudson, K. M. Itoh, A. Morello et A. S. Dzurak. An addressable quantum dot qubit with fault-tolerant control-fidelity. *Nature Nanotechnology*, 9(12):981–985, oct. 2014. DOI:10.1038/nnano.2014.216.
- [108] L. Petit, M. Russ, G. H. G. J. Eenink, W. I. L. Lawrie, J. S. Clarke, L. M. K. Vandersypen et M. Veldhorst. Design and integration of single-qubit rotations and two-qubit gates in silicon above one Kelvin. *Communications Materials*, 3(1), nov. 2022. DOI:10.1038/s43246-022-00304-9.
- [109] K. Takeda, J. Kamioka, T. Otsuka, J. Yoneda, T. Nakajima, M. R. Delbecq, S. Amaha, G. Allison, T. Kodera, S. Oda et S. Tarucha. A fault-tolerant address-
-

- sable spin qubit in a natural silicon quantum dot. *Science Advances*, 2(8), août 2016. DOI:10.1126/sciadv.1600694.
- [110] J. Yoneda, K. Takeda, T. Otsuka, T. Nakajima, M. R. Delbecq, G. Allison, T. Honda, T. Koderu, S. Oda, Y. Hoshi, N. Usami, K. M. Itoh et S. Tarucha. A quantum-dot spin qubit with coherence limited by charge noise and fidelity higher than 99.9%. *Nature Nanotechnology*, 13(2):102–106, déc. 2017. DOI:10.1038/s41565-017-0014-x.
- [111] A. R. Mills, C. R. Guinn, M. J. Gullans, A. J. Sigillito, M. M. Feldman, E. Nielsen et J. R. Petta. Two-qubit silicon quantum processor with operation fidelity exceeding 99%. *Science Advances*, 8(14), avr. 2022. DOI:10.1126/sciadv.abn5130.
- [112] A. Noiri, K. Takeda, T. Nakajima, T. Kobayashi, A. Sammak, G. Scappucci et S. Tarucha. A shuttling-based two-qubit logic gate for linking distant silicon quantum processors. *Nature Communications*, 13(1), sept. 2022. DOI:10.1038/s41467-022-33453-z.
- [113] X. Xue, M. Russ, N. Samkharadze, B. Undseth, A. Sammak, G. Scappucci et L. M. K. Vandersypen. Quantum logic with spin qubits crossing the surface code threshold. *Nature*, 601(7893):343–347, jan. 2022. DOI:10.1038/s41586-021-04273-w.
- [114] P. Steinacker, N. D. Stuyck, W. H. Lim, T. Tanttu, M. Feng, A. Nickl, S. Serrano, M. Candido, J. D. Cifuentes, F. E. Hudson, K. W. Chan, S. Kubicek, J. Jussot, Y. Canvel, S. Beyne, Y. Shimura, R. Loo, C. Godfrin, B. Raes, S. Baudot, D. Wan, A. Laucht, C. H. Yang, A. Saraiva, C. C. Escott *et al.* A 300 mm foundry silicon spin qubit unit cell exceeding 99% fidelity in all operations. *ArXiv*, 2024. DOI:10.48550/arxiv.2410.15590. Preprint.
- [115] B. Cheng, X.-H. Deng, X. Gu, Y. He, G. Hu, P. Huang, J. Li, B.-C. Lin, D. Lu, Y. Lu, C. Qiu, H. Wang, T. Xin, S. Yu, M.-H. Yung, J. Zeng, S. Zhang, Y. Zhong, X. Peng, F. Nori et D. Yu. Noisy intermediate-scale quantum computers. *Frontiers of Physics*, 18(2), mars 2023. DOI:10.1007/s11467-022-1249-z.
- [116] J. Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, août 2018. DOI:10.22331/q-2018-08-06-79.
- [117] A. K. Fedorov, N. Gisin, S. M. Beloussov et A. I. Lvovsky. Quantum computing at the quantum advantage threshold: A down-to-business review. *ArXiv*, 2022. DOI:10.48550/arxiv.2203.17181. Preprint.
- [118] J. Preskill. Quantum computing and the entanglement frontier. *ArXiv*, 2012. DOI:10.48550/arxiv.1203.5813. Preprint.
- [119] S. Boixo, S. V. Isakov, V. N. Smelyanskiy, R. Babbush, N. Ding, Z. Jiang, M. J. Bremner, J. M. Martinis et H. Neven. Characterizing quantum supremacy in near-term devices. *Nature Physics*, 14(6):595–600, avr. 2018. DOI:10.1038/s41567-018-0124-x.
- [120] E. Pednault, D. Maslov, J. Gunnels et J. Gambetta. On “quantum supremacy”, oct. 2019. URL:<https://www.ibm.com/quantum/blog/on-quantum-supremacy>. Visité le 20-01-2025.
- [121] C. Gidney et M. Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, avr. 2021. DOI:10.22331/q-2021-04-15-433.
-

-
- [122] L. Gaudreau, A. Kam, G. Granger, S. A. Studenikin, P. Zawadzki et A. S. Sachrajda. A tunable few electron triple quantum dot. *Applied Physics Letters*, 95(19), nov. 2009. DOI:10.1063/1.3258663.
- [123] E. D. Commins. Electron spin and its history. *Annual Review of Nuclear and Particle Science*, 62(1):133–157, nov. 2012. DOI:10.1146/annurev-nucl-102711-094908.
- [124] F. R. Waugh, M. J. Berry, C. H. Crouch, C. Livermore, D. J. Mar, R. M. Westervelt, K. L. Campman et A. C. Gossard. Measuring interactions between tunnel-coupled quantum dots. *Physical Review B*, 53(3):1413–1420, jan. 1996. DOI:10.1103/physrevb.53.1413.
- [125] F. Borsoi, N. W. Hendrickx, V. John, M. Meyer, S. Motz, F. van Riggelen, A. Sammak, S. L. de Snoo, G. Scappucci et M. Veldhorst. Shared control of a 16 semiconductor quantum dot crossbar array. *Nature Nanotechnology*, 19(1):21–27, août 2023. DOI:10.1038/s41565-023-01491-3.
- [126] A. Zubchenko, D. Middlebrooks, T. Rasmussen, L. Lausen, F. Kuemmeth, A. Chatterjee et J. P. Zwolak. Autonomous bootstrapping of quantum dot devices. *ArXiv*, 2024. DOI:10.48550/arxiv.2407.20061. Preprint.
- [127] S. S. Kalantre, J. P. Zwolak, S. Ragole, X. Wu, N. M. Zimmerman, M. D. Stewart et J. M. Taylor. Machine learning techniques for state recognition and auto-tuning in quantum dots. *NPJ Quantum Information*, 5(1), jan. 2019. DOI:10.1038/s41534-018-0118-7.
- [128] J. P. Zwolak, T. McJunkin, S. S. Kalantre, J. Dodson, E. MacQuarrie, D. Savage, M. Lagally, S. Coppersmith, M. A. Eriksson et J. M. Taylor. Autotuning of double-dot devices in situ with machine learning. *Physical Review Applied*, 13(3), mars 2020. DOI:10.1103/physrevapplied.13.034075.
- [129] H. Moon, D. T. Lennon, J. Kirkpatrick, N. M. van Esbroeck, L. C. Camenzind, L. Yu, F. Vigneau, D. M. Zumbühl, G. A. D. Briggs, M. A. Osborne, D. Sejdinovic, E. A. Laird et N. Ares. Machine learning enables completely automatic tuning of a quantum device faster than human experts. *Nature Communications*, 11(1), août 2020. DOI:10.1038/s41467-020-17835-9.
- [130] Y. Muto, T. Nakaso, M. Shinozaki, T. Aizawa, T. Kitada, T. Nakajima, M. R. Delbecq, J. Yoneda, K. Takeda, A. Noiri, A. Ludwig, A. D. Wieck, S. Tarucha, A. Kanemura, M. Shiga et T. Otsuka. Visual explanations of machine learning model estimating charge states in quantum dots. *APL Machine Learning*, 2(2), avr. 2024. DOI:10.1063/5.0193621.
- [131] B. Severin, D. T. Lennon, L. C. Camenzind, F. Vigneau, F. Fedele, D. Jirovec, A. Ballabio, D. Chrastina, G. Isella, M. de Kruijf, M. J. Carballido, S. Svab, A. V. Kuhlmann, S. Geyer, F. N. M. Froning, H. Moon, M. A. Osborne, D. Sejdinovic, G. Katsaros, D. M. Zumbühl, G. A. D. Briggs et N. Ares. Cross-architecture tuning of silicon and SiGe-based quantum devices using machine learning. *Scientific Reports*, 14(1), juil. 2024. DOI:10.1038/s41598-024-67787-z.
- [132] J. Ziegler, F. Luthi, M. Ramsey, F. Borjans, G. Zheng et J. P. Zwolak. Tuning arrays with rays: Physics-informed tuning of quantum dot charge states. *Physical Review Applied*, 20(3), sept. 2023. DOI:10.1103/physrevapplied.20.034067.
-

-
- [133] A. Paurevic. *Automated Tuning and Optimal Control of Spin Qubits in Quantum Dot Devices*. Thèse de doctorat, University of Waterloo, 2024. URL:<https://hdl.handle.net/10012/21021>.
- [134] R. Durrer, B. Kratochwil, J. Koski, A. Landig, C. Reichl, W. Wegscheider, T. Ihn et E. Greplova. Automated tuning of double quantum dots into specific charge states using neural networks. *Physical Review Applied*, 13(5), mai 2020. DOI:10.1103/physrevapplied.13.054019.
- [135] M. Lapointe-Major, O. Germain, J. Camirand Lemyre, D. Lachance-Quirion, S. Rochette, F. Camirand Lemyre et M. Pioro-Ladrière. Algorithm for automated tuning of a quantum dot into the single-electron regime. *Physical Review B*, 102(8), août 2020. DOI:10.1103/physrevb.102.085301.
- [136] S. Czischek, V. Yon, M.-A. Genest, M.-A. Roux, S. Rochette, J. Camirand Lemyre, M. Moras, M. Pioro-Ladrière, D. Drouin, Y. Beilliard et R. G. Melko. Miniaturizing neural networks for charge state autotuning in quantum dots. *Machine Learning: Science and Technology*, 3(1):015001, nov. 2021. DOI:10.1088/2632-2153/ac34db.
- [137] S. Daraeizadeh, S. P. Premaratne et A. Y. Matsuura. Designing high-fidelity multi-qubit gates for semiconductor quantum dots through deep reinforcement learning. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, p. 30–36. IEEE, oct. 2020. DOI:10.1109/qce49297.2020.00014.
- [138] J. Schuff, M. J. Carballido, M. Kotzagiannidis, J. C. Calvo, M. Caselli, J. Rawling, D. L. Craig, B. van Straaten, B. Severin, F. Fedele, S. Svab, P. C. Kwon, R. S. Eggi, T. Patlatiuk, N. Korda, D. Zumbühl et N. Ares. Fully autonomous tuning of a spin qubit. *ArXiv*, 2024. DOI:10.48550/arxiv.2402.03931. Preprint.
- [139] A. Voulodimos, N. Doulamis, A. Doulamis et E. Protopapadakis. Deep learning for computer vision: A brief review. *Computational Intelligence and Neuroscience*, 2018:1–13, 2018. DOI:10.1155/2018/7068349.
- [140] J. Darulová, M. Troyer et M. C. Cassidy. Evaluation of synthetic and experimental training data in supervised machine learning applied to charge-state detection of quantum dots. *Machine Learning: Science and Technology*, 2(4):045023, sept. 2021. DOI:10.1088/2632-2153/ac104c.
- [141] L. Ding et A. Goshtasby. On the canny edge detector. *Pattern Recognition*, 34(3):721–725, mars 2001. DOI:10.1016/s0031-3203(00)00023-6.
- [142] C. M. Bishop. *Pattern recognition and machine learning*. Information Science and Statistics. Springer, 1 éd., août 2006. URL:<https://core.ac.uk/download/pdf/44152170.pdf>.
- [143] P. Mukhopadhyay et B. B. Chaudhuri. A survey of Hough transform. *Pattern Recognition*, 48(3):993–1010, mars 2015. DOI:10.1016/j.patcog.2014.08.027.
- [144] R. Sun, T. Lei, Q. Chen, Z. Wang, X. Du, W. Zhao et A. K. Nandi. Survey of image edge detection. *Frontiers in Signal Processing*, 2, mars 2022. DOI:10.3389/frsip.2022.826967.
- [145] J. P. Zwolak, T. McJunkin, S. S. Kalantre, S. F. Neyens, E. MacQuarrie, M. A. Eriksson et J. M. Taylor. Ray-based framework for state identification in quantum dot devices. *PRX Quantum*, 2(2), juin 2021. DOI:10.1103/prxquantum.2.020335.
-

- [146] A. Morvan, B. Villalonga, X. Mi, S. Mandrà, A. Bengtsson, P. V. Klimov, Z. Chen, S. Hong, C. Erickson, I. K. Drozdov, J. Chau, G. Laun, R. Movassagh, A. Asfaw, L. T. A. N. Brandão, R. Peralta, D. Abanin, R. Acharya, R. Allen, T. I. Andersen, K. Anderson, M. Ansmann, F. Arute, K. Arya, J. Atalaya *et al.* Phase transitions in random circuit sampling. *Nature*, 634(8033):328–333, oct. 2024. DOI:10.1038/s41586-024-07998-6.
- [147] W. K. Wootters et W. H. Zurek. A single quantum cannot be cloned. *Nature*, 299(5886):802–803, oct. 1982. DOI:10.1038/299802a0.
- [148] D. Dieks. Communication by EPR devices. *Physics Letters A*, 92(6):271–272, nov. 1982. DOI:10.1016/0375-9601(82)90084-6.
- [149] J. Preskill. Quantum computation and quantum information: Lecture notes for physics 229, 1998. URL:<http://theory.caltech.edu/~preskill/ph229/>. Chapter 1: section 1.8 - Visité le 20-01-2025.
- [150] P. W. Shor. Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4):R2493–r2496, oct. 1995. DOI:10.1103/physreva.52.r2493.
- [151] D. Gottesman. Stabilizer codes and quantum error correction. *ArXiv*, 1997. DOI:10.48550/arxiv.quant-ph/9705052. Preprint.
- [152] M. H. Abobeih, Y. Wang, J. Randall, S. J. H. Loenen, C. E. Bradley, M. Markham, D. J. Twitchen, B. M. Terhal et T. H. Taminiau. Fault-tolerant operation of a logical qubit in a diamond quantum processor. *Nature*, 606(7916):884–889, mai 2022. DOI:10.1038/s41586-022-04819-6.
- [153] A. Grimm, N. E. Frattini, S. Puri, S. O. Mundhada, S. Touzard, M. Mirrahimi, S. M. Girvin, S. Shankar et M. H. Devoret. Stabilization and operation of a Kerr-cat qubit. *Nature*, 584(7820):205–209, août 2020. DOI:10.1038/s41586-020-2587-z.
- [154] N. Sundaresan, T. J. Yoder, Y. Kim, M. Li, E. H. Chen, G. Harper, T. Thorbeck, A. W. Cross, A. D. Córcoles et M. Takita. Demonstrating multi-round subsystem quantum error correction using matching and maximum likelihood decoders. *Nature Communications*, 14(1), mai 2023. DOI:10.1038/s41467-023-38247-5.
- [155] C. Ryan-Anderson, J. G. Bohnet, K. Lee, D. Gresh, A. Hankin, J. P. Gaebler, D. Francois, A. Chernoguzov, D. Lucchetti, N. C. Brown, T. M. Gatterman, S. K. Halit, K. Gilmore, J. A. Gerber, B. Neyenhuis, D. Hayes et R. P. Stutz. Realization of real-time fault-tolerant quantum error correction. *Physical Review X*, 11(4), déc. 2021. DOI:10.1103/physrevx.11.041058.
- [156] L. Egan, D. M. Debroy, C. Noel, A. Risinger, D. Zhu, D. Biswas, M. Newman, M. Li, K. R. Brown, M. Cetina et C. Monroe. Fault-tolerant control of an error-corrected qubit. *Nature*, 598(7880):281–286, oct. 2021. DOI:10.1038/s41586-021-03928-y.
- [157] L. Postler, S. Heußen, I. Pogorelov, M. Rispler, T. Feldker, M. Meth, C. D. Marciniak, R. Stricker, M. Ringbauer, R. Blatt, P. Schindler, M. Müller et T. Monz. Demonstration of fault-tolerant universal quantum gate operations. *Nature*, 605(7911):675–680, mai 2022. DOI:10.1038/s41586-022-04721-1.
- [158] R. Acharya, I. Aleiner, R. Allen, T. I. Andersen, M. Ansmann, F. Arute, K. Arya, A. Asfaw, J. Atalaya, R. Babbush, D. Bacon, J. C. Bardin, J. Basso, A. Bengtsson, S. Boixo, G. Bortoli, A. Bourassa, J. Bovaird, L. Brill, M. Broughton, B. B. Buckley, D. A. Buell, T. Burger, B. Burkett, N. Bushnell *et al.* Suppressing quantum

- errors by scaling a surface code logical qubit. *Nature*, 614(7949):676–681, fév. 2023. DOI:10.1038/s41586-022-05434-1.
- [159] S. J. Griffiths et D. E. Browne. Union-find quantum decoding without union-find. *ArXiv*, 2023. DOI:10.48550/arxiv.2306.09767. Preprint.
- [160] N. Delfosse et N. H. Nickerson. Almost-linear time decoding algorithm for topological codes. *Quantum*, 5:595, déc. 2021. DOI:10.22331/q-2021-12-02-595.
- [161] Y. Wu, N. Liyanage et L. Zhong. An interpretation of union-find decoder on weighted graphs. *ArXiv*, 2022. DOI:10.48550/arxiv.2211.03288. Preprint.
- [162] C. Chamberland, A. Kubica, T. J. Yoder et G. Zhu. Triangular color codes on trivalent graphs with flag qubits. *New Journal of Physics*, 22(2):023019, fév. 2020. DOI:10.1088/1367-2630/ab68fd.
- [163] N. Liyanage, Y. Wu, A. Deters et L. Zhong. Scalable quantum error correction for surface codes using FPGA. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, p. 916–927. IEEE, sept. 2023. DOI:10.1109/qce57702.2023.00106.
- [164] H. Liao, D. S. Wang, I. Sitdikov, C. Salcedo, A. Seif et Z. K. Mineev. Machine learning for practical quantum error mitigation. *Nature Machine Intelligence*, 6(12):1478–1486, nov. 2024. DOI:10.1038/s42256-024-00927-2.
- [165] C. Chamberland et P. Ronagh. Deep neural decoders for near term fault-tolerant experiments. *Quantum Science and Technology*, 3(4):044002, juil. 2018. DOI:10.1088/2058-9565/aad1f7.
- [166] P. Baireuther, T. E. O’Brien, B. Tarasinski et C. W. J. Beenakker. Machine-learning-assisted correction of correlated qubit errors in a topological code. *Quantum*, 2:48, 2018. DOI:10.22331/q-2018-01-29-48.
- [167] D. J. Reilly. Challenges in scaling-up the control interface of a quantum computer. In *2019 IEEE International Electron Devices Meeting (IEDM)*. IEEE, déc. 2019. DOI:10.1109/iedm19573.2019.8993497.
- [168] J. C. Bardin, T. White, M. Giustina, K. J. Satzinger, K. Arya, P. Roushan, B. Chiaro, J. Kelly, Z. Chen, B. Burkett, Y. Chen, E. Jeffrey, A. Dunsworth, A. Fowler, B. Foxen, C. Gidney, R. Graff, P. Klimov, J. Mutus, M. J. McEwen, M. Neeley, C. J. Neill, E. Lucero, C. Quintana, A. Vainsencher *et al.* Design and characterization of a 28-nm bulk-CMOS cryogenic quantum controller dissipating less than 2mW at 3K. *IEEE Journal of Solid-State Circuits*, 54(11):3043–3060, nov. 2019. DOI:10.1109/jssc.2019.2937234.
- [169] L. Geck, A. Kruth, H. Bluhm, S. v. Waasen et S. Heinen. Control electronics for semiconductor spin qubits. *Quantum Science and Technology*, 5(1):015004, déc. 2019. DOI:10.1088/2058-9565/ab5e07.
- [170] X. Xue, B. Patra, J. P. G. van Dijk, N. Samkharadze, S. Subramanian, A. Corna, B. Paquelet Wuetz, C. Jeon, F. Sheikh, E. Juarez-Hernandez, B. P. Esparza, H. Ramapurawala, B. Carlton, S. Ravikumar, C. Nieva, S. Kim, H.-J. Lee, A. Sammak, G. Scappucci, M. Veldhorst, F. Sebastiano, M. Babaie, S. Pellerano, E. Charbon et L. M. K. Vandersypen. CMOS-based cryogenic control of silicon quantum circuits. *Nature*, 593(7858):205–210, mai 2021. DOI:10.1038/s41586-021-03469-4.
-

-
- [171] P.-A. Mouny. *Développement d'électronique froide à base de memristors pour la mise à l'échelle des calculateurs quantiques à base de boîtes quantiques*. Thèse de doctorat, Université de Sherbrooke (Québec, Canada), juin 2024. URL:<https://hal.science/tel-04676762>.
 - [172] S. Legg et M. Hutter. Universal intelligence: A definition of machine intelligence. *Minds and Machines*, 17(4):391–444, nov. 2007. DOI:10.1007/s11023-007-9079-x.
 - [173] D. Jannai, A. Meron, B. Lenz, Y. Levine et Y. Shoham. Human or not? A gamified approach to the Turing test. *ArXiv*, 2023. DOI:10.48550/arxiv.2305.20010. Preprint.
 - [174] C. Biever. ChatGPT broke the Turing test – The race is on for new ways to assess AI. *Nature*, 619(7971):686–689, juil. 2023. DOI:10.1038/d41586-023-02361-7.
 - [175] A. Srivastava, A. Rastogi, A. Rao, A. A. M. Shueb, A. Abid, A. Fisch, A. R. Brown, A. Santoro, A. Gupta, A. Garriga-Alonso, A. Kluska, A. Lewkowycz, A. Agarwal, A. Power, A. Ray, A. Warstadt, A. W. Kocurek, A. Safaya, A. Tazarv, A. Xiang, A. Parrish, A. Nie, A. Hussain, A. Askell, A. Dsouza *et al.* Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *ArXiv*, 2022. DOI:10.48550/arxiv.2206.04615. Preprint.
 - [176] M. Pandey, M. Fernandez, F. Gentile, O. Isayev, A. Tropsha, A. C. Stern et A. Cherkasov. The transformational role of GPU computing and deep learning in drug discovery. *Nature Machine Intelligence*, 4(3):211–221, mars 2022. DOI:10.1038/s42256-022-00463-x.
 - [177] W. McCulloch et W. Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, déc. 1943. DOI:10.1007/bf02478259.
 - [178] G. Piccinini. The first computational theory of mind and brain: A close look at McCulloch and Pitts's “logical calculus of ideas immanent in nervous activity”. *Synthese*, 141(2):175–215, août 2004. DOI:10.1023/b:synt.0000043018.52445.3e.
 - [179] K. Hornik, M. Stinchcombe et H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, jan. 1989. DOI:10.1016/0893-6080(89)90020-8.
 - [180] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, déc. 1989. DOI:10.1007/bf02551274.
 - [181] F. Informatik, Y. Bengio, P. Frasconi et J. Schmidhuber. Gradient flow in recurrent nets: The difficulty of learning long-term dependencies. *A Field Guide to Dynamical Recurrent Neural Networks*, 03 2003. URL:<https://www.bioinf.jku.at/publications/older/ch7.pdf>.
 - [182] S. R. Dubey, S. K. Singh et B. B. Chaudhuri. Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 503:92–108, sept. 2022. DOI:10.1016/j.neucom.2022.06.111.
 - [183] A. Byerly, T. Kalganova et I. Dear. No routing needed between capsules. *ArXiv*, 2020. DOI:10.48550/arxiv.2001.09136. Preprint.
 - [184] M. Auli, M. Galley, C. Quirk et G. Zweig. Joint language and translation modeling with recurrent neural networks. *In Proc. of EMNLP*, oct.
-

2013. URL:<https://www.microsoft.com/en-us/research/publication/joint-language-and-translation-modeling-with-recurrent-neural-networks/>.
- [185] J. L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, mars 1990. DOI:10.1207/s15516709cog1402_1.
- [186] P. Rodriguez, J. Wiles et J. L. Elman. A recurrent neural network that learns to count. *Connection Science*, 11(1):5–40, mars 1999. DOI:10.1080/095400999116340.
- [187] A. Tyagi et A. Abraham. *Recurrent Neural Networks: Concepts and Applications*. CRC Press, 2022. URL:<https://books.google.ca/books?id=YJ4IEQAAQBAJ>.
- [188] A. Graves, A.-r. Mohamed et G. Hinton. Speech recognition with deep recurrent neural networks. In *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, mai 2013. DOI:10.1109/icassp.2013.6638947.
- [189] I. Sutskever, J. Martens et G. Hinton. Generating text with recurrent neural networks. In *Proceedings of the 28th International Conference on International Conference on Machine Learning, ICML’11*, p. 1017–1024, Madison, WI, USA, 2011. Omnipress. URL:https://www.cs.toronto.edu/~jmartens/docs/RNN_Language.pdf.
- [190] J. Joyce. Bayes’ theorem. In E. N. Zalta, éd., *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, fall 2021 édn, 2021. URL:<https://plato.stanford.edu/archives/fall2021/entries/bayes-theorem/>.
- [191] Y. Kwon, J.-H. Won, B. J. Kim et M. C. Paik. Uncertainty quantification using Bayesian neural networks in classification: Application to biomedical image segmentation. *Computational Statistics & Data Analysis*, 142:106816, fév. 2020. DOI:10.1016/j.csda.2019.106816.
- [192] L. V. Jospin, H. Laga, F. Boussaid, W. Buntine et M. Bennamoun. Hands-on Bayesian neural networks – A tutorial for deep learning users. *IEEE Computational Intelligence Magazine*, 17(2):29–48, mai 2022. DOI:10.1109/mci.2022.3155327.
- [193] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher, M. Shahzad, W. Yang, R. Bamler et X. X. Zhu. A survey of uncertainty in deep neural networks. *ArXiv*, 2021. DOI:10.48550/arxiv.2107.03342. Preprint.
- [194] R. M. Neal. Bayesian training of backpropagation networks by the hybrid Monte Carlo method. Rapport technique, Citeseer, 1992. URL:<https://citeseerx.ist.psu.edu/document?doi=38b7a4d7d9646c4474c893fc53a606dee3264fec>.
- [195] R. M. Neal. *Bayesian learning for neural networks*. Thèse de doctorat, University of Toronto, 1995. URL:<https://librarysearch.library.utoronto.ca/permalink/01UTORONTO%5FINST/14bjeso/alma991106438365706196>.
- [196] M. Welling et Y. W. Teh. Bayesian learning via stochastic gradient langevin dynamics. In *Proceedings of the 28th International Conference on International Conference on Machine Learning, Icm1’11*, p. 681–688, Madison, WI, USA, 2011. Omnipress. URL:<https://www.stats.ox.ac.uk/~teh/research/compstats/WelTeh2011a.pdf>.
- [197] C. Nemeth et P. Fearnhead. Stochastic gradient markov chain Monte Carlo. *Journal of the American Statistical Association*, 116(533):433–450, jan. 2021. DOI:10.1080/01621459.2020.1847120.

-
- [198] J. Lee, M. Humt, J. Feng et R. Triebel. Estimating model uncertainty of neural networks in sparse information form. *ArXiv*, 2020. DOI:10.48550/arxiv.2006.11631. Preprint.
 - [199] H. Rue, S. Martino et N. Chopin. Approximate Bayesian inference for latent gaussian models by using integrated nested Laplace approximations. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 71(2):319–392, avr. 2009. DOI:10.1111/j.1467-9868.2008.00700.x.
 - [200] G. E. Hinton et D. van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the Sixth Annual Conference on Computational Learning Theory*, Colt '93, p. 5–13, New York, NY, USA, 1993. Association for Computing Machinery. DOI:10.1145/168304.168306.
 - [201] D. Barber et C. Bishop. Ensemble learning in Bayesian neural networks. In *Generalization in Neural Networks and Machine Learning*, p. 215–237. Springer Verlag, jan. 1998. URL:<https://www.microsoft.com/en-us/research/publication/ensemble-learning-in-bayesian-neural-networks/>.
 - [202] A. Graves. Practical variational inference for neural networks. In J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira et K. Weinberger, édés, *Advances in Neural Information Processing Systems*, vol. 24. Curran Associates, Inc., 2011. URL:<https://proceedings.neurips.cc/paper/2011/hash/7eb3c8be3d411e8ebfab08eba5f49632-Abstract.html>.
 - [203] C. Blundell, J. Cornebise, K. Kavukcuoglu et D. Wierstra. Weight uncertainty in neural networks. *ArXiv*, 2015. DOI:10.48550/arxiv.1505.05424. Preprint.
 - [204] Y. Gal et Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In M. F. Balcan et K. Q. Weinberger, édés, *Proceedings of The 33rd International Conference on Machine Learning*, vol. 48 de *Proceedings of Machine Learning Research*, p. 1050–1059, New York, New York, USA, 20–22 Jun 2016. PMLR. URL:<https://proceedings.mlr.press/v48/gal16.html>.
 - [205] Y. Gal. *Uncertainty in Deep Learning*. Thèse de doctorat, University of Cambridge, 2016. URL:<https://www.cs.ox.ac.uk/people/yarin.gal/website/thesis/thesis.pdf>.
 - [206] S. Kullback et R. A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, mars 1951. DOI:10.1214/aoms/1177729694.
 - [207] M. Campbell, A. Hoane et F. hsiung Hsu. Deep blue. *Artificial Intelligence*, 134(1-2):57–83, jan. 2002. DOI:10.1016/s0004-3702(01)00129-1.
 - [208] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel et D. Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, jan. 2016. DOI:10.1038/nature16961.
 - [209] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel et D. Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, oct. 2017. DOI:10.1038/nature24270.
-

- [210] M. Bowling, N. Burch, M. Johanson et O. Tammelin. Heads-up limit hold'em poker is solved. *Science*, 347(6218):145–149, jan. 2015. DOI:10.1126/science.1259433.
- [211] M. Moravčík, M. Schmid, N. Burch, V. L. D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson et M. Bowling. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508–513, mars 2017. DOI:10.1126/science.aam6960.
- [212] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky *et al.* Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, oct. 2019. DOI:10.1038/s41586-019-1724-z.
- [213] OpenAI, C. Berner, G. Brockman, B. Chan, V. Cheung, P. Dębiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. Józefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H. P. de Oliveira Pinto, J. Raiman, T. Salimans, J. Schlatter, J. Schneider, S. Sidor, I. Sutskever, J. Tang, F. Wolski *et al.* Dota 2 with large scale deep reinforcement learning. *ArXiv*, déc. 2019. DOI:10.48550/arxiv.1912.06680. Preprint.
- [214] N. Maslej, L. Fattorini, R. Perrault, V. Parli, A. Reuel, E. Brynjolfsson, J. Etchemendy, K. Ligett, T. Lyons, J. Manyika, J. C. Niebles, Y. Shoham, R. Wald et J. Clark. The AI index 2024 annual report. Rapport technique, Stanford University, Stanford, CA, avr. 2024. URL:<https://aiindex.stanford.edu/report>.
- [215] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko *et al.* GPT-4 technical report. *ArXiv*, 2023. DOI:10.48550/arxiv.2303.08774. Preprint.
- [216] S. S. Biswas. Role of ChatGPT in public health. *Annals of Biomedical Engineering*, 51(5):868–869, mars 2023. DOI:10.1007/s10439-023-03172-7.
- [217] E. Waisberg, J. Ong, M. Masalkhi, S. A. Kamran, N. Zaman, P. Sarker, A. G. Lee et A. Tavakkoli. GPT-4: A new era of artificial intelligence in medicine. *Irish Journal of Medical Science (1971 -)*, 192(6):3197–3200, avr. 2023. DOI:10.1007/s11845-023-03377-8.
- [218] I. Adeshola et A. P. Adepoju. The opportunities and challenges of ChatGPT in education. *Interactive Learning Environments*, p. 1–14, sept. 2023. DOI:10.1080/10494820.2023.2253858.
- [219] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, S. Iqbal, J.-B. Lespiau, A. Ahern, T. Köppe, K. Millikin, S. Gaffney, S. Elster, J. Broshear, C. Gamble, K. Milan, R. Tung, M. Hwang, T. Cemgil, M. Barekatin, Y. Li, A. Mandhane, T. Hubert, J. Schrittwieser *et al.* Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618(7964):257–263, juin 2023. DOI:10.1038/s41586-023-06004-9.
- [220] T. Ha, D. Lee, Y. Kwon, M. S. Park, S. Lee, J. Jang, B. Choi, H. Jeon, J. Kim, H. Choi, H.-T. Seo, W. Choi, W. Hong, Y. J. Park, J. Jang, J. Cho, B. Kim, H. Kwon,

- G. Kim, W. S. Oh, J. W. Kim, J. Choi, M. Min, A. Jeon, Y. Jung *et al.* AI-driven robotic chemist for autonomous synthesis of organic molecules. *Science Advances*, 9(44), nov. 2023. DOI:10.1126/sciadv.adj0461.
- [221] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu, A. Merose, S. Hoyer, G. Holland, O. Vinyals, J. Stott, A. Pritzel, S. Mohamed et P. Battaglia. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, déc. 2023. DOI:10.1126/science.adi2336.
- [222] G. Nearing, D. Cohen, V. Dube, M. Gauch, O. Gilon, S. Harrigan, A. Hassidim, D. Klotz, F. Kratzert, A. Metzger, S. Nevo, F. Pappenberger, C. Prudhomme, G. Shalev, S. Shenzis, T. Tekalign, D. Weitzner et Y. Matias. AI increases global access to reliable flood forecasts. *ArXiv*, 2023. DOI:10.48550/arxiv.2307.16104. Preprint.
- [223] A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon et E. D. Cubuk. Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85, nov. 2023. DOI:10.1038/s41586-023-06735-9.
- [224] J. Betker, G. Goh, L. Jing, T. Brooks, J. Wang, L. Li, L. Ouyang, J. Zhuang, J. Lee, Y. Guo, W. Manassra, P. Dhariwal, C. Chu, Y. Jiao et A. Ramesh. Improving image generation with better captions. 2023. URL:<https://cdn.openai.com/papers/dall-e-3.pdf>.
- [225] L. G. Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, nov. 1984. DOI:10.1145/1968.1972.
- [226] Z. Ji et M. Telgarsky. Directional convergence and alignment in deep learning. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan et H. Lin, édés, *Advances in Neural Information Processing Systems*, vol. 33, p. 17176–17186. Curran Associates, Inc., 2020. URL:<https://proceedings.neurips.cc/paper%5Ffiles/paper/2020/file/c76e4b2fa54f8506719a5c0dc14c2eb9-Paper.pdf>.
- [227] C. Zhang, S. Bengio, M. Hardt, B. Recht et O. Vinyals. Understanding deep learning requires rethinking generalization. *ArXiv*, 2016. DOI:10.48550/arxiv.1611.03530. Preprint.
- [228] D. Castelvechi. Can we open the black box of AI? *Nature*, 538(7623):20–23, oct. 2016. DOI:10.1038/538020a.
- [229] P. S. Chib et P. Singh. Recent advancements in end-to-end autonomous driving using deep learning: A survey. *IEEE Transactions on Intelligent Vehicles*, 9(1):103–118, jan. 2024. DOI:10.1109/tiv.2023.3318070.
- [230] B. Badjie, J. Cecilio et A. Casimiro. Adversarial attacks and countermeasures on image classification-based deep learning models in autonomous driving systems: A systematic review. *ACM Computing Surveys*, août 2024. DOI:10.1145/3691625.
- [231] S. Atakishiyev, M. Salameh, H. Yao et R. Goebel. Explainable artificial intelligence for autonomous driving: A comprehensive overview and field guide for future research directions. *IEEE Access*, 12:101603–101625, 2024. DOI:10.1109/access.2024.3431437.
-

- [232] N. Akhtar et A. Mian. Threat of adversarial attacks on deep learning in computer vision: A survey. *IEEE Access*, 6:14410–14430, 2018. DOI:10.1109/access.2018.2807385.
 - [233] X. Liu, L. Xie, Y. Wang, J. Zou, J. Xiong, Z. Ying et A. V. Vasilakos. Privacy and security issues in deep learning: A survey. *IEEE Access*, 9:4566–4593, 2021. DOI:10.1109/access.2020.3045078.
 - [234] M. Nasr, N. Carlini, J. Hayase, M. Jagielski, A. F. Cooper, D. Ippolito, C. A. Choquette-Choo, E. Wallace, F. Tramèr et K. Lee. Scalable extraction of training data from (production) language models. *ArXiv*, 2023. DOI:10.48550/arxiv.2311.17035. Preprint.
 - [235] A. Alwosheel, S. van Cranenburgh et C. G. Chorus. Is your dataset big enough? Sample size requirements when using artificial neural networks for discrete choice analysis. *Journal of Choice Modelling*, 28:167–182, sept. 2018. DOI:10.1016/j.jocm.2018.07.002.
 - [236] Z. Allen-Zhu, Y. Li et Z. Song. A convergence theory for deep learning via overparameterization. *ArXiv*, 2018. DOI:10.48550/arxiv.1811.03962. Preprint.
 - [237] S. Arora, N. Cohen et E. Hazan. On the optimization of deep networks: Implicit acceleration by overparameterization. In J. Dy et A. Krause, édés, *Proceedings of the 35th International Conference on Machine Learning*, vol. 80 de *Proceedings of Machine Learning Research*, p. 244–253. PMLR, 10–15 Jul 2018. URL:<https://proceedings.mlr.press/v80/arora18a.html>.
 - [238] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier et J. Dean. Carbon emissions and large neural network training. *ArXiv*, 2021. DOI:10.48550/arxiv.2104.10350. Preprint.
 - [239] S. Luccioni, Y. Jernite et E. Strubell. Power hungry processing: Watts driving the cost of AI deployment? In *The 2024 ACM Conference on Fairness, Accountability, and Transparency*, FAccT '24. ACM, juin 2024. DOI:10.1145/3630106.3658542.
 - [240] L. F. W. Anthony, B. Kanding et R. Selvan. Carbontracker: Tracking and predicting the carbon footprint of training deep learning models. *ArXiv*, 2020. DOI:10.48550/arxiv.2007.03051. Preprint.
 - [241] P. Li, J. Yang, M. A. Islam et S. Ren. Making AI less "thirsty": Uncovering and addressing the secret water footprint of AI models. *ArXiv*, 2023. DOI:10.48550/arxiv.2304.03271. Preprint.
 - [242] I. d'Aramon, B. Ruf et M. Detyniecki. *Assessing Carbon Footprint Estimations of ChatGPT*, p. 127–133. Springer Nature Switzerland, 2024. DOI:10.1007/978-3-031-59005-4_15.
 - [243] A. A. Chien, L. Lin, H. Nguyen, V. Rao, T. Sharma et R. Wijayawardana. Reducing the carbon impact of generative AI inference (today and in 2035). In *Proceedings of the 2nd Workshop on Sustainable Computer Systems*, HotCarbon '23. ACM, juil. 2023. DOI:10.1145/3604930.3605705.
 - [244] M. Baude et J.-L. Pasquier. L’empreinte carbone de la france de 1995 à 2022, oct. 2023. URL:<https://www.statistiques.developpement-durable.gouv.fr/lempreinte-carbone-de-la-france-de-1995-2022>. Visité le 20-01-2025.
-

-
- [245] C. Sung, H. Hwang et I. K. Yoo. Perspective: A review on memristive hardware for neuromorphic computation. *Journal of Applied Physics*, 124(15):151903, oct. 2018. DOI:10.1063/1.5037835.
 - [246] L. Luo, Z. Dong, S. Duan et C. S. Lai. Memristor-based stateful logic gates for multi-functional logic circuit. *IET Circuits, Devices & Systems*, 14(6):811–818, sept. 2020. DOI:10.1049/iet-cds.2019.0422.
 - [247] L. Goux, A. Fantini, G. Kar, Y.-Y. Chen, N. Jossart, R. Degraeve, S. Clima, B. Govoreanu, G. Lorenzo, G. Pourtois, D. Wouters, J. Kittl, L. Altissimo et M. Jurczak. Ultralow sub-500nA operating current high-performance TiN\Al₂O₃\HfO₂\Hf\TiN bipolar RRAM achieved through understanding-based stack-engineering. In *2012 Symposium on VLSI Technology (VLSIT)*. IEEE, juin 2012. DOI:10.1109/vlsit.2012.6242510.
 - [248] B. J. Choi, A. C. Torrezan, J. P. Strachan, P. G. Kotula, A. J. Lohn, M. J. Marinella, Z. Li, R. S. Williams et J. J. Yang. High-speed and low-energy nitride memristors. *Advanced Functional Materials*, 26(29):5290–5296, mai 2016. DOI:10.1002/adfm.201600680.
 - [249] S. Pi, C. Li, H. Jiang, W. Xia, H. Xin, J. J. Yang et Q. Xia. Memristor crossbar arrays with 6-nm half-pitch and 2-nm critical dimension. *Nature Nanotechnology*, 14(1):35–39, nov. 2018. DOI:10.1038/s41565-018-0302-0.
 - [250] K.-H. Kim, S. Gaba, D. Wheeler, J. M. Cruz-Albrecht, T. Hussain, N. Srinivasa et W. Lu. A functional hybrid memristor crossbar-array/CMOS system for data storage and neuromorphic applications. *Nano Letters*, 12(1):389–395, déc. 2011. DOI:10.1021/nl203687n.
 - [251] H.-S. P. Wong, H.-Y. Lee, S. Yu, Y.-S. Chen, Y. Wu, P.-S. Chen, B. Lee, F. T. Chen et M.-J. Tsai. Metal–oxide RRAM. *Proceedings of the IEEE*, 100(6):1951–1970, juin 2012. DOI:10.1109/jproc.2012.2190369.
 - [252] D. V. Christensen, R. Dittmann, B. Linares-Barranco, A. Sebastian, M. Le Gallo, A. Redaelli, S. Slesazeck, T. Mikolajick, S. Spiga, S. Menzel, I. Valov, G. Milano, C. Ricciardi, S.-J. Liang, F. Miao, M. Lanza, T. J. Quill, S. T. Keene, A. Salleo, J. Grollier, D. Marković, A. Mizrahi, P. Yao, J. J. Yang, G. Indiveri *et al.* 2022 roadmap on neuromorphic computing and engineering. *Neuromorphic Computing and Engineering*, 2(2):022501, mai 2022. DOI:10.1088/2634-4386/ac4a83.
 - [253] F. Aguirre, A. Sebastian, M. Le Gallo, W. Song, T. Wang, J. J. Yang, W. Lu, M.-F. Chang, D. Ielmini, Y. Yang, A. Mehonic, A. Kenyon, M. A. Villena, J. B. Roldán, Y. Wu, H.-H. Hsu, N. Raghavan, J. Suñé, E. Miranda, A. Eltawil, G. Setti, K. Smagulova, K. N. Salama, O. Krestinskaya, X. Yan *et al.* Hardware implementation of memristor-based artificial neural networks. *Nature Communications*, 15(1), mars 2024. DOI:10.1038/s41467-024-45670-9.
 - [254] D. S. Jeong, R. Thomas, R. S. Katiyar, J. F. Scott, H. Kohlstedt, A. Petraru et C. S. Hwang. Emerging memories: Resistive switching mechanisms and current status. *Reports on Progress in Physics*, 75(7):076502, juin 2012. DOI:10.1088/0034-4885/75/7/076502.
-

- [255] T. Schenk, M. Pešić, S. Slesazeck, U. Schroeder et T. Mikolajick. Memory technology — A primer for material scientists. *Reports on Progress in Physics*, 83(8):086501, juil. 2020. DOI:10.1088/1361-6633/ab8f86.
- [256] G. W. Burr, R. M. Shelby, A. Sebastian, S. Kim, S. Kim, S. Sidler, K. Virwani, M. Ishii, P. Narayanan, A. Fumarola, L. L. Sanches, I. Boybat, M. L. Gallo, K. Moon, J. Woo, H. Hwang et Y. Leblebici. Neuromorphic computing using non-volatile memory. *Advances in Physics: X*, 2(1):89–124, déc. 2016. DOI:10.1080/23746149.2016.1259585.
- [257] L. Wang, W. Li et D. Wen. Soybean-based memristor for multilevel data storage and emulation of synaptic behavior. *Microelectronic Engineering*, 267–268:111911, jan. 2023. DOI:10.1016/j.mee.2022.111911.
- [258] A. Amirsoleimani, F. Alibart, V. Yon, J. Xu, M. R. Pazhouhandeh, S. Ecoffey, Y. Beilliard, R. Genov et D. Drouin. In-memory vector-matrix multiplication in monolithic complementary metal–oxide–semiconductor–memristor integrated circuits: Design choices, challenges, and perspectives. *Advanced Intelligent Systems*, p. 2000115, août 2020. DOI:10.1002/aisy.202000115.
- [259] Q. Xia et J. J. Yang. Memristive crossbar arrays for brain-inspired computing. *Nature Materials*, 18(4):309–323, mars 2019. DOI:10.1038/s41563-019-0291-x.
- [260] P. Narayanan, S. Ambrogio, A. Okazaki, K. Hosokawa, H. Tsai, A. Nomura, T. Yasuda, C. Mackin, S. C. Lewis, A. Friz, M. Ishii, Y. Kohda, H. Mori, K. Spoon, R. Khaddam-Aljameh, N. Saulnier, M. Bergendahl, J. Demarest, K. W. Brew, V. Chan, S. Choi, I. Ok, I. Ahsan, F. L. Lie, W. Haensch *et al.* Fully on-chip MAC at 14 nm enabled by accurate row-wise programming of PCM-based weights and parallel vector-transport in duration-format. *IEEE Transactions on Electron Devices*, 68(12):6629–6636, déc. 2021. DOI:10.1109/ted.2021.3115993.
- [261] P. Yao, H. Wu, B. Gao, J. Tang, Q. Zhang, W. Zhang, J. J. Yang et H. Qian. Fully hardware-implemented memristor convolutional neural network. *Nature*, 577(7792):641–646, jan. 2020. DOI:10.1038/s41586-020-1942-4.
- [262] M. Le Gallo, R. Khaddam-Aljameh, M. Stanisavljevic, A. Vasilopoulos, B. Kersting, M. Dazzi, G. Karunaratne, M. Brändli, A. Singh, S. M. Müller, J. Büchel, X. Timoneda, V. Joshi, M. J. Rasch, U. Egger, A. Garofalo, A. Petropoulos, T. Antonakopoulos, K. Brew, S. Choi, I. Ok, T. Philip, V. Chan, C. Silvestre, I. Ahsan *et al.* A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nature Electronics*, 6(9):680–693, août 2023. DOI:10.1038/s41928-023-01010-1.
- [263] C. Wang, D. Feng, W. Tong, J. Liu, Z. Li, J. Chang, Y. Zhang, B. Wu, J. Xu, W. Zhao, Y. Li et R. Ren. Cross-point resistive memory: Nonideal properties and solutions. *ACM Transactions on Design Automation of Electronic Systems*, 24(4):1–37, juil. 2019. DOI:10.1145/3325067.
- [264] D. Efnusheva, A. Cholakoska et A. Tentov. A survey of different approaches for overcoming the processor - Memory bottleneck. *International Journal of Computer Science and Information Technology*, 9(2):151–163, avr. 2017. DOI:10.5121/ijcsit.2017.9214.
-

-
- [265] V. Sze, Y.-H. Chen, J. Emer, A. Suleiman et Z. Zhang. Hardware for machine learning: Challenges and opportunities. In *2017 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, avr. 2017. DOI:10.1109/cicc.2017.7993626.
- [266] G. W. Burr, P. Narayanan, R. M. Shelby, S. Sidler, I. Boybat, C. di Nolfo et Y. Leblebici. Large-scale neural networks implemented with non-volatile memory as the synaptic weight element: Comparative performance analysis (accuracy, speed, and power). In *2015 IEEE International Electron Devices Meeting (IEDM)*. IEEE, déc. 2015. DOI:10.1109/iedm.2015.7409625.
- [267] D. S. Jeong, K. M. Kim, S. Kim, B. J. Choi et C. S. Hwang. Memristors for energy-efficient new computing paradigms. *Advanced Electronic Materials*, 2(9):1600090, août 2016. DOI:10.1002/aelm.201600090.
- [268] G. C. Adam, A. Khat et T. Prodromakis. Challenges hindering memristive neuromorphic hardware from going mainstream. *Nature Communications*, 9(1), déc. 2018. DOI:10.1038/s41467-018-07565-4.
- [269] V. Yon, A. Amirsoleimani, F. Alibart, R. G. Melko, D. Drouin et Y. Beilliard. Exploiting non-idealities of resistive switching memories for efficient machine learning. *Frontiers in Electronics*, 3, mars 2022. DOI:10.3389/felec.2022.825077.
- [270] C. Li, D. Belkin, Y. Li, P. Yan, M. Hu, N. Ge, H. Jiang, E. Montgomery, P. Lin, Z. Wang, W. Song, J. P. Strachan, M. Barnell, Q. Wu, R. S. Williams, J. J. Yang et Q. Xia. Efficient and self-adaptive in-situ learning in multilayer memristor neural networks. *Nature Communications*, 9(1), juin 2018. DOI:10.1038/s41467-018-04484-2.
- [271] Y. Liao, B. Gao, F. Xu, P. Yao, J. Chen, W. Zhang, J. Tang, H. Wu et H. Qian. A compact model of analog RRAM with device and array nonideal effects for neuromorphic systems. *IEEE Transactions on Electron Devices*, 67(4):1593–1599, avr. 2020. DOI:10.1109/ted.2020.2975314.
- [272] E. Gale. TiO₂-based memristors and ReRAM: Materials, mechanisms and models (a review). *Semiconductor Science and Technology*, 29(10):104004, sept. 2014. DOI:10.1088/0268-1242/29/10/104004.
- [273] F. Alibart, E. Zamanidoost et D. B. Strukov. Pattern classification by memristive crossbar circuits using ex situ and in situ training. *Nature Communications*, 4(1), juin 2013. DOI:10.1038/ncomms3072.
- [274] M. Ernoult, J. Grollier et D. Querlioz. Using memristors for robust local learning of hardware restricted Boltzmann machines. *Scientific Reports*, 9(1), fév. 2019. DOI:10.1038/s41598-018-38181-3.
- [275] W.-Q. Pan, J. Chen, R. Kuang, Y. Li, Y.-H. He, G.-R. Feng, N. Duan, T.-C. Chang et X.-S. Miao. Strategies to improve the accuracy of memristor-based convolutional neural networks. *IEEE Transactions on Electron Devices*, 67(3):895–901, mars 2020. DOI:10.1109/ted.2019.2963323.
- [276] Y. Wang, S. Wu, L. Tian et L. Shi. SSM: A high-performance scheme for in situ training of imprecise memristor neural networks. *Neurocomputing*, 407:270–280, sept. 2020. DOI:10.1016/j.neucom.2020.04.130.
-

- [277] M. H. Maruf et S. I. Ali. Review and comparative study of I-V characteristics of different memristor models with sinusoidal input. *International Journal of Electronics*, 107(3):349–375, sept. 2019. DOI:10.1080/00207217.2019.1661021.
- [278] F. O. Rzig, K. Mbarek, S. Ghedira et K. Besbes. The basic I–V characteristics of memristor model: Simulation and analysis. *Applied Physics A*, 123(4), mars 2017. DOI:10.1007/s00339-017-0902-9.
- [279] M. R. Mahmoodi, M. Prezioso et D. B. Strukov. Versatile stochastic dot product circuits based on nonvolatile memories for high performance neuro-computing and neurooptimization. *Nature Communications*, 10(1), nov. 2019. DOI:10.1038/s41467-019-13103-7.
- [280] A. Fantini, G. Gorine, R. Degraeve, L. Goux, C. Y. Chen, A. Redolfi, S. Clima, A. Cabrini, G. Torelli et M. Jurczak. Intrinsic program instability in HfO₂ RRAM and consequences on program algorithms. In *2015 IEEE International Electron Devices Meeting (IEDM)*. IEEE, déc. 2015. DOI:10.1109/iedm.2015.7409648.
- [281] R. Thamankar, F. M. Puglisi, A. Ranjan, N. Raghavan, K. Shubhakar, J. Molina, L. Larcher, A. Padovani, P. Pavan, S. J. O'Shea et K. L. Pey. Localized characterization of charge transport and random telegraph noise at the nanoscale in HfO₂ films combining scanning tunneling microscopy and multi-scale simulations. *Journal of Applied Physics*, 122(2):024301, juil. 2017. DOI:10.1063/1.4991002.
- [282] C. Claeys, M. G. C. de Andrade, Z. Chai, W. Fang, B. Govoreanu, B. Kaczer, W. Zhang et E. Simoen. Random telegraph signal noise in advanced high performance and memory devices. In *2016 31st Symposium on Microelectronics Technology and Devices (SBMicro)*. IEEE, août 2016. DOI:10.1109/sbmicro.2016.7731315.
- [283] J.-K. Lee, J.-W. Lee, J. Park, S.-W. Chung, J. S. Roh, S.-J. Hong, I. whan Cho, H.-I. Kwon et J.-H. Lee. Extraction of trap location and energy from random telegraph noise in amorphous TiOx resistance random access memories. *Applied Physics Letters*, 98(14):143502, avr. 2011. DOI:10.1063/1.3575572.
- [284] D. Ielmini, F. Nardi et C. Cagli. Resistance-dependent amplitude of random telegraph-signal noise in resistive switching memories. *Applied Physics Letters*, 96(5):053503, fév. 2010. DOI:10.1063/1.3304167.
- [285] J. J. Yang, M.-X. Zhang, J. P. Strachan, F. Miao, M. D. Pickett, R. D. Kelley, G. Medeiros-Ribeiro et R. S. Williams. High switching endurance in TaOx memristive devices. *Applied Physics Letters*, 97(23):232102, déc. 2010. DOI:10.1063/1.3524521.
- [286] P. S. Georgiou, I. Koymen et E. M. Drakakis. Noise properties of ideal memristors. In *2015 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, mai 2015. DOI:10.1109/iscas.2015.7168841.
- [287] X.-F. Cheng, W.-H. Qian, J. Wang, C. Yu, J.-H. He, H. Li, Q.-F. Xu, D.-Y. Chen, N.-J. Li et J.-M. Lu. Environmentally robust memristor enabled by lead-free double perovskite for high-performance information storage. *Small*, 15(49):1905731, oct. 2019. DOI:10.1002/sml.201905731.
- [288] Z. Wang, M. Yin, T. Zhang, Y. Cai, Y. Wang, Y. Yang et R. Huang. Engineering incremental resistive switching in TaO_x based memristors for brain-inspired computing. *Nanoscale*, 8(29):14015–14022, 2016. DOI:10.1039/c6nr00476h.
-

-
- [289] J. Spring, E. Sediva, Z. D. Hood, J. C. Gonzalez-Rosillo, W. O'Leary, K. J. Kim, A. J. Carrillo et J. L. M. Rupp. Toward controlling filament size and location for resistive switches via nanoparticle exsolution at oxide interfaces. *Small*, 16(41):2003224, sept. 2020. DOI:10.1002/smll.202003224.
 - [290] Nature. Big data needs a hardware revolution. *Nature*, 554(7691):145–146, fév. 2018. DOI:10.1038/d41586-018-01683-1.
 - [291] W. Wang, W. Song, P. Yao, Y. Li, J. Van Nostrand, Q. Qiu, D. Ielmini et J. J. Yang. Integration and co-design of memristive devices and algorithms for artificial intelligence. *iScience*, 23(12):101809, déc. 2020. DOI:10.1016/j.isci.2020.101809.
 - [292] P. Drolet, R. Dawant, V. Yon, P.-A. Mouny, M. Valdenaire, J. A. Zapata, P. Gliech, S. U. N. Wood, S. Ecoffey, F. Alibart, Y. Beilliard et D. Drouin. Hardware-aware training techniques for improving robustness of ex-situ neural network transfer onto passive TiO₂ ReRAM crossbars. *ArXiv*, 2023. DOI:10.48550/arxiv.2305.18495. Preprint.
 - [293] G.-q. Bi et M.-m. Poo. Synaptic modifications in cultured hippocampal neurons: Dependence on spike timing, synaptic strength, and postsynaptic cell type. *The Journal of Neuroscience*, 18(24):10464–10472, déc. 1998. DOI:10.1523/jneurosci.18-24-10464.1998.
 - [294] T. Iakymchuk, A. Rosado-Muñoz, J. F. Guerrero-Martínez, M. Bataller-Mompeán et J. V. Francés Villora. Simplified spiking neural network architecture and STDP learning algorithm applied to image classification. *EURASIP Journal on Image and Video Processing*, 2015(1), fév. 2015. DOI:10.1186/s13640-015-0059-4.
 - [295] E. O. Neftci, H. Mostafa et F. Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, nov. 2019. DOI:10.1109/msp.2019.2931595.
 - [296] C. Alessandri. *Ferroelectric Memory and Architecture for Deep Neural Network Training in Resistive Crossbar Arrays*. Thèse de doctorat, University of Notre Dame, Indiana, avr. 2019. URL:<https://curate.nd.edu/show/2514nk34r7x>.
 - [297] V. Yon, B. Galaup, C. Rohrbacher, J. Rivard, C. Godfrin, R. Li, S. Kubicek, K. De Greve, L. Gaudreau, E. Dupont-Ferrier, Y. Beilliard, R. G. Melko et D. Drouin. Robust quantum dots charge autotuning using neural network uncertainty. *Machine Learning: Science and Technology*, 2024. DOI:10.1088/2632-2153/ad88d5.
 - [298] A. Elsayed, M. M. K. Shehata, C. Godfrin, S. Kubicek, S. Massar, Y. Canvel, J. Jussot, G. Simion, M. Mongillo, D. Wan, B. Govoreanu, I. P. Radu, R. Li, P. Van Dorpe et K. De Greve. Low charge noise quantum dots with industrial CMOS manufacturing. *NPJ Quantum Information*, 10(1), juil. 2024. DOI:10.1038/s41534-024-00864-3.
 - [299] M. F. Gonzalez-Zalba, S. de Franceschi, E. Charbon, T. Meunier, M. Vinet et A. S. Dzurak. Scaling silicon-based quantum computing using CMOS technology. *Nature Electronics*, 4(12):872–884, déc. 2021. DOI:10.1038/s41928-021-00681-y.
 - [300] C. Rohrbacher, J. Rivard, R. Ritzenthaler, B. Bureau, C. Lupien, H. Mertens, N. Horiguchi et E. Dupont-Ferrier. Dual operation of gate-all-around silicon
-

- nanowires at cryogenic temperatures: FET and quantum dot. *ArXiv*, 2023. DOI:10.48550/arxiv.2312.00903. Preprint.
- [301] X. Liu et M. C. Hersam. 2D materials for quantum information science. *Nature Reviews Materials*, 4(10):669–684, août 2019. DOI:10.1038/s41578-019-0136-x.
- [302] A. Saraiva, W. H. Lim, C. H. Yang, C. C. Escott, A. Laucht et A. S. Dzurak. Materials for silicon quantum dots and their impact on electron spin qubits. *Advanced Functional Materials*, 32(3), déc. 2021. DOI:10.1002/adfm.202105488.
- [303] R. Patel, Y. Agrawal et R. Parekh. Single-electron transistor: Review in perspective of theory, modelling, design and fabrication. *Microsystem Technologies*, 27(5):1863–1875, sept. 2020. DOI:10.1007/s00542-020-05002-5.
- [304] C. B. Simmons, M. Thalakulam, N. Shaji, L. J. Klein, H. Qin, R. H. Blick, D. E. Savage, M. G. Lagally, S. N. Coppersmith et M. A. Eriksson. Single-electron quantum dot in Si/SiGe with integrated charge sensing. *Applied Physics Letters*, 91(21), nov. 2007. DOI:10.1063/1.2816331.
- [305] T. A. Baart, P. T. Eendebak, C. Reichl, W. Wegscheider et L. M. K. Vandersypen. Computer-automated tuning of semiconductor double quantum dots into the single-electron regime. *Applied Physics Letters*, 108(21), mai 2016. DOI:10.1063/1.4952624.
- [306] J. Ziegler, T. McJunkin, E. Joseph, S. S. Kalantre, B. Harpt, D. Savage, M. Lagally, M. Eriksson, J. M. Taylor et J. P. Zwolak. Toward robust autotuning of noisy quantum dot devices. *Physical Review Applied*, 17(2), fév. 2022. DOI:10.1103/physrevapplied.17.024069.
- [307] E. Goan et C. Fookes. *Bayesian Neural Networks: An Introduction and Survey*, p. 45–87. Springer International Publishing, 2020. DOI:10.1007/978-3-030-42553-1_3.
- [308] H. Liu, B. Wang, N. Wang, Z. Sun, H. Yin, H. Li, G. Cao et G. Guo. An automated approach for consecutive tuning of quantum dot arrays. *Applied Physics Letters*, 121(8), août 2022. DOI:10.1063/5.0111128.
- [309] J. K. Perron, M. D. S. Jr et N. M. Zimmerman. A quantitative study of bias triangles presented in chemical potential space. *Journal of Physics: Condensed Matter*, 27(23):235302, mai 2015. DOI:10.1088/0953-8984/27/23/235302.
- [310] T. Hensgens. *Emulating Fermi-Hubbard physics with quantum dots*. Thèse de doctorat, Delft University of Technology, 2018. DOI:10.4233/uuid:b71f3b0b-73a0-4996-896c-84ed43e72035.
- [311] J. D. Teske, S. S. Humpohl, R. Otten, P. Bethke, P. Cerfontaine, J. Dedden, A. Ludwig, A. D. Wieck et H. Bluhm. A machine learning approach for automated fine-tuning of semiconductor spin qubits. *Applied Physics Letters*, 114(13), avr. 2019. DOI:10.1063/1.5088412.
- [312] S. Rochette, M. Rudolph, A.-M. Roy, M. J. Curry, G. A. T. Eyck, R. P. Manginell, J. R. Wendt, T. Pluym, S. M. Carr, D. R. Ward, M. P. Lilly, M. S. Carroll et M. Pioro-Ladrière. Quantum dots with split enhancement gate tunnel barrier control. *Applied Physics Letters*, 114(8), fév. 2019. DOI:10.1063/1.5091111.
-

-
- [313] J. P. Zwolak, S. S. Kalantre, X. Wu, S. Ragole et J. M. Taylor. QFlow lite dataset: A machine-learning approach to the charge states in quantum dot experiments. *Plos One*, 13(10):e0205844, oct. 2018. DOI:10.1371/journal.pone.0205844.
 - [314] V. Yon, B. Galaup, M.-A. Roux, M.-A. Genest, C. Rohrbacher, J. Rivard, C. Godfrin, R. Li, K. De Greve, S. Rochette, J. Camirand-Lemire, L. Gaudreau, M. Pioro-Ladrière, Y. Beilliard, R. G. Melko, E. Dupont-Ferrier et D. Drouin. Quantum dots stability diagrams dataset, mai 2024. DOI:10.5281/zenodo.11402792.
 - [315] S. Minaee, Y. Y. Boykov, F. Porikli, A. J. Plaza, N. Kehtarnavaz et D. Terzopoulos. Image segmentation using deep learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, p. 1, 2021. DOI:10.1109/tpami.2021.3059968.
 - [316] Y. Zhang, Z. Shen et R. Jiao. Segment anything model for medical image segmentation: Current applications and future directions. *Computers in Biology and Medicine*, 171:108238, mars 2024. DOI:10.1016/j.combiomed.2024.108238.
 - [317] Y. Bengio. Deep learning of representations for unsupervised and transfer learning. In I. Guyon, G. Dror, V. Lemaire, G. Taylor et D. Silver, édés, *Proceedings of ICML Workshop on Unsupervised and Transfer Learning*, vol. 27 de *Proceedings of Machine Learning Research*, p. 17–36, Bellevue, Washington, USA, 02 Jul 2012. PMLR. URL:<https://proceedings.mlr.press/v27/bengio12a.html>.
 - [318] A. Krizhevsky, I. Sutskever et G. E. Hinton. ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, mai 2017. DOI:10.1145/3065386.
 - [319] A. Patil et M. Rane. *Convolutional Neural Networks: An Overview and Its Applications in Pattern Recognition*, p. 21–30. Springer Singapore, oct. 2020. DOI:10.1007/978-981-15-7078-0_3.
 - [320] L. Chen, S. Li, Q. Bai, J. Yang, S. Jiang et Y. Miao. Review of image classification algorithms based on convolutional neural networks. *Remote Sensing*, 13(22):4712, nov. 2021. DOI:10.3390/rs13224712.
 - [321] Z. Li, F. Liu, W. Yang, S. Peng et J. Zhou. A survey of convolutional neural networks: Analysis, applications, and prospects. *IEEE Transactions on Neural Networks and Learning Systems*, 33(12):6999–7019, déc. 2022. DOI:10.1109/tnnls.2021.3084827.
 - [322] B. Lienhard, A. Vepsäläinen, L. C. Govia, C. R. Hoffer, J. Y. Qiu, D. Ristè, M. Ware, D. Kim, R. Winik, A. Melville, B. Niedzielski, J. Yoder, G. J. Ribeill, T. A. Ohki, H. K. Krovi, T. P. Orlando, S. Gustavsson et W. D. Oliver. Deep-neural-network discrimination of multiplexed superconducting-qubit states. *Physical Review Applied*, 17(1), jan. 2022. DOI:10.1103/physrevapplied.17.014024.
 - [323] L. Smith et Y. Gal. Understanding measures of uncertainty for adversarial example detection. *ArXiv*, 2018. DOI:10.48550/arxiv.1803.08533. Preprint.
 - [324] H. Zaragoza et F. d’Alché Buc. Confidence measures for neural network classifiers. In *Proceedings of the Seventh Int. Conf. Information Processing and Management of Uncertainty in Knowledge Based Systems*, vol. 9. Citeseer, 1998. URL:https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/hugoz_ipmu98.pdf.
 - [325] A. Mandelbaum et D. Weinshall. Distance-based confidence score for neural network classifiers. *ArXiv*, 2017. DOI:10.48550/arxiv.1709.09844. Preprint.
-

- [326] D. Ramos, J. Franco-Pedroso, A. Lozano-Diez et J. Gonzalez-Rodriguez. Deconstructing cross-entropy for probabilistic binary classifiers. *Entropy*, 20(3):208, mars 2018. DOI:10.3390/e20030208.
 - [327] A. Mao, M. Mohri et Y. Zhong. Cross-entropy loss functions: Theoretical analysis and applications. In *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 de *Proceedings of Machine Learning Research*, p. 23803–23828. PMLR, 23–29 Jul 2023. URL:<https://proceedings.mlr.press/v202/mao23b.html>.
 - [328] P. Auer. Using confidence bounds for exploitation-exploration trade-offs. *Journal of Machine Learning Research*, 3(Nov):397–422, 2002. URL:<https://www.jmlr.org/papers/v3/auer02a.html>.
 - [329] A. Simpkins, R. de Callafon et E. Todorov. Optimal trade-off between exploration and exploitation. In *2008 American Control Conference*. IEEE, juin 2008. DOI:10.1109/acc.2008.4586462.
 - [330] C. Guo, G. Pleiss, Y. Sun et K. Q. Weinberger. On calibration of modern neural networks. *ArXiv*, 2017. DOI:10.48550/arxiv.1706.04599. Preprint.
 - [331] J. Vaicenavicius, D. Widmann, C. Andersson, F. Lindsten, J. Roll et T. B. Schön. Evaluating model calibration in classification. *ArXiv*, 2019. DOI:10.48550/arxiv.1902.06977. Preprint.
 - [332] T. Silva Filho, H. Song, M. Perello-Nieto, R. Santos-Rodriguez, M. Kull et P. Flach. Classifier calibration: A survey on how to assess and improve predicted class probabilities. *Machine Learning*, 112(9):3211–3260, mai 2023. DOI:10.1007/s10994-023-06336-7.
 - [333] S. Raschka. Model evaluation, model selection, and algorithm selection in machine learning. *ArXiv*, 2018. DOI:10.48550/arxiv.1811.12808. Preprint.
 - [334] P. Izmailov, S. Vikram, M. D. Hoffman et A. G. G. Wilson. What are Bayesian neural network posteriors really like? In M. Meila et T. Zhang, édés, *Proceedings of the 38th International Conference on Machine Learning*, vol. 139 de *Proceedings of Machine Learning Research*, p. 4629–4640. PMLR, 18–24 Jul 2021. URL:<https://proceedings.mlr.press/v139/izmailov21a.html>.
 - [335] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. Fieguth, X. Cao, A. Khosravi, U. R. Acharya, V. Makarenkov et S. Nahavandi. A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information Fusion*, 76:243–297, déc. 2021. DOI:10.1016/j.inffus.2021.05.008.
 - [336] B. Lakshminarayanan, A. Pritzel et C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *ArXiv*, 2016. DOI:10.48550/arxiv.1612.01474. Preprint.
 - [337] V. Fortuin, A. Garriga-Alonso, S. W. Ober, F. Wenzel, G. Rätsch, R. E. Turner, M. van der Wilk et L. Aitchison. Bayesian neural network priors revisited. *ArXiv*, 2021. DOI:10.48550/arxiv.2102.06571. Preprint.
 - [338] D. Silvestro et T. Andermann. Prior choice affects ability of Bayesian neural networks to identify unknowns. *ArXiv*, 2020. DOI:10.48550/arxiv.2005.04987. Preprint.
-

-
- [339] V. Gebhart, R. Santagati, A. A. Gentile, E. M. Gauger, D. Craig, N. Ares, L. Banchi, F. Marquardt, L. Pezzè et C. Bonato. Learning quantum systems. *Nature Reviews Physics*, 5(3):141–156, fév. 2023. DOI:10.1038/s42254-022-00552-1.
 - [340] L. Szulakowska et J. Dai. Bayesian autotuning of Hubbard model quantum simulators. *ArXiv*, 2022. DOI:10.48550/arxiv.2210.03077. Preprint.
 - [341] O. Krause, B. Brovang, T. Rasmussen, A. Chatterjee et F. Kuemmeth. Estimation of convex polytopes for automatic discovery of charge state transitions in quantum dot arrays. *Electronics*, 11(15):2327, juil. 2022. DOI:10.3390/electronics11152327.
 - [342] P.-A. Mouny, R. Dawant, B. Galaup, S. Ecoffey, M. Pioro-Ladrière, Y. Beilliard et D. Drouin. Analog programming of CMOS-compatible $\text{Al}_2\text{O}_3/\text{TiO}_{2-x}$ memristor at 4.2K after metal-insulator transition suppression by cryogenic reforming. *Applied Physics Letters*, 123(16), oct. 2023. DOI:10.1063/5.0170058.
 - [343] V. Yon, F. Marcotte, P.-A. Mouny, G. A. Dagnew, B. Kulchytskyy, S. Rochette, Y. Beilliard, D. Drouin et P. Ronagh. A cryogenic memristive neural decoder for fault-tolerant quantum error correction. *ArXiv*, 2024. DOI:10.48550/arxiv.2307.09463. Preprint.
 - [344] R. Dawant, M. Gaudreau, M.-A. Roy, P.-A. Mouny, M. Valdenaire, P. Gliech, J. A. Zapata, M. Zegaoui, F. Alibart, D. Drouin et S. Ecoffey. Damascene versus subtractive line CMP process for resistive memory crossbars BEOL integration. *Micro and Nano Engineering*, 23:100251, juin 2024. DOI:10.1016/j.mne.2024.100251.
 - [345] V. Yon, B. Galaup, R. G. Melko, Y. Beilliard et D. Drouin. Robust quantum dots charge autotuning using neural network uncertainty - Output data, mai 2024. DOI:10.5281/zenodo.11403192.
 - [346] V. Yon, B. Galaup, C. Rohrbacher, J. Rivard, A. Morel, D. Leclerc, C. Godfrin, R. Li, S. Kubicek, K. De Greve, E. Dupont-Ferrier, Y. Beilliard, R. G. Melko et D. Drouin. Experimental online quantum dots charge autotuning using neural networks. *ArXiv*, 2024. DOI:10.48550/arxiv.2409.20320. Preprint.
 - [347] F. A. Zwanenburg, A. S. Dzurak, A. Morello, M. Y. Simmons, L. C. L. Hollenberg, G. Klimeck, S. Rogge, S. N. Coppersmith et M. A. Eriksson. Silicon quantum electronics. *Reviews of Modern Physics*, 85(3):961–1019, juil. 2013. DOI:10.1103/revmodphys.85.961.
 - [348] A. J. Weinstein, M. D. Reed, A. M. Jones, R. W. Andrews, D. Barnes, J. Z. Blumoff, L. E. Euliss, K. Eng, B. H. Fong, S. D. Ha, D. R. Hulbert, C. A. C. Jackson, M. Jura, T. E. Keating, J. Kerckhoff, A. A. Kiselev, J. Matten, G. Sabbir, A. Smith, J. Wright, M. T. Rakher, T. D. Ladd et M. G. Borselli. Universal logic with encoded spin qubits in silicon. *Nature*, 615(7954):817–822, fév. 2023. DOI:10.1038/s41586-023-05777-3.
 - [349] W. Gilbert, T. Tanttu, W. H. Lim, M. Feng, J. Y. Huang, J. D. Cifuentes, S. Serrano, P. Y. Mai, R. C. C. Leon, C. C. Escott, K. M. Itoh, N. V. Abrosimov, H.-J. Pohl, M. L. W. Thewalt, F. E. Hudson, A. Morello, A. Laucht, C. H. Yang, A. Saraiva et A. S. Dzurak. On-demand electrical control of spin qubits. *Nature Nanotechnology*, 18(2):131–136, jan. 2023. DOI:10.1038/s41565-022-01280-4.
 - [350] C. H. Yang, R. C. C. Leon, J. C. C. Hwang, A. Saraiva, T. Tanttu, W. Huang, J. Camirand Lemyre, K. W. Chan, K. Y. Tan, F. E. Hudson, K. M. Itoh, A. Mo-
-

- rello, M. Pioro-Ladrière, A. Laucht et A. S. Dzurak. Operation of a silicon quantum processor unit cell above one Kelvin. *Nature*, 580(7803):350–354, avr. 2020. DOI:10.1038/s41586-020-2171-6.
- [351] J. Y. Huang, R. Y. Su, W. H. Lim, M. Feng, B. van Straaten, B. Severin, W. Gilbert, N. Dumoulin Stuyck, T. Tanttu, S. Serrano, J. D. Cifuentes, I. Hansen, A. E. Seedhouse, E. Vahapoglu, R. C. C. Leon, N. V. Abrosimov, H.-J. Pohl, M. L. W. Thewalt, F. E. Hudson, C. C. Escott, N. Ares, S. D. Bartlett, A. Morello, A. Saraiva, A. Laucht *et al.* High-fidelity spin qubit operation and algorithmic initialization above 1K. *Nature*, 627(8005):772–777, mars 2024. DOI:10.1038/s41586-024-07160-2.
- [352] A. M. J. Zwerver, T. Krähenmann, T. F. Watson, L. Lampert, H. C. George, R. Pillarisetty, S. A. Bojarski, P. Amin, S. V. Amitonov, J. M. Boter, R. Caudillo, D. Correas-Serrano, J. P. Dehollain, G. Droulers, E. M. Henry, R. Kotlyar, M. Lodari, F. Lüthi, D. J. Michalak, B. K. Mueller, S. Neyens, J. Roberts, N. Samkharadze, G. Zheng, O. K. Zietz *et al.* Qubits made by advanced semiconductor manufacturing. *Nature Electronics*, 5(3):184–190, mars 2022. DOI:10.1038/s41928-022-00727-9.
- [353] S. Neyens, O. K. Zietz, T. F. Watson, F. Luthi, A. Nethwewala, H. C. George, E. Henry, M. Islam, A. J. Wagner, F. Borjans, E. J. Connors, J. Corrigan, M. J. Curry, D. Keith, R. Kotlyar, L. F. Lampert, M. T. Mądzik, K. Millard, F. A. Mohiyaddin, S. Pellerano, R. Pillarisetty, M. Ramsey, R. Savytskyy, S. Schaal, G. Zheng *et al.* Probing single electrons across 300-mm spin qubit wafers. *Nature*, 629(8010):80–85, mai 2024. DOI:10.1038/s41586-024-07275-6.
- [354] C. Rohrbacher, D. Leclerc, J. Rivard, R. Ritzenthaler, C. Lupien, H. Mertens, N. Horiguchi et E. Dupont-Ferrier. Nanosheet transistors produced in 300 mm fabrication platform for quantum computing. 2024. To be published.
- [355] J. P. Dodson, N. Holman, B. Thorgrimsson, S. F. Neyens, E. R. MacQuarrie, T. McJunkin, R. H. Foote, L. F. Edge, S. N. Coppersmith et M. A. Eriksson. Fabrication process and failure analysis for robust quantum dots in silicon. *Nanotechnology*, 31(50):505001, oct. 2020. DOI:10.1088/1361-6528/abb559.
- [356] C. Tahan. Opinion: Democratizing spin qubits. *Quantum*, 5:584, nov. 2021. DOI:10.22331/q-2021-11-18-584.
- [357] J. Michniewicz et M. S. Kim. Leveraging off-the-shelf silicon chips for quantum computing. *Applied Physics Letters*, 124(26), juin 2024. DOI:10.1063/5.0207162.
- [358] P.-A. Mouny, R. Dawant, P. Dufour, M. Valdenaire, S. Ecoffey, M. Pioro-Ladrière, Y. Beillard et D. Drouin. Towards scalable cryogenic quantum dot biasing using memristor-based DC sources. *ArXiv*, 2024. DOI:10.48550/arxiv.2404.10694. Preprint.
- [359] T.-K. Hsiao, C. van Diepen, U. Mukhopadhyay, C. Reichl, W. Wegscheider et L. Vandersypen. Efficient orthogonal control of tunnel couplings in a quantum dot array. *Physical Review Applied*, 13(5), mai 2020. DOI:10.1103/physrevapplied.13.054018.
- [360] C. J. van Diepen, P. T. Eendebak, B. T. Buijtendorp, U. Mukhopadhyay, T. Fujita, C. Reichl, W. Wegscheider et L. M. K. Vandersypen. Automated tuning of inter-dot tunnel coupling in double quantum dots. *Applied Physics Letters*, 113(3), juil. 2018. DOI:10.1063/1.5031034.
-

- [361] M. Meyer, C. Déprez, I. N. Meijer, F. K. Unseld, S. Karwal, A. Sammak, G. Scappucci, L. M. K. Vandersypen et M. Veldhorst. Single-electron occupation in quantum dot arrays at selectable plunger gate voltage. *Nano Letters*, 23(24):11593–11600, déc. 2023. DOI:10.1021/acs.nanolett.3c03349.
- [362] T. Botzem, M. D. Shulman, S. Foletti, S. P. Harvey, O. E. Dial, P. Bethke, P. Cerfontaine, R. P. G. McNeil, D. Mahalu, V. Umansky, A. Ludwig, A. Wieck, D. Schuh, D. Bougeard, A. Yacoby et H. Bluhm. Tuning methods for semiconductor spin qubits. *Physical Review Applied*, 10(5), nov. 2018. DOI:10.1103/physrevapplied.10.054026.
- [363] I. J. Goodfellow, J. Shlens et C. Szegedy. Explaining and harnessing adversarial examples. *ArXiv*, 2014. DOI:10.48550/arxiv.1412.6572. Preprint.
- [364] R. Mehrotra, K. Namuduri et N. Ranganathan. Gabor filter-based edge detection. *Pattern Recognition*, 25(12):1479–1494, déc. 1992. DOI:10.1016/0031-3203(92)90121-x.
- [365] K. O’Shea et R. Nash. An introduction to convolutional neural networks. *ArXiv*, 2015. DOI:10.48550/arxiv.1511.08458. Preprint.
- [366] L. Cai, J. Gao et D. Zhao. A review of the application of deep learning in medical image classification and segmentation. *Annals of Translational Medicine*, 8(11):713–713, juin 2020. DOI:10.21037/atm.2020.02.44.
- [367] R. Rojas. *The Backpropagation Algorithm*, p. 149–182. Springer Berlin Heidelberg, 1996. DOI:10.1007/978-3-642-61068-4_7.
- [368] J. Stehlik, Y.-Y. Liu, C. Quintana, C. Eichler, T. Hartke et J. Petta. Fast charge sensing of a cavity-coupled double quantum dot using a Josephson parametric amplifier. *Physical Review Applied*, 4(1), juil. 2015. DOI:10.1103/physrevapplied.4.014018.
- [369] D. J. Reilly, C. M. Marcus, M. P. Hanson et A. C. Gossard. Fast single-charge sensing with a rf quantum point contact. *Applied Physics Letters*, 91(16), oct. 2007. DOI:10.1063/1.2794995.
- [370] V. Yon, B. Galaup, R. G. Melko, Y. Beilliard et D. Drouin. Experimental online quantum dots charge autotuning using neural networks - Output data, 2024. DOI:10.5281/zenodo.13381665.
- [371] Y. Beilliard, F. Paquette, F. Brousseau, S. Ecoffey, F. Alibart et D. Drouin. Investigation of resistive switching and transport mechanisms of Al₂O₃/TiO₂-x memristors under cryogenic conditions (1.5K). *AIP Advances*, 10(2), fév. 2020. DOI:10.1063/1.5140994.
- [372] Y. Beilliard, F. Paquette, F. Brousseau, S. Ecoffey, F. Alibart et D. Drouin. Conductive filament evolution dynamics revealed by cryogenic (1.5 K) multilevel switching of CMOS-compatible Al₂O₃/TiO₂ resistive memories. *Nanotechnology*, 31(44):445205, août 2020. DOI:10.1088/1361-6528/aba6b4.
- [373] B. Manchon, D. Coffineau, G. Segantini, N. Baboux, P. Rojo Romeo, R. Barhoumi, I. Cañero Infante, F. Alibart, B. Vilquin, D. Drouin et D. Deleruyelle. Study of polarisation and conduction mechanisms in ferroelectric Hf_{0.5}Zr_{0.5}O₂ down to deep cryogenic temperature 4.2K. In *Isaf-pfm-ecapd 2022*, Tours, France, juin 2022. IEEE UFFC. URL:<https://hal.science/hal-03719069>.

- [374] P.-A. Mouny, Y. Beilliard, S. Graveline, M.-A. Roux, A. E. Mesoudy, R. Dawant, P. Gliech, S. Ecoffey, F. Alibart, M. Pioro-Ladrière et D. Drouin. Memristor-based cryogenic programmable DC sources for scalable in situ quantum-dot control. *IEEE Transactions on Electron Devices*, 70(4):1989–1995, avr. 2023. DOI:10.1109/ted.2023.3244133.
 - [375] R. W. J. Overwater, M. Babaie et F. Sebastiano. Neural-network decoders for quantum error correction using surface codes: A space exploration of the hardware cost-performance tradeoffs. *IEEE Transactions on Quantum Engineering*, 3:1–19, 2022. DOI:10.1109/tqe.2022.3174017.
 - [376] J. Preskill. Reliable quantum computers. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, 454(1969):385–410, jan. 1998. DOI:10.1098/rspa.1998.0167.
 - [377] M. E. Beverland, P. Murali, M. Troyer, K. M. Svore, T. Hoefler, V. Kliuchnikov, G. H. Low, M. Soeken, A. Sundaram et A. Vashillo. Assessing requirements to scale to practical quantum advantage. *ArXiv*, 2022. DOI:10.48550/arxiv.2211.07629. Preprint.
 - [378] D. Camps, E. Rrapaj, K. Klymko, B. Austin et N. J. Wright. Evaluation of the classical hardware requirements for large-scale quantum computations. In *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*, p. 1–12. IEEE, mai 2024. DOI:10.23919/isc.2024.10528937.
 - [379] A. Holmes, M. R. Jokar, G. Pasandi, Y. Ding, M. Pedram et F. T. Chong. NISQ+: Boosting quantum computing power by approximating quantum error correction. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, p. 556–569. IEEE, mai 2020. DOI:10.1109/isca45697.2020.00053.
 - [380] Y. Ueno, M. Kondo, M. Tanaka, Y. Suzuki et Y. Tabuchi. QECool: On-line quantum error correction with a superconducting decoder for surface code. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*, p. 451–456. IEEE, déc. 2021. DOI:10.1109/dac18074.2021.9586326.
 - [381] Y. Ueno, M. Kondo, M. Tanaka, Y. Suzuki et Y. Tabuchi. QULATIS: A quantum error correction methodology toward lattice surgery. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, p. 274–287. IEEE, avr. 2022. DOI:10.1109/hpca53966.2022.00028.
 - [382] P. Das, C. A. Pattison, S. Manne, D. M. Carmean, K. M. Svore, M. Qureshi et N. Delfosse. AFS: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, p. 259–273. IEEE, avr. 2022. DOI:10.1109/hpca53966.2022.00027.
 - [383] N. Liyanage, Y. Wu, S. Tagare et L. Zhong. FPGA-based distributed union-find decoder for surface codes. *IEEE Transactions on Quantum Engineering*, 5:1–18, 2024. DOI:10.1109/tqe.2024.3467271.
 - [384] X. Tan, F. Zhang, R. Chao, Y. Shi et J. Chen. Scalable surface-code decoders with parallelization in time. *PRX Quantum*, 4(4), déc. 2023. DOI:10.1103/prxquantum.4.040344.
-

- [385] L. Skoric, D. E. Browne, K. M. Barnes, N. I. Gillespie et E. T. Campbell. Parallel window decoding enables scalable fault tolerant quantum computation. *Nature Communications*, 14(1), nov. 2023. DOI:10.1038/s41467-023-42482-1.
- [386] H. Bombín, C. Dawson, Y.-H. Liu, N. Nickerson, F. Pastawski et S. Roberts. Modular decoding: Parallelizable real-time decoding for quantum computers. *ArXiv*, 2023. DOI:10.48550/arxiv.2303.04846. Preprint.
- [387] N. Verma, H. Jia, H. Valavi, Y. Tang, M. Ozatay, L.-Y. Chen, B. Zhang et P. Dea-ville. In-memory computing: Advances and prospects. *IEEE Solid-State Circuits Magazine*, 11(3):43–55, 2019. DOI:10.1109/mssc.2019.2922889.
- [388] A. El Mesoudy, G. Lamri, R. Dawant, J. Arias-Zapata, P. Gliech, Y. Beilliard, S. Ecoffey, A. Ruediger, F. Alibart et D. Drouin. Fully CMOS-compatible passive TiO₂-based memristor crossbars for in-memory computing. *Microelectronic Engineering*, 255:111706, fév. 2022. DOI:10.1016/j.mee.2021.111706.
- [389] K. Berggren, Q. Xia, K. K. Likharev, D. B. Strukov, H. Jiang, T. Mikolajick, D. Querlioz, M. Salinga, J. R. Erickson, S. Pi, F. Xiong, P. Lin, C. Li, Y. Chen, S. Xiong, B. D. Hoskins, M. W. Daniels, A. Madhavan, J. A. Liddle, J. J. McClelland, Y. Yang, J. Rupp, S. S. Nonnenmann, K.-T. Cheng, N. Gong *et al.* Roadmap on emerging hardware and technology for machine learning. *Nanotechnology*, 32(1):012002, oct. 2020. DOI:10.1088/1361-6528/aba70f.
- [390] M.-K. Song, J.-H. Kang, X. Zhang, W. Ji, A. Ascoli, I. Messaris, A. S. Demirkol, B. Dong, S. Aggarwal, W. Wan, S.-M. Hong, S. G. Cardwell, I. Boybat, J. sun Seo, J.-S. Lee, M. Lanza, H. Yeon, M. Onen, J. Li, B. Yildiz, J. A. del Alamo, S. Kim, S. Choi, G. Milano, C. Ricciardi *et al.* Recent advances and future prospects for memristive materials, devices, and systems. *ACS Nano*, 17(13):11994–12039, juin 2023. DOI:10.1021/acsnano.3c03505.
- [391] D. Kumar, R. Aluguri, U. Chand et T. Tseng. Metal oxide resistive switching memory: Materials, properties and switching mechanisms. *Ceramics International*, 43:S547–s556, août 2017. DOI:10.1016/j.ceramint.2017.05.289.
- [392] J. Woo et S. Yu. Resistive memory-based analog synapse: The pursuit for linear and symmetric weight update. *IEEE Nanotechnology Magazine*, 12(3):36–44, sept. 2018. DOI:10.1109/mnano.2018.2844902.
- [393] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh et E. Eleftheriou. Memory devices and applications for in-memory computing. *Nature Nanotechnology*, 15(7):529–544, mars 2020. DOI:10.1038/s41565-020-0655-z.
- [394] M. Hu, J. P. Strachan, Z. Li, E. M. Grafals, N. Davila, C. Graves, S. Lam, N. Ge, J. J. Yang et R. S. Williams. Dot-product engine for neuromorphic computing: Programming 1T1M crossbar to accelerate matrix-vector multiplication. In *Proceedings of the 53rd Annual Design Automation Conference, DAC '16*, p. 1–6. ACM, juin 2016. DOI:10.1145/2897937.2898010.
- [395] P. Wang, X. Peng, W. Chakraborty, A. I. Khan, S. Datta et S. Yu. Cryogenic benchmarks of embedded memory technologies for recurrent neural network based quantum error correction. In *2020 IEEE International Electron Devices Meeting (IEDM)*, p. 38.5.1–38.5.4. IEEE, déc. 2020. DOI:10.1109/iedm13553.2020.9371912.

- [396] Y. Ichikawa, A. Goda, C. Matsui et K. Takeuchi. Non-volatile memory application to quantum error correction with non-uniformly quantized CiM. *In 2022 IEEE International Memory Workshop (IMW)*, p. 1–4. IEEE, mai 2022. DOI:10.1109/imw52921.2022.9779280.
 - [397] H. Ali, J. Marques, O. Crawford, J. Majaniemi, M. Serra-Peralta, D. Byfield, B. Varbanov, B. M. Terhal, L. DiCarlo et E. T. Campbell. Reducing the error rate of a superconducting logical qubit using analog readout information. *Physical Review Applied*, 22(4), oct. 2024. DOI:10.1103/physrevapplied.22.044031.
 - [398] B. Barber, K. M. Barnes, T. Bialas, O. Buğdaycı, E. T. Campbell, N. I. Gillespie, K. Johar, R. Rajan, A. W. Richardson, L. Skoric, C. Topal, M. L. Turner et A. B. Ziad. A real-time, scalable, fast and resource-efficient decoder for a quantum computer. *Nature Electronics*, jan. 2025. DOI:10.1038/s41928-024-01319-5.
 - [399] P.-A. Mouny, M. Benhouria, V. Yon, P. Dufour, L. Huang, Y. Beilgaard, S. Rochette, D. Drouin et P. Ronagh. Towards a cryogenic CMOS-memristor neural decoder for quantum error correction. *In 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, p. 1258–1263. IEEE, sept. 2024. DOI:10.1109/qce60285.2024.00149.
 - [400] L. Caune, L. Skoric, N. S. Blunt, A. Ruban, J. McDaniel, J. A. Valery, A. D. Patterson, A. V. Gramolin, J. Majaniemi, K. M. Barnes, T. Bialas, O. Buğdaycı, O. Crawford, G. P. Gehér, H. Krovi, E. Matekole, C. Topal, S. Poletto, M. Bryant, K. Snyder, N. I. Gillespie, G. Jones, K. Johar, E. T. Campbell et A. D. Hill. Demonstrating real-time and low-latency quantum error correction with superconducting qubits. *ArXiv*, 2024. DOI:10.48550/arxiv.2410.05202. Preprint.
 - [401] A. Stillmaker et B. Baas. Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration*, 58:74–81, juin 2017. DOI:10.1016/j.vlsi.2017.02.002.
 - [402] V. Joshi, M. Le Gallo, S. Haefeli, I. Boybat, S. R. Nandakumar, C. Piveteau, M. Dazzi, B. Rajendran, A. Sebastian et E. Eleftheriou. Accurate deep neural network inference using computational phase-change memory. *Nature Communications*, 11(1):2473, mai 2020. DOI:10.1038/s41467-020-16108-9.
 - [403] Y. Xi, B. Gao, J. Tang, A. Chen, M.-F. Chang, X. S. Hu, J. V. D. Spiegel, H. Qian et H. Wu. In-memory learning with analog resistive switching memory: A review and perspective. *Proceedings of the IEEE*, 109(1):14–42, jan. 2021. DOI:10.1109/jproc.2020.3004543.
 - [404] I. Chakraborty, M. Ali, A. Ankit, S. Jain, S. Roy, S. Sridharan, A. Agrawal, A. Raghunathan et K. Roy. Resistive crossbars as approximate hardware building blocks for machine learning: Opportunities and challenges. *Proceedings of the IEEE*, 108(12):2276–2310, déc. 2020. DOI:10.1109/jproc.2020.3003007.
 - [405] H. Bombin et M. A. Martin-Delgado. Optimal resources for topological two-dimensional stabilizer codes: Comparative study. *Phys. Rev. A*, 76:012305, juil. 2007. DOI:10.1103/PhysRevA.76.012305.
 - [406] D. Horsman, A. G. Fowler, S. Devitt et R. V. Meter. Surface code quantum computing by lattice surgery. *New Journal of Physics*, 14(12):123011, déc. 2012. DOI:10.1088/1367-2630/14/12/123011.
-

-
- [407] Y. Tomita et K. M. Svore. Low-distance surface codes under realistic quantum noise. *Physical Review A*, 90(6), déc. 2014. DOI:10.1103/physreva.90.062320.
 - [408] D. Litinski. A game of surface codes: Large-scale quantum computing with lattice surgery. *Quantum*, 3:128, 2019. DOI:<https://doi.org/10.22331/q-2019-03-05-128>.
 - [409] S. Varsamopoulos, K. Bertels et C. G. Almudever. Decoding surface code with a distributed neural network-based decoder. *Quantum Machine Intelligence*, 2(1):3, 2020. DOI:10.1007/s42484-020-00015-9.
 - [410] N. Delfosse. Hierarchical decoding to reduce hardware requirements for quantum computing. *ArXiv*, 2020. DOI:10.48550/arxiv.2001.11427. Preprint.
 - [411] V. Yon et G. Dagneu. A memristive neural decoder for cryogenic fault-tolerant quantum error correction - Syndromes dataset, mai 2024. DOI:10.5281/zenodo.11166209.
 - [412] C. Chamberland et M. E. Beverland. Flag fault-tolerant error correction with arbitrary distance codes. *Quantum*, 2:53, fév. 2018. DOI:10.22331/q-2018-02-08-53.
 - [413] T. Gokmen, M. J. Rasch et W. Haensch. Training LSTM networks with resistive cross-point devices. *Frontiers in Neuroscience*, 12, oct. 2018. DOI:10.3389/fnins.2018.00745.
 - [414] V. Milo, G. Malavena, C. Monzio Compagnoni et D. Ielmini. Memristive and CMOS devices for neuromorphic computing. *Materials*, 13(1):166, jan. 2020. DOI:10.3390/ma13010166.
 - [415] F. Alibart, L. Gao, B. D. Hoskins et D. B. Strukov. High precision tuning of state for memristive devices by adaptable variation-tolerant algorithm. *Nanotechnology*, 23(7):075201, jan. 2012. DOI:10.1088/0957-4484/23/7/075201.
 - [416] D. Ielmini et H.-S. P. Wong. In-memory computing with resistive switching devices. *Nature Electronics*, 1(6):333–343, juin 2018. DOI:10.1038/s41928-018-0092-2.
 - [417] Y. Zhang, P. Huang, B. Gao, J. Kang et H. Wu. Oxide-based filamentary RRAM for deep learning. *Journal of Physics D: Applied Physics*, 54(8):083002, déc. 2020. DOI:10.1088/1361-6463/abc5e7.
 - [418] C.-Y. Chen, H.-C. Shih, C.-W. Wu, C.-H. Lin, P.-F. Chiu, S.-S. Sheu et F. T. Chen. RRAM defect modeling and failure analysis based on march test and a novel squeeze-search scheme. *IEEE Transactions on Computers*, 64(1):180–190, jan. 2015. DOI:10.1109/tc.2014.12.
 - [419] H. Kim, M. R. Mahmoodi, H. Nili et D. B. Strukov. 4K-memristor analog-grade passive crossbar circuit. *Nature Communications*, 12(1), août 2021. DOI:10.1038/s41467-021-25455-0.
 - [420] J. Jang, S. Gi, I. Yeo, S. Choi, S. Jang, S. Ham, B. Lee et G. Wang. A Learning-Rate Modulable and Reliable TiO_x Memristor Array for Robust, Fast, and Accurate Neuromorphic Computing. *Advanced Science*, 9(22):2201117, juin 2022. DOI:10.1002/adv.202201117.
 - [421] H. Yeon, P. Lin, C. Choi, S. H. Tan, Y. Park, D. Lee, J. Lee, F. Xu, B. Gao, H. Wu, H. Qian, Y. Nie, S. Kim et J. Kim. Alloying conducting channels for reliable neuromorphic computing. *Nature Nanotechnology*, 15(7):574–579, juin 2020. DOI:10.1038/s41565-020-0694-5.
-

- [422] M. J. Rasch, D. Moreda, T. Gokmen, M. Le Gallo, F. Carta, C. Goldberg, K. El Maghraoui, A. Sebastian et V. Narayanan. A flexible and fast PyTorch toolkit for simulating training and inference on analog crossbar arrays. In *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, p. 1–4. IEEE, juin 2021. DOI:10.1109/aicas51828.2021.9458494.
 - [423] L. Wan, M. Zeiler, S. Zhang, Y. Le Cun et R. Fergus. Regularization of neural networks using dropconnect. In S. Dasgupta et D. McAllester, édés, *Proceedings of the 30th International Conference on Machine Learning*, vol. 28 de *Proceedings of Machine Learning Research*, p. 1058–1066, Atlanta, Georgia, USA, 17–19 Jun 2013. PMLR. URL:<https://proceedings.mlr.press/v28/wan13.html>.
 - [424] V. Yon. A memristive neural decoder for cryogenic fault-tolerant quantum error correction - Simulation data, mai 2024. DOI:10.5281/zenodo.11164068.
 - [425] A. van de Goor et Y. Zorian. Effective march algorithms for testing single-order addressed memories. In *1993 European Conference on Design Automation with the European Event in ASIC Design, EDAC-93*, p. 499–505. IEEE Computer Society Press. DOI:10.1109/edac.1993.386425.
 - [426] Y.-X. Chen et J.-F. Li. Fault modeling and testing of 1T1R memristor memories. In *2015 IEEE 33rd VLSI Test Symposium (VTS)*. IEEE, avr. 2015. DOI:10.1109/vts.2015.7116247.
 - [427] O. Krestinskaya, B. Choubey et A. P. James. Memristive GAN in analog. *Scientific Reports*, 10(1), avr. 2020. DOI:10.1038/s41598-020-62676-7.
 - [428] I. Surekcigil Pesch, E. Bestelink, O. de Sagazan, A. Mehonic et R. A. Sporea. Multi-modal transistors as ReLU activation functions in physical neural network classifiers. *Scientific Reports*, 12(1), jan. 2022. DOI:10.1038/s41598-021-04614-9.
 - [429] B. Liu, H. Li, Y. Chen, X. Li, T. Huang, Q. Wu et M. Barnell. Reduction and IR-drop compensations techniques for reliable neuromorphic computing systems. In *2014 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, p. 63–70. IEEE, nov. 2014. DOI:10.1109/iccad.2014.7001330.
 - [430] B. Liu, H. Li, Y. Chen, X. Li, Q. Wu et T. Huang. Vortex: Variation-aware training for memristor X-bar. In *Proceedings of the 52nd Annual Design Automation Conference, DAC '15*. ACM, juin 2015. DOI:10.1145/2744769.2744930.
 - [431] K. Meinerz, C.-Y. Park et S. Trebst. Scalable neural decoder for topological surface codes. *Physical Review Letters*, 128(8), fév. 2022. DOI:10.1103/physrevlett.128.080505.
 - [432] S. Varsamopoulos, K. Bertels et C. G. Almudever. Comparing neural network based decoders for the surface code. *IEEE Transactions on Computers*, 69(2):300–311, fév. 2020. DOI:10.1109/tc.2019.2948612.
 - [433] C. Chamberland, L. Goncalves, P. Sivarajah, E. Peterson et S. Grimberg. Techniques for combining fast local decoders with global decoders under circuit-level noise. *ArXiv*, 2022. URL:<https://arxiv.org/abs/2208.01178>. Preprint.
 - [434] A. Bayerstadler, G. Becquin, J. Binder, T. Botter, H. Ehm, T. Ehmer, M. Erdmann, N. Gaus, P. Harbach, M. Hess, J. Klepsch, M. Leib, S. Luber, A. Luckow, M. Mansky, W. Maurer, F. Neukart, C. Niedermeier, L. Palackal, R. Pfeif-
-

- fer, C. Polenz, J. Sepulveda, T. Sievers, B. Standen, M. Streif *et al.* Industry quantum computing applications. *EPJ Quantum Technology*, 8(1), nov. 2021. DOI:10.1140/epjqt/s40507-021-00114-x.
- [435] P.-A. Mouny, V. Yon, S. Rochette, P. Ronagh, D. Drouin, Y. Beilliard et F. Marcotte. A memristor-based circuit for implementing a quantum error correction decoder and methods for training and inference of a memristor-based quantum error correction decoder neural network, nov. 2024. URL:<https://patentscope.wipo.int/search/en/detail.jsf?docId=W02024234109>. Brevet.
- [436] I. Labelbox. Labelbox, 2024. URL:<https://labelbox.com>.
- [437] D. P. Kingma et J. Ba. Adam: A method for stochastic optimization. *ArXiv*, 2014. DOI:10.48550/arxiv.1412.6980. Preprint.
- [438] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai et S. Chintala. PyTorch: An imperative style, high-performance deep learning library. *ArXiv*, 2019. DOI:10.48550/arxiv.1912.01703. Preprint.
- [439] P. Esposito. BLiTZ - Bayesian layers in Torch Zoo (a Bayesian deep learning library for Torch). *GitHub repository*, 2020. URL:<https://github.com/piEsposito/blitz-bayesian-deep-learning>.
- [440] V. T. Vasudevan, A. Sethy et A. R. Ghias. Towards better confidence estimation for neural models. In *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, mai 2019. DOI:10.1109/icassp.2019.8683359.
- [441] D. Hendrycks et K. Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. *ArXiv*, 2016. DOI:10.48550/arxiv.1610.02136. Preprint.
- [442] M. Sensoy, L. Kaplan et M. Kandemir. Evidential deep learning to quantify classification uncertainty. *ArXiv*, 2018. DOI:10.48550/arxiv.1806.01768. Preprint.
- [443] M. Możejko, M. Susik et R. Karczewski. Inhibited softmax for uncertainty estimation in neural networks. *ArXiv*, 2018. DOI:10.48550/arxiv.1810.01861. Preprint.
- [444] O. Higgott. PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. *ACM Transactions on Quantum Computing*, 3(3):1–16, juin 2022. DOI:10.1145/3505637.
- [445] M. Spear, J. E. Kim, C. H. Bennett, S. Agarwal, M. J. Marinella et T. P. Xiao. The impact of analog-to-digital converter architecture and variability on analog neural network accuracy. *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, 9(2):176–184, déc. 2023. DOI:10.1109/jxcdc.2023.3315134.
- [446] C. Wang, H. Wu, B. Gao, T. Zhang, Y. Yang et H. Qian. Conduction mechanisms, dynamics and stability in ReRAMs. *Microelectronic Engineering*, 187–188:121–133, fév. 2018. DOI:10.1016/j.mee.2017.11.003.